

Design Whitepaper

Architecture & Design

How the QtRocket Simulator Works

A rigorous exposition of the QtRocket simulation core — its layered architecture, its data model, the physics behind its algorithms, and the areas still to be built — written as a reference for engineers joining the project.

Status	Describes the working tree at commit 254cd41 (“Major comment cleanup”)
Scope	Every layer: <code>model/</code> , <code>sim/</code> , <code>utils/</code> , <code>gui/</code> , <code>cli/</code> , <code>visualizer/</code> , <code>build</code> , <code>tests</code> , <code>CI</code>
Target	QtRocket simulation core (C++23); baseline branch <code>development</code>
Audience	Engineers new to the project, with or without a rocketry background

Abstract

QtRocket is an open-source model-rocket simulator: a C++23 simulation core shared by a Qt6 Widgets GUI, a Qt-free headless REPL, and a standalone OpenGL design viewer. This whitepaper is the project's design reference — a rigorous exposition of how the software works, written for an engineer joining the project with or without a rocketry background. It describes the working tree at commit [254cd41](#): the architecture, the data model, the algorithms, and the physics underneath them, with every claim about the code pinned to a [file:line](#) citation into that revision.

The document opens with a first-principles primer on model-rocket flight — forces, stability, the atmosphere, and why simulation means numerical integration — and then descends through the system: the strict four-layer link graph under three executables; the composite part tree with its lazily cached, time-aware mass, center-of-mass, and inertia composition; the placement system that stores physical intent and derives all geometry through a single resolver; the motor subsystem that turns three ingest formats into thrust and mass curves; the propagator with its fixed-step RK4 and embedded-pair adaptive RKF45 solvers and typed termination taxonomy; the seven-layer US Standard Atmosphere 1976 and pluggable gravity models; the live quadratic-drag force path beside the fully built but dormant Barrowman stability layer; the three user-facing surfaces; the versioned XML persistence formats; and the 246-test, five-platform verification infrastructure. UML class, sequence, and component diagrams accompany each subsystem.

QtRocket is not a finished simulator, and the document says so precisely: each chapter marks what is dormant, partial, or absent where it lives, and a consolidated inventory orders the distance to a full-featured flight simulator — flight dynamics first, recovery second — around the seams the architecture has already built and tested. Four appendices compare QtRocket with OpenRocket, the de facto standard the project treats as a calibration baseline rather than a specification: comparison intent and project histories, a sixteen-area feature matrix, simulation methods side by side, and the two software architectures.

The text was produced from full deep-reads of every subsystem and verified adversarially: every citation re-opened against the source, every listing diffed verbatim, every equation re-derived, every figure checked element-by-element ([Section F](#)). The companion Architecture Review assesses the same revision's capabilities and gaps; this document is the map of how it works.

The document in one sentence

A thesis-style reference that teaches QtRocket's layered core — part tree, placement resolver, motors, propagator, environment, aerodynamics, interfaces, persistence, and tests — from the physics up, marks every not-yet-built area in place, and calibrates the whole against OpenRocket in four appendices.

Contents

Abstract	ii
1 Introduction & scope	1
1.1 Three documents, one series	1
1.2 Ground truth and citations	1
1.3 The state of the project, honestly	2
1.4 How to read this document	2
1.5 Document roadmap	3
2 A model-rocket flight, from first principles	4
2.1 Anatomy of a rocket, anatomy of a flight	4
2.2 The three forces	5
2.3 Newton’s second law as the equation the simulator integrates	6
2.4 Stability: the center of gravity, the center of pressure, and the arrow	9
2.5 The atmosphere	9
2.6 Why numerical integration	10
2.7 Motors: impulse classes, designations, and the thrust curve	11
2.8 From concept to code	12
3 System overview & layering	12
3.1 Three executables, one core	14
3.2 The layering is the link graph	14
3.3 The QtRocket controller	15
3.4 The Propagatable bridge	17
3.5 Build system in brief	19
4 The part tree & mass properties	20
4.1 The composite tree	20
4.2 The concrete catalog	23
4.3 Mass properties: the two-pass composite walk	25
4.4 Time-awareness and caching	27
4.5 The closed-form tensor library	29
4.6 Diagnostics gating on the composite readers	30
5 Placement: stations, seats, and the resolver	30
5.1 Philosophy: placement is intent, never coordinates	31
5.2 The data model: stations, links, and seats	32
5.3 The resolver, term by term	33
5.4 Diagnostics: the envelope sweep and the fail-closed gate	36
5.5 Authoring today: one verb reaches the front ends	39
5.6 The invariance gate: the system’s contract	39
6 The motor subsystem	40
6.1 The runtime model: MotorModel	40
6.2 ThrustCurve: piecewise-linear thrust	43
6.3 The mass model: constant- I_{sp} depletion	44

6.4	Ignition	46
6.5	MotorModelDatabase: one keyed surface	46
6.6	Three ingest paths	48
6.6.1	RockSim .rse: lenient XML	48
6.6.2	RASP .eng: strict text	50
6.6.3	thrustcurve.org: REST behind a seam	50
6.7	Impulse-class derivation and its one seam	51
7	The simulation core	52
7.1	The pieces and their contracts	52
7.2	The equations of motion, as coded	55
7.3	The run loop, step by step	56
7.4	The termination taxonomy	58
7.5	Numerical methods	59
7.5.1	The fixed-step backend	59
7.5.2	The adaptive backend: the Fehlberg 4(5) embedded pair	60
7.6	Recording subtleties	62
7.7	Flight statistics and their consumers	63
8	Environment models	64
8.1	The container: registries, selection, and injection	64
8.2	Gravity	66
8.3	The US Standard Atmosphere 1976	68
8.4	The constant and vacuum atmospheres	72
8.5	Wind: a dormant seam	73
9	Aerodynamics	73
9.1	The live force path	75
9.2	The reference area: derived correctly, delivered nowhere	77
9.3	The Barrowman method	78
9.4	The dormant implementation	79
9.5	Dormancy, verified	82
9.6	The fin set's mirrored chordwise frame	82
9.7	From seam to consumer	83
10	User interfaces	84
10.1	The Qt6 Widgets GUI	84
10.1.1	The entry point: QApplication on a spawned thread	85
10.1.2	MainWindow: the shell, the disabled File menu, and the stub panes	85
10.1.3	CannonballTab and SimOptionsTab	86
10.1.4	Motor search over the network	87
10.1.5	AnalysisWindow: exactly three fixed plots	88
10.2	The command-line REPL	88
10.2.1	Process model and the stdout protocol	88
10.2.2	The command inventory	89
10.2.3	Launching and the CSV artifact	90
10.2.4	Design authoring from the REPL	91
10.3	The standalone visualizer	91

10.3.1 Startup and the GL-context workaround	92
10.3.2 From .qrd to pixels: the shared loader and the shared resolver	92
10.4 The maturity asymmetry	93
11 Design persistence	94
11.1 The .qrd design file	94
11.2 What is deliberately — and not so deliberately — absent	96
11.3 The save path	97
11.4 The load path: build detached, install once	97
11.5 Round-trip guarantees	98
11.6 The fixture corpus	98
11.7 The motor database on disk: .qmd	100
11.8 Consumers and interoperability	100
12 Testing & infrastructure	101
12.1 Six binaries, two registrations	101
12.2 Oracle patterns	102
12.3 The light/heavy registration split	104
12.4 Test data and fixtures	105
12.5 Coverage	106
12.6 Continuous integration	107
12.7 Warning hygiene	108
12.8 What the infrastructure does not do	108
13 Yet to be implemented	109
13.1 Dormant seams	109
13.2 The 6-DOF cluster	111
13.3 Partial areas	112
13.4 Absent subsystems	113
13.5 A suggested order of work	114
13.6 The placement system's forward hook	115
A OpenRocket as comparison baseline	116
A.1 Why this baseline, and what it is not	116
A.2 OpenRocket in brief	117
A.3 Four philosophical differences	117
A.4 Reading guide	118
B Feature comparison with OpenRocket	118
B.1 The sixteen-area matrix	119
B.2 Where QtRocket is ahead	122
C Simulation-methods comparison with OpenRocket	124
C.1 Flight dynamics	124
C.2 Aerodynamic coefficients	125
C.3 Drag	126
C.4 Atmosphere and wind	127
C.5 Gravity and geodesy	127
C.6 Numerical integration	128

C.7	Motors	129
C.8	Recovery physics	130
C.9	Validation posture	131
C.10	Summary	131
D	Architecture comparison with OpenRocket	131
D.1	Language, runtime, and build	133
D.2	The design tree and mass properties	134
D.3	Positioning models	135
D.4	The simulation engine	136
D.5	Extension surfaces	137
D.6	Persistence	137
D.7	Motor data	138
D.8	GUI architecture	138
D.9	What each architecture makes easy	139
E	Glossary	139
E.1	Rocketry and physics terms	140
E.2	QtRocket terms of art	141
F	Reproducibility & colophon	143
F.1	Directory layout and pipeline	144
F.2	The revision under description	144
F.3	How this document was produced and verified	144

1 Introduction & scope

QtRocket is an open-source model-rocket simulator: a C++23 simulation core shared by three front ends — a Qt6 Widgets GUI (`qtrocket`), a Qt-free headless REPL (read-eval-print loop, `qtrocket-cli`), and a standalone OpenGL design viewer (`qtrocket-visualizer`). All three executables link one static library, `qtrocket_core`, which contains the application controller and pulls in the `model`, `sim`, and `utils` layer libraries beneath it `CMakeLists.txt:224`. The core describes a rocket as a composite tree of parts, resolves each part's absolute position from stored placement intent, computes time-aware mass, center of gravity (CG), and inertia while a motor burns, and integrates the resulting three-degree-of-freedom (3-DOF) point-mass flight with a fixed-step fourth-order Runge–Kutta (RK4) or adaptive Runge–Kutta–Fehlberg (RKF45) solver until a typed termination condition ends it.

This document is the project's design reference: a rigorous exposition of how that software works — its layered architecture, its data model, its algorithms, and the physics underneath them — written so that an engineer new to the project, with or without a rocketry background, can navigate the source and extend it without first reverse-engineering the design from the code. Each chapter therefore explains not only what a subsystem does but what it stores, what it derives, why the boundary sits where it does, and what the code deliberately does not do yet. Physics is developed before it is used: a reader who has never heard of a center of pressure will meet the concept in [Section 2](#) and in background callouts before any chapter leans on it.

1.1 Three documents, one series

This whitepaper is the third volume of a series with a deliberate division of labor. The Architecture Review ([docs/ArchitectureReviewLatex/](#)) is a capabilities-and-gaps review performed at the same revision this document describes: it measures what the program can actually do, adversarially verifies every claim against the source, and ranks what is missing against OpenRocket as a baseline. The Part Placement whitepaper ([docs/PartPlacementDesignLatex/](#)) is the design specification for the placement feature that now ships: the argument, written before implementation, for storing placement as physical intent and deriving all absolute geometry through a single resolver. Those two documents assess and specify; this one teaches. Where a companion's verified finding is load-bearing here — a located defect, a dormant-status verdict — this document states it in place with the same citation rather than re-deriving it, and it does not re-litigate the review's rankings or the specification's rejected alternatives except where the rationale is part of the design being taught.

1.2 Ground truth and citations

Every statement about the code is pinned to one revision: commit [254cd41](#) (“Major comment cleanup”), the tip of the `development` branch at the time of writing, reviewed against a clean working tree on 2026-07-07. That commit includes the completed Part Placement feature; work in flight on other branches is mentioned only where a chapter says so explicitly. Code citations take the form `model/parts/Part.h:130` and are placed directly after the claim they ground. Line numbers are anchors into that revision, not absolutes — they will drift as the tree moves, but they resolve exactly at the pinned commit.

Ground truth policy

Three sources are authoritative for this document: the source tree at pinned commit [254cd41](#), the Architecture Review ([docs/ArchitectureReviewLatex/](#)), and the Part Placement design whitepaper ([docs/PartPlacementDesignLatex/](#)). Every other artifact under `docs/` — the architecture audit, the

phase specifications, the implementation plans — is historical: the review verified that those files have drifted from the shipped code, and this document never cites them as fact. Where any document, including this one, disagrees with the tree, the tree wins. General physics background carries no citation and is framed explicitly as theory ([Section 1.4](#)); it is never attributed to the code.

1.3 The state of the project, honestly

QtRocket at this commit is a finished, well-tested 3-DOF point-mass simulator surrounded by machinery staged for six degrees of freedom (6-DOF). What flies is small: thrust pinned along the world vertical, “assumed through the CM” `model/RocketModel.cpp:70`, pluggable gravity, and a single constant- C_d quadratic drag term; there is no angle of attack, no restoring moment, no recovery deployment, and every flight ends ballistically, descending below the launch site `model/RocketModel.cpp:62`. Around that core sits what the companion review calls “a ring of machinery that is built, unit-tested, and consumed by nothing”: the per-part Barrowman center-of-pressure composition `model/parts/Part.cpp:231`, a torque interface that returns zero and has no caller `model/RocketModel.cpp:94`, quaternion orientation slots in the state vector `sim/StateData.h:46`, a commented-out orientation integrator `sim/Propagator.h:103`, and a wind model that returns the zero vector `sim/WindModel.cpp:16`. The staging is deliberate, and the tree says so itself: “nothing in production consumes this in P2 – these tests pin the seam for P5” `model/tests/AeroTests.cpp:18`. This document teaches the dormant machinery alongside the live code, at the same depth, because the dormant seams are the shape of the next tier of the simulator and a new engineer will extend them, not replace them.

How this document marks unbuilt work

Three status words recur, with the companion review’s exact senses. *Dormant* means built and tested but with zero production consumers — unreachable from any shipping code path. *Partial* means reachable but with named gaps. *Absent* means no trace in the tree. A *seam* is an interface or capability built and tested ahead of its consumer; QtRocket’s development order is seams first, consumers later. Each chapter marks its dormant, partial, and absent areas plainly, in place, where the surrounding machinery is taught; [Section 13](#) consolidates all of them into a single inventory. Expository chapters do not editorialize about what should be built next — that is the review’s job.

1.4 How to read this document

Four conventions hold everywhere, and the code enforces or documents each of them.

- **SI units throughout:** meters, kilograms, seconds, newtons. The constant gravity model, for example, returns an acceleration of $(0, 0, -9.80665) \text{ m/s}^2$ `sim/ConstantGravityModel.h:18`.
- **World frame:** z up. Altitude is the z component of position, the launch site is the origin, and the nominal end of flight is descending below $z = 0$ `model/RocketModel.cpp:62`.
- **Part frame:** $+z = \text{forward}$. Every part is modeled in its own frame with $+z$ toward the nose, its fore plane at the local origin, and its body occupying $z \in [-L, 0]$ `model/parts/Part.h:130`. When the placement resolver plants the root part’s fore plane at the world origin, the nose tip becomes the datum for the whole assembly; composite CG and center of pressure (CP) are both reported against it ([Sections 5 and 9](#)).

- **Quaternions are stored** (x, y, z, w) — vector part first, scalar last. `Quaternion` is `Eigen::Quaterniond` `utils/math/MathTypes.h:16`, and the state vector’s orientation members carry the “(vector, scalar)” annotation `sim/StateData.h:46`. Eigen’s value constructor takes (w, x, y, z) ; [Section 13](#) flags that hazard where the dormant slots are inventoried.

Inline identifiers are set in colored teletype (`getCompositeMass`); verbatim code fragments use real C++ syntax (`getMass(t)`). Listings quote the source verbatim and cite it in the caption. Acronyms are expanded at first use in each chapter. Beyond the running text, five callout boxes carry distinct semantics:

Design rule (blue). A rule the code enforces, or an invariant the design commits to — quoting the in-tree comment where it binds whenever one exists.

Key correctness detail (teal). A subtlety that is easy to miss and expensive to rediscover: why a cache key is sound, why a clamp must exist, how a guard prevents a failure that once occurred.

Hard, located failure (red). A verified defect present at the pinned commit, stated with its mechanism and its file:line location. These are inherited from the companion review’s adversarial verification and restated here where the affected machinery is taught.

Rejected alternative (amber). A design alternative that was considered and turned down, kept because the argument that killed it is part of the design.

Physical background (dark blue). Pure physics: theory a newcomer needs, developed from first principles. The separation is deliberate — anything inside a physics box is background and carries no code citation; anything outside one that describes behavior is a statement about the code and does.

1.5 Document roadmap

[Section 2](#) opens with physics, not code: a model-rocket flight from first principles — thrust, drag, stability, and the vocabulary (center of pressure, static margin, apogee, recovery) the rest of the document uses freely. Readers who already know rocketry can skip it. [Section 3](#) maps the system: the three executables, the layer libraries and the link graph that enforces their ordering, the application controller, and the `Propagatable` bridge between model and simulation. Three chapters then build the rocket: [Section 4](#) covers the composite part tree and its time-aware mass, CG, and inertia machinery; [Section 5](#) the placement model — placement intent stored as fractional stations and seat kinds, the single resolver that turns intent into absolute geometry, and the fail-closed overlap diagnostics gating it; [Section 6](#) the motor subsystem, from thrust-curve interpolation and the reconstructed mass curve to the database and its three ingest paths. Three more chapters fly it: [Section 7](#) the propagator loop, the two solvers, and the typed termination taxonomy; [Section 8](#) the pluggable atmosphere, gravity, and geoid models; [Section 9](#) the aerodynamics — the small consumed force path and the dormant Barrowman layer beside it. [Section 10](#) covers the three user-facing surfaces, from the GUI’s launch panel to the 33-command scriptable REPL `cli/Repl.cpp:245`; [Section 11](#) the design and motor-database file formats and their round-trip guarantees; [Section 12](#) the test suites, coverage tooling, and continuous integration. [Section 13](#) closes the body with the consolidated inventory of everything dormant, partial, or absent.

Four appendices place QtRocket next to OpenRocket, the de facto standard open-source model-rocket simulator. The intent is calibration, not conformance: QtRocket is an independent project with similar goals, OpenRocket is the baseline because it defines what a mature tool in this space offers, and the comparison measures distance from that surface — never compliance with it as a specification. [Section A](#) establishes the baseline and that framing; [Section B](#) compares feature surface; [Section C](#) simulation methods; [Section D](#) architecture. OpenRocket-side claims are researched from

that project’s published documentation and sources rather than verified file-by-line the way QtRocket claims are, and the appendices say so. [Section E](#) collects the series’ terms of art, and [Section F](#) records how this document itself builds and the ground rules under which it was written.

2 A model-rocket flight, from first principles

This chapter is for the engineer who has never flown a model rocket. It builds, from first principles, every physical concept the rest of the document leans on. Pure theory is set apart in *Physical background* boxes so it is never conflated with claims about the code; every statement about what QtRocket actually does carries a source citation into the pinned commit [254cd41](#), and the chapter closes with a table mapping each concept to the class that implements it and the chapter that examines it.

2.1 Anatomy of a rocket, anatomy of a flight

A sport model rocket is a slender axisymmetric stack: a *nose cone* at the top, one or more cylindrical *body tubes*, a set of three or four *fins* at the aft end, and a replaceable solid-propellant *motor* in a mount at the tail. Real airframes also carry a recovery device (parachute or streamer) packed in the body tube and a launch lug or rail button that guides the rocket along a rod or rail for its first meter of flight, before the fins have enough airspeed to work. QtRocket’s part catalog covers the structural core of this anatomy — nose cone, body tube, fin set, plus a hollow sphere and the motor leaf — assembled into an ownership tree ([Section 4](#)); recovery devices and launch guides are among the part types that do not exist yet ([Section 13](#)).

A flight divides into four phases punctuated by one instant, and the vocabulary recurs throughout this document ([Fig. 1](#)):

Pad. The rocket sits on the launch rod at altitude zero until the igniter fires. QtRocket models the pad as a kinematic clamp: until the first time step that ends above $z = 0$ under power, position and velocity are held at rest, standing in for the normal force a real pad exerts [sim/Propagator.cpp:105](#). A rocket that cannot lift off is detected — more than 3 s without exceeding 1 m — and the run ends with a typed `NoLiftoff` verdict rather than silently [sim/Propagator.cpp:146](#), [sim/Propagator.h:26](#).

Boost. The motor burns, typically for half a second to a few seconds, producing a thrust force that varies strongly over the burn. Production code ignites the motor at exactly $t = 0$ [model/RocketModel.cpp:108](#). During boost the rocket both accelerates and *lightens*: the propellant leaves through the nozzle.

Coast. After burnout the rocket is a projectile decelerating under gravity and drag. In a real motor a slow-burning *delay charge* smolders through this phase; QtRocket’s motor model reports zero thrust after the last thrust sample [model/ThrustCurve.cpp:51](#) and the vehicle simply coasts.

Apogee. The instant of peak altitude, where vertical velocity passes through zero. This is where a real flight fires the ejection charge and deploys recovery. QtRocket records apogee as a running maximum folded into the flight statistics every step [sim/TrajectoryStatistics.h:26](#); nothing else happens there.

Descent. A real rocket descends under parachute at a few meters per second. QtRocket has no recovery events at the pinned commit: every flight descends *ballistically*, under the same drag law as ascent, and terminates when the model reports that it is both below the launch site and descending — $z < 0$ and $v_z < 0$ [model/RocketModel.cpp:65](#). Ejection-delay data is parsed by the local motor-file loaders and persisted, but no flight logic consumes it [model/MotorModel.h:303](#).

The consolidated inventory of unbuilt flight events lives in [Section 13](#).

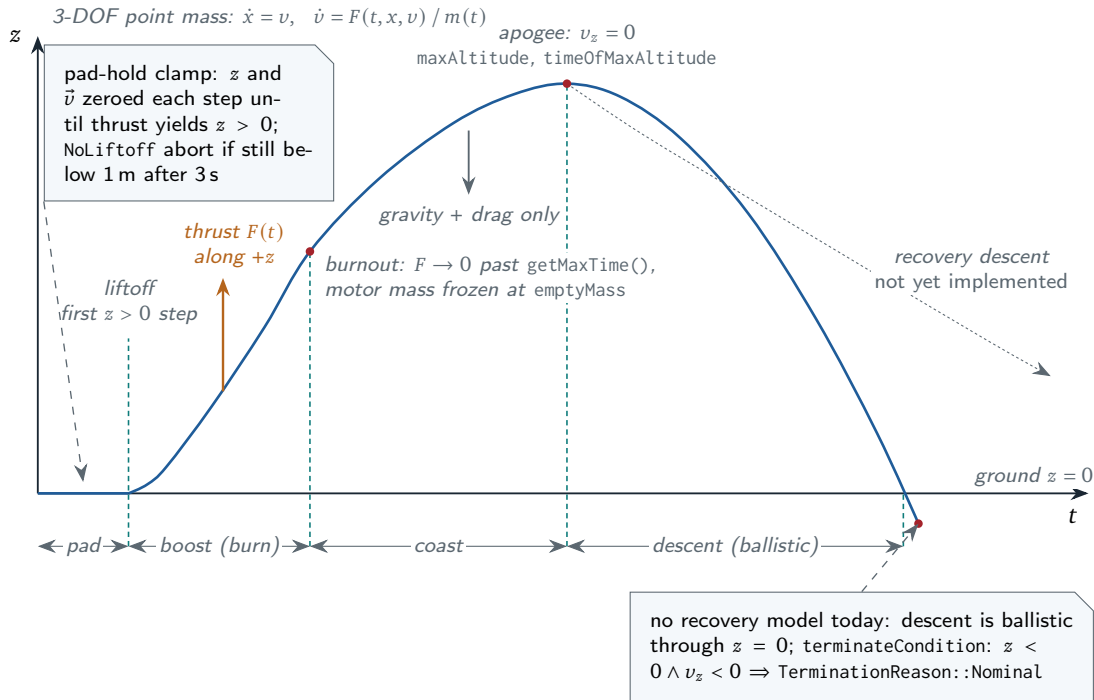


Figure 1. The phases of a model-rocket flight, annotated with the code that observes each one. QtRocket holds the vehicle on the pad until the first powered step ends above $z = 0$ `sim/Propagator.cpp:105`, ignites the motor at $t = 0$ `model/RocketModel.cpp:108`, tracks apogee as a running maximum `sim/TrajectoryStatistics.h:26`, and — because no recovery events exist at the pinned commit — flies the descent ballistically until the termination predicate $z < 0$ and $v_z < 0$ fires `model/RocketModel.cpp:65`.

2.2 The three forces

Three forces act on a rocket in flight, and QtRocket’s production force model assembles exactly these three terms and no others `model/RocketModel.cpp:68`.

Thrust. A solid motor’s thrust is not a constant; it is a measured function of time $F(t)$, published as a *thrust curve* — a table of (time, force) samples from a certification test stand. A typical curve spikes to its peak in the first tenth of a second (high initial acceleration gets the rocket moving fast enough for fin stabilization), then settles to a sustainer plateau, then tails off at burnout. QtRocket stores the curve as sampled pairs and interpolates linearly between samples, returning zero outside the burn window `model/ThrustCurve.cpp:47`. In flight, the thrust force is applied along the world vertical and assumed to act through the center of mass `model/RocketModel.cpp:71`; since no attitude state exists in the dynamics ([Section 2.3](#)), the rocket’s axis and the world z -axis are the same thing.

Weight. Weight is $m(t)g$ — and the time argument matters. The propellant that produces the thrust leaves the vehicle as it burns, so the mass in Newton’s second law falls throughout boost: propellant can be up to half of a small motor’s loaded mass. QtRocket obtains $m(t)$ from the part tree’s composite mass, with the motor as a time-aware leaf part ([Sections 4 and 6](#)), and evaluates gravity through a pluggable model — uniform $-g_0\hat{z}$ or Newtonian inverse-square — at the integrator’s trial position `model/RocketModel.cpp:77` ([Section 8](#)).

Drag. Air resistance is the force a newcomer is most likely to underestimate: for a lightweight airframe at a hundred meters per second it can rival or exceed the rocket’s weight, and it is the reason drag-free vacuum apogees run to multiples of real ones.

The quadratic drag law, symbol by symbol

For a blunt body moving faster than a slow walk, drag is well modeled by

$$D = \frac{1}{2} \rho v^2 C_d A. \quad (1)$$

Each symbol earns its place:

- ρ is the local *air density* (kg/m³). Drag is momentum transferred to the air the vehicle displaces, so it scales with how much air mass occupies each swept meter; density falls with altitude ([Section 2.5](#)).
- v^2 : the vehicle sweeps through air proportionally to v and imparts to that air a velocity change also proportional to v — hence the square, which makes the equation of motion nonlinear ([Section 2.6](#)).
- C_d is the dimensionless *drag coefficient*, the shape’s efficiency at evading the momentum bill: about 1.2 for a flat disc, 0.3–0.75 for typical model rockets.
- A is the *reference area* (m²) — a bookkeeping convention, not independent physics: C_d is only meaningful relative to a stated area, and the hobby convention is the airframe’s maximum frontal cross-section (the widest body disc, never inflated by fin span).

A useful corollary is *terminal velocity*, the speed at which drag balances weight in steady descent: $v_t = \sqrt{2mg/(\rho C_d A)}$.

QtRocket implements the vector form of this law in one line, written with $|v| \vec{v}$ rather than $v^2 \hat{v}$ so that zero velocity produces exactly zero drag with no division [model/RocketModel.cpp:88](#):

$$\vec{F}_{\text{drag}} = -\frac{1}{2} \rho(z) \|\vec{v}\| \text{dragCoefficient} \cdot \text{referenceArea} \vec{v}, \quad (2)$$

where $\rho(z)$ is sampled from the selected atmosphere model at the trial altitude, clamped to $z \geq 0$ [model/RocketModel.cpp:85](#). Both coefficients are user-set scalars with defaults $C_d = 1.0$ [model/RocketModel.h:138](#) and $A = 1.134 \times 10^{-3} \text{ m}^2$ — the frontal disc of a 38 mm body tube [model/RocketModel.h:141](#). A helper that derives the reference area from the assembled geometry under the max-frontal-disc convention exists [model/RocketModel.cpp:48](#), but its wiring status belongs to [Section 9](#). The drag term is also the best-tested physics in the tree: the terminal-velocity force balance above is pinned analytically by an integration test [tests/PhysicsIntegrationTests.cpp:262](#), and drag-versus-vacuum apogee sweeps run across every reference airframe and the full motor ladder [tests/DesignMatrixTests.cpp:515](#).

2.3 Newton’s second law as the equation the simulator integrates

Summing the three forces gives the equation of motion. Because the mass changes, it is worth writing carefully:

$$\dot{\vec{x}} = \vec{v}, \quad \dot{\vec{v}} = \frac{\vec{F}_{\text{thrust}}(t) + \vec{F}_{\text{gravity}}(\vec{x}, t) + \vec{F}_{\text{drag}}(\vec{x}, \vec{v})}{m(t)}. \quad (3)$$

The second-order law $\ddot{\vec{x}} = \vec{F}/m$ has been reduced to a coupled first-order pair — the standard trick that lets a general-purpose ODE (ordinary differential equation) solver advance position and velocity

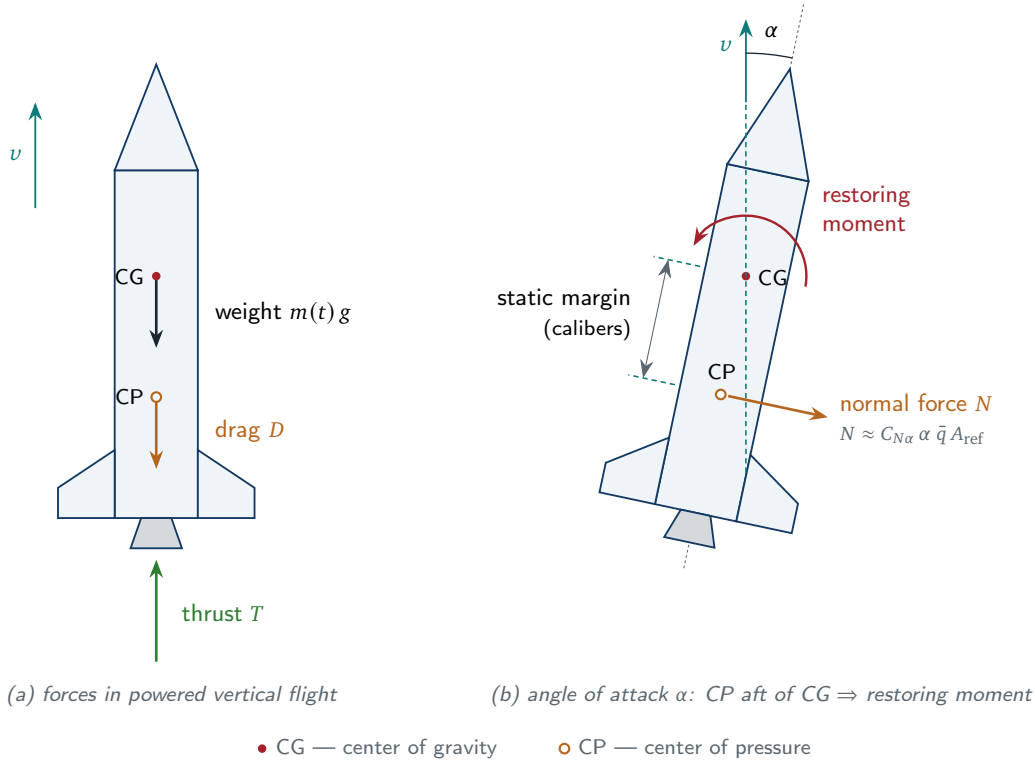


Figure 2. Left: the free-body diagram of the point-mass flight model — thrust $F(t)$ along the world vertical `model/RocketModel.cpp:71`, weight $m(t)g$ `model/RocketModel.cpp:77`, and quadratic drag opposing the velocity vector `model/RocketModel.cpp:88`. Right: the geometry of static stability — when a gust tilts the rocket to a small angle of attack, the fins' normal force acts at the center of pressure (CP); with the CP aft of the center of gravity (CG) the resulting moment restores the rocket toward zero angle, and the CP–CG gap measured in body diameters is the static margin. QtRocket computes both stations on a shared datum (`sim/Aero.h:25`) but no torque acts in flight at the pinned commit (`model/RocketModel.cpp:94`); [Section 9](#) carries the analysis.

together. The pair $(\vec{x}, \vec{v}) \in \mathbb{R}^6$ is the *state vector*: the minimal set of numbers from which the future of the system is computable. QtRocket builds [Eq. \(3\)](#) verbatim, as a lambda constructed once in the propagator’s constructor ([Listing 1](#)); this closure is the entire integrated physics, and [Section 7](#) dissects the loop that drives it.

```
// Linear ODEs: dx/dt = v, dv/dt = F/m. t is passed through so getForces() can evaluate a
// time-dependent forcing function (motor thrust curve) at the actual simulation time.
std::function<std::pair<Vector3, Vector3>(double, Vector3&, Vector3&)> linearODEs = [this](double t,
↳ Vector3& state, Vector3& rate) -> std::pair<Vector3, Vector3>
{
    Vector3 dPosition;
    Vector3 dVelocity;
    dPosition = rate;
    dVelocity = object->getForces(t, state, rate, *environment) / object->getMass(t);

    return std::make_pair(dPosition, dVelocity);
};
```

Listing 1. Newton’s second law with time-varying mass, as QtRocket states it `sim/Propagator.cpp:36`. The time argument is threaded through so the thrust curve and the burning mass are sampled at each solver stage’s node time rather than frozen at the step start.

Where the rocket equation went

Textbook treatments of variable-mass motion add a momentum-flux term $\dot{m} v_e$ (exhaust mass flow times exhaust velocity) — the term that becomes the Tsiolkovsky rocket equation. QtRocket carries no such term, and that is correct, not an omission: the thrust curve is a *measured force* from a test stand, so the momentum flux is already embedded in $F(t)$. Mass loss enters the dynamics only through the divisor $m(t)$ `sim/Propagator.cpp:41` and the weight term `model/RocketModel.cpp:77`; adding the flux on top of a measured curve would double-count it.

Degrees of freedom, precisely. A 3-DOF (*degree-of-freedom*) point-mass model integrates the three translational coordinates only: the state is $(\vec{x}, \vec{v}) \in \mathbb{R}^6$, forces are applied at a single point, and the vehicle has no orientation — it cannot pitch, yaw, roll, fly at an angle of attack, or respond to a torque. A 6-DOF *rigid-body* model adds the three rotational degrees of freedom: an attitude (conventionally a unit quaternion) and an angular velocity, giving a thirteen-element state, with torques and the time-varying inertia tensor entering through Euler’s rigid-body equations. This is the single most important fidelity boundary in flight simulation: stability, weathercocking, gravity turns, and spin all live on the rotational side.

QtRocket today integrates exactly the three linear degrees of freedom ([Listing 1](#)). The rotational half exists as deliberate, typed scaffolding with no writers: the state record reserves quaternion, direction-cosine-matrix, and Euler-angle slots that no code writes `sim/StateData.h:46`; the model’s torque accessor returns the zero vector and has no caller in the simulation loop `model/RocketModel.cpp:94`; and the orientation integrator is a commented-out member with a design note `sim/Propagator.h:103`. One rotational input is live: composite mass, center of gravity, and inertia tensor are recorded into every state sample, expressly as “the seam the 6-DOF rotational ODE will read” `sim/StateData.h:59`. [Section 7](#) covers the integrated path; [Section 13](#) inventories the dormant half.

2.4 Stability: the center of gravity, the center of pressure, and the arrow

Nothing about a finless tube makes it fly straight. Point-mass simulation sidesteps the question by construction; understanding what it sidesteps requires two points on the airframe.

Static stability and the Barrowman method

The *center of gravity* (CG) is where the vehicle’s mass balances; the rocket rotates about it. The *center of pressure* (CP) is where the net aerodynamic *normal* force — the sideways lift that appears when the airflow meets the body at a small *angle of attack* α — effectively acts. The classic intuition is the arrow: feathers at the back add lifting area far aft, dragging the CP behind the CG, so any disturbance that tilts the arrow produces a force behind the pivot that weathervanes it back into the wind. Fins are the rocket’s feathers. If the CP sits *ahead* of the CG, the same moment amplifies the disturbance and the rocket tumbles.

The gap is quoted in *calibers* — body diameters: $SM = (x_{CP} - x_{CG})/d$ with stations measured tip-aft-positive; the hobby rule of thumb is one to two calibers of positive margin. The same mechanism has a cost: *weathercocking*. A stable rocket leaving the rod in a crosswind flies at a momentary angle of attack, and its own restoring moment turns it *into* the wind, trading altitude for upwind travel.

Computing the CP from geometry alone was solved for hobby rockets by James and Judith Barrowman (1966). Their method linearizes each component’s normal force at small α — only the slope C_{N_α} per radian matters — and applies slender-body theory: a constant-diameter tube contributes zero normal-force slope; any nose cone contributes exactly $C_{N_\alpha} = 2$ referenced to its base area, with its CP at a fixed fraction of its length; fins contribute a closed-form term with a body-interference factor. Because all components share one α and one reference area, the slopes add, and the composite CP is the slope-weighted average of the component CPs. The result is a closed-form, microsecond-cheap calculation that is accurate in exactly the regime sport rockets occupy — slender, axisymmetric, small α — which is why essentially every hobby simulator, OpenRocket included, is built on it.

QtRocket implements the Barrowman method faithfully at the component level — the cone’s $C_{N_\alpha} = 2$ at base area `model/parts/ConicalNoseCone.cpp:64`, the N -fin term with the interference factor `model/parts/FinSet.cpp:77` — and composes the pieces onto the same datum as the composite CG so that the static margin is literally a subtraction, `cp() - cg()` `sim/Aero.h:25`, `model/parts/Part.cpp:231`. At the pinned commit this machinery is *dormant*: built and unit-tested with zero production consumers — the test suite states “nothing in production consumes this” `model/tests/AeroTests.cpp:18` — and since no torque acts and no angle of attack is ever computed, no flight is affected by stability or its absence. Weathercocking is doubly out of reach: it needs both the restoring moment and a wind field, and the wind model is a zero-returning stub with no call sites `sim/WindModel.cpp:16`. [Section 9](#) gives the Barrowman layer the full treatment, including a verified sign defect in the fin-set frame that must be fixed before the seam is consumed.

2.5 The atmosphere

Drag needs $\rho(z)$, and ρ is not a constant: a rocket that climbs 3 km flies through air about a quarter thinner than at the pad.

Hydrostatic balance and the standard atmosphere

Air density falls with altitude because each layer of the atmosphere must carry the weight of all the air above it. A static column obeys the *hydrostatic equation*

$$\frac{dP}{dh} = -\rho g, \quad (4)$$

and the ideal gas law $P = \rho \frac{R^*}{M} T$ ties pressure, density, and temperature together. Substituting one into the other shows pressure decaying roughly exponentially with altitude, with a scale height near 8 km — but the exact profile depends on the temperature $T(h)$, which real weather varies daily. A *standard atmosphere* resolves this by decree: it prescribes a fixed, piecewise-linear temperature profile (the troposphere cools at 6.5 K/km; the stratosphere is first isothermal, then warms as ozone absorbs ultraviolet), which makes the hydrostatic equation integrable in closed form layer by layer — an exponential in the isothermal layers, a power law in the gradient layers. The result is a reproducible reference profile, the same for every user on every day, which is exactly what a simulator wants.

QtRocket ships three pluggable atmospheres — constant sea-level, vacuum, and the US Standard Atmosphere 1976 — selectable at runtime behind one interface, with constant sea-level as the default `sim/Environment.h:31`. The US Standard 1976 implementation codes the seven layer tables `sim/USStandardAtmosphere.cpp:21` and the closed-form per-layer solutions `sim/USStandardAtmosphere.cpp:93`, and is validated against the published NOAA report tables at 0.1–0.5% relative tolerance from 0 to 80 km `sim/tests/USStandardAtmosphereTests.cpp`. Of the five properties the atmosphere interface exposes, the flight path consumes exactly one — density `model/RocketModel.cpp:86`; temperature, pressure, speed of sound, and viscosity are computed and await a Mach- or Reynolds-aware drag model. [Section 8](#) covers the environment layer, including gravity models and the geoid.

2.6 Why numerical integration

[Equation \(3\)](#) has no closed-form solution for any realistic flight. Three ingredients each independently destroy solvability: the drag term is quadratic in v (nonlinear), the thrust forcing $F(t)$ is an arbitrary tabulated curve (not an elementary function), and both $m(t)$ and $\rho(z)$ vary along the trajectory. Even the drastically simplified case — vertical flight, constant thrust, constant mass, constant density — yields hyperbolic-tangent and logarithmic expressions that fall apart the moment thrust varies. So every flight simulator marches the state forward numerically, and the craft lies in doing so accurately and cheaply.

From Euler to Runge–Kutta, and why steps adapt

The simplest scheme is Euler’s: sample the derivative at the step start and walk straight along it, $x_{n+1} = x_n + h f(t_n, x_n)$. Its global error shrinks only linearly with the step size h — halving the error costs double the work. The classical fourth-order Runge–Kutta method (RK4) instead probes the derivative four times per step (at the start, twice at midpoint estimates, and at the end) and takes a weighted average; the leading error terms cancel, so global error scales as h^4 : halving h buys sixteen times the accuracy. The remaining inefficiency is the *fixed* step: a flight needs tiny steps during the violent first tenth of a second of the burn and could take huge ones during a placid coast. *Embedded* methods such as Runge–Kutta–Fehlberg 4(5) compute a fourth- and a fifth-order estimate from the same six derivative samples; their difference is a free

estimate of the local error, which a controller compares against a tolerance — shrinking the step where the physics is fast, growing it where it is smooth.

QtRocket provides both: a textbook fixed-step RK4 `sim/RK4Solver.h:45` and an adaptive Runge–Kutta–Fehlberg 4(5) with the classical tableau `sim/RK45Solver.h:75`, at a default tolerance of 10^{-6} that no production caller overrides `sim/RK45Solver.h:158`, selectable by name at runtime from either front end. The payoff of adaptivity is measured in the tree: the same test flight that costs RK4 roughly 4050 steps costs the adaptive solver about 530 `tests/PhysicsIntegrationTests.cpp:163`. Two details from the box above are load-bearing in the code. First, every solver stage passes its own node time into the ODE, so the thrust curve is sampled at the right instants within a step `sim/RK4Solver.h:55` — freezing thrust at the step start once inflated adaptive-run apogees by roughly 8% `tests/PhysicsIntegrationTests.cpp:149`. Second, adaptive step growth must be capped `sim/RK45Solver.h:144`: coasting in vacuum is integrated *exactly* by both embedded orders, the error estimate collapses to zero, and an uncapped controller would grow the step without bound and leap past apogee and the ground. [Section 7](#) examines both solvers, the run loop, and its typed termination taxonomy `sim/Propagator.h:35`.

2.7 Motors: impulse classes, designations, and the thrust curve

Reading a motor designation

Hobby motors are certified and labeled by *total impulse*, the area under the thrust curve, $I_{\text{tot}} = \int F dt$ in newton-seconds. Impulse classes are letters, and each full letter doubles the ceiling of the one before: class A tops out at 2.5 Ns, B at 5, C at 10, and so on up through G (160, the top of the “model rocket” range), H–I (level-1 high power), J–L (level 2), and M (10 240, level 3); fractional classes 1/4A and 1/2A sit below A. A designation like C6-5 reads: class C (total impulse in the 5–10 Ns bracket), average thrust 6 N, and a 5 s ejection delay between burnout and the recovery charge. Manufacturer suffixes, such as the T in *G80T*, name the propellant formulation. Two motors of the same class can fly very differently: a high-thrust short burn and a gentle long burn deliver the same impulse with different peak accelerations — which is why simulators consume the full thrust curve, never just the letter.

QtRocket’s motor subsystem ([Section 6](#)) mirrors this taxonomy directly. The impulse class is derived from the leading letter of the motor code, with the fractional classes 1/4A and 1/2A special-cased `model/MotorModel.cpp:112`; the heavy test suite flies the complete 1/4A→M ladder across every reference airframe `tests/DesignMatrixTests.cpp:515`. Delay options are parsed from the local ingest formats and stored — with 1000 as the sentinel for a plugged motor `model/MotorModel.h:303` — though, as [Section 2.1](#) noted, no flight logic yet consumes them. The thrust curve is sampled with linear interpolation and careful edge handling: curves whose first published sample sits at $t > 0$ ramp from an implicit origin `model/ThrustCurve.cpp:76`. And because $m(t)$ matters ([Section 2.2](#)), the motor also derives a burning-mass curve: a 128-point table that depletes propellant in proportion to the fraction of total impulse delivered — the standard constant-specific-impulse assumption `model/MotorModel.cpp:132`, `model/MotorModel.cpp:141` — so mass falls fastest exactly when the curve burns hardest. The motor’s three mass regimes — loaded before ignition, depleting during the burn, casing after burnout `model/MotorModel.cpp:28` — are the single time-varying input the part tree’s composite mass walk sees ([Section 4](#)).

2.8 From concept to code

[Table 1](#) closes the primer by mapping each physical concept introduced above to the code that owns it and the chapter that examines it. The table doubles as a capability summary at the pinned commit: the point-mass rows are complete and consumed; the rotational and recovery rows name machinery that is built but dormant, or absent outright.

Physical concept	Where QtRocket implements it	Chapter
Thrust curve $F(t)$	<code>ThrustCurve</code> sampling + <code>MotorModel</code> burnout latch model/ <code>ThrustCurve.cpp</code>	Section 6
Burning mass $m(t)$	128-point impulse-proportional mass curve; <code>part::Motor</code> leaf in the tree <code>model/MotorModel.cpp</code>	Section 6
Mass, CG, inertia of the airframe	<code>Part</code> composite tree, two-pass walk behind the mass-delta cache <code>model/parts/Part.cpp</code>	Section 4
Geometry: where parts sit	<code>StationLink</code> intent + the placement resolver <code>model/parts/ Placement.cpp</code>	Section 5
Weight / gravity field	<code>ConstantGravityModel</code> , <code>SphericalGravityModel</code> sim/ <code>GravityModel.h</code>	Section 8
Air density $\rho(z)$	<code>AtmosphericModel</code> registry; US Standard Atmosphere 1976 sim/ <code>USStandardAtmosphere.cpp</code>	Section 8
Quadratic drag	Three-term force model in <code>RocketModel::getForces()</code> <code>model/ RocketModel.cpp</code>	Section 9
Newton’s law, state, flight phases	<code>Propagator</code> run loop, pad hold, typed termination sim/ <code>Propagator.cpp</code>	Section 7
Numerical integration	<code>RK4Solver</code> (fixed), <code>RK45Solver</code> (adaptive) behind <code>DESolver</code> sim/ <code>RK45Solver.h</code>	Section 7
CP, static margin (Barrowman)	<code>Part::getCompositeAero()</code> onto the shared datum — dormant sim/ <code>Aero.h</code>	Section 9
Attitude / 6-DOF, wind	Reserved seams: unwritten <code>StateData</code> slots, zero torques, stub <code>WindModel</code>	Section 13
Recovery events, ejection delays	Delays parsed and persisted; no flight logic consumes them	Section 13

Table 1. The primer’s concepts mapped to their implementations at commit [254cd41](#). Every row’s owning chapter carries the citations and the analysis; rows marked dormant or absent are consolidated in [Section 13](#).

3 System overview & layering

QtRocket is organized as three executables over one Qt-free core. `qtrocket` is the Qt6 Widgets desktop application; `qtrocket-cli` is a headless, line-oriented read-eval-print loop (REPL) over the same simulation engine; `qtrocket-visualizer` is a standalone OpenGL viewer for saved designs. All three link a single static library, `qtrocket_core`, which contains exactly one class — the QtRocket application controller `CMakeLists.txt:224` — and pulls in the three layer libraries beneath it: `model`, `sim`, and `utils` `CMakeLists.txt:229`. [Figure 3](#) shows the arrangement at the pinned commit [254cd41](#); this section walks it from the top down and closes with the build system that enforces it.

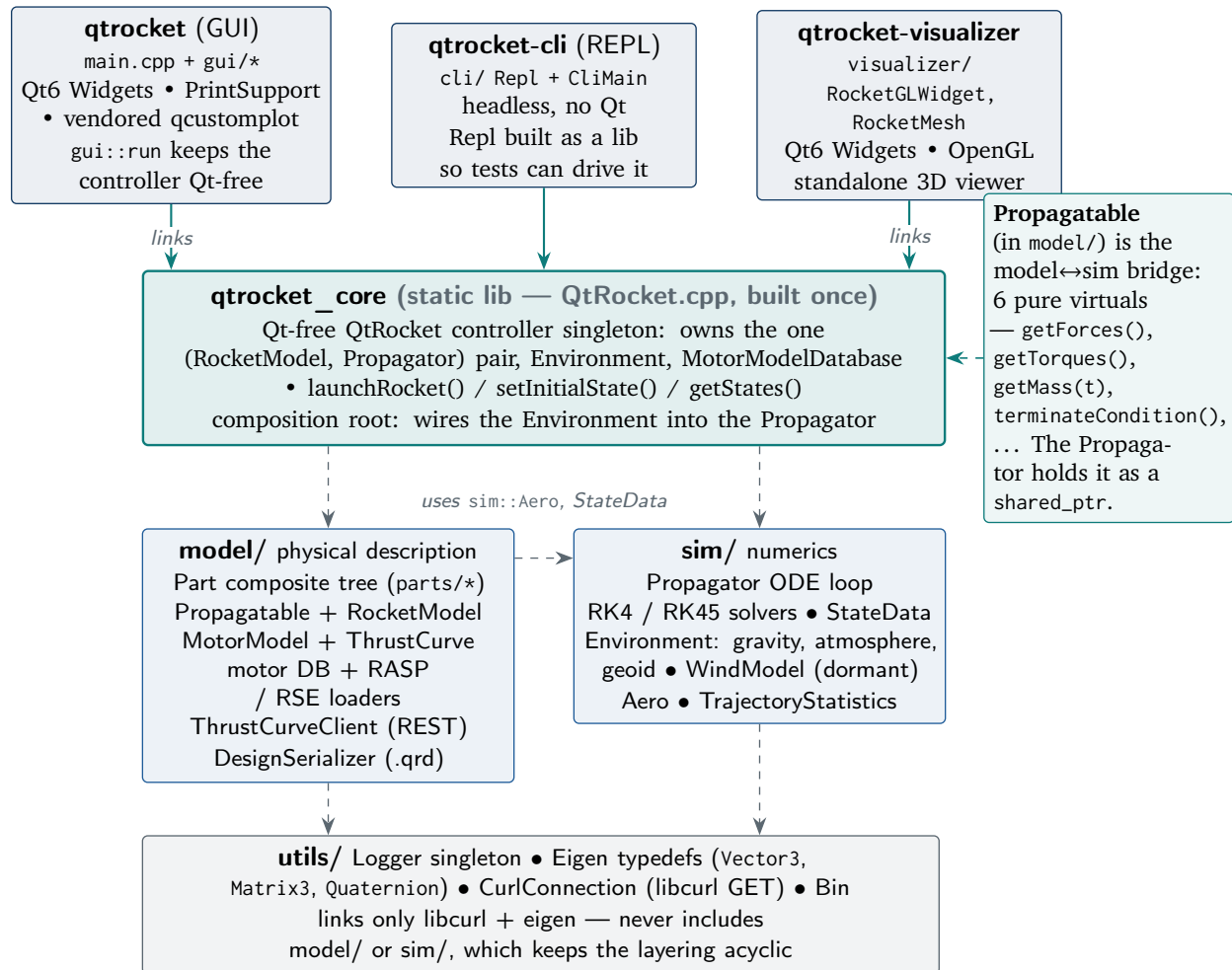


Figure 3. Three executables over one Qt-free core. Every link edge points strictly downward: the front ends link `qtrocket_core` — the static library holding the `QtRocket` controller `CMakeLists.txt:224` — the core links the `model`, `sim`, and `utils` layer libraries `CMakeLists.txt:229`, and `utils` links only the vendored Eigen and libcurl `utils/CMakeLists.txt:14`. Qt appears only in the two graphical executables; nothing at or below the controller can name a Qt type.

3.1 Three executables, one core

qtrocket (GUI). The entry point sets the `Logger` to `INFO` `main.cpp:16`, obtains the controller singleton `main.cpp:20`, and blocks in `gui::run(qtrocket, argc, argv)` `main.cpp:25`. `gui::run` exists precisely so the controller stays Qt-free: it owns `QApplication`, and everything Qt-dependent in this executable lives under `gui/` `gui/GuiRunner.h:13`. The executable links `Qt6::Widgets`, `Qt6::PrintSupport`, and `qtrocket_core` `CMakeLists.txt:286`.

qtrocket-cli (headless REPL). The same engine with no display requirement. Its entry point silences the logger to `ERROR` because `stdout` doubles as a machine-readable `OK/ERR/CSV` protocol `cli/CliMain.cpp:17`, then obtains the same singleton `cli/CliMain.cpp:19`. The REPL is compiled into its own `cli` static library so the test binaries drive exactly the object code users type into `CMakeLists.txt:234`; the executable links that library and nothing else `CMakeLists.txt:297`.

qtrocket-visualizer (OpenGL viewer). A self-contained Qt application under `visualizer/` with its own `find_package(Qt6 ...)` for the OpenGL components `visualizer/CMakeLists.txt:9`; it links `qtrocket_core` for the shared design loader `CMakeLists.txt:241` and renders saved `.qrd` designs ([Section 11](#)).

All three front ends are examined in depth in [Section 10](#). The core they share is deliberately minimal: `qtrocket_core` compiles `QtRocket.cpp` exactly once — “Application controller shared by the GUI, the CLI, and the integration tests” `CMakeLists.txt:222` — and every consumer links the archive rather than recompiling the translation unit ([Listing 2](#)). The integration-test binary is the fourth consumer `tests/CMakeLists.txt:19`, so the controller’s singleton statics exist once per process no matter which front end, or which test, is running.

```
# Application controller shared by the GUI, the CLI, and the
# integration tests.
add_library(qtrocket_core STATIC
    QtRocket.cpp
    QtRocket.h)
qtrocket_enable_coverage(qtrocket_core)

target_link_libraries(qtrocket_core PUBLIC
    model
    sim
    utils)
```

Listing 2. The keystone of the link graph `CMakeLists.txt:222`. The controller compiles once into a static library; the three executables and the integration tests all link it, and its `PUBLIC` dependencies hand every consumer the layer libraries below.

3.2 The layering is the link graph

The layering is not a documentation convention; it is the link graph itself, declared target by target in CMake. `utils` sits at the bottom and links only the two vendored third-party libraries, `libcurl` and `eigen` `utils/CMakeLists.txt:14` — it supplies the math typedefs (`Vector3`, `Matrix3`, `Quaternion`), the `Logger`, and the HTTP transport everything above shares. `sim` links only `utils` `sim/CMakeLists.txt:28`. `model` links `utils`, `sim`, `Boost::property_tree`, and `jsoncpp_static` `model/CMakeLists.txt:44`; the build file states why the middle edge exists — “sim is a real dependency: Propagatable/RocketModel use `sim::Aero` and `sim/StateData.h`” `model/CMakeLists.txt:41`. `model` and `sim` are therefore layered rather than peers: the model consumes the simulation layer’s value types, and no `sim/` code consumes

a model value type in return. The one upward reference is the bridge interface itself — `sim/Propagator.h` includes `model/Propagatable.h` `sim/Propagator.h:18` to hold the abstract rocket it integrates, an abstraction whose own includes point strictly downward `model/Propagatable.h:12` — so `sim` still links only `utils`, and the `model` archive supplies the concrete rocket when a final executable links both. Include resolution everywhere uses one global `include_directories(${PROJECT_SOURCE_DIR})` `CMakeLists.txt:147`, which is what makes the `model/...`, `sim/...`, and `utils/...` path prefixes resolve uniformly from any file in the tree.

The layering invariant, stated where it binds

“Bottom layer: `utils` must not depend on `model/` or `sim/`” `utils/CMakeLists.txt:13`. The comment sits directly above the `target_link_libraries` call it constrains, and it holds: every `#include` under `utils/` resolves to the standard library, `curl`, `Eigen`, or another `utils/` header. This single rule keeps the dependency graph acyclic — every layer above may share the bottom layer’s typedefs and logging without ever creating a back-edge.

The invariant holds upward as well: nothing below the controller reaches back up to it. The simulation environment enters the model as an explicit parameter of the force query `model/Propagatable.h:29`, passed by the propagator at the call site `sim/Propagator.cpp:41`; there is no `QtRocket::getInstance()` call anywhere in `model/` or `sim/`, so the layers below the controller are usable — and are unit-tested — without a controller existing at all.

3.3 The QtRocket controller

`QtRocket` is the composition root: a lazily created, mutex-guarded singleton that lives for the whole process and owns, through `std::shared_ptr`, one `(RocketModel, Propagator)` pair `QtRocket.h:79`, one `sim::Environment` `QtRocket.h:82`, and one `model::MotorModelDatabase` `QtRocket.h:83`. Its constructor wires the system in four moves `QtRocket.cpp:39`: the environment, whose default constructor supplies working gravity and atmosphere models `QtRocket.cpp:43` (the alternatives are the subject of [Section 8](#)); the rocket model, which boots with a placeholder body so the composite mass is real from the first query `model/RocketModel.cpp:30`; the propagator, constructed around that model with the shared environment injected in a single statement `QtRocket.cpp:48`; and the motor database `QtRocket.cpp:51`. Because the propagator is constructed together with the model it integrates — and the one API that would rebuild the pair, `addRocket`, has no callers anywhere in the tree `QtRocket.h:41` — the model and the loop can never drift apart, and the environment injection has exactly one site.

The singleton machinery is small enough to state completely. Three statics — `instance`, `mtx`, and a plain `bool initialized` `QtRocket.cpp:14` — implement double-checked initialization: `getInstance()` reads the flag with no lock `QtRocket.cpp:21`; on false, `init()` acquires the mutex, re-checks under the lock so a racing second caller cannot construct twice `QtRocket.cpp:31`, and only after allocating the controller publishes `initialized = true` `QtRocket.cpp:35`. The instance is never deleted; it lives until the process exits.

The non-atomic fast-path read

The first check reads `initialized` — a plain `bool`, not `std::atomic<bool>` — outside the mutex `QtRocket.cpp:21`, while `init()` writes it under the lock `QtRocket.cpp:35`. Concurrent first calls from two threads would be a data race under the C++ memory model, and on weakly-ordered hardware a second thread could observe the flag before the construction it guards. The race is latent and benign today because every entry point that reaches the controller calls `getInstance()`

on the main thread before any other thread exists `main.cpp:20`, `cli/CliMain.cpp:19` — safe by usage, not by construction. The fix idiom already exists one layer down: the `Logger` is a Meyers singleton, initialized exactly once by the language itself `utils/Logger.cpp:19`.

The public surface is deliberately small, and almost all of it delegates: `setTimeStep` and `setIntegratorModel` forward to the propagator `QtRocket.h:35`; `setInitialState` and `getStates` to the model `QtRocket.h:68`, `QtRocket.h:48`; `getTrajectoryStatistics` `QtRocket.h:55` and the typed `getTerminationReason` `QtRocket.h:62` return whole-flight results; plain accessors expose the environment, the rocket, and the motor database `QtRocket.h:34`. The one method with a body worth reading is `launchRocket()` `QtRocket.h:43`, reproduced verbatim in [Listing 3](#): a flight is exactly four calls.

```
void QtRocket::launchRocket()
{
    // initialize the propagator
    rocket.first->clearStates();
    rocket.second->setCurrentTime(0.0);

    // start the rocket motor
    rocket.first->launch();

    // run the propagator until it terminates
    rocket.second->runUntilTerminate();
}
```

Listing 3. The entire launch path in the controller `QtRocket.cpp:54`: clear the recorded history, rewind simulation time, ignite the model, run the loop to termination. Everything else — staging launch parameters, selecting models, exporting results — lives in the front ends.

[Figure 4](#) places those four calls in the full request flow. In order:

1. **Clear history.** `clearStates()` empties the $(t, \text{StateData})$ vector stored on the model `model/Propagatable.h:51`. Any element reference still held from a previous `getStates()` is invalidated here, which is why both front ends re-fetch the history after every launch `gui/AnalysisWindow.cpp:39`, `cli/Repl.cpp:690`.
2. **Rewind time.** `setCurrentTime(0.0)` `sim/Propagator.h:67`. Every flight starts at $t = 0$, which is also the ignition convention — the simulation clock and the thrust-curve clock agree by construction.
3. **Ignite.** `launch()` copies the staged initial state into the live state and arms the motor’s thrust curve with `startMotor(0.0)` `model/RocketModel.cpp:105`. With no motor this is a no-op; the propagator’s no-liftoff guard then ends the run with a typed verdict rather than a hang ([Section 7](#)).
4. **Propagate.** `runUntilTerminate()` integrates until the model’s own `terminateCondition()` or one of the loop’s guards fires `sim/Propagator.cpp:58`, resetting the whole-flight statistics at the top of the run `sim/Propagator.cpp:65`. At every integrator stage the loop evaluates $\vec{v} = \text{getForces}(t, \vec{x}, \vec{v}, \text{env}) / \text{getMass}(t) `sim/Propagator.cpp:41`; per recorded step it snapshots mass properties and appends to the history `sim/Propagator.cpp:132`.$

The whole sequence is synchronous on the calling thread — no worker thread, no lock around the history, and nothing reads it concurrently (the consequences for the GUI are taken up in [Section 10](#)). The canonical drive sequence — `setInitialState` → `setIntegratorModel` →

`setTimeStep` → `launchRocket` → `getStates` — appears verbatim in the integration harness `tests/PhysicsIntegrationTests.cpp:90`; the REPL issues the same calls, the integrator and timestep staged by their own commands before its launch handler runs `cli/Repl.cpp:687`.

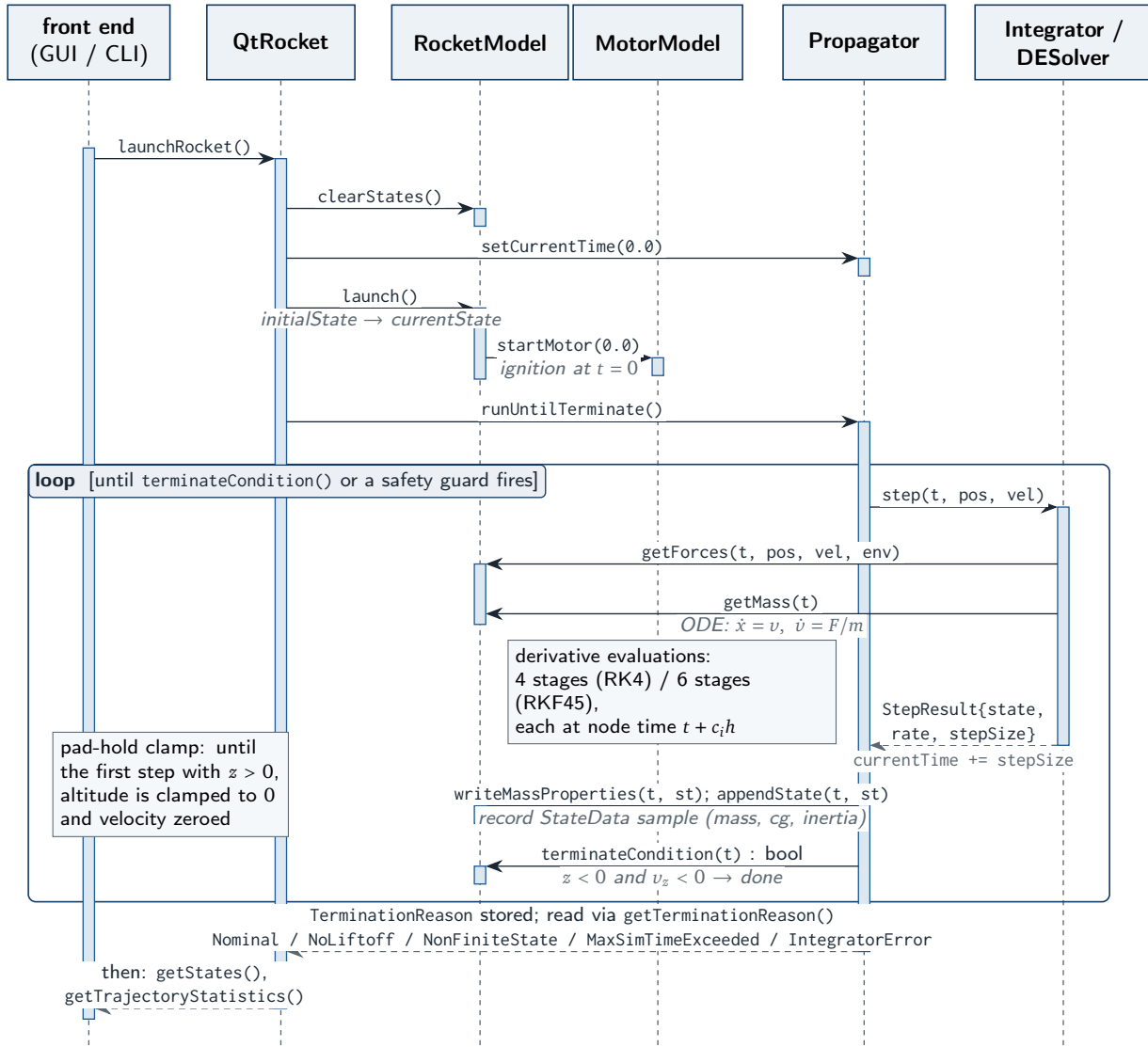


Figure 4. The launch sequence. A front end stages the flight through the controller’s delegating setters, then calls `launchRocket()`, whose entire body is the four calls of [Listing 3](#) `QtRocket.cpp:54`. Inside `runUntilTerminate()` `sim/Propagator.cpp:58` the rocket is visible only through the `Propagatable` interface: force and mass at every integrator stage, a mass-property snapshot per recorded step, and the termination predicate once per step. Results flow back through const-reference accessors after the run completes.

3.4 The Propagatable bridge

The seam between the two halves of the core is the abstract class `model::Propagatable` `model/Propagatable.h:23`: the only view of the rocket the simulation loop ever sees. `RocketModel` is its only production implementation; the `Propagator` holds it as `std::shared_ptr<model::Propagatable>` `sim/Propagator.h:105` and interrogates it through six pure virtuals, reproduced verbatim in [Listing 4](#).

What a simulation loop needs from a rocket

A point-mass flight simulator integrates Newton’s second law: given position and velocity, it needs the net force $\vec{F}(t, \vec{x}, \vec{v})$ and the current mass $m(t)$ — mass is a function of time because propellant burns away. A rigid-body simulator with all six degrees of freedom (6-DOF: three translational plus three rotational) additionally needs the net torque and the inertia tensor for the rotational equations. That is the entire shape of the interface below: two questions for the translational problem, two for the rotational one, plus a snapshot hook and an end-of-flight predicate.

```
virtual Vector3 getForces(double t, const Vector3& position, const Vector3& velocity, sim::Environment&
    ↪ environment) = 0;
virtual Vector3 getTorques(double t) = 0;

virtual double getMass(double t) = 0;
virtual Matrix3 getCompositeInertiaTensor(double t) = 0;

/// Fill st.mass/cg/inertia with the body's composite mass properties at @p t. Called by the
/// Propagator once per recorded step (before appendState).
virtual void writeMassProperties(double t, StateData& st) = 0;

virtual bool terminateCondition(double t) = 0;
```

Listing 4. The model↔sim bridge, complete `model/Propagatable.h:29`. The integrator needs forces, torques, and time-varying mass properties — and nothing else about the rocket. `terminateCondition` lets the model, not the loop, decide when a flight is over.

The bridge is more than a pure interface: it is also the state store. The base class carries the current, initial, and recorded states, so the flight history lives on the model side of the seam — the propagator commits each accepted step through `setCurrentState` `sim/Propagator.cpp:124` and appends to the history exposed by `getStates()` `model/Propagatable.h:47`, which is why the controller’s `getStates()` delegates to the rocket rather than to the propagator `QtRocket.h:48`. The minimal contract is pinned by the propagator test suite: a conforming mock implements only the six pure virtuals and inherits state storage, history, and statistics from the base class `tests/PropagatorTests.cpp:32`.

Of the six virtuals, four carry the current flight and two are dormant — implemented, with zero production consumers. The propagator’s ordinary-differential-equation (ODE) lambda consumes `getForces` and `getMass` at every integrator stage `sim/Propagator.cpp:41`; `writeMassProperties` runs once per recorded step `sim/Propagator.cpp:132`; `terminateCondition` is tested once per step after the state is committed `sim/Propagator.cpp:137`. `getTorques` is implemented as constant zero `model/RocketModel.cpp:94` and called by no simulation code — the comment beside the propagator’s commented-out orientation integrator names it as the future 6-DOF consumer `sim/Propagator.h:102` — and `getCompositeInertiaTensor` likewise has no production caller; the per-step snapshot reaches the part tree directly rather than through the virtual `model/RocketModel.cpp:59`. Two protected members are dead outright: `aeroData` is default-constructed and never read `model/Propagatable.h:61` — the `sim::Aero` alias survives specifically so it keeps compiling `sim/Aero.h:63` — and `nextState` is never touched; the propagator uses its own local of the same name `model/Propagatable.h:65`, `sim/Propagator.cpp:90`. The loop that drives the live half is the subject of [Section 7](#); what the dormant halves are waiting for is taken up in [Section 9](#); [Section 13](#) carries the consolidated inventory.

3.5 Build system in brief

The build is one CMake project — `cmake_minimum_required(VERSION 3.25) CMakeLists.txt:1`, project `VERSION 0.1 CMakeLists.txt:3` — pinned to C++23 `CMakeLists.txt:5` with `compile_commands.json` exported for tooling `CMakeLists.txt:7`. Generator, compiler, and build type are all pinned by `CMakePresets.json`: Ninja everywhere `CMakePresets.json:7`, eight configure presets (debug/release for the default compiler, Clang, and GCC; a release-only MSVC preset `CMakePresets.json:70`; and `coverage-clang`, which redirects to a separate `build-coverage/` tree `CMakePresets.json:35`), each with matching build and test presets. The presets are the single sanctioned entry point — the command line, the editor tasks, and continuous integration (CI) all configure through them — a pinning adopted after a generator mismatch broke the build directory `TODO.md:24`.

Dependency policy is strict vendoring (Table 2): every dependency except Qt6 is fetched by `FetchContent` at an exact git tag and compiled from source at first configure. A directory-scoped `set(BUILD_SHARED_LIBS OFF) CMakeLists.txt:46` makes every in-tree library a static archive as well, so Fig. 3 is a graph of static libraries resolved at link time, not a runtime loading scheme.

Dependency	Pinned tag	Build shape and role	
GoogleTest	<code>v1.17.0</code>	test framework for the six test binaries	<code>CMakeLists.txt:17</code>
jsoncpp	<code>1.9.7</code>	static-only, tests off; thrustcurve.org JSON (Section 6)	<code>CMakeLists.txt:32</code>
curl	<code>curl-8_20_0</code>	static, HTTP-only; the sole network transport, under <code>utils</code>	<code>CMakeLists.txt:44</code>
Eigen	<code>5.0.1</code>	vectors, matrices, quaternions beneath <code>utils</code>	<code>CMakeLists.txt:72</code>
Boost	<code>boost-1.91.0-1</code>	restricted to <code>property_tree</code> ; XML persistence (Section 11)	<code>CMakeLists.txt:79</code>
Qt6	<code>system</code>	Widgets/PrintSupport for the GUI; OpenGL modules for the visualizer	<code>CMakeLists.txt:140</code>

Table 2. Dependency vendoring at the pinned commit. Everything except Qt6 is fetched at an exact tag via `FetchContent`, so the dependency closure is a function of the repository, not of the build host. The first configure downloads and compiles all of it — slow once, hermetic thereafter.

Hermetic dependency closure

The curl block is the most deliberate piece of the vendoring. curl is built static and HTTP-only `CMakeLists.txt:46`, and every optional third-party backend (`libssh2`, `libssh`, `nghttp2`, `libidn2`, `brotili`, `zstd`) is explicitly forced off, with the rationale recorded in the tree: several of those options default to ON/AUTO, so curl links whatever it auto-detects on the build host and the final link then fails on machines that lack those libraries — “Forcing them off keeps the build identical on every platform” `CMakeLists.txt:56`. The payoff is stated where it pays: the FreeBSD CI job builds in a bare virtual machine: “The hermetic curl config means OpenSSL + zlib (both in the FreeBSD base system) are the only external libs curl needs” `.github/workflows/cmake-multi-platform.yml:99`.

Warning hygiene is tree-wide and strict: `-Wall -Wextra -Wpedantic -Werror` on GCC and Clang `CMakeLists.txt:97`, `/W4 /WX` on MSVC `CMakeLists.txt:88`. The placement of those flags is itself the policy: they are added after the five `FetchContent` blocks but before the project’s own subdirectories, so vendored code compiles with its own flags while every project target must be warning-clean; the one vendored source inside a project target, `gui/qcustomplot.cpp`, is exempted per-source with `-w` `CMakeLists.txt:154`. The rest of the infrastructure — the six GoogleTest binaries and their dual ctest

registration, the light/heavy test split, the four-entry CI matrix plus the FreeBSD virtual-machine job, and the llvm-cov coverage pipeline — is the subject of [Section 12](#).

Logging and API documentation. Diagnostics everywhere in the tree flow through the `utils::Logger` Meyers singleton `utils/Logger.cpp:18`, which writes every message to two sinks — `stdout` and a `log.txt` opened in the process working directory `utils/Logger.cpp:25` — at five cumulative levels up to `PERF_` `utils/Logger.h:28`. Emission is mutex-guarded `utils/Logger.cpp:35`; the level setter is not `utils/Logger.cpp:77`, which is safe under the current call pattern (each front end pins the level once at startup, [Section 10](#)). A root `Doxyfile` generates API documentation from the tree’s `///` comments.

Getting a build. For the reader who wants the executables before the theory:

```
cmake --preset debug-clang          # configure (Ninja, into build/)
cmake --build --preset debug-clang  # compile everything
ctest --test-dir build -R 'qtrocket_*' -LE heavy # the fast test loop
```

The first configure downloads and compiles the vendored closure of [Table 2](#), so it is slow once and incremental thereafter. The executables land at `build/qtrocket` and `build/qtrocket-cli`, with the viewer under `build/visualizer/`; the test binaries and their ctest registration are mapped in [Section 12](#).

With the map established, the remaining sections descend into the boxes: the part tree ([Section 4](#)) and placement ([Section 5](#)), motors ([Section 6](#)), the simulation loop ([Section 7](#)) and its environment models ([Section 8](#)), aerodynamics ([Section 9](#)), the front ends ([Section 10](#)), and persistence ([Section 11](#)).

4 The part tree & mass properties

Everything the simulator knows about a rocket’s mass — how heavy it is, where its center of mass (CM) sits, how hard it resists rotation — comes from one data structure: a composite tree of `Part` nodes under `model/parts/`. Each node is simultaneously one physical component and the root of a sub-tree, so it carries both its own mass properties and the lazily cached composite of itself plus every descendant `model/parts/Part.h:26`. This chapter walks the tree from the outside in: ownership and identity ([Section 4.1](#)), the five concrete part types ([Section 4.2](#)), the two-pass mass-property algorithm at the chapter’s core ([Section 4.3](#)), the caching that makes those properties time-aware ([Section 4.4](#)), the closed-form tensor library ([Section 4.5](#)), and the diagnostics gate on the composite readers ([Section 4.6](#)). How a child’s position is *stored and resolved* — the `StationLink`, the resolver, and the overlap sweep — is the subject of [Section 5](#); this chapter treats placement as an opaque intent value riding alongside each child. The motor’s internals live in [Section 6](#). All quantities are SI — meters, kilograms — with one deliberate exception flagged in [Section 4.2](#).

4.1 The composite tree

`Part` is the abstract node of a classic Composite pattern ([Fig. 5](#)). A node owns its children as a vector of (`shared_ptr<Part>`, `StationLink`) pairs in attachment order `model/parts/Part.h:288`: the smart pointer carries ownership of the child sub-tree, and the `StationLink` beside it stores the child’s *placement intent* — a feature-to-feature relationship, never a resolved coordinate ([Section 5](#)). Each node also keeps a non-owning back-pointer to its parent, used to propagate cache-invalidation flags up the tree `model/parts/Part.h:222`. Two members are pure virtual, and they define what a concrete part

must supply: `typeName()`, the stable string tag that doubles as the factory key and the design-file type attribute `model/parts/Part.h:181`, and `cloneShallow()`, the type-preservation hook for deep copies `model/parts/Part.h:217`.

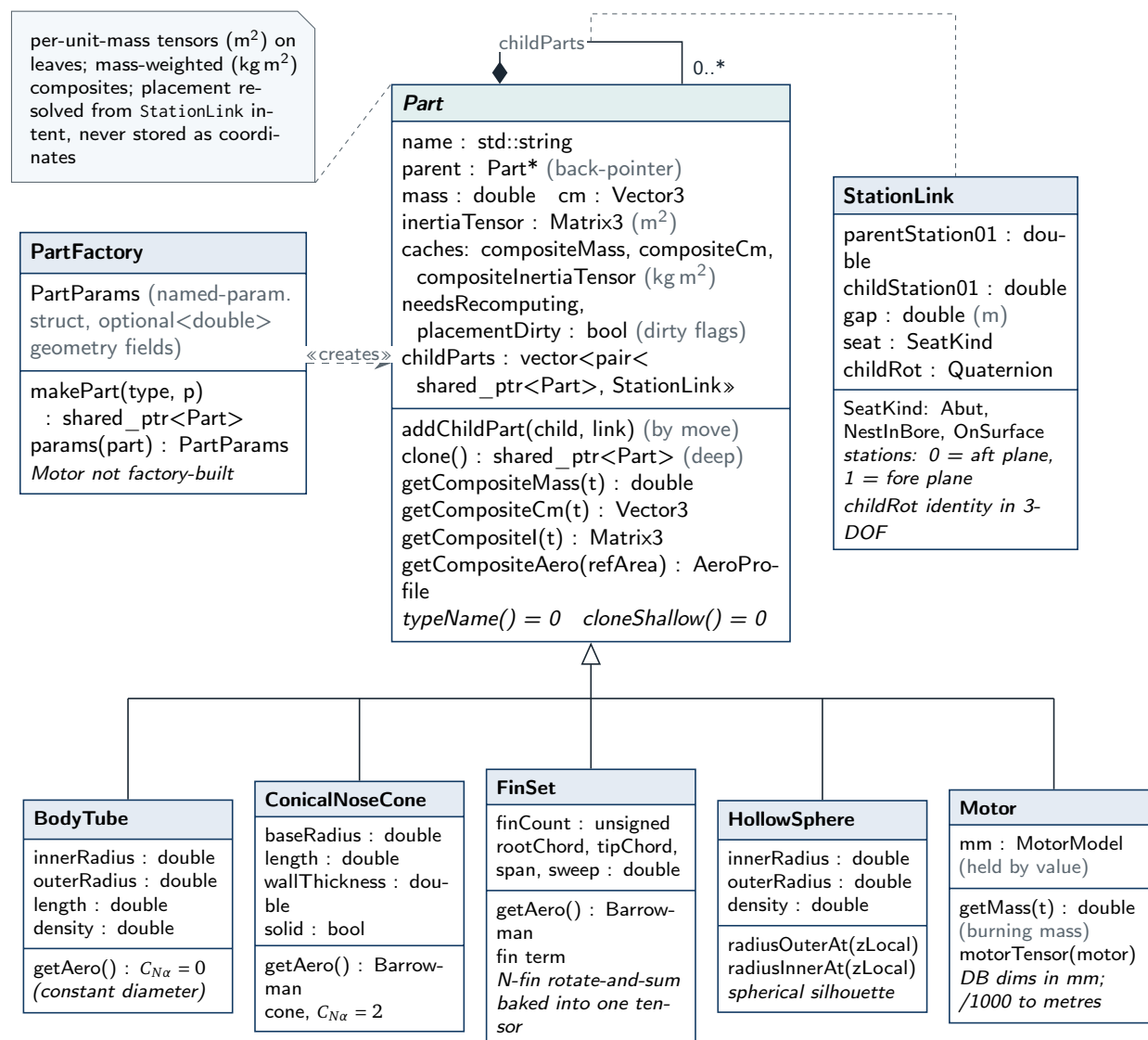


Figure 5. The `Part` composite tree. The abstract base owns children as (`shared_ptr`, `StationLink`) pairs in attachment order `model/parts/Part.h:288` and keeps a non-owning parent back-pointer for upward cache invalidation `model/parts/Part.h:222`. Five concrete types override the pure virtuals `typeName()` and `cloneShallow()`; `PartFactory` is the sanctioned construction path for the four geometry types, while `Motor` is attached only through `RocketModel::setMotorModel()` `model/parts/PartFactory.cpp:73`.

Identity is the id, never the name. Every part gets a process-unique `Part::Id` (a `std::uint64_t`) from a relaxed atomic counter that starts at 1, reserving 0 as “none” `model/parts/Part.cpp:26`; the id is assigned fresh in every constructor — including the copy constructor — and never reassigned `model/parts/Part.h:224`. Names are human-facing labels and need not be unique `model/parts/Part.h:176`; ids are deliberately not serialized, because they are only meaningful within a single run `model/parts/Part.h:171`. The placement diagnostics of [Section 5](#) identify parts by id for exactly this reason: names collide and pointers dangle.

A part lives at one place in one tree

Copy and move assignment are deleted, and the protected copy constructor suppresses the implicit move, so a `Part` cannot be copied, moved, or sliced at the value level `model/parts/Part.h:66`. A part enters a tree only by moving its owning `shared_ptr` into `addChildPart()`, and is duplicated only via `clone()` — “the only way to duplicate a part” `model/parts/Part.h:193`. Aliasing a node into two trees, or double-attaching it within one, is unrepresentable rather than merely discouraged.

`addChildPart(child, link)` enforces that rule with three guards, each a logged no-op rather than an exception: a null child `model/parts/Part.cpp:73`; a child that already has a parent, with the log message pointing at the remedy (“clone() it or detach first”) `model/parts/Part.cpp:78`; and a child that is this part or one of its ancestors, which would form both a graph cycle and an ownership cycle through the child `shared_ptrs` `model/parts/Part.cpp:84`. Adoption is then re-parenting followed by a move — no copy, so the child’s dynamic type and id are preserved `model/parts/Part.cpp:97`. Detachment is the inverse: `removeChildById()` moves the owning pointer out, clears the child’s parent, and returns the intact sub-tree to the caller `model/parts/Part.cpp:277`. Both operations end by invalidating *both* caches — composite mass properties and resolved placement — up the whole ancestor chain `model/parts/Part.cpp:100`; removal drives the rebuild through the dirty flag rather than the mass-delta gate of [Section 4.4](#), so even detaching a zero-mass sub-tree refreshes the composite `model/parts/Part.h:205`.

Duplication is a two-level protocol ([Listing 5](#)). `cloneShallow()` copies just one node with the correct dynamic type; each concrete class implements it as `new X(*this)` through a protected, defaulted copy constructor (e.g. `model/parts/BodyTube.h:63`), and the base copy constructor copies the node’s own mass properties, mints a fresh id, and leaves the copy parentless and childless `model/parts/Part.cpp:55`. `clone()` then recurses polymorphically over the children, copying each `StationLink` verbatim — the placement intent is plain data and clones exactly. The result is a deep, independent, type-preserving copy of the whole sub-tree, pinned per concrete type by the test suite `model/tests/PartTests.cpp:252`.

```
std::shared_ptr<Part> Part::clone() const
{
    std::shared_ptr<Part> copy = cloneShallow(); // this node: correct dynamic type, fresh id, no kids
    for(const auto& [child, link] : childParts)
    {
        std::shared_ptr<Part> childCopy = child->clone(); // recurse polymorphically (no slicing)
        childCopy->parent = copy.get();
        copy->childParts.emplace_back(std::move(childCopy), link); // the placement intent clones verbatim
    }
    return copy;
}
```

Listing 5. Deep copy without slicing `model/parts/Part.cpp:106`: the pure-virtual `cloneShallow()` reproduces each node’s dynamic type, the recursion reproduces the structure, and the copied `StationLinks` reproduce the placement intent. Every cloned node receives a fresh id.

After construction a part is close to immutable: the only setters are `setMass()` and `setI()`, both of which flag the ancestor chain for recompute `model/parts/Part.h:74`; there are no geometry setters on any concrete type — changing a dimension means building a replacement part (and losing the id). Geometry is exposed read-only through a family of virtuals: `getLength()` (a part occupies $z \in [-L, 0]$ in its own $+z$ =forward frame, fore plane at the local origin) `model/parts/Part.h:130`, the silhou-

ette profiles `radiusOuterAt()/radiusInnerAt()` `model/parts/Part.h:135`, `axialLength()`, `isSolid()`, and `stationAt()` `model/parts/Part.h:144`, the solid-host capacity rule `innerCapacityAt()` `model/parts/Part.h:158`, and the frontal-disc reference areas `model/parts/Part.h:162`. [Section 5](#) gives these their semantics; the catalog below records what each concrete type returns.

The sanctioned construction path is `PartFactory`. One type-agnostic `PartParams` struct spans every factory-buildable part, every geometry field a `std::optional` so a missing key differs from a supplied zero `model/parts/PartFactory.h:27`; the CLI’s key=value parser, `makePart()`, and the reflection function `params()` all share it, “so the field vocabulary can’t drift between the front-ends and the on-disk format” `model/parts/PartFactory.h:23`. `makePart(type, p)` dispatches on the exact `typeName()` string `model/parts/PartFactory.cpp:27`: a missing required field throws a type-named `std::invalid_argument` `model/parts/PartFactory.cpp:16`, constructor range validation propagates, and an unknown type — including “Motor”, attached through `RocketModel::setMotorModel()`, never built directly — throws as well `model/parts/PartFactory.cpp:73`. The factory always passes a zero CM offset: “placement is the caller’s attach offset, not a baked-in CM” `model/parts/PartFactory.h:54`. `params()` reflects a part’s own scalars back through a `dynamic_cast` chain `model/parts/PartFactory.cpp:79`, and the round trip reproduces mass and full composite CM/inertia to within four ulps (`EXPECT_DOUBLE_EQ`) `model/tests/PartFactoryTests.cpp:139`. New part headers register in one place, the umbrella header `model/parts/Parts.h` `model/parts/Parts.h:4`.

4.2 The concrete catalog

Five concrete types derive from `Part` ([Table 3](#)). The four geometry types compute their mass, per-unit-mass inertia tensor, and CM offset from geometry plus density at construction, delegating the tensor algebra to `model::InertiaTensors` ([Section 4.5](#)); their constructors validate ranges and throw `std::invalid_argument` on non-physical geometry, so a constructed part is always internally consistent. The `Motor` takes its mass and dimensions from the wrapped catalog entry instead.

<code>typeName()</code>	Geometry parameters	Own mass	Tensor	$C_{N\alpha}$
<code>BodyTube</code>	r_i, r_o, L, ρ	$\rho \pi (r_o^2 - r_i^2) L$	<code>Tube</code>	0
<code>NoseCone</code>	$R, L, \text{wall } t, \rho, \text{solid flag}$	$\rho \frac{1}{3} \pi R^2 L$ (solid); $\rho \pi R \sqrt{R^2 + L^2} t$ (shell)	<code>SolidCone / ConicalShell</code>	2
<code>FinSet</code>	$N, c_r, c_t, s, X_t, t, r_b, \rho$	$N \rho \frac{1}{2} (c_r + c_t) s t$	<code>TrapezoidalFinSet</code>	fin term
<code>HollowSphere</code>	r_i, r_o, ρ	$\rho \frac{4}{3} \pi (r_o^3 - r_i^3)$	<code>HollowSphere</code>	inert
<code>Motor</code>	wraps a <code>MotorModel</code>	<code>mm.getMass(t)</code>	cylinder from motor d, L	inert

Table 3. The concrete part catalog at the pinned commit 254cd41. Mass closed forms: `model/parts/BodyTube.cpp:28`, `model/parts/ConicalNoseCone.cpp:32`, `model/parts/FinSet.cpp:46`, `model/parts/HollowSphere.cpp:42`, `model/parts/Motor.h:40`. The $C_{N\alpha}$ column is each part’s Barrowman normal-force-slope contribution, analyzed in [Section 9](#).

BodyTube — a uniform-density hollow circular cylinder, axis on z ; a solid rod when $r_i = 0$. The constructor validates $0 \leq r_i < r_o, L > 0, \rho > 0$ `model/parts/BodyTube.cpp:22`; the CM sits at the geometric center `model/parts/BodyTube.h:25` and the silhouette is constant walls `model/parts/BodyTube.h:55`. It contributes a reference area πr_o^2 `model/parts/BodyTube.h:52` and a wetted area $2\pi r_o L$ held ready for a future skin-friction drag term `model/parts/BodyTube.h:51`; aerodynamically it reports $C_{N\alpha} = 0$ — a constant-diameter body produces no normal force, so it drops out of the center-of-pressure average `model/parts/BodyTube.cpp:38`.

ConicalNoseCone — a straight right circular cone, either solid (the default) or a thin lateral shell (open base, wall thickness $t \ll R$) `model/parts/ConicalNoseCone.h:22`. The two variants differ in everything derived: mass `model/parts/ConicalNoseCone.cpp:32`, tensor (`SolidCone` vs. `ConicalShell` `model/parts/ConicalNoseCone.cpp:44`), and CM — the solid centroid is $L/4$ from the base, the shell's $L/3$, reported as offsets $-L/4$ and $-L/6$ from the component middle in the $+z$ =forward frame `model/parts/ConicalNoseCone.cpp:49`. The silhouette is the linear taper $R \cdot (-z/L)$ with a zero-length divide guard `model/parts/ConicalNoseCone.h:61`, and `isSolid()` returns the *authored* flag, because a shell cone's bore vanishes at the tip and bore inference would misclassify it `model/parts/ConicalNoseCone.h:55`. Its Barrowman contribution is the classic nose $C_{N\alpha} = 2$ at its own base area, rescaled to the shared reference area, with the center of pressure at $2L/3$ aft of the tip `model/parts/ConicalNoseCone.cpp:57`. It is the only nose profile in the tree — no ogive, parabolic, or Haack shapes exist at this commit ([Section 13](#)).

FinSet — $N \geq 3$ identical symmetric trapezoidal flat-plate fins, modeled as *one analytic leaf*, not N child parts `model/parts/FinSet.h:22`. The geometry vocabulary is the standard planform set: root chord c_r , tip chord c_t (zero allowed — a triangular fin), semi-span s , plate thickness, mounting body radius r_b , and the leading-edge (LE) sweep *length* X_t in meters — an axial offset, not an angle `model/parts/FinSet.h:45`. Its axial extent is the root chord `model/parts/FinSet.h:61`, and its CM lies on the axis at the trapezoid's axial mass centroid, reported as $x_c - c_r/2$ from the middle `model/parts/FinSet.cpp:54`. The envelope is deliberately honest: `radiusOuterAt()` reports only the body disc r_b , never $r_b + s$ `model/parts/FinSet.h:73`, and the reference area is the body disc πr_b^2 — fins do not inflate the reference area `model/parts/FinSet.h:71`. A fin count below three is genuinely anisotropic and unsupported: the constructor warns and falls back to the $N \geq 3$ isotropic tensor rather than throwing `model/parts/FinSet.cpp:36`. Its Barrowman contribution is the standard N -fin term with the mid-chord-line factor and the fin-body interference factor $K_{fb} = 1 + r_b/(s + r_b)$ `model/parts/FinSet.cpp:60` ([Section 9](#)).

HollowSphere — a uniform-density thick-walled hollow sphere. The constructor rejects $r_i \geq r_o$ and $\rho \leq 0$ up front, because $r_i = r_o$ would divide by zero inside the tensor formula `model/parts/HollowSphere.cpp:33`. Its length is pole to pole, $2r_o$ `model/parts/HollowSphere.h:51`; its silhouette is the circle about a center at $z = -r_o$, with the cavity as an inner band `model/parts/HollowSphere.cpp:53`; and it overrides `isSolid()` because the bore vanishes at the poles, where fore-plane inference would call it solid `model/parts/HollowSphere.h:59`. It doubles as the boot placeholder body a `RocketModel` constructs before any design is authored or loaded `model/RocketModel.cpp:30`.

Motor — a zero-length leaf that wraps a `MotorModel` by *value* `model/parts/Motor.h:18`, so a burning motor's mass participates in composite mass, CM, and inertia like any other part. Its single physics contribution is the `getMass(t)` override, which delegates to the wrapped model: loaded weight before ignition, falling to casing mass over the burn, constant after burnout `model/parts/Motor.h:40` ([Section 6](#)). Swapping motors goes through `setMotorModel()`, which mutates in place — re-seeding the tensor and mass so the tree recomputes — rather than re-attaching, so the `RocketModel`'s borrowed `Motor*` handle stays valid `model/parts/Motor.cpp:16`. It is not factory-constructible; `RocketModel::setMotorModel()` attaches it at the airframe root, and since a `Motor` carries no geometric envelope it never trips the overlap gate `model/RocketModel.cpp:118`.

Fin sets are one leaf, not N children

“The Part tree shares one body-frame orientation and never rotates child tensors (`Part.h`), so an azimuthally-arrayed component can't be N children at different angles — each fin's tensor

would need an R_z rotation the tree won't apply. A `FinSet` instead bakes the N-fin rotate-and-sum into its own per-unit-mass tensor" `model/parts/FinSet.h:25`. This is the composite walk's omitted $R I R^T$ rotation (Section 4.3) surfacing as a modeling constraint: the one component whose sub-parts genuinely sit at different azimuths must pre-sum them analytically in `InertiaTensors::TrapezoidalFinSet` (Section 4.5) and present as a single transversely isotropic leaf.

The motor is the tree's one millimeter island

Every part API in `model/parts/` speaks SI meters — the factory contract says so explicitly: “Lengths SI meters, densities kg/m^3 — never mm” `model/parts/PartFactory.h:25`. The lone exception is the motor's *catalog* geometry: `MotorModel` stores diameter and length in millimeters, the convention of the RockSim files and the motor database that feed it (Section 6). The unit boundary is crossed at exactly two lines, both inside `Motor::motorTensor(): motor.data.diameter / 1000.0` and its length twin, annotated “RSE / DB store mm” `model/parts/Motor.cpp:27`. From those two divisions onward everything the tree sees is meters: the per-unit-mass tensor is a solid cylinder built from the converted dimensions, degrading to a zero tensor — a point mass that still contributes its time-varying mass — when the catalog geometry is unknown `model/parts/Motor.cpp:31`.

4.3 Mass properties: the two-pass composite walk

The tree's tensor bookkeeping rests on one convention: a part's *own* stored tensor (`getI()/setI()`) is *per unit mass* — the inertia integral divided by mass, a purely geometric quantity in m^2 — while every *composite* tensor (`getCompositeI()`) is the full mass-weighted tensor in kg m^2 about the composite CM `model/parts/Part.h:30`. The constructor seeds the childless composite as $m \cdot I$ `model/parts/Part.cpp:41`, and the composite walk multiplies by `getMass(t)` at read time `model/parts/Part.cpp:225`. The payoff is that density and geometry are factored: the closed forms in `InertiaTensors` are pure geometry, and a `Motor`'s tensor stays constant while its mass burns down — the time variation enters once, through the mass factor. A white-box test pins the convention so it cannot silently regress: for a 2 kg unit sphere, `getI()` reads 0.4 and `getCompositeI()` reads 0.8 `model/tests/PartTests.cpp:145`.

The parallel-axis (Huygens–Steiner) theorem

The inertia tensor I measures a rigid body's resistance to angular acceleration; unlike mass, it depends on the reference point it is taken about, and it is smallest about the body's own CM. To combine several bodies into one composite, each body's centroidal tensor must first be re-expressed about the *common* point — the composite CM. For a body of mass m whose CM is displaced by the vector d from that point, the theorem gives

$$I' = I_{\text{cm}} + m((d \cdot d) I_3 - d d^T),$$

where I_3 is the 3×3 identity. The first added term, $m(d \cdot d)I_3$, is the familiar scalar rule $I = I_{\text{cm}} + m d^2$ applied to every axis at once; the subtracted $m d d^T$ removes the component along d itself, because displacing a body *along* an axis does not change its inertia about that axis. The correction is even in d — only the offset matters, not its sign. One subtlety governs composite algorithms: the map is valid only from a body's own CM to the new point — shifting through an intermediate point does *not* add up, so each body must be shifted in one step, from its own CM directly to the composite CM.

All three composite quantities come from one time-aware walk, `computeCompositeAt(t)` `model/parts/Part.h:230`. It runs over `resolvedCache` — the resolver output for this sub-tree with the sub-tree root planted at the local origin `model/parts/Part.cpp:166` — so geometry is resolved once per structural change and the walk merely *re-weights* it by `getMass(t)` `model/parts/Part.cpp:177`. Before any arithmetic, the walk consults the cached placement verdict and refuses a failed solve ([Section 4.6](#)).

Pass 1: the mass-weighted composite CM. For each resolved placement, the part’s CM is located in the sub-tree-root frame by composing its resolved fore-plane origin with its local CM station `model/parts/Part.cpp:196`. Writing the code’s own names, with pm_i the part’s `getMass(t)` and $cmOffset_i$ the CM-versus-middle offset of [Section 4.2](#):

$$cmLocal_i = \left(cmOffset_{i,x}, cmOffset_{i,y}, -\frac{L_i}{2} + cmOffset_{i,z} \right), \quad (5)$$

$$cmInRoot_i = pose.origin_i + pose.orient_i \cdot cmLocal_i, \quad (6)$$

$$m = \sum_i pm_i, \quad temp_cm = \frac{1}{m} \sum_i pm_i cmInRoot_i \quad (m > 0). \quad (7)$$

The $-L_i/2$ appears because every part reports its CM relative to the component *middle*, while its resolved origin is its *fore plane*; the uniform expression $-L/2 + cmOffset_z$ converts one to the other for every type with no per-part adapter `model/parts/Part.h:81`. A fully massless sub-tree has no meaningful CM, so the divide is guarded and `temp_cm` stays at the origin `model/parts/Part.cpp:210`. Because the sub-tree root’s own pose is the identity, the result is expressed in the root’s fore-plane frame — for the whole rocket, the nose tip. That *tip datum* is shared by the composite center-of-pressure fold ([Section 9](#)), which is what later makes the static margin a plain subtraction.

Read the composite-CM datum from the tests, not the header

Three doc comments in `model/parts/Part.h` still describe the composite CM as “relative to this part’s own CM” — on the `CompositeProperties.cm` field `model/parts/Part.h:90`, on `getCompositeCm()` `model/parts/Part.h:104`, and on the `compositeCm` member `model/parts/Part.h:258` — and `sim/Aero.h:24` carries the same stale wording. The implementation reports the tip datum: the walk plants the sub-tree root at the local origin `model/parts/Part.cpp:166` and folds each part at `pose.origin + orient * (-L/2 + cmOffset)` `model/parts/Part.cpp:203`, so even a childless nonzero-length part reports $-L/2 + cmOffset_z$, not zero. The tests state the truth explicitly — “Composite CG is now reported in the root’s fore-plane (tip) datum, not the root’s own CM” `model/tests/BodyTubeTests.cpp:92` — and the migration-invariance suite is built around it `tests/PlacementInvarianceTests.cpp:113`. This is a stale comment, not a code defect — the tensor is taken about the composite CM regardless of which datum that CM is reported in, so no arithmetic is wrong — but a reader who trusts those three lines will misplace every CG by half the root’s length.

Pass 2: inertia about the composite CM. The walk then shifts each part’s mass-weighted tensor to `temp_cm` with the parallel-axis theorem, exactly as the physics box states it ([Listing 6](#)). The displacement tensor is a three-line helper, `return d.dot(d) * Matrix3::Identity() - d * d.transpose();` `model/parts/Part.cpp:21`, noted in the source as even in d “so the sign of d is irrelevant” `model/parts/Part.cpp:19`. In the code’s names,

$$I = \sum_i \left[c.mass_i \cdot I_i + c.mass_i ((d_i \cdot d_i) I_3 - d_i d_i^T) \right], \quad d_i = cmInRoot_i - temp_cm, \quad (8)$$

where I_i is `c.part->getI()`, the per-unit-mass centroidal tensor, so $c.mass_i \cdot I_i$ is the part’s full centroidal tensor in kg m^2 and the second term is its Huygens–Steiner shift to the composite CM `model/parts/Part.cpp:225`. The physics box’s subtlety — shift each body from its *own* CM directly to the composite CM, never through an intermediate point — is exactly the bug class the tests pin: a two-point-mass fixture whose inertia about the composite CM is the reduced-mass value μL^2 (“the pre-fix code computed this about the PARENT’s CM” `model/tests/PartTests.cpp:199`), a depth-2 chain checked against its flattened equivalent `model/tests/PartTests.cpp:219`, and the sharpest oracle of all: two coaxial tubes stacked end-to-end must reproduce one longer tube’s mass, CM, and full tensor to 10^{-12} — the transverse moment grows as L^2 , so any shift error is loud `model/tests/PartTests.cpp:308`.

```
// Pass 2: inertia about the composite CM. Shift each part's own mass-weighted tensor to temp_cm via
// the parallel-axis theorem. 6-DOF would first rotate the tensor by pl.pose.orient (the R*I*R^T term,
// identity today, omitted to stay bit-stable).
// TODO(6-DOF): when that rotation is added, add a test pinning that it reduces to identity in 3-DOF.
Matrix3 I = Matrix3::Zero();
for(const Contribution& c : parts)
{
    const Vector3 d = c.cmInRoot - temp_cm;
    I += c.mass * c.part->getI() + c.mass * parallelAxisTerm(d);
}
```

Listing 6. Pass 2 of the composite walk `model/parts/Part.cpp:217`: the parallel-axis accumulation of Eq. (8), with the in-tree comment naming the one deliberately omitted term — the child-tensor rotation that six-degree-of-freedom flight will require.

The omitted RIR^T rotation

Full generality would rotate each child’s tensor into the composite frame before shifting it: $I'_i = R_i I_i R_i^T$ with R_i from `pl.pose.orient`. The code deliberately omits that step. In three-degree-of-freedom (3-DOF) flight every resolved orientation is the identity — the placement layer carries the quaternions but never sets them (Section 5) — so the rotation would be an exact no-op, and it is skipped “to stay bit-stable” `model/parts/Part.cpp:217`. The omission is total: the only use of `pl.pose.orient` anywhere in the composite path is the CM translation of pass 1 `model/parts/Part.cpp:205`. The class doc calls the inertia shift “the one piece still assuming identity” `model/parts/Part.h:39`, and the companion design document specifies the one-line 6-DOF form `docs/PartPlacementDesignLatex/sections/09-sixdof.tex:83`. Today’s visible consequence is structural, not numerical: azimuthally arrayed components cannot be modeled as rotated children, which is precisely why `FinSet` is one analytic leaf (Section 4.2). Section 13 carries the consolidated 6-DOF inventory.

4.4 Time-awareness and caching

The single override point for time-varying mass is `getMass(t)` `model/parts/Part.h:94`. Exactly one production type overrides it — `Motor` `model/parts/Motor.h:40` — and everything downstream follows from the composite walk with no motor-specific inertia code: as propellant burns, the composite mass falls, the composite CM migrates away from the emptying motor, and the composite tensor re-weights, all through the same two passes. During a burn the composite tensor tracks an independent parallel-axis oracle `model/tests/MotorTests.cpp:162`, and after burnout the composites freeze bitwise `model/tests/MotorTests.cpp:231`. The consumer cadence is per recorded step: `RocketModel::writeMassProperties()` snapshots the triple — mass, CG, inertia, “mutually consistent

at this t ” — into the recorded `StateData` `model/RocketModel.cpp:53`, the seam the 6-DOF rotational ODE will read `sim/StateData.h:59`; the loop that drives it is [Section 7](#)’s subject.

Freshness is governed by two independent gates with two cadences. The first is structural: `markAsNeedsRecomputing()` sets a per-node flag and walks the parent chain `model/parts/Part.h:238`, invoked by `setMass()/setI()` `model/parts/Part.h:74` and by both tree mutators `model/parts/Part.cpp:100`. A white-box fixture (a declared test-only `friend` `model/parts/Part.h:44`) pins the propagation: a grandchild’s `setMass()` dirties the root `model/tests/PartTests.cpp:538`, and removal dirties the ex-parent’s whole ancestor chain `model/tests/PartTests.cpp:585`. The second gate is the mass-delta gate ([Listing 7](#)): absent structural change, the cached CM and tensor rebuild only when the composite mass has *moved* since the last build.

```
const double mNow = getCompositeMass(t);
if(needsRecomputing || mNow != builtAtCompositeMass)
{
    const CompositeProperties c = computeCompositeAt(t);
    compositeMass      = c.mass;
    compositeCm        = c.cm;
    compositeInertiaTensor = c.inertia;
    builtAtCompositeMass = mNow;
    needsRecomputing     = false;
}
```

Listing 7. The mass-delta gate in `ensureCompositeCache()` `model/parts/Part.cpp:130`: a structural edit always rebuilds; otherwise the walk re-runs exactly when the composite mass differs — by exact floating-point `!=` — from the mass it was last built at.

Why `exact ==` on a double is the right gate

The gate key `builtAtCompositeMass` is seeded with a quiet NaN, and NaN compares unequal to everything — so the first read always builds `model/parts/Part.h:248`. During a burn, `getMass( $t$ )` differs at every query, so the CM and tensor re-weight every step. After burnout, the motor’s mass function returns the *bit-identical* empty mass for every t , so `mNow == builtAtCompositeMass` holds exactly and the cache freezes, bit-stably, with zero recomputation for the rest of the flight `model/parts/Part.cpp:132`. Keying on mass rather than on time is deliberate: an adaptive integrator evaluates repeated and *rejected* stage times ([Section 7](#)), and a time-keyed cache would thrash on them, while a mass-keyed one is immune `model/parts/Part.h:108`. Two couplings are worth stating plainly: the gate keys on *total* mass alone, so a hypothetical second time-varying part whose burn kept the total constant while shifting the CM would be served stale values — unrealizable under the single-motor assumption (`model/RocketModel.cpp:13`; [Section 6](#)), but coupled to it; and the freeze relies on post-burnout `getMass` being idempotent float math — a future mass model without that property would silently degrade the cache to rebuild-every-step, a performance hazard rather than a correctness one.

The gate’s key, `getCompositeMass( $t$ )`, is itself never cached: it is a cheap live recursive sum — this node plus every descendant, no tensor work `model/parts/Part.cpp:118` — because it is both the gate key and the hot divisor in the equation of motion. The placement cache, finally, runs on the structural flag alone: `placementDirty` is set by add/remove, never by a mass or inertia edit — geometry is invariant under a burn `model/parts/Part.h:264`. The two cadences are witnessed directly: over a simulated burn, an instrumented tube’s radius-query counter stays frozen while the composite CM and mass move every step `model/tests/DiagnosticsGateTests.cpp:97`. [Section 5](#) covers what that

placement cache holds.

4.5 The closed-form tensor library

The concrete parts take their per-unit-mass tensors from `model::InertiaTensors`, a static-method class of closed forms, each diagonal, about the body’s own CM, with z the symmetry axis `model/InertiaTensors.h:11` (Table 4).

Shape	$I_{xx}/m = I_{yy}/m$	I_{zz}/m	
Solid sphere, radius r	$\frac{2}{5}r^2$	$\frac{2}{5}r^2$	<code>model/InertiaTensors.h:17</code>
Thin-shell sphere, radius r	$\frac{2}{3}r^2$	$\frac{2}{3}r^2$	<code>model/InertiaTensors.h:28</code>
Thick hollow sphere, r_i, r_o	$\frac{2}{5} \frac{r_o^5 - r_i^5}{r_o^3 - r_i^3}$	$\frac{2}{5} \frac{r_o^5 - r_i^5}{r_o^3 - r_i^3}$	<code>model/InertiaTensors.h:45</code>
Tube (solid cylinder at $r_i=0$)	$\frac{1}{12}(3(r_i^2 + r_o^2) + L^2)$	$\frac{1}{2}(r_i^2 + r_o^2)$	<code>model/InertiaTensors.h:63</code>
Solid cone (CM at $L/4$ from base)	$\frac{3}{20}R^2 + \frac{3}{80}L^2$	$\frac{3}{10}R^2$	<code>model/InertiaTensors.h:80</code>
Conical shell (CM at $L/3$ from base)	$\frac{1}{4}R^2 + \frac{1}{18}L^2$	$\frac{1}{2}R^2$	<code>model/InertiaTensors.h:97</code>
Trapezoidal fin set	Eq. (9)	Eq. (9)	<code>model/InertiaTensors.cpp:11</code>

Table 4. The closed-form tensor library: per-unit-mass (m^2), diagonal, about each body’s own CM, z the symmetry axis. The thick-sphere form reduces to the solid sphere as $r_i \rightarrow 0$ and to the thin shell as $r_i \rightarrow r_o$ `model/InertiaTensors.h:39`; the conical shell’s $\frac{1}{18}L^2$ term is noted in the source as verified exact.

The one non-textbook entry is the trapezoidal fin set. The model treats one fin as a uniform trapezoidal lamina in a meridian plane, thickness circumferential, and forms the set tensor as the azimuthal average of one fin’s tensor over the N mounting angles $2\pi k/N$; for $N \geq 3$ that average is transversely isotropic *and independent of N* — which is why the fin count parameter is `[[maybe_unused]]` in the implementation `model/InertiaTensors.cpp:11`. With $sum = c_r + c_t$, the code computes the radial mass centroid of one fin, $y_c = \frac{s}{3}(c_r + 2c_t)/sum$, and its lever to the body axis, $d = r_b + y_c$ `model/InertiaTensors.cpp:26`, then the single-fin centroidal second moments — spanwise P (sweep-independent), chordwise Q (sweep-dependent through the LE sweep X_t , as the mean square minus the squared centroid), and through-thickness T —

$$P = \frac{s^2(c_r^2 + 4c_rc_t + c_t^2)}{18sum^2}, \quad x_c = \frac{X_t(c_r + 2c_t) + c_r^2 + c_rc_t + c_t^2}{3sum},$$

$$\text{meanXsq} = \frac{X_t^2(c_r + 3c_t) + X_t(c_r^2 + 2c_rc_t + 3c_t^2) + sum(c_r^2 + c_t^2)}{6sum}, \quad Q = \text{meanXsq} - x_c^2, \quad T = \frac{t^2}{12}$$

`model/InertiaTensors.cpp:29`, and finally the azimuthal average, in which the radial–axial product and z -coupling terms cancel for $N \geq 3$:

$$I_{zz}/m = P + T + d^2, \quad I_{xx}/m = I_{yy}/m = \frac{1}{2}(P + 2Q + T + d^2) \quad \text{model/InertiaTensors.cpp:45.} \quad (9)$$

The library’s acceptance gate is not the algebra but a set of independent numeric oracles — the implementation says so itself: “The mesh oracle in `InertiaTensorsTests`, not this closed form, is the acceptance gate” `model/InertiaTensors.cpp:15`. The solid cone is checked against a 400,000-slice disk-stack integral, with the literal constants $\frac{3}{10}R^2$ and $\frac{3}{20}R^2 + \frac{3}{80}L^2$ pinned at 10^{-15} `model/tests/InertiaTensorsTests.cpp:147`; the conical shell against a surface integral `model/tests/`

`InertiaTensorsTests.cpp:169`; and the fin set against a volume-mesh integration at a 0.5% tolerance chosen between the mesh’s own error ($\sim 0.01\%$) and the 1.2–11% signature of a wrong (rectangle-only) chordwise term `model/tests/InertiaTensorsTests.cpp:186`. Cross-checks tighten the net: I_{zz} must equal an independently derived closed form $J + t^2/12$ at 10^{-12} `model/tests/InertiaTensorsTests.cpp:139`, the tensor must be identical across $N \in \{3, 4, 6\}$ `model/tests/InertiaTensorsTests.cpp:227`, and the rectangle degeneracy ($c_t = c_r$, $X_t = 0$) must recover $P = s^2/12$, $Q = c^2/12$ `model/tests/FinSetTests.cpp:139`. The mesh also demonstrates the approximation’s honest edge: a 2-fin set is genuinely anisotropic ($I_{xx} \neq I_{yy}$ by more than 1%), while the helper knowingly returns the forced-isotropic $N \geq 3$ tensor — the case the `FinSet` constructor warns about `model/tests/InertiaTensorsTests.cpp:259`.

4.6 Diagnostics gating on the composite readers

The composite walk opens with a gate, not a computation. `computeCompositeAt()` first ensures the placement cache is fresh, then reads the cached overlap verdict; if the sub-tree’s geometry failed its solve, it throws a `std::runtime_error` carrying the first located diagnostic — “A self-intersecting design is a hard, located failure, never a silently-wrong mass/inertia” `model/parts/Part.cpp:182`. The check costs a flag read, not a re-sweep: the verdict is computed once per structural change and cached alongside the resolved geometry `model/parts/Part.cpp:160`.

Fail closed — except for the ODE divisor

The gate is asymmetric by design. `getCompositeCm()` and `getCompositeI()` both pass through the gated cache and therefore throw on a failed solve, and the test suite pins both refusals `model/tests/DiagnosticsGateTests.cpp:78`. `getCompositeMass()` deliberately does *not*: it is the hot divisor in the equation of motion, queried at every integrator stage, and it performs no placement-dependent work — a mass sum is correct whether or not two parts interpenetrate. The same tests pin that it keeps answering on a failed design `model/tests/DiagnosticsGateTests.cpp:83`. The verdict itself is exposed read-only through `placementDiagnostics()` `model/parts/Part.h:125`, and the visualizer reads that same cached object to paint the offender `visualizer/RocketMesh.cpp:440` — one verdict, two consumers, so the simulator’s refusal and the on-screen highlight can never disagree `model/parts/Part.h:274`. What the sweep actually checks — intervals, breakpoints, the solid-host rule — is [Section 5](#)’s subject.

One more composite fold rides on the same resolved geometry, and it is worth naming here because it completes the tree’s API. `getCompositeAero(refArea)` folds each part’s Barrowman contribution additively over the resolved placements, re-expressing every part’s center-of-pressure moment onto the tip datum — the same datum as the composite CM, so that the static margin is literally `cp() - cg()` `model/parts/Part.cpp:231`. It is a separate, pure-geometry pass from the mass walk — it reads no time-varying state `model/parts/Part.h:119`. At the pinned commit it is also dormant: built, correct on its tests, and consumed by no production code path — the test file says so in as many words (“nothing in production consumes this in P2” `model/tests/AeroTests.cpp:18`). The mathematics it folds, the datum trick that makes the fold additive, and what consuming it would take are the subject of [Section 9](#); [Section 13](#) places it in the consolidated inventory.

5 Placement: stations, seats, and the resolver

[Section 4](#) described the rocket as an ownership tree of parts and the machinery that folds that tree into mass, center of mass (CM), and inertia. This section describes the other half of the model layer’s geometry story: how the tree knows *where* each part sits. The answer is deliberately indirect.

QtRocket never stores a coordinate for any part. What it stores, on every parent→child edge, is a small value called a `StationLink` — a statement of *physical intent* (“this rim seats against that rim, inserted this deep”) — and every absolute position in the rocket is recomputed from those statements by exactly one resolver function whenever the structure changes `model/parts/Placement.h:147`. The section walks the design from philosophy to enforcement: the rules, the data model (stations, seats, links), the resolver arithmetic term by term, the diagnostics that refuse impossible geometry, the narrow authoring surface that exists today, and the invariance suite that pins the system’s observable behavior.

5.1 Philosophy: placement is intent, never coordinates

The placement system was designed in its own whitepaper (`docs/PartPlacementDesignLatex/`) and implemented essentially as designed; where the document and the shipped code disagree, this section reports the code and says so. The design opens with three rules stated before any type is introduced, because they constrain every later decision.

Three non-negotiable rules

(1) *Placement is derived*: absolute position is recomputed by exactly one resolver — neither an absolute coordinate nor a resolved pose is ever stored as truth. (2) *The stored link is physical intent*: a feature-to-feature relationship between two parts, never an opaque offset; the center of mass falls out of geometry as a consequence. (3) *The common case costs zero authored numbers*: a child attached with no link simply abuts its parent `docs/PartPlacementDesignLatex/sections/02-philosophy.tex:21`.

The rules are a reaction to an observed failure. The representation they replaced stored one `Vector3` per child, defined as the child’s CM relative to the parent’s CM. A CM is a derived quantity — a solid nose cone’s sits a quarter length from the base, not at mid-length — so every offset was a hand calculation, and the value’s two consumers quietly disagreed about what it meant: the simulator applied it CM-to-CM as documented, the visualizer as a geometric-center-to-center translation. The two reconstructions of the same file diverged by exactly the cone’s CM offset, and a committed design shipped with a coupler silently interpenetrating the nose cone — no check fired, and a self-intersecting solid was fed to the integrator `docs/PartPlacementDesignLatex/sections/01-introduction.tex:65`. The link recorded a derived quantity where the author actually knew a geometric fact — two rims touch. The replacement stores the geometric fact.

Storing intent also leaves nothing to go stale. Persisting the resolved transform alongside the intent was rejected in the design document as the same staleness bug reflected: edit a length and the stored transform is silently wrong `docs/PartPlacementDesignLatex/sections/02-philosophy.tex:43`. Resolved poses exist in the shipped code only as a cache gated on a structural dirty flag ([Section 5.3](#)) — an optimization that can never be wrong, not a second copy of the truth that can.

One resolver, two consumers, identical numbers

“The single authority for absolute placement: intent (`StationLink`) in, geometry (`Pose`) out. Both the simulator and the visualizer consume its output, so the two cannot disagree” `model/parts/Placement.h:148`. The mass-properties walk reads the resolver’s cached output when it folds the tree `model/parts/Part.cpp:199`, and the visualizer’s mesh walk reads the same output when it draws it `visualizer/RocketMesh.cpp:434` — so the number the integrator uses to place a part’s mass is, by construction, the number the renderer uses to draw it. The legacy divergence is

structurally impossible.

One paragraph of contrast situates the choice. OpenRocket — the comparison baseline of [Section D](#) — positions each component with a signed axial offset plus an `AxialMethod` enumerator (top, middle, bottom, absolute) selecting which datum of the parent the offset is measured from. That scheme also re-derives absolute coordinates from live parent geometry, so it is not a staleness hazard; the difference is semantic. The stored number is a coordinate displacement whose meaning is switched by a flag, and the physical relationship it encodes — *is this coupler inserted into that tube, or merely near it?* — is not represented and cannot be checked. QtRocket’s `StationLink` stores the mating itself: which stations of parent and child sit together, and in what radial relationship. That extra semantic content is what gives the diagnostics layer of [Section 5.4](#) something to enforce, and it is why QtRocket can afford to fail closed where OpenRocket’s tradition is to warn and continue.

5.2 The data model: stations, links, and seats

Stations, fore and aft

Aerospace drafting locates features on a vehicle by *station*: a cross-section plane at a stated distance from a reference datum — “fuselage station 172” names a plane, not a part. *Fore* means toward the nose, *aft* toward the tail. A rocket airframe is, to good approximation, a stack of axisymmetric bodies, so a station plus the radius profile there — outer skin and, for hollow parts, inner bore — captures everything about the cross-section that matters for fit. QtRocket’s placement model is this drafting idea made executable: parts are mated station-to-station, and the radius profiles at the mated stations decide whether the mating is physically possible.

Every part lives in one shared local frame, established in [Section 4](#): $+z$ = forward, the part’s own origin at its *fore* plane, the body occupying $z \in [-L, 0]$ `model/parts/Part.h:130`. Stations are expressed as fractions: `station01` $\in [0, 1]$ with 0 the aft plane and 1 the fore plane, mapped to a local coordinate by

$$z(\text{station01}) = (\text{station01} - 1) \cdot L, \quad (10)$$

where L is the part’s *live* length, re-read from `getLength()` on every resolve `model/parts/Part.cpp:305`. `Part::stationAt` clamps the fraction to $[0, 1]$, evaluates [Eq. \(10\)](#), and returns a `Station` landmark — $\{z, r_{\text{outer}}, r_{\text{inner}}\}$, the coordinate plus the radius profile read at it — which is always derived, never stored `model/parts/Placement.h:62`. Fractions of live length are the anti-staleness property in miniature: lengthen a tube and every seam expressed against it moves to follow, because no stored number baked the old length in (pinned by test: `model/tests/ResolverSweepTests.cpp:168`). One honest caveat: at the pinned commit [254cd41](#), concrete parts are geometrically immutable after construction — no length or radius setters exist `model/parts/BodyTube.h:69` — so the only structural edits are attach and detach, and the live-length machinery is future-proofing for an editing path not yet built ([Section 13](#)).

The stored value on each edge is the `StationLink` ([Listing 8](#)): a fractional station on each of the two mated parts, a gap in meters, a seat kind, and a per-seam quaternion reserved for six-degree-of-freedom (6-DOF) work — identity today, so a stored link is bit-identical to a pure translation `model/parts/Placement.h:44`.

The `SeatKind` enumeration is closed at three values by design: a rigid axisymmetric stack admits exactly three radial relationships `model/parts/Placement.h:32`, and leaving the enum open “would

```

struct StationLink
{
    double    parentStation01{0.0};           ///< 0 = aft plane, 1 = fore plane, on the PARENT
    double    childStation01{1.0};            ///< 0 = aft plane, 1 = fore plane, on the CHILD
    double    gap{0.0};                       ///< meters; meaning per SeatKind
    SeatKind  seat{SeatKind::Abut};           ///< selects the radial check and the gap sign
    Quaternion childRot{Quaternion::Identity()}; ///< (x,y,z,w) seam orientation; identity in 3-DOF
};

```

Listing 8. The only stored spatial relationship in the model `model/parts/Placement.h:49`. The defaults are load-bearing: a default-constructed link means “child fore plane abuts parent aft plane,” the natural nose→body→fins build order, so the common case costs zero authored numbers.

only invite half-defined cases” `docs/PartPlacementDesignLatex/sections/03-datamodel.tex:126`. The seat does double duty: it names the radial compatibility semantics of the seam, and it fixes the *sign* of the gap — the direction in which the gap displaces the child (Table 5). Encoding direction in the seat, rather than asking the author to sign the number, is what lets an insertion depth of 0.04 m and a forward standoff of 0.04 m both be written as a plain positive number. The non-negativity of a `NestInBore` depth is a documented contract of the authoring helpers, not a runtime check `model/parts/Placement.h:133`; an out-of-contract value surfaces only if the resulting geometry trips the envelope sweep.

Seat kind	Radial compatibility semantics	Gap meaning and sign
Abut	Rim to rim: the two outer radii at the mated stations must match, $ p.r_{\text{outer}} - c.r_{\text{outer}} \leq \text{tol}$ <code>model/parts/Placement.cpp:66</code>	Forward standoff along $+z$; flush at zero. The default seat.
NestInBore	Child outer diameter (OD) fits inside the parent’s inner diameter (ID): $c.r_{\text{outer}} \leq p.r_{\text{inner}} + \text{tol}$ <code>model/parts/Placement.cpp:72</code>	Insertion <i>depth</i> , driving the child aft ($-z$); the one sign-flipping seat.
OnSurface	Child seats radially on the parent’s outer wall: child radius equals the parent’s outer radius at the seat station <code>model/parts/Placement.cpp:69</code>	Axial standoff along the wall.

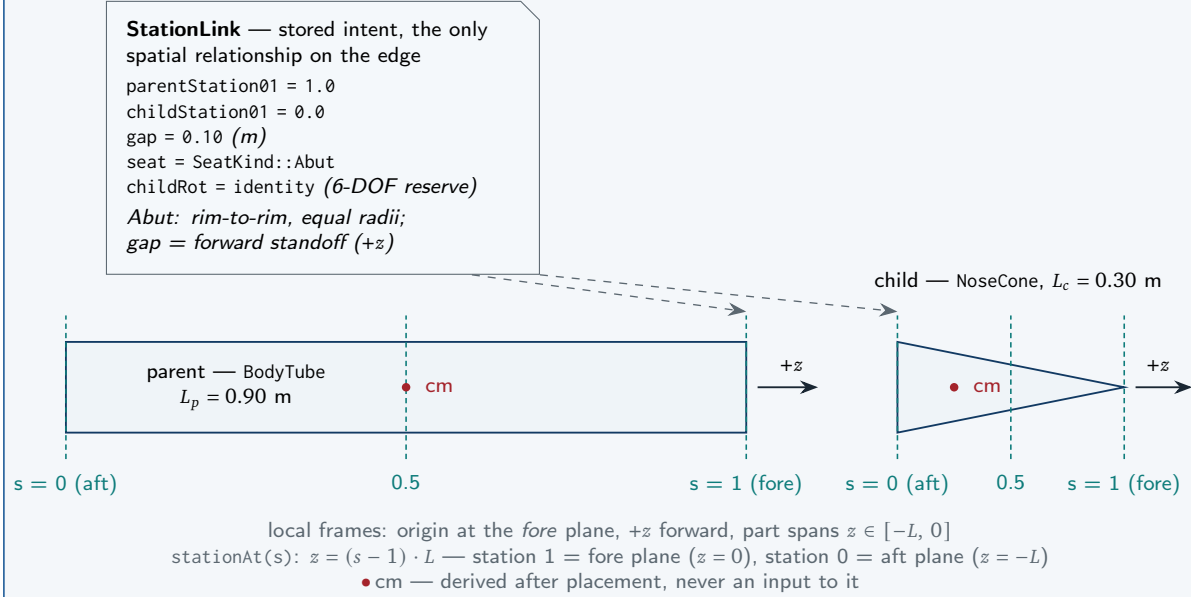
Table 5. The three seat kinds `model/parts/Placement.h:32`. Each seat selects both a radial check between the two mated `Station` landmarks and the direction the gap displaces the child. The radial semantics are implemented in `radialSeamCheck`; the enforcement status of that check is examined in Section 5.4.

Three snap-operator verbs provide sugar over the brace-initializer form: `abut(gap)`, `nestInBore(depth)`, and `seatOnWall(parentStation01)` each produce a `StationLink` by value `model/parts/Placement.h:127`. Their defining property is what they do *not* do: no verb reads a length, computes a station, stores a pose, or mutates a part `model/parts/Placement.h:122`. Verbs name intent; only the resolver resolves it — so a verb called before its operands are sized, or a link reused across designs, cannot bake in a stale coordinate.

5.3 The resolver, term by term

The resolver’s output frame is fixed by one convention: `resolvePlacements(root, Pose{})` plants the root part’s fore plane — for a conventional rocket, the nose tip — at the origin, and the stack

Stored intent — two local frames, fractional stations, one StationLink



Resolved — one resolver: placeChild(parentPose, parent, child, link) → Pose

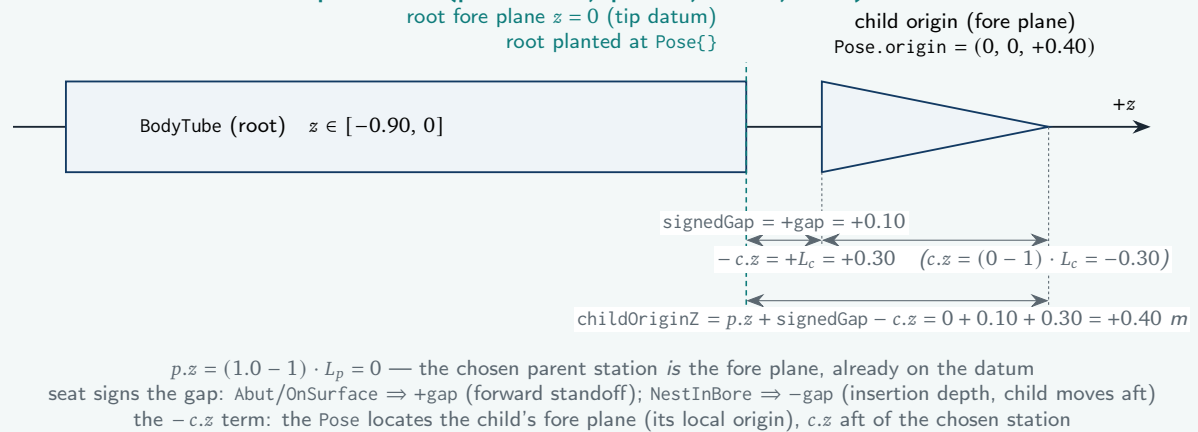


Figure 6. Anatomy of a `StationLink`, shown on the `xl75_multi` stack that exercises all three seat kinds. Each parent→child edge stores fractional stations on both parts, a seat kind, and a seat-signed gap; the resolver turns each link into a fore-plane pose in the nose-tip frame. The coupler's link (`NestInBore`, depth 0.04 m into the body's fore bore) resolves to origin $z = -0.26 \text{ m}$; the fins' link (`OnSurface` at parent station 0.06) seats them on the tube wall near its aft end. No stored number is an absolute coordinate. `model/parts/Placement.h:49`

grows aft into `-z` `model/parts/Placement.cpp:38`. Every derived quantity downstream inherits this *nose-tip datum*: resolved part origins, the composite CM, and the composite center of pressure are all coordinates on the same tip-anchored axis, which is what makes the static margin of [Section 9](#) a literal subtraction. The datum is a fixed, recognizable geometric feature rather than anything mass-dependent — a deliberate contrast with the retired CM-relative convention.

Placing one child is a pure function of the parent’s resolved pose, the two parts’ geometry, and the link ([Listing 9](#)).

```
const Station p = parent.stationAt(link.parentStation01); // landmark on the parent
const Station c = child.stationAt(link.childStation01);   // landmark on the child

// signedGap encodes seat direction in +z = forward: NestInBore inserts aft (-z, gap is depth),
// Abut/OnSurface stand off forward (+z). The author always writes a non-negative number.
const double signedGap = (link.seat == SeatKind::NestInBore) ? -link.gap : link.gap;

// Line the child's station up with the parent's, then displace by the gap. The pose locates the
// child's fore plane, which sits c.z from its chosen station -- hence the '- c.z'.
const double childOriginZ = p.z + signedGap - c.z;

const Pose childInParent(Vector3(0.0, 0.0, childOriginZ), link.childRot); // coaxial in 3-DOF
return parentPose.compose(childInParent);
```

Listing 9. The entire resolver core `model/parts/Placement.cpp:23`: two station lookups, one seat-signed gap, one subtraction, one rigid-transform composition. Everything else in the placement system exists to feed this arithmetic or to check its output.

$$\text{childOriginZ} = p.z + \text{signedGap} - c.z, \quad \text{signedGap} = \begin{cases} -\text{gap} & \text{NestInBore,} \\ +\text{gap} & \text{Abut, OnSurface.} \end{cases} \quad (11)$$

Each term earns its place:

- $p.z$ is the parent-local coordinate of the chosen parent station, $(\text{parentStation01} - 1) \cdot L_{\text{parent}}$ by [Eq. \(10\)](#) — *where on the parent the seam sits*.
- signedGap is the authored gap with the seat’s direction applied. A *NestInBore* gap is an insertion depth, so it moves the child aft ($-z$); an *Abut* or *OnSurface* gap is a standoff, so it moves the child forward ($+z$) `model/parts/Placement.cpp:28`. The author never signs the number; the seat does.
- $-c.z$ converts a seam position into a fore-plane position. The *Pose* locates the child’s *fore plane* — its local origin — but the seam was expressed against the child’s chosen station, which sits $c.z$ from that origin. Subtracting $c.z$ re-expresses the one as the other `model/parts/Placement.cpp:30`.

The resulting child-in-parent pose is $(0, 0, \text{childOriginZ})$ with orientation `link.childRot`: $x = y = 0$ because placement is coaxial in three degrees of freedom (3-DOF). Radial position is deliberately a *check*, not a coordinate — the radii recovered from the two stations feed the diagnostics, but no pose is ever displaced off axis `model/parts/Placement.cpp:34`. The design document rejects off-axis seating in 3-DOF explicitly: without orientation and a body-frame CM offset it would store a half-correct placement, the exact defect class the design exists to eliminate, relocated from the axial axis to the radial one `docs/PartPlacementDesignLatex/sections/04-resolver.tex:132`. The pose is composed into the root frame by `Pose::compose` — rotate-then-translate, which under identity quaternions

degenerates bit-identically to vector addition `model/parts/Placement.h:81`; carrying the quaternion anyway is the zero-schema path to 6-DOF ([Section 13](#)).

`resolvePlacements` is then a depth-first search (DFS) from the root pose, calling `placeChild` for each (`child`, `link`) pair in attachment order and emitting one `Placed` — a non-owning part pointer plus its resolved pose — per part, in deterministic pre-order `model/parts/Placement.cpp:38`. The determinism is pinned by test `model/tests/ResolverSweepTests.cpp:151` because the overlap sweep’s reproducibility depends on it. The resolver reads “no CM, time, or mass” `model/parts/Placement.h:158`: it is pure geometry, which is what allows it to run on a completely different cadence from the mass machinery.

Worked numbers make the arithmetic concrete, on the `xl75_multi` stack of [Fig. 6](#); every value below is pinned in `model/tests/ResolverSweepTests.cpp:95` and reproduced end-to-end through the design file reader in `tests/PokeThroughRegressionTests.cpp:70`. The nose (root, length 0.30 m) plants at the identity pose: tip at $z = 0$, base rim at -0.30 m. The body tube (length 0.90 m) attaches with the default link — zero authored numbers — so $p.z = -0.30$, $c.z = 0$, gap 0: its fore plane lands at -0.30 m and it spans $[-1.20, -0.30]$. The fins seat on the wall at parent station 0.06: $p.z = (0.06 - 1) \cdot 0.90 = -0.846$ (world -1.146), $c.z = -0.10$ (root chord), so the fin origin resolves to world -1.046 m. The interesting case is the coupler (length 0.08 m), nested `nestInBore(0.04)` in the body’s fore bore:

$$\text{childOriginZ} = \underbrace{0}_{p.z \text{ (bore mouth)}} \underbrace{-0.04}_{\text{signedGap}} \underbrace{+0.08}_{c.z \text{ (aft plane)}} = +0.04$$

in the body frame — world -0.26 m, spanning $[-0.34, -0.26]$. Its aft 0.04 m is correctly buried in the bore; its fore 0.04 m projects forward past the body rim, a fact the resolver happily records and [Section 5.4](#) refuses.

Where do the poses land? Each `Part` memoizes its own sub-tree’s resolution in a mutable `resolvedCache`, rebuilt by `ensurePlacementCache` only when the *structural* dirty flag `placementDirty` is set — by attach and detach, never by a mass or inertia edit `model/parts/Part.h:264`. The rebuild does two things in one breath: it re-resolves the poses (this part planted at the local origin) and re-runs the envelope sweep of [Section 5.4](#), caching the verdict alongside the poses `model/parts/Part.cpp:160`. Two in-tree consumers read the cache: the two-pass composite mass/CM/inertia walk of [Section 4](#) `model/parts/Part.cpp:199` and the composite aerodynamics fold of [Section 9](#) `model/parts/Part.cpp:231`. The visualizer’s mesh walk invokes the same resolver on the same tree — “consume the same absolute placement the simulator does,” its own comment says — and reads the cached diagnostics verdict alongside it `visualizer/RocketMesh.cpp:434`.

Two gates, two cadences

The mass-delta gate of [Section 4](#) fires every step while a motor burns, because composite mass moves every step; the placement gate fires only when the rocket is structurally rebuilt, because geometry is invariant under a burn `model/parts/Part.cpp:162`. A white-box test proves the split: during a simulated burn the composite CM shifts every step while a counting part’s radius-profile call count stays frozen — the resolver and the sweep never re-fire inside the ordinary-differential-equation (ODE) loop `model/tests/DiagnosticsGateTests.cpp:97`.

5.4 Diagnostics: the envelope sweep and the fail-closed gate

Deriving placement from intent removes the consumers-disagree error class, but not the other one: a set of links can be individually sensible and still describe a physically impossible rocket — the

worked coupler above pokes 0.04 m out of its bore into the solid nose. The resolver places it without complaint; placement is pure geometry. The diagnostics layer is the component that refuses such designs, and refuses them with a *located* reason rather than a silent miscomputation.

The enforced check is the *envelope sweep*, `sweepOverlaps model/parts/Placement.cpp:107`: a global one-dimensional spatial query over the resolved tree, run from `ensurePlacementCache` once per structural change `model/parts/Part.cpp:160`. Its steps:

1. *Intervals*. Each resolved part contributes a world axial interval $[z_{\text{aft}}, z_{\text{fore}}] = [\text{originZ} - \text{axialLength}, \text{originZ}]$ — valid because orientation is identity in 3-DOF `model/parts/Placement.cpp:95`.
2. *Sort*. Intervals sort by aft station, making host lookup a sweep — the $O(N \log N)$ in the design's name `model/parts/Placement.cpp:151`.
3. *Feature breakpoints*. Each offender is sampled at its own two endpoints plus every other interval endpoint within its span `model/parts/Placement.cpp:160` — exact rather than sampled, *given* the premise that radius profiles are closed-form and monotone between breakpoints (examined below).
4. *Host selection*. At each breakpoint the host is the smallest-id covering interval, excluding the offender itself, its descendants (legitimate nesting), and its direct seat parent `model/parts/Placement.cpp:142`. Smallest-id is a deterministic, reload-stable tie-break, pinned by test `model/tests/ResolverSweepTests.cpp:325`; stated plainly, the check is single-host per breakpoint — a violation against a larger-id covering host goes unevaluated at that station.
5. *Capacity test*. The offender's outer radius is tested against the host's capacity under the solid-host rule, with a guard exempting an offender on or outside a hollow host's own skin — a modeling artifact (a fin set's body disc over a co-radial aft coupler), not a collision `model/parts/Placement.cpp:192`.
6. *Verdict*. The deepest violation per (offender, host) pair is kept; `SolveResult.ok` holds exactly when the diagnostics list is empty `model/parts/Placement.cpp:230`.

The solid-host rule

What radius does a host *offer* an intruder? For a bored part, its bore; for a solid part, its outer skin. `return isSolid() ? radiusOuterAt(zLocal) : radiusInnerAt(zLocal); model/parts/Part.cpp:314` — and an offender of radius r fits iff $r \leq \text{innerCapacityAt}(z) + \text{tol}$ `model/parts/Part.h:158`. Without the rule, a naive OD-versus-bore predicate reads a solid cone's bore as zero everywhere and flags every neighbor; with it, the same single predicate is simultaneously the nesting check and the poke-through check, and call sites never branch on solidity.

On the worked stack the sweep catches exactly the violation the resolver recorded. Over $[-0.34, -0.30]$ the only interval covering the coupler is the body's — its direct seat parent, excluded from host selection — so no capacity test fires there. Over $[-0.30, -0.26]$ the covering envelope that survives exclusion is the solid nose — a *non-tree neighbor*: the coupler is linked to the body, not the nose, and the sweep sees the collision precisely because it does not restrict itself to the ownership tree. At the breakpoint $z = -0.26$ the nose skin has tapered to $0.0395 \cdot (0.26/0.30) = 0.0342$ m, so the coupler's OD exceeds capacity by 0.0034 m: the diagnostic names offender and host by stable id, the world station, and the penetration depth `model/tests/ResolverSweepTests.cpp:249`. Ids, not names or pointers, are the only safe handle: part names need not be unique, and a raw pointer would dangle if the tree were edited between resolve and report `model/parts/Placement.h:99`.

The verdict is consumed fail-closed. `computeCompositeAt` — the single walk behind the

composite mass-property reads — refuses a failed solve before doing any arithmetic (Listing 10), so both `getCompositeCm(t)` and `getCompositeI(t)` throw on a self-intersecting design `model/parts/Part.cpp:148`; the behavior is pinned directly `model/tests/DiagnosticsGateTests.cpp:66` and end-to-end through the design-file reader on a frozen poke-through fixture `tests/PokeThroughRegressionTests.cpp:46`. Exactly one composite reader is deliberately exempt: `getCompositeMass(t)` stays an ungated live sum, because it is the hot ODE divisor and does no placement work `model/tests/DiagnosticsGateTests.cpp:81`. The visualizer reads the same cached verdict through `placementDiagnostics()` `model/parts/Part.h:125` and flags the offending part `visualizer/RocketMesh.cpp:440` — one verdict, two consumers, computed where the geometry changes rather than where it is consumed, so guarding the inner loop costs a single boolean read.

```
// A self-intersecting design is a hard, located failure, never a silently-wrong mass/inertia. The
// verdict is cached (computed once per structural resolve), so this is a flag read, not a re-sweep.
if(!resolvedDiagnostics.ok)
{
    const std::string detail = resolvedDiagnostics.diagnostics.empty()
        ? std::string("self-intersecting geometry")
        : resolvedDiagnostics.diagnostics.front().message;
    throw std::runtime_error("Part::computeCompositeAt: placement solve failed -- " + detail);
}
```

Listing 10. The fail-closed gate at the top of the composite walk `model/parts/Part.cpp:182`: the thrown error carries the first located diagnostic, so the caller learns who intruded into what, where, and by how much.

The monotonicity premise fails for the sphere

The breakpoint scheme’s exactness rests on the claim that every radius profile is “closed-form and monotone between breakpoints” `model/parts/Placement.cpp:161`. True for tubes (constant), cones (linear taper), and fin sets (body disc) — false for `HollowSphere`, whose silhouette $\sqrt{r_o^2 - (z + r_o)^2}$ is zero at both poles (its own interval endpoints) and maximal at the mid-span equator `model/parts/HollowSphere.cpp:53`. A sphere nested in a too-narrow tube evaluates $r \approx 0$ at every breakpoint and passes undetected; no sweep test exercises a sphere `model/tests/ResolverSweepTests.cpp`. The placement design document itself listed “a sphere bulges” among the monotone profiles — one of the places the companion documents disagree with the shipped code, and the Architecture Review’s code-verified finding wins. **FAIL**

Two further layers of the designed diagnostics stack deserve an honest status report. The *radial seam check* — Layer 1 in the design document, the per-seam radial compatibility test of Table 5 — is implemented in full as `radialSeamCheck` `model/parts/Placement.cpp:56` and unit-tested across all three seats `model/tests/ResolverSweepTests.cpp:188`, but it is *dormant*: the cached verdict contains Layer-2 sweep output only `model/parts/Part.cpp:170`, and no production code calls the function. The design document places it “on every resolve” `docs/PartPlacementDesignLatex/sections/06-diagnostics.tex:66`; the as-built resolver does not run it, and its `zWorld` field is left parent-local behind the comment “the root-frame z is added once a pose is known” `model/parts/Placement.cpp:87` — a promise no caller exists to fulfill. The practical consequence: a pure rim-radius mismatch producing no axial-interval overlap is not in the cached verdict today. Layer 3 — degree-of-freedom accounting, a detector warning when a child is under-constrained under free orientation — exists only in the design document `docs/PartPlacementDesignLatex/sections/06-diagnostics.tex:179`; no corresponding code exists anywhere in `model/parts/`. Both gaps are inventoried in Section 13.

5.5 Authoring today: one verb reaches the front ends

The placement model’s vocabulary is far richer than any interface that speaks it. At the pinned commit, exactly one placement operation is reachable from a front end: the command-line interface’s (CLI’s) `addpart` command, which attaches every new part with `model::part::abut(offset.z())` — the child’s fore plane against the parent’s aft plane, with an optional forward standoff taken from a parsed `z=` token [cli/Repl.cpp:872](#). The grammar accepts `x=` and `y=` too, but they are parsed and discarded: placement is coaxial in 3-DOF, and only the axial component reaches the link [cli/Repl.cpp:189](#). The graphical interface authors no structure at all at this commit ([Section 10](#)), so the CLI’s single verb *is* the interactive authoring surface.

Everything else arrives by other roads. The full seat vocabulary — `NestInBore`, `OnSurface`, non-default stations, arbitrary gaps — is reachable from a design file, where the serializer reads a `<link>` element’s seat, stations, and gap [model/DesignSerializer.cpp:186](#) and rejects an unknown seat string fail-closed [model/DesignSerializer.cpp:184](#) ([Section 11](#)), and from C++, where the one production author of a link with non-default stations is the motor attachment: `RocketModel::setMotorModel` joins the airframe root’s CM station to the motor’s CM station with an `Abut` link whose gap is a motor offset that is zero today [model/RocketModel.cpp:124](#). Nothing in production sets the reserved `childRot` to a non-identity value. A richer authoring surface — interactive seat selection, station editing, the geometry mutation that would finally exercise the live-length machinery — is future work the data model was expressly shaped for; [Section 13](#) carries the inventory.

5.6 The invariance gate: the system’s contract

The placement system replaced a load-bearing representation underneath a working simulator, and its correctness argument is embodied in a test suite that predates the implementation: [tests/PlacementInvarianceTests.cpp](#), written as Phase 0 of the migration before any production type changed [tests/PlacementInvarianceTests.cpp:1](#). The gate loads the 24 committed design fixtures [tests/PlacementInvarianceTests.cpp:66](#) and compares, per design, the quantities the integrator consumes against an immutable committed baseline generated by the *legacy* CM-to-CM reader before the refactor:

- composite mass and composite inertia about the CM — datum-independent quantities — match the frozen baseline to a relative 10^{-9} , a tolerance chosen to admit unit-in-the-last-place drift and nothing else [tests/PlacementInvarianceTests.cpp:147](#);
- the composite CM matches after one known, deliberate correction: the datum change from the root’s own CM to the nose tip, re-expressed by the single fixed offset of the root’s local CM station [tests/PlacementInvarianceTests.cpp:286](#);
- every part’s resolved axial station matches under the same single shift [tests/PlacementInvarianceTests.cpp:299](#);
- the static margin is deliberately *not* compared: the legacy center of pressure was contaminated by part CMs, and correcting it was part of the point [tests/PlacementInvarianceTests.cpp:318](#);
- a save–reload round trip through the current design writer reproduces the same snapshot, pinning idempotence of the intent serialization [tests/PlacementInvarianceTests.cpp:358](#).

Two hand-computed pins guard the specific defects the migration corrected: the solid nose cone’s local CM at -0.225 m for $L = 0.30$ m — three quarters of the length aft of the tip, computed by hand independently of both code paths, isolating the cone’s historical reversed-frame sign [tests/PlacementInvarianceTests.cpp:445](#) — and the fin set’s axial CM reference, worth half a root chord, guarding the end-versus-mid-chord confusion [tests/PlacementInvarianceTests.cpp:458](#).

Invariance is defined by the incumbent, not the spec

The baseline is the legacy reader’s literal output — including its mis-placements — not a re-derivation from each design’s intended geometry, because a re-derivation could silently share whatever bug the legacy path contained. The file’s own header declares it immutable ground truth that must never be regenerated to make a failing migration pass `tests/PlacementInvarianceTests.cpp:30`. Correcting a design’s actual geometry (the interpenetrating coupler) was done separately and visibly: the fixed corpus resolves clean, and the original defect survives as a frozen regression fixture that must keep failing `tests/PokeThroughRegressionTests.cpp:46`.

The gate is the placement system’s contract in the strongest sense available to a refactor: every observable the flight path consumes is pinned across the representation change, every deliberate deviation (the tip datum, the corrected CP) is named and accounted for, and the one behavior that was supposed to change — a self-intersecting design failing loudly instead of simulating silently — is regression-guarded from the design file all the way to the thrown, located error. Placement intent in, geometry out, and a test suite standing over the seam.

6 The motor subsystem

The motor subsystem is a translation layer: it turns catalog motor data — a burn’s worth of measured thrust samples plus manufacturer metadata — into the two time functions the simulation core consumes during the powered ascent of [Section 2](#): thrust $F(t)$ in newtons and motor mass $m(t)$ in kilograms. Thrust enters the force model as a body- z force through the center of mass `model/RocketModel.cpp:71`; mass enters through the part tree, where a `part::Motor` leaf reports the burning motor’s mass into the composite walk of [Section 4](#) `model/parts/Motor.h:40`. Everything else here — piecewise-linear curves, a derived mass table, a keyed store, three ingest paths — exists to produce those two functions reliably from three interchange formats. [Figure 7](#) maps the classes.

What a hobby motor ships as data

A commercial model-rocket motor is a sealed solid-propellant cartridge, certified with a measured thrust-versus-time curve, a total impulse $I_{tot} = \int F dt$ in newton-seconds (N s), a propellant mass, and a loaded mass. The catalog grammar is compact: a code like “G80T” reads as impulse class G, average thrust roughly 80 N, propellant type letter T. Impulse classes are letters, each doubling the ceiling of the one before — A is at most 2.5 N s, B at most 5, up through M at 10240 N s — with fractional classes 1/4A and 1/2A below A. Motors also list *delay* options: seconds between burnout and the recovery-deploying ejection charge, with “plugged” (no charge) as a special case.

6.1 The runtime model: `MotorModel`

`model::MotorModel` `model/MotorModel.h:26` is a plain value type — copyable, movable, owned by value everywhere: in the database’s map, in the `part::Motor` tree leaf, in the loaders’ output vectors. It composes three pieces of state: a public `MetaData` record of catalog fields `model/MotorModel.h:285` ([Table 6](#); public with a standing `TODO` to privatize `model/MotorModel.h:282`), a `ThrustCurve` owned by value `model/MotorModel.h:343`, and a derived `massCurve` — a 128-point table of remaining propellant versus burn time `model/MotorModel.h:345`, recomputed whenever metadata is set ([Section 6.3](#)).

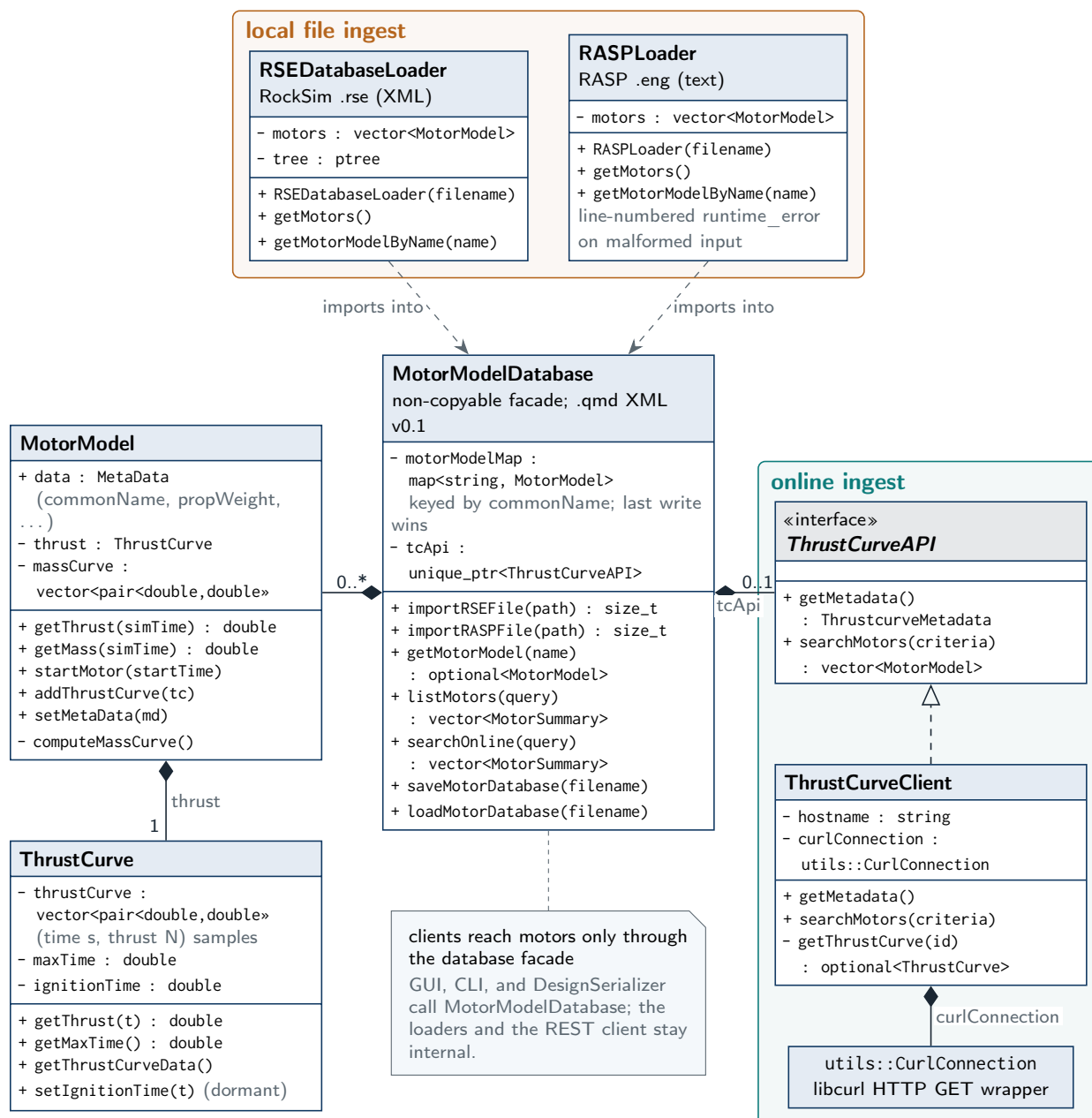


Figure 7. The motor subsystem. `MotorModelDatabase` is the single client-facing surface: a `std::map` of `MotorModel` values keyed by common name `model/MotorModelDatabase.h:127`. Two local file loaders — RockSim .rse and RASP .eng — are constructed stack-locally inside the import methods `model/MotorModelDatabase.cpp:66`, and the thrustcurve.org REST client sits behind the abstract `ThrustCurveAPI` seam `model/ThrustCurveClient.h:68`, injectable only through a private, friend-only constructor. Each `MotorModel` owns one `ThrustCurve` by value `model/MotorModel.h:343` and derives a 128-point propellant mass curve from it.

Field	Units	Meaning	
<code>commonName</code>	—	the database key; e.g. “A8”, “G80T”	model/MotorModel.h:301
<code>impulseClass</code>	—	motor letter: “1/4A”, “1/2A”, “A”...“M”	model/MotorModel.h:306
<code>diameter, length</code>	mm	casing geometry — deliberately <i>not</i> SI; hobby-catalog convention	model/MotorModel.h:305
<code>propWeight</code>	kg	propellant mass; “loaders convert; consumed as kg”	model/MotorModel.h:314
<code>totalWeight</code>	kg	loaded motor mass including casing	model/MotorModel.h:317
<code>totalImpulse</code>	N s	certified total impulse	model/MotorModel.h:316
<code>avgThrust, maxThrust</code>	N	catalog thrust summary values	model/MotorModel.h:298
<code>burnTime</code>	s	burn duration; gates the mass model	model/MotorModel.h:299
<code>delays</code>	s	ejection-delay options; 1000 is the plugged sentinel	model/MotorModel.h:303
<code>manufacturer, type, availability, certOrg</code>	—	enum wrappers, each with a <code>str()/toEnum()</code> string round trip	model/MotorModel.h:309
<code>motorIdTC</code>	—	24-character thrustcurve.org hex identifier	model/MotorModel.h:312

Table 6. Selected `MotorModel::MetaData` fields [model/MotorModel.h:285](#). The units column is the SI story in miniature: weights are kilograms by documented contract — every ingest path normalizes to kg at the boundary — while `diameter` and `length` stay in millimeters, converted to meters only where physics needs them [model/parts/Motor.cpp:27](#).

The unit discipline deserves one explicit sentence, because it is the one place QtRocket’s SI-throughout convention (Section 3) admits a deliberate exception. Weights are kilograms *at the MotorModel boundary* — each ingest path normalizes from its source’s native unit (Table 7) — while `diameter` and `length` stay in millimeters because every source catalog quotes them that way; the one consumer that needs meters, the `part::Motor` inertia tensor, converts at the point of use `model/parts/Motor.cpp:27`. A regression pins the kilogram contract: a 1/4A micro-rocket flown from an online-sourced database must lift off with an ignition mass under 0.05 kg, which the gram-as-kilogram failure mode (a 6.1 kg “motor”) cannot do `tests/DesignMatrixTests.cpp:269`.

6.2 ThrustCurve: piecewise-linear thrust

`ThrustCurve model/ThrustCurve.h:12` — the one motor type in the global namespace rather than `model::` — stores the samples as a `std::vector<std::pair<double, double>>` of (time s, thrust N) pairs `model/ThrustCurve.h:38` plus `maxTime`, the largest sample time `model/ThrustCurve.cpp:24`. A default-constructed curve is a single (0, 0) point, and an empty sample vector degenerates to the same, “so `getThrust()` always has an interval to walk” `model/ThrustCurve.cpp:19`.

`getThrust(t) model/ThrustCurve.cpp:47` first shifts into motor-relative time (`t -= ignitionTime`; `model/ThrustCurve.cpp:50` — always a zero shift in practice, Section 6.4), then clamps: outside `[0, maxTime]` thrust is identically zero `model/ThrustCurve.cpp:51`. No thrust before ignition, none after the last sample — after burnout the rocket coasts until the termination condition of Section 7. Inside the window, the code walks for the first sample strictly after `t`; an exact hit on a sample time returns that sample directly `model/ThrustCurve.cpp:61`, and running off the end returns the last sample’s thrust `model/ThrustCurve.cpp:67`. The interior case is linear interpolation, in the code’s own names:

$$\text{slope} = \frac{\text{thrustEnd} - \text{thrustStart}}{t\text{End} - t\text{Start}}, \quad F(t) = \text{thrustStart} + (t - t\text{Start}) \cdot \text{slope} \quad (12)$$

`model/ThrustCurve.cpp:85`. The subtlety is the interval’s *start* (Listing 11): normally the previous sample — but real catalog curves often begin at $t > 0$ rather than at the origin, and then there is no previous sample to read.

```
// Interval start is the previous sample -- unless t precedes the first sample (i == begin), where
// the curve has no (0,0) origin and we ramp from it. std::prev(begin()) here would read out of
// bounds.
double tStart = 0.0;
double thrustStart = 0.0;
if(i != thrustCurve.cbegin())
{
    tStart = std::prev(i)->first;
    thrustStart = std::prev(i)->second;
}
const double tEnd = i->first;
const double thrustEnd = i->second;
const double slope = (thrustEnd - thrustStart) / (tEnd - tStart);
return thrustStart + (t - tStart) * slope;
```

Listing 11. The implicit-origin ramp `model/ThrustCurve.cpp:73`. When the query time precedes the first measured sample, the curve interpolates from an implicit (0, 0) origin instead of dereferencing `std::prev(begin())`.

Why the origin ramp exists

Curves whose first sample sits at $t > 0$ used to send the interpolation walk through `std::prev(begin())` — an out-of-bounds heap read whose value depended on allocator state. The regression test preserves the symptom in its comment: “an order-dependent, non-reproducible apogee” `model/tests/ThrustCurveTests.cpp:45`, and pins the fix’s exact ramp — first sample at $t = 0.05$, oracles $F(0) = 0$, $F(0.025) = 5$, $F(0.05) = 10$ `model/tests/ThrustCurveTests.cpp:46`. **OK**

One level up, `MotorModel::getThrust(simTime)` adds the burnout latch: once `simTime` exceeds `thrust.getMaxTime() + ignitionTime` the motor returns zero forever, logging “motor burnout occurred” exactly once through the `burnOutOccurred` flag `model/MotorModel.cpp:72`. Otherwise it delegates the *absolute* simulation time straight to the curve `model/MotorModel.cpp:82` — correct only because ignition always happens at $t = 0$ (Section 6.4).

6.3 The mass model: constant- I_{sp} depletion

Why mass flow is proportional to thrust

A rocket motor produces thrust by expelling mass: $F = \dot{m} v_e$, where v_e is the effective exhaust velocity. If v_e is constant over the burn — the standard hobby-simulator assumption, equivalent to constant specific impulse $I_{sp} = v_e/g_0$ — the propellant depletion rate is proportional to instantaneous thrust: $\dot{m}_{prop}(t) = -F(t)/(g_0 I_{sp})$. Substituting the definition $I_{sp} = I_{tot}/(g_0 m_{prop,0})$, where $m_{prop,0}$ is the initial propellant mass, makes I_{sp} cancel entirely:

$$\dot{m}_{prop}(t) = -F(t) \frac{m_{prop,0}}{I_{tot}}. \quad (13)$$

The proportionality constant is exactly the one that makes the propellant integrate to $m_{prop,0}$ when the thrust integrates to I_{tot} : hard-burning moments consume propellant fast, the tail-off slowly, and the books balance at burnout by construction. What constant I_{sp} ignores — chamber-pressure variation, mass expelled by delay and ejection charges — this model ignores too.

QtRocket implements Eq. (13) as a precomputed table. `computeMassCurve()` `model/MotorModel.cpp:125` first splits the loaded motor into casing and propellant, `emptyMass = totalWeight - propWeight` `model/MotorModel.cpp:127`, computes `isp = totalImpulse/(g_0 * propWeight)` with $g_0 = 9.80665 \text{ m/s}^2$ `model/MotorModel.cpp:130` `utils/math/Constants.h:10` — a member that, as Eq. (13) predicts, cancels out of the depletion arithmetic and is in fact never read anywhere (Section 13) — then builds the 128-point table (Listing 12). With $\Delta t = \text{burnTime}/127$ `model/MotorModel.cpp:135` and `propMass` seeded to `propWeight` `model/MotorModel.cpp:137`, the loop is a left Riemann sum over Eq. (13):

$$\text{propMass}_{i+1} = \text{propMass}_i - F(t_i) \Delta t \frac{\text{propWeight}}{\text{totalImpulse}}, \quad t_i = i \Delta t \quad (14)$$

`model/MotorModel.cpp:141`. The 128th point is then forced to exact depletion: `massCurve.push_back(std::make_pair(data.burnTime, 0.0));` `model/MotorModel.cpp:143`. Pinning the endpoint zeroes out whatever residue the Riemann sum accumulated, so the tabulated propellant mass is first-order-approximate between samples but *exact* at ignition and burnout — post-burn mass is exactly `emptyMass`, bit for bit, which is what lets the part tree’s composite cache freeze stably after burnout (Section 4).

```

// Precompute the mass curve as a lookup table: keeps repeated getMass()/getThrust() calls within
// one time step consistent, and speeds up the sim. 128 sample points (RASP files cap at 32).
massCurve.reserve(128);
double timeStep = data.burnTime / 127.0;
double t = 0.0;
double propMass{data.propWeight};
for(std::size_t i = 0; i < 127; ++i)
{
    massCurve.push_back(std::make_pair(t + i*timeStep, propMass));
    propMass -= thrust.getThrust(t + i*timeStep) * timeStep * data.propWeight / data.totalImpulse;
}
massCurve.push_back(std::make_pair(data.burnTime, 0.0));

```

Listing 12. The 128-point depletion table `model/MotorModel.cpp:132`: a left Riemann sum of the constant- I_{sp} law with the final point pinned to exact depletion. Note what is absent — the vector is reserved but never cleared.

`getMass(simTime)` serves three regimes — “empty mass + remaining propellant mass” `model/MotorModel.cpp:30`. Before ignition: `totalWeight`, the loaded motor `model/MotorModel.cpp:32`. During the burn (`simTime - ignitionTime ≤ burnTime` `model/MotorModel.cpp:36`): walk the table — an entry matching within 4 units in the last place (`utils::math::floatingPointEqual` `utils/math/UtilityMathFunctions.h:21`) returns directly `model/MotorModel.cpp:43`, otherwise the bracketing interval interpolates exactly as the thrust curve does `model/MotorModel.cpp:52`. After: `emptyMass` `model/MotorModel.cpp:65`. The walk carries no end-iterator guard `model/MotorModel.cpp:41`; it is safe because the branch guard caps `thrustTime` at `burnTime` and the table’s last entry is the literal pair (`burnTime, 0.0`), so at the boundary the exact-hit test fires on the final element before the iterator can run off the end.

One asymmetry deserves plain statement: `getMass` gates the burn on the metadata’s `burnTime` while `getThrust` gates on the curve’s own `maxTime` `model/MotorModel.cpp:36` `model/MotorModel.cpp:72` — two clocks that agree only if the metadata is honest. RASP guarantees agreement by construction, deriving `burnTime` from the last sample time `model/RASPLoader.cpp:210`; RockSim reads `burn-time` as an independent file attribute `model/RSEDatabaseLoader.cpp:54`, so a malformed file could make thrust and mass disagree about when the burn ends.

Curve before metadata

`setMetaData()/moveMetaData()` trigger `computeMassCurve()` `model/MotorModel.cpp:85`, and the mass curve integrates the thrust curve — so the curve must be attached first, or the mass table silently integrates the default (0,0) curve and the motor holds its full propellant mass until the pinned burnout point. The rule is obeyed at every construction site — both file loaders `model/RSEDatabaseLoader.cpp:96` `model/RASPLoader.cpp:242`, the REST client `model/ThrustCurveClient.cpp:328`, and the XML database loader: “Order matters: `setMetaData()` triggers `computeMassCurve()`, which integrates the thrust curve, so the curve must be in place first” `model/MotorModelDatabase.cpp:303`. It is enforced by convention and comment, not by the type system.

A latent trap: `massCurve` is never cleared

`computeMassCurve()` calls `massCurve.reserve(128)` and appends `model/MotorModel.cpp:134`; it never clears. Calling `setMetaData()` twice on the same instance would leave the stale 128 points

in front of the new ones, and `getMass()`'s front-to-back walk would read the stale curve. No current call path does this — every loader builds a fresh instance, and `part::Motor::setMotorModel` replaces the whole owned object `model/parts/Motor.cpp:18` — but a future metadata-editing feature that mutates a live `MotorModel` in place would trip it.

6.4 Ignition

Ignition is a boolean latch, set exactly once per flight. The only production call site of `startMotor` is `RocketModel::launch(): motorPart->getMotorModel().startMotor(0.0); model/RocketModel.cpp:108`, invoked by `QtRocket::launchRocket()` immediately before the propagation loop `QtRocket.cpp:54`. `startMotor` sets `ignitionOccurred` and records `ignitionTime` `model/MotorModel.h:328`; the member's declaration warns that `ignitionTime` is read by the burnout test even before ignition, so garbage there would make “the flight depend on stack/heap layout” `model/MotorModel.h:341` — it is zero-initialized in the class definition. From ignition onward, thrust flows to `RocketModel::getForces` `model/RocketModel.cpp:71` and mass flows through the `part::Motor` leaf into the composite walk `model/RocketModel.cpp:36` (Section 4; Section 7 covers the loop that queries both).

The ignition-time seam is half-wired

`startMotor(startTime)` accepts any ignition time, and both the mass lookup and the burnout latch honor it `model/MotorModel.cpp:36` `model/MotorModel.cpp:72`. But `MotorModel` never forwards the ignition time to its curve: `ThrustCurve::setIgnitionTime` has zero call sites in the entire tree `model/ThrustCurve.h:31`, its body carries a commented-out `//maxTime += ignitionTime;` `model/ThrustCurve.cpp:44`, and `getThrust` passes *absolute* simulation time to the curve `model/MotorModel.cpp:82`, whose own `ignitionTime` is permanently zero. Consequence: a hypothetical `startMotor(t0)` with $t_0 \neq 0$ — an air-start or staged ignition — would shift the mass curve and the burnout window by t_0 while thrust samples were still looked up at absolute time. Thrust and mass would disagree for the whole burn. The hazard is latent solely because the single production caller passes 0.0 `model/RocketModel.cpp:108`; a naive future consumer of the dormant plumbing inherits it intact. **FAIL**

Two neighboring fields share this dormant status, stated once here and inventoried in Section 13. The `delays` vector is parsed by both file ingest paths and persisted through the XML round trip, but nothing in `sim/`, `cli/`, or `gui/` consumes it — there is no ejection-charge or recovery-deployment model, so the plugged sentinel 1000 `model/MotorModel.h:303` guards behavior that does not yet exist. And `isp` is computed on every metadata set `model/MotorModel.cpp:130` and read by nothing; the depletion formula embeds it implicitly (Eq. (13)).

6.5 MotorModelDatabase: one keyed surface

`model::MotorModelDatabase` `model/MotorModelDatabase.h:68` is the subsystem's single client-facing surface: a non-copyable, non-movable `model/MotorModelDatabase.h:75` store owned as a `shared_ptr` by the `QtRocket` controller `QtRocket.cpp:51`. Its heart is disarmingly small — `std::map<std::string, MotorModel>` keyed by common name, the in-tree comment admitting “The ‘database’ is really just a map. :)” `model/MotorModelDatabase.h:127`.

Clients reach motors only through the database

The loaders are constructed stack-locally inside `importRSEFile/importRASFile` — “owned here, never seen by GUI/CLI” `model/MotorModelDatabase.cpp:63` — and the REST client hides behind the private `ThrustCurveAPI` injection constructor `model/MotorModelDatabase.h:114`. Every motor, whatever its source, funnels into the same keyed map with the same query model, and the front ends prove the rule: all GUI and CLI motor traffic goes through database methods (`gui/CannonballTab.cpp:137`, `gui/ThrustCurveMotorSelector.cpp:52`, `cli/Repl.cpp:315`), and a saved design references its motor by common name alone, re-resolved against whatever database the user has loaded `model/DesignSerializer.cpp:271` (Section 11). One practical payoff: the loaders’ miss semantics — a default-constructed, zero-thrust, zero-mass `MotorModel` `model/RSEDatabaseLoader.cpp:44` — never reach application code, which sees the database’s `std::optional` instead `model/MotorModelDatabase.cpp:84`.

Keying and collisions. `addMotorModel` keys on `data.commonName` and assigns unconditionally: `motorModelMap[name] = m; model/MotorModelDatabase.cpp:50` — last write wins, replace-versus-add distinguished only in the logs `model/MotorModelDatabase.cpp:44`. The import methods report *net new* entries — the map’s size delta — so “same-name motors collapse onto one entry, and a re-import correctly reports 0” `model/MotorModelDatabase.cpp:68`. The bundled `data/Aerotech.rse` makes it concrete: 252 `<engine>` elements, but three codes (G77R, G79W, I65W) appear twice each, so a first import reports 249 and a second reports 0 — the idempotent re-import pinned by test `tests/MotorDatabasePersistenceTests.cpp:348`.

Queries. `listMotors` takes a `MotorQuery` whose fields are all `std::optional` — unset does not constrain, so a default query matches everything `model/MotorModelDatabase.h:33`. The filters: exact match on manufacturer and impulse class `model/MotorModelDatabase.cpp:102`, case-sensitive substring on the common name `model/MotorModelDatabase.cpp:110`, and a diameter tolerance of ± 0.5 mm — motor diameters are nominal integer millimeters at least 1 mm apart, so the tolerance distinguishes sizes while forgiving doubles round-tripped through XML `model/MotorModelDatabase.cpp:106`; the boundary is pinned at 29.49 matching a 29 mm motor and 29.51 not `tests/MotorDatabasePersistenceTests.cpp:445`. Results are `MotorSummary` rows — a deliberately thin type that lets a UI populate a list without holding full, possibly not-yet-downloaded thrust curves; callers fetch the full model on selection `model/MotorModelDatabase.h:41`. Sorting comes free: `std::map` iterates in key order `model/MotorModelDatabase.cpp:116`.

Persistence: the .qmd format. `saveMotorDatabase` writes a Boost property-tree XML document rooted at `QtRocketMotorDatabase` with `version="0.1"` `model/MotorModelDatabase.cpp:184`: one `<motor>` element per map entry with twenty-two child fields — enums serialized through their `str()` wrappers `model/MotorModelDatabase.cpp:190`, delays as a CSV string whose writing loop is index-guarded because “the old `size()-1` form underflowed” on the empty delay vectors online-sourced motors carry `model/MotorModelDatabase.cpp:211`, and the thrust curve as `<thrust time=".." force=".."/>` children `model/MotorModelDatabase.cpp:222`. Numerics round-trip exactly because `property_tree` writes `max_digits10` `tests/MotorDatabasePersistenceTests.cpp:315`. `loadMotorDatabase` mirrors it: skip any node that is not `motor` `model/MotorModelDatabase.cpp:251`, default every missing field `model/MotorModelDatabase.cpp:255`, re-parse delays skipping empty tokens (so “4,,7,” yields {4, 7} `model/MotorModelDatabase.cpp:281`), rebuild curve-first `model/MotorModelDatabase.cpp:302`, and merge through `addMotorModel` `model/MotorModelDatabase.cpp:307` — a load is an import, with the same last-write-wins

semantics. The full round trip is pinned end to end: import the real 249-motor Aerotech file, save, reload, compare field by field `tests/MotorDatabasePersistenceTests.cpp:297`. [Listing 13](#) shows the format on a committed fixture.

```
<QtRocketMotorDatabase version="0.1">
  <MotorModels>
    <motor name="1/2A3">
      <availability>regular</availability>
      <avgThrust>3.0299999999999998</avgThrust>
      <burnTime>0.3599999999999999</burnTime>
      <certOrg>Uncertified</certOrg>
      <commonName>1/2A3</commonName>
      <designation>1/2A3</designation>
      <diameter>13</diameter>
      <impulseClass>A</impulseClass>
```

Listing 13. Opening of the committed `.qmd` fixture `tests/data/motors/estes_small.qmd:2`. Enums travel as their `str()` strings, numerics at `max_digits10`. Note the 1/2A3 motor carrying `<impulseClass>A</impulseClass>` — the server-verbatim class discussed in [Section 6.7](#).

The typos are load-bearing

`CertOrg::str()` emits “Austrialian Model Rocket Society Inc.” `model/MotorModel.h:128` and “Unkown” `model/MotorModel.h:138` — misspelled, and deliberately preserved. The `.qmd` format’s losslessness rests on every enum wrapper’s `str()`→`toEnum()` round trip, so `toEnum` explicitly accepts the typo’d full names alongside the short codes: “The ‘Austrialian’/‘Unkown’ spellings match `str()`’s typo’d output” `model/MotorModel.h:144`. Correcting the spelling without a migration would orphan the AMRS values in every existing saved database; the round-trip test — every wrapper, every enum value — would catch the break `tests/MotorDatabasePersistenceTests.cpp:245`. Format-compatibility strings are part of the interface, typos included.

6.6 Three ingest paths

Three paths feed the map, each translating one interchange format to the `MetaData` contract of [Table 6](#). They differ sharply in error posture — [Table 7](#) contrasts them — but converge on the same assembly ritual: build the thrust curve, attach it, then move the metadata in.

6.6.1 RockSim `.rse`: lenient XML

The `.rse` format is the XML engine-list format of the commercial RockSim simulator; QtRocket’s only bundled motor data, `data/Aerotech.rse`, uses it. The loader parses eagerly in its constructor via `boost::property_tree::read_xml` `model/RSEDatabaseLoader.cpp:24` and iterates every child of `engine-database.engine-list` without a key filter `model/RSEDatabaseLoader.cpp:27`; the format is assumed well-formed, and parse failures propagate as exceptions `model/MotorModelDatabase.h:84`. Metadata comes from XML attributes with defaults: `code` becomes the common name `model/RSEDatabaseLoader.cpp:56`, `initWt/propWt` divide by 1000 `model/RSEDatabaseLoader.cpp:81`, delays parse from a CSV string defaulting to the plugged sentinel `"1000"` `model/RSEDatabaseLoader.cpp:59`. One exception to leniency is accidental: `Type` is read with no default, so a file lacking that attribute throws from inside `property_tree` `model/RSEDatabaseLoader.cpp:85`.

	RockSim .rse	RASP .eng	thrustcurve.org REST
Parser	RSEDatabaseLoader, Boost property_tree XML model/ RSEDatabaseLoader.cpp:24	RASPLoader, hand-rolled line parser model/ RASPLoader.cpp:253	ThrustCurveClient, jsoncpp over utils::CurlConnection model/ ThrustCurveClient.h:99
Error posture	lenient: nearly every attribute defaults model/ RSEDatabaseLoader.cpp:53; Type is the one non-defaulted metadata read	strict: typed, line-numbered std::runtime_error on any violation model/ RASPLoader.cpp:65	tolerant: malformed JSON → nullopt + log; server error field → empty-but-valid model/ ThrustCurveClient.cpp:37
Weights in source	grams, ÷1000 model/ RSEDatabaseLoader.cpp:81	kilograms, as-is model/ RASPLoader.cpp:237	grams, ÷1000 model/ ThrustCurveClient.cpp:208
Impulse class	derived from the motor code model/ RSEDatabaseLoader.cpp:73	derived from the motor code model/ RASPLoader.cpp:233	server field, verbatim model/ ThrustCurveClient.cpp:201
burnTime	file attribute model/ RSEDatabaseLoader.cpp:54	last sample time model/ RASPLoader.cpp:210	server field model/ ThrustCurveClient.cpp:196
Data discarded	per-sample mass/CG, comments model/ RSEDatabaseLoader.cpp:91	—	all simfiles beyond the chosen one model/ ThrustCurveClient.cpp:67

Table 7. The three ingest paths behind the database facade. All converge on kilogram weights, millimeter dimensions, and the curve-before-metadata assembly order; they diverge in how much they trust their input.

What the loader *discards* matters as much as what it keeps. Each `<eng-data>` sample in an `.rse` file carries four attributes — time, force, and measured remaining mass and center-of-gravity (`m=`, `cg=`) — but only `t` and `f` are read `model/RSEDatabaseLoader.cpp:91`. QtRocket deliberately reconstructs motor mass from thrust via the constant- I_{sp} model of [Section 6.3](#) rather than consuming the file’s measured mass samples, and it models the motor’s own center of mass as fixed (the `part::Motor` leaf’s offset is `Vector3::Zero()` `model/parts/Motor.cpp:12`): propellant burn moves the *composite* center of mass only through motor mass loss, not through intra-motor CG travel. A modeling simplification, chosen with the data’s existence in plain view.

6.6.2 RASP .eng: strict text

RASP `.eng` is the line-oriented text format of the hobby’s oldest simulator lineage, and the loader treats it with the opposite posture: every violation is a typed, located error — lines are numbered from 1 as read `model/RASPLoader.cpp:262`, and every failure throws `std::runtime_error("RASP parse error on line N: message")` `model/RASPLoader.cpp:65`. The header must contain exactly seven whitespace-separated fields — code, diameter (mm), length (mm), delays, propellant weight (kg), total weight (kg), manufacturer `model/RASPLoader.cpp:145` — with positivity and `totalWeight ≥ propWeight` enforced `model/RASPLoader.cpp:157`. Delays are dash-separated (4-7); each token must be a non-negative integer or the case-insensitive `P/NONE`, which map to the plugged sentinel 1000 `model/RASPLoader.cpp:98`. Sample lines must hold exactly two finite numbers `model/RASPLoader.cpp:171`, times strictly increasing, thrust non-negative `model/RASPLoader.cpp:303`. A zero-thrust sample marks the end of a motor — a sample after it is the error “zero-thrust sample must be the final data point” `model/RASPLoader.cpp:292`, a motor that never reaches zero is “motor thrust data must end with a zero-thrust sample” `model/RASPLoader.cpp:315` — and a non-sample line after the final zero starts the next motor in a multi-motor file `model/RASPLoader.cpp:289`.

Because a RASP file carries no summary metadata beyond the header, the loader derives what the other formats read. After prepending the implicit origin if the first sample sits at $t > 0$ `model/RASPLoader.cpp:207` — the parse-time counterpart of [Listing 11](#)’s runtime ramp:

$$\text{burnTime} = t_{\text{last}}, \quad \text{totalImpulse} = \sum_{i \geq 1} \frac{1}{2} (F_{i-1} + F_i) (t_i - t_{i-1}), \quad \text{avgThrust} = \frac{\text{totalImpulse}}{\text{burnTime}} \quad (15)$$

— trapezoid integration of the samples `model/RASPLoader.cpp:189`, with `maxThrust` the sample maximum `model/RASPLoader.cpp:217` and both derived quantities checked positive `model/RASPLoader.cpp:212`. A spec-style Estes D12 test pins the pipeline end to end `model/tests/RASPLoaderTests.cpp:45`. No `.eng` file ships in `data/`; RASP coverage runs entirely on inline fixtures.

6.6.3 thrustcurve.org: REST behind a seam

The third path is the community motor-data service `thrustcurve.org`, reached over its REST (representational state transfer) API from base URL `https://www.thrustcurve.org/` `model/ThrustCurveClient.cpp:235` via the libcurl wrapper `utils::CurlConnection`, whose `get()` returns the body on success and an empty string on any failure — transport error, timeout, or non-2xx status `utils/CurlConnection.h`. Three endpoints are consumed. `GET api/v1/metadata.json` `model/ThrustCurveClient.cpp:268` yields the filterable vocabulary. `GET api/v1/search.json` `model/ThrustCurveClient.cpp:284` turns each criteria pair into a query parameter and, absent a caller-supplied limit, appends `maxResults=100`: the server default is 20, and the comment explains the ceiling — each result costs one `download.json` round trip, performed synchronously on the caller’s (often GUI) thread `model/ThrustCurveClient.cpp:292`. The search parser maps each result into a `MetaData`: gram

fields divided by 1000 `model/ThrustCurveClient.cpp:208`, the server’s `motorId` kept as `motorIdTC` `model/ThrustCurveClient.cpp:206`, the `impulseClass` field taken verbatim `model/ThrustCurveClient.cpp:201`, and `certOrg` and `delays` left at defaults under explicit TODOs `model/ThrustCurveClient.cpp:197` — the empty delay vectors those TODOs produce are what the `.qmd` CSV underflow guard exists for.

Finally, `GET api/v1/download.json?motorId=<id>&data=samples` `model/ThrustCurveClient.cpp:248` fetches the curve itself. A motor typically has several “simfiles” — alternative measured descriptions of the same burn, in arbitrary order — and the parser selects exactly one: the first RASP-format entry with samples wins immediately, else the first non-empty entry of any format is the fallback `model/ThrustCurveClient.cpp:84`. The in-code rationale is blunt: “their sample sets are alternative descriptions of the same burn, so we must pick exactly one... Concatenating them would interleave time series” `model/ThrustCurveClient.cpp:67`. A motor whose download produced no usable samples is still added — it keeps the default (0, 0) curve `model/ThrustCurveClient.cpp:324`. `searchMotors` assembles each full `MotorModel` curve-first `model/ThrustCurveClient.cpp:328` and warns when the server’s `matches` count exceeds the returned set — the truncation signal `model/ThrustCurveClient.cpp:312`.

The database wraps this behind two methods. `getOnlineSearchFacets()` maps the metadata to filter values, keeping only manufacturer *codes* because the search API keys on the code `model/MotorModelDatabase.cpp:151`. `searchOnline(q)` translates the source-agnostic `MotorQuery` into raw `SearchCriteria` `model/MotorModelDatabase.cpp:159` — diameter formatted with `std::format` so 29.0 travels as “29” `tests/MotorDatabasePersistenceTests.cpp:559`, and `nameContains` deliberately omitted, documented as ignored for online search `model/MotorModelDatabase.h:104` — then merges every returned motor into the map via `addMotorModel` so subsequent *local* queries see it `model/MotorModelDatabase.cpp:171`. An online search is an import.

The seam that makes this testable is `ThrustCurveAPI` `model/ThrustCurveClient.h:68`: a two-method pure-virtual interface that exists, per its own header, so the online wrappers can be tested with a fake source — “the injection point is private/friend-only in `MotorModelDatabase`, so application code never uses this directly” `model/ThrustCurveClient.h:63`. The public default constructor delegates with a null API `model/MotorModelDatabase.cpp:24`, the production `ThrustCurveClient` is constructed lazily on first online use `model/MotorModelDatabase.cpp:137`, and the befriended `MotorModelDatabaseTestAccess` lets the integration tests inject a `FakeThrustCurveAPI` returning canned motors `tests/MotorDatabasePersistenceTests.cpp:106`. The three JSON parsers are split out as free functions precisely so canned response bodies can exercise them with no HTTP at all `model/ThrustCurveClient.h:88` — [Section 12](#) returns to both layers of this arrangement.

6.7 Impulse-class derivation and its one seam

The impulse-class string is search vocabulary — `listMotors` filters on it exactly `model/MotorModelDatabase.cpp:104` — and neither local file format carries it explicitly, so QtRocket derives it from the motor code. `MetaData::deriveImpulseClass` `model/MotorModel.cpp:97` skips leading whitespace, uppercases, checks the two fractional prefixes first — `starts_with("1/4A")` and `starts_with("1/2A")` `model/MotorModel.cpp:112` — and otherwise returns the first alphabetic character (“G80T” → “G”), or the empty string if none exists `model/MotorModel.cpp:117`. The RockSim and RASP paths share this one parser `model/RSEDatabaseLoader.cpp:73` `model/RASPLoader.cpp:233`, and tests pin the fractional cases through both `model/tests/RASPLoaderTests.cpp:102` `model/tests/RASPLoaderTests.cpp:174`.

Fractional classes are source-dependent

The `thrustcurve.org` path does not derive; it stores the server's `impulseClass` field verbatim `model/ThrustCurveClient.cpp:201` — and the committed fixture, captured from the real API, shows a 1/2A3 motor carrying `<impulseClass>A</impulseClass>` `tests/data/motors/estes_small.qmd:12` (Listing 13): the server reports the fractional-class motor under its parent letter. The same physical motor therefore lands in the database with class "1/2A" when imported from an `.rse` or `.eng` file and class "A" when fetched online, and a `listMotors` filter on `impulseClass = "1/2A"` `model/MotorModelDatabase.cpp:104` matches the former and misses the latter. The discrepancy is data-visible today in any mixed-source database. **FAIL**

That seam aside, the subsystem delivers what the rest of the document consumes: a keyed catalog with one query model, and per-motor $F(t)$ and $m(t)$ functions whose books balance exactly at ignition and at burnout. Section 4 showed how the mass function enters the composite tree through the `part::Motor` leaf; Section 7 follows both functions into the propagation loop, where thrust is sampled every integrator stage and mass gates the composite-property cache; Section 11 covers the design file that names a motor and expects this database to resolve it; and the dormant inventory — non-zero ignition, delays, `isp`, the unread cert-org and type facets — is consolidated with everything else in Section 13.

7 The simulation core

The preceding sections describe the rocket as an object: a part tree with mass properties (Sections 4 and 5) and a motor that pushes it (Section 6). This section describes the machine that flies it. The simulation core — the `sim/` layer — turns a `model::Propagatable` (in production, `RocketModel`) into a time history of flight states. `sim::Propagator` owns the ordinary-differential-equation (ODE) loop: it builds the coupled first-order system $\dot{x} = v$, $\dot{v} = F(t, x, v)/m(t)$ as a lambda `sim/Propagator.cpp:36`, hands it to a `sim::Integrator` — a string-keyed runtime selector over two `DESolver<Vector3>` backends, a fixed-step classical Runge–Kutta 4 and an adaptive Runge–Kutta–Fehlberg 4(5) embedded pair `sim/Integrator.h:25` — and drives `runUntilTerminate()` until the model reports end of flight or a safety guard converts a would-be infinite loop into a typed verdict `sim/Propagator.cpp:58`.

Two facts frame everything that follows. First, only the three linear degrees of freedom (DOF) are integrated: the moving state is center-of-mass position and velocity, nothing else. `StateData` reserves quaternion orientation slots, but no code writes them, and the orientation integrator exists only as a commented-out member `sim/Propagator.h:103` — Section 7.2 states this plainly and Section 13 carries the consolidated inventory. Second, every run ends with a reason: the loop cannot exit without assigning one of five `TerminationReason` enumerators `sim/Propagator.h:35`, and the whole loop body is wrapped in a try/catch so even a throwing integrator lands on a verdict rather than escaping uncaught `sim/Propagator.cpp:81`.

7.1 The pieces and their contracts

Figure 8 shows the static structure. Five types carry the design; each has a contract narrow enough to state in a sentence or two.

`model::Propagatable`. The `model↔sim` bridge (Section 3) is the only view of the rocket the loop ever sees `model/Propagatable.h:23`. Its pure-virtual heart is `getForces(t, position, velocity, environment)` `model/Propagatable.h:29`, `getMass(t)` `model/Propagatable.h:32`, `writeMassProperties(t,`

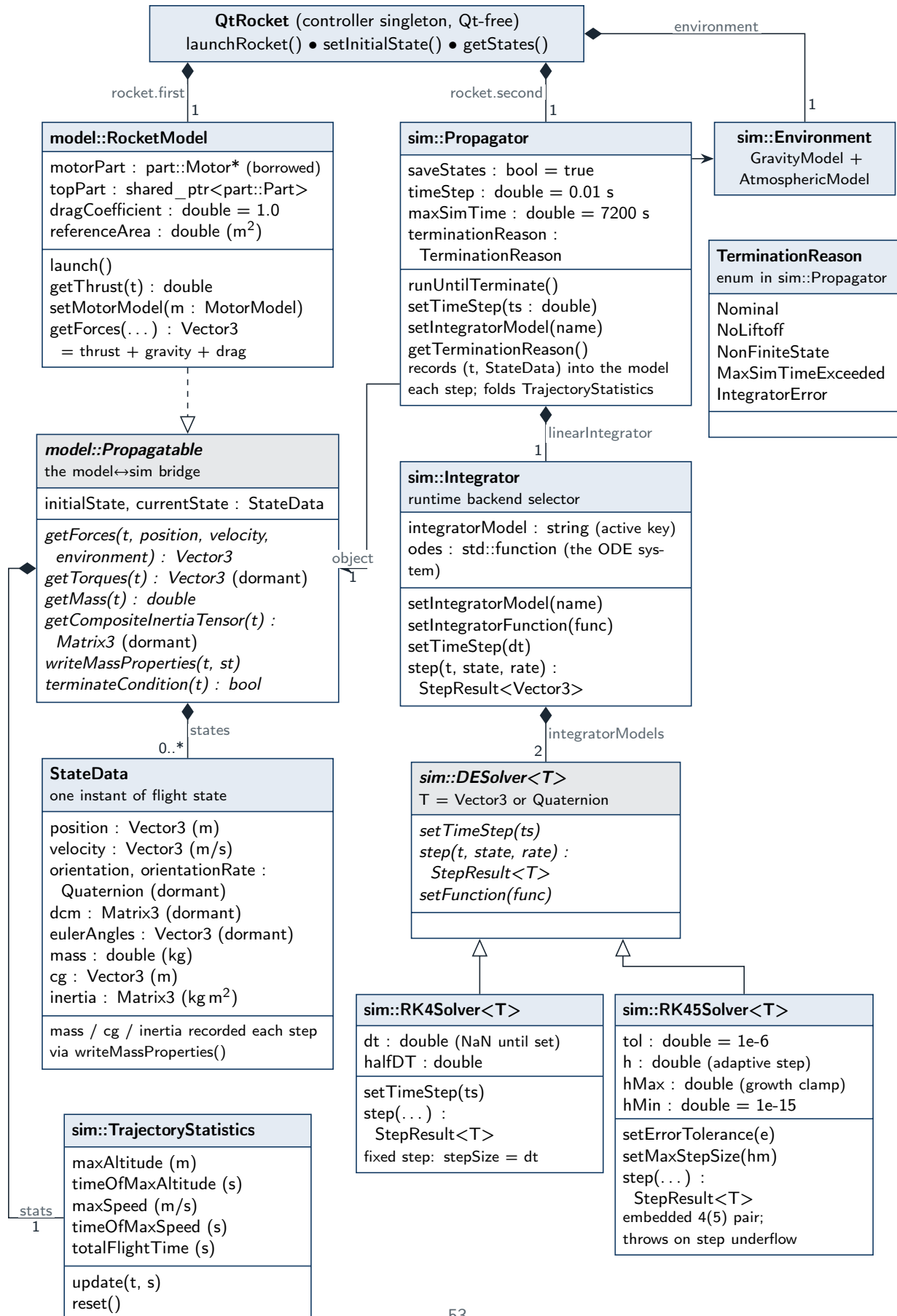


Figure 8. Static structure of the simulation core. `sim::Propagator` drives a `model::Propagatable` (implemented by `RocketModel`) and owns one `sim::Integrator` `sim/Propagator.h:100`, which in turn owns the two `DESolver<Vector3>` backends in a string-keyed map and holds the ODE callback `sim/Integrator.h:86`.

st) `model/Propagatable.h:37`, and `terminateCondition(t)` `model/Propagatable.h:39` — the model, not the loop, decides when a flight is over. Two more virtuals exist without a consumer: `getTorques(t)` is implemented as a zero vector and never called `model/RocketModel.cpp:94` — the torque half of the bridge waits for the aerodynamic moments of [Section 9](#) — and `getCompositeInertiaTensor(t)` `model/Propagatable.h:33` has no production caller, because per-step inertia recording flows through `writeMassProperties` instead ([Section 13](#)). The interface also carries the non-virtual plumbing the loop relies on: state accessors, the recorded history as (time, `StateData`) pairs `model/Propagatable.h:49`, and the statistics fold `model/Propagatable.h:55`.

sim::Propagator. The run-loop owner. It shares ownership of the flown body and the injected Environment `sim/Propagator.h:105` and owns exactly one Integrator `sim/Propagator.h:100`. Its knobs are few: `timeStep` defaults to 0.01 s `sim/Propagator.h:110`, and `setTimeStep` rejects any `ts` failing `ts > 0.0` (which also catches NaN) with a logged warning, keeping the previous value `sim/Propagator.h:73`, then pushes the accepted step into the integrator so the loop's time axis and the solver's step cannot diverge `sim/Propagator.h:82`. `setMaxSimTime` bounds a run at 7200 s by default `sim/Propagator.h:115`; the no-liftoff thresholds (3 s, 1 m) and the 10^8 -iteration ceiling are namespace-scope constants `sim/Propagator.h:26`.

sim::Integrator. A runtime selector, non-copyable and non-movable `sim/Integrator.h:38`. Its constructor seeds a map with exactly two keys — "Runge-Kutta 4th Order" and "Runge-Kutta-Fehlberg" — and immediately selects RK4 as the default `sim/Integrator.h:32`. `setIntegratorModel` constructs a fresh solver for a recognized name from the stored ODE callback `sim/Integrator.h:57`; an unknown name is a logged no-op that keeps the current model `sim/Integrator.h:64`, pinned by a unit test `sim/tests/IntegratorTests.cpp:32`. `step` forwards to the active solver `sim/Integrator.h:82`. Front ends never hard-code the model list: they enumerate `getAvailableIntegratorModels()` `sim/Integrator.h:43` — the GUI fills its combo box from a throwaway Integrator `gui/SimOptionsTab.cpp:42` — and selection flows front end → QtRocket `QtRocket.h:36` → Propagator → Integrator ([Section 10](#)).

sim::DESolver<T> and StepResult<T>. The abstract solver interface `sim/DESolver.h:29`: `setTimeStep` (for a fixed-step solver, the step; for an adaptive one, the seed guess `sim/DESolver.h:36`), `setFunction`, and `step(t, state, rate)`, whose stages evaluate at their node times $t + c_i h$ `sim/DESolver.h:39`. `T` is restricted to `Vector3` or `Quaternion` by a `static_assert` in both concrete solvers `sim/RK4Solver.h:34`, `sim/RK45Solver.h:51`; only `<Vector3>` is ever instantiated in production. Every step returns a `StepResult<T>`: the advanced state, the advanced rate, and `stepSize`, the step actually taken `sim/DESolver.h:21`.

Advance by the step actually taken

The interface contract says it outright: callers advance their clock by `StepResult::stepSize`, never by an assumed `dt` `sim/DESolver.h:18`. The run loop honors it — `currentTime += result.␣stepSize` is its only clock advance `sim/Propagator.cpp:166` — as does the solver test helper `sim/tests/RK45SolverTests.cpp:32`. This one rule is what makes the backends interchangeable: under RK4 the reported step is always `dt`, under RK45 it is whatever the controller accepted, and the loop's time axis follows the solver instead of presuming it.

StateData and sim::TrajectoryStatistics. `StateData` (global namespace `sim/StateData.h:15`) is one instant of physical state: all-public value semantics, copy/move explicitly defaulted — member-wise, so a new member cannot be silently dropped `sim/StateData.h:24`. Its live fields are world-frame position (m) and velocity (m/s) `sim/StateData.h:42`; its recorded fields are the composite mass,

cg, and full 3×3 inertia tensor written once per retained step `sim/StateData.h:60`; its dormant fields are the orientation slots taken up in [Section 7.2](#). `TrajectoryStatistics` is a plain struct of five doubles — `maxAltitude`, `timeOfMaxAltitude`, `maxSpeed`, `timeOfMaxSpeed`, `totalFlightTime` `sim/TrajectoryStatistics.h:18` — folded up every step and examined in [Section 7.7](#).

7.2 The equations of motion, as coded

The integrated physics is one lambda, built in the `Propagator` constructor and closing over the model and the injected environment ([Listing 14](#)). The comment above it states the design intent: linear ODEs, with t passed through so `getForces()` can evaluate a time-dependent forcing function — the motor thrust curve — at the actual simulation time `sim/Propagator.cpp:34`.

```
std::function<std::pair<Vector3, Vector3>(double, Vector3&, Vector3&)> linearODEs = [this](double t,
↳ Vector3& state, Vector3& rate) -> std::pair<Vector3, Vector3>
{
    Vector3 dPosition;
    Vector3 dVelocity;
    dPosition = rate;
    dVelocity = object->getForces(t, state, rate, *environment) / object->getMass(t);

    return std::make_pair(dPosition, dVelocity);
};
```

Listing 14. The integrated physics in its entirety `sim/Propagator.cpp:36`: one lambda closing over the model and the environment. Everything the solvers ever learn about rockets passes through this pair of derivatives.

In the code’s own names, with `state` the center-of-mass position (m) and `rate` its velocity (m/s):

$$dPosition = rate, \quad dVelocity = \frac{getForces(t, state, rate, environment)}{getMass(t)}. \quad (16)$$

This is Newton’s second law with time-varying mass, reduced from one second-order equation to a coupled first-order pair: $\dot{x} = v$, $\dot{v} = F(t, x, v)/m(t)$. There is no separate $\dot{m} v_e$ rocket-equation term in the simulation core: thrust arrives already as a force from the motor’s measured thrust curve ([Section 6](#)), so the momentum flux is embedded in the data, and mass loss enters only through the divisor and the gravity term. The force side assembles three contributions per evaluation — thrust along $+z$ through the center of mass `model/RocketModel.cpp:71`, gravity evaluated at the integrator’s *trial* position so each stage sees a consistent state `model/RocketModel.cpp:73`, and quadratic drag $-\frac{1}{2} \rho(alt) |v| C_d A v$ `model/RocketModel.cpp:88` — with the gravity and atmosphere models behind them the subject of [Section 8](#) and the drag law itself of [Section 9](#).

Mass is sampled *twice* per derivative evaluation: once in the lambda as the divisor `sim/Propagator.cpp:41`, and once more inside `getForces` itself, where the gravity force is `gravityModel->getAccel(position)*getMass(t)` `model/RocketModel.cpp:77`. Both samples carry the same stage node time, so thrust and burning mass stay mutually consistent within a step; under RK4 that is four evaluations — eight mass reads — per step, under RKF45 six evaluations per attempt. The composite accessors behind `getMass` are gated ([Section 4](#)), so post-burnout reads hit a frozen cache.

The solvers do not see a six-component state vector. Each backend advances *two* T-valued variables — `state` and `rate` — whose derivatives arrive from one callback returning a `std::pair`, first element the state derivative, second the rate derivative `sim/DESolver.h:44`. For the linear DOF this pairing is the order reduction of [Eq. \(16\)](#); the same shape is intended to serve a future orientation ODE with `T = Quaternion`, which is why the `static_assert` admits the type `sim/RK45Solver.h:51`.

That future is visible in the tree but is not running. `StateData` reserves a quaternion `orientation` and `orientationRate` (stored (x, y, z, w) per project convention; the field comment reads “(vector, scalar)”) `sim/StateData.h:46`, a direction-cosine matrix (DCM) `sim/StateData.h:49`, and 3-2-1 yaw-pitch-roll Euler angles `sim/StateData.h:51`. None of these has a writer anywhere: a repository-wide search finds only the declarations. The quaternions are brace-initialized to all zeros — not a valid unit rotation — which is safe only because nothing reads them. And the orientation integrator itself is a commented-out member whose design note already fixes its interface ([Listing 15](#)). These are reserved 6-DOF seams, not live state; [Section 13](#) inventories them alongside the dormant `getTorques`.

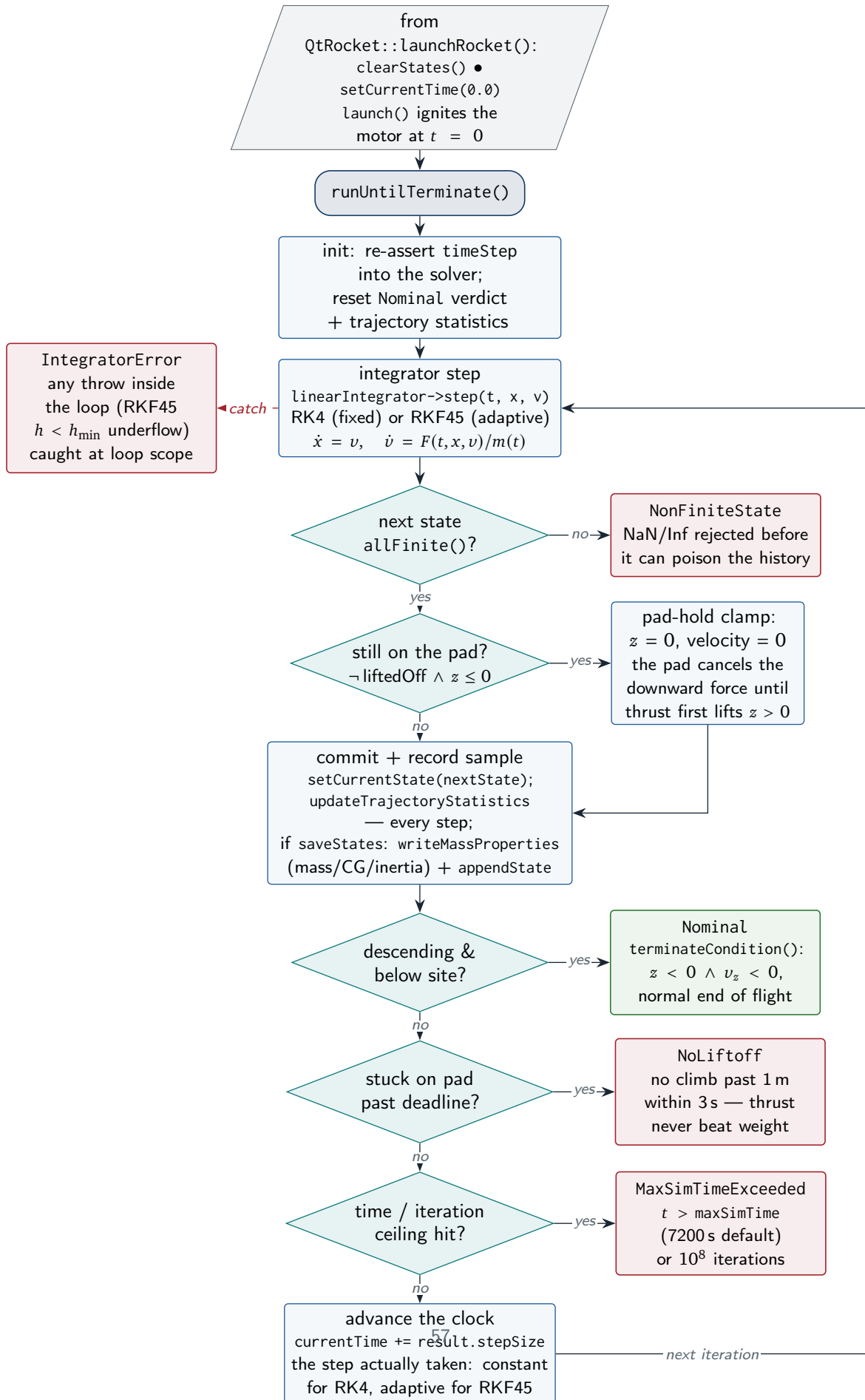
```
std::unique_ptr<sim::Integrator> linearIntegrator;
// The 6-DOF orientation integrator will use the same DESolver<Quaternion> interface, with a
// time-keyed ODE callback so getTorques() can be sampled at each stage's node time like getForces().
// std::unique_ptr<sim::RK4Solver<Quaternion>> orientationIntegrator;
```

Listing 15. The other half of the problem, waiting `sim/Propagator.h:100`: the linear integrator is live; the orientation integrator is a commented-out member whose comment already names the interface and the sampling discipline it will use.

7.3 The run loop, step by step

A flight reaches the loop through `QtRocket::launchRocket()`, which is four calls: clear the state history, reset the propagator clock to zero, `launch()` the model — copy the initial state into the current state and ignite the motor at $t = 0$ `model/RocketModel.cpp:105` — and `runUntilTerminate()` `QtRocket.cpp:54`. [Figure 9](#) shows the loop; the walk below follows it top to bottom.

1. **Re-assert the timestep, start clean.** The first statement pushes the stored `timeStep` into the integrator so a run always steps with it `sim/Propagator.cpp:61` — redundant-looking but load-bearing, because a model switch silently resets a fresh solver’s step ([Section 7.6](#)). The verdict resets to `Nominal` and the statistics to zero `sim/Propagator.cpp:64`; a per-run local `bool liftedOff = false` tracks whether the rocket has yet risen above the pad `sim/Propagator.cpp:77`.
2. **Step.** Each iteration reads the model’s current position and velocity `sim/Propagator.cpp:85`, takes one integrator step `sim/Propagator.cpp:88`, and builds a candidate `nextState` from the returned `StepResult` `sim/Propagator.cpp:90`.
3. **Guard: non-finite state.** If any coefficient of the stepped position or velocity fails Eigen’s `allFinite()`, the run aborts as `NonFiniteState` `sim/Propagator.cpp:97`. The in-tree rationale is exact: a NaN coordinate fails every comparison, so $z < 0$ would never trip and the loop would spin forever; breaking here also keeps the poisoned sample out of the state history `sim/Propagator.cpp:94`, which the guard test confirms `tests/PropagatorTests.cpp:91`.
4. **Pad-hold clamp.** Until the first genuinely positive-altitude step, the rocket is held on the pad: if `position[2] > 0` the flag flips `sim/Propagator.cpp:113`; otherwise z is clamped to 0 and *all three* velocity components are zeroed `sim/Propagator.cpp:119`. The clamp exists because a thrust curve starts with an implicit $(0, 0)$ sample ([Section 6](#)), so over the first step thrust is effectively zero: left free, a from-rest launch at $z = 0$ would accelerate downward, dip below the launch site, and satisfy the nominal “descending below $z = 0$ ” condition after one step `sim/Propagator.cpp:105`. The clamp is a kinematic stand-in for the pad’s normal force, not a contact model, and it has a consequence worth knowing: an angled launch that fails to reach $z > 0$ on its early steps loses its horizontal velocity too — defensible as static friction, but a modeling choice, not an accident. A rocket that can never lift stays put with `maxAltitude = 0`, so the



no-liftoff guard fires instead of the run ending silently `sim/Propagator.cpp:109`.

5. **Commit and record.** The accepted state becomes the model’s current state `sim/Propagator.cpp:124`; the statistics fold then runs *unconditionally* — the in-tree comment: every step “regardless of `saveStates`, so the flight summary and the no-liftoff guard work even when the state history isn’t retained” `sim/Propagator.cpp:125`. Only when `saveStates` is on are the composite mass properties snapshotted into the sample and the sample appended to the history `sim/Propagator.cpp:128`. `writeMassProperties` fills `st.mass`, `st.cg`, and `st.inertia` from the part tree through the gated accessors, so the three are mutually consistent at this t and cost no recompute after burnout `model/RocketModel.cpp:53`. This per-step recording is what makes $CG(t)$ and $I(t)$ observable in the output — the integration suite watches the recorded I_{xx} shrink through the burn and freeze bit-stably after it `tests/PhysicsIntegrationTests.cpp:356`. (`retainStates(false)` exists `sim/Propagator.h:62` but has no production caller; history saving is effectively always on.)
6. **Guard: nominal termination.** If `terminateCondition(t)` returns true the run ends as `Nominal` `sim/Propagator.cpp:137`. `RocketModel`’s condition is descending *and* below the launch site: `position[2] < 0.0 && velocity[2] < 0.0` `model/RocketModel.cpp:65` — the conjunction prevents ending a flight that is still climbing as it passes $z = 0$ from below.
7. **Guard: no liftoff.** After `noLiftoffTime` (3 s) without `maxAltitude` exceeding `noLiftoffAltitude` (1 m), the run aborts as `NoLiftoff` with a logged hint to check thrust against weight `sim/Propagator.cpp:146`. This catches an underpowered motor, or a thrust/weight balance that hovers instead of descending.
8. **Guard: hard backstops.** If `currentTime` exceeds `maxSimTime` or the iteration count reaches 10^8 , the run aborts as `MaxSimTimeExceeded` `sim/Propagator.cpp:157`. The iteration ceiling exists for a flight whose adaptive steps collapse toward zero — sim time then advances negligibly and the time cap alone would never fire `sim/Propagator.cpp:155`.
9. **Advance the clock.** `currentTime += result.stepSize` — constant `dt` for RK4, whatever was accepted for RKF45 `sim/Propagator.cpp:166`.

Around the whole loop sits the catch: any `std::exception` escaping the body — the RKF45 step-size underflow throw, or a throwing force model — becomes `IntegratorError` with the exception text logged `sim/Propagator.cpp:169`. The guard test drives a force model that throws mid-flight and asserts `runUntilTerminate()` itself never throws `tests/PropagatorTests.cpp:140`.

7.4 The termination taxonomy

The five enumerators of `Propagator::TerminationReason` partition every possible exit `sim/Propagator.h:35`; the header’s own summary is that `Nominal` is the normal end of flight and the rest are safety aborts that turn a would-be infinite loop into a reportable stop `sim/Propagator.h:33`. Table 8 gives each enumerator its exact trigger.

One asymmetry deserves a plain statement: the two hard backstops — the sim-time cap and the iteration cap — share the single `MaxSimTimeExceeded` enumerator `sim/Propagator.cpp:157`, so a consumer reading the verdict cannot distinguish a flight that flew too long from one whose steps collapsed. The taxonomy is one enumerator short of its guard count. Everything else about the verdict is disciplined: it resets to `Nominal` at the top of every run `sim/Propagator.cpp:64`, is read back through `QtRocket::getTerminationReason()` `QtRocket.h:62`, and the CLI renders each value through a switch that deliberately has no `default`, so adding a sixth enumerator fails to compile there until it is handled `cli/Repl.cpp:39` — fail-closed at the vocabulary level. Each abort path is pinned by a mock-driven guard test: zero net force ends as `NoLiftoff` `tests/PropagatorTests.cpp:72`, an injected NaN

Enumerator	Exact trigger	Set at
Nominal	<code>terminateCondition(t)</code> returns true; for <code>RocketModel</code> : $z < 0$ and $v_z < 0$ (descending below the launch site)	<code>sim/Propagator.cpp:137</code>
NoLiftoff	<code>currentTime > 3.0 s</code> and <code>maxAltitude < 1.0 m</code>	<code>sim/Propagator.cpp:146</code>
NonFiniteState	any NaN/Inf coefficient in the stepped position or velocity (Eigen <code>allFinite()</code>)	<code>sim/Propagator.cpp:97</code>
MaxSimTimeExceeded	<code>currentTime > maxSimTime</code> (default 7200 s) or <code>iterations ≥ 10⁸</code>	<code>sim/Propagator.cpp:157</code>
IntegratorError	any <code>std::exception</code> escaping the loop body, e.g. the RKF45 step-size underflow throw <code>sim/RK45Solver.h:80</code>	<code>sim/Propagator.cpp:169</code>

Table 8. The termination taxonomy: five enumerators `sim/Propagator.h:35` and their exact triggers. The nominal condition lives in the model `model/RocketModel.cpp:65`; the four aborts live in the loop.

as `NonFiniteState tests/PropagatorTests.cpp:91`, a never-descending flight as `MaxSimTimeExceeded tests/PropagatorTests.cpp:121` — the test is also `setMaxSimTime`’s only caller in the tree — and a throwing force model as a clean `IntegratorError tests/PropagatorTests.cpp:140`.

7.5 Numerical methods

From Euler to embedded pairs

Given $\dot{y} = f(t, y)$ and the state y_n at time t_n , a one-step method estimates $y(t_n + h)$. Euler’s method, $y_{n+1} = y_n + h f(t_n, y_n)$, follows the slope at the start of the step for the whole step: per-step error $O(h^2)$, accumulating to $O(h)$ — halving the step merely halves the error. Runge–Kutta methods probe f at trial points *inside* the step and weight the sampled slopes so the combination matches the true solution’s Taylor expansion to higher order; the classical fourth-order scheme (RK4) uses four samples for a local error of $O(h^5)$ and a global error of $O(h^4)$, so halving the step buys roughly sixteen times the accuracy.

A fixed step, however, must be sized for the worst transient in the flight — a motor’s sharp thrust curve — and is then wasted on the long smooth coast that follows. Adaptive methods pick h per step, which requires an error estimate. Fehlberg’s insight: a carefully chosen set of six stage evaluations yields *two* estimates of the step, one fourth-order and one fifth-order, and their difference is a nearly free estimate of the lower-order method’s local error. Compare it against a tolerance; shrink and retry when too large, accept and grow when comfortably small. Because an order- p method’s local error scales as h^{p+1} , the step multiplier that would have just met a tolerance `tol` is $(\text{tol}/\text{err})^{1/(p+1)}$.

7.5.1 The fixed-step backend

`RK4Solver::step` guards first: `dt` is initialized to a quiet-NaN sentinel `sim/RK4Solver.h:89` and checked with `std::isnan` — the in-tree comment notes that `dt == quiet_NaN()` would always be false — logging an error and returning an empty `StepResult<T>{}` if the step was never set `sim/RK4Solver.h:49`. That error path is a sharp edge for direct users (the returned `stepSize` is 0.0, and the Eigen members are default-constructed, hence uninitialized), but it is unreachable through the `Propagator`, which re-asserts `dt` at the top of every run.

The four stages are the textbook scheme, evaluated at nodes t , $t + dt/2$, $t + dt/2$, $t + dt$, with

`halfDT = dt/2` precomputed in `setTimeStep sim/RK4Solver.h:42`. Writing k_i for the derivative *pair* the callback returns $(k_i\text{State}, k_i\text{Rate})$, with the additions applying componentwise to the pair $(\text{state}, \text{rate})$ `sim/RK4Solver.h:57`:

$$\begin{aligned} k_1 &= \text{odes}(t, \text{state}, \text{rate}), \\ k_2 &= \text{odes}(t + \text{halfDT}, (\text{state}, \text{rate}) + k_1 \text{halfDT}), \\ k_3 &= \text{odes}(t + \text{halfDT}, (\text{state}, \text{rate}) + k_2 \text{halfDT}), \\ k_4 &= \text{odes}(t + \text{dt}, (\text{state}, \text{rate}) + k_3 \text{dt}), \end{aligned} \quad (17)$$

combined with the classical $\frac{1}{6}(1, 2, 2, 1)$ weights and returned with the constant step `sim/RK4Solver.h:69`:

$$(\text{state}, \text{rate})_{n+1} = (\text{state}, \text{rate})_n + \frac{\text{dt}}{6} (k_1 + 2k_2 + 2k_3 + k_4), \quad \text{stepSize} = \text{dt}. \quad (18)$$

Passing each node time into the callback is not a nicety: the forcing is explicitly time-dependent (motor thrust, burning mass), and the comment above the stages says exactly that `sim/RK4Solver.h:55`. Fixed step also means a predictable output cadence — the recorded samples land exactly `dt` apart, which the integration suite verifies `tests/PhysicsIntegrationTests.cpp:131`.

7.5.2 The adaptive backend: the Fehlberg 4(5) embedded pair

`RK45Solver::step` is an adapt-until-accept loop `sim/RK45Solver.h:78`. Every attempt begins with the underflow guard: if h has been driven below $\text{hMin} = 10^{-15}$ s, the solver throws `std::runtime_error` — “cannot meet the requested error tolerance” `sim/RK45Solver.h:80` — which the `Propagator` converts into `IntegratorError`. Then come the six Fehlberg stages: stage i evaluates the callback at node time $t + C_i h$ on a trial pair built from the previous stage derivatives `sim/RK45Solver.h:92`,

$$(s_i, r_i) = \text{odes}\left(t + C_i h, (\text{state}, \text{rate}) + h \sum_{j=1}^{i-1} K_i[j-1] (s_j, r_j)\right), \quad i = 1, \dots, 6, \quad (19)$$

with $C_1 = 0$ and the empty sum for the first stage. The coefficients are `static constexpr` members `sim/RK45Solver.h:172`, and they are the classic Fehlberg tableau, verified digit for digit — the node fractions C_i down the left, the stage couplings K_i in the body (each C_i equals its row sum, as the in-source comment notes `sim/RK45Solver.h:169`), and the two solution-weight rows E_4/E_5 beneath the line:

0						
$\frac{1}{4}$	$\frac{1}{4}$					
$\frac{3}{8}$	$\frac{3}{32}$	$\frac{9}{32}$				
$\frac{12}{13}$	$\frac{1932}{2197}$	$-\frac{7200}{2197}$	$\frac{7296}{2197}$			
1	$\frac{439}{216}$	-8	$\frac{3680}{513}$	$-\frac{845}{4104}$		
$\frac{1}{2}$	$-\frac{8}{27}$	2	$-\frac{3544}{2565}$	$\frac{1859}{4104}$	$-\frac{11}{40}$	
E_4	$\frac{25}{216}$	0	$\frac{1408}{2565}$	$\frac{2197}{4104}$	$-\frac{1}{5}$	0
E_5	$\frac{16}{135}$	0	$\frac{6656}{12825}$	$\frac{28561}{56430}$	$-\frac{9}{50}$	$\frac{2}{55}$

(20)

From the same six stage derivatives the solver forms both estimates of the advanced pair `sim/RK45Solver.h:114`:

$$y^{(4)} = (\text{state}, \text{rate}) + h \sum_{i=1}^6 E_4[i-1](s_i, r_i), \quad y^{(5)} = (\text{state}, \text{rate}) + h \sum_{i=1}^6 E_5[i-1](s_i, r_i), \quad (21)$$

and their difference feeds a single absolute Euclidean norm over the concatenated state and rate (Listing 16):

$$\text{err} = \sqrt{\|y5\text{State} - y4\text{State}\|^2 + \|y5\text{Rate} - y4\text{Rate}\|^2}. \quad (22)$$

```
// Local error estimate: magnitude of the 5th-vs-4th-order difference across state and rate.
const double err = std::sqrt((y5State - y4State).squaredNorm()
    + (y5Rate - y4Rate).squaredNorm());

if(err > tol)
{
    // Reject: shrink the step and retry, flooring the shrink factor at 0.1.
    const double scale = std::pow(tol / err, 0.25);
    h *= (scale < 0.1) ? 0.1 : scale;
    continue;
}
```

Listing 16. The embedded pair’s error estimate and rejection branch `sim/RK45Solver.h:120`: one absolute norm across position and velocity, compared directly against `tol`, with the shrink factor floored at 0.1.

One norm, two units

Equation (22) adds squared meters to squared meters-per-second and compares the result against a single absolute tolerance (default `tol = 10-6`, `sim/RK45Solver.h:158`). Two consequences follow. First, effective accuracy is scale-dependent: a `tol` that is tight for a hundred-meter hop is proportionally looser for a kilometers-high flight, and with no per-component scaling a small-magnitude component rides unprotected alongside a large one. Second, `tol` bounds the *whole-step* local error, not the textbook per-unit-time criterion `err/h`. The header concedes the point in place: the norm matches “the reference algorithm,” and “a scaled/relative norm is a possible refinement” `sim/RK45Solver.h:36`.

Step control is a shrink/grow pair with hard clamps. On rejection (`err > tol`), the step shrinks and the whole stage cascade retries `sim/RK45Solver.h:127`:

$$h \leftarrow h \cdot \max\left((\text{tol}/\text{err})^{1/4}, 0.1\right). \quad (23)$$

On acceptance the solver remembers `usedStep = h`, then grows the guess for next time: if `err` is below machine epsilon it grows maximally (`h ← 5h`, `sim/RK45Solver.h:134`), otherwise

$$h \leftarrow h \cdot \min\left((\text{tol}/\text{err})^{1/5}, 5.0\right), \quad (24)$$

and finally the guess is clamped to `hMax` `sim/RK45Solver.h:147`. The exponents are the classical ones — $1/5$ is $1/(p+1)$ for the order-4 member, and the $1/4$ used on rejection shrinks slightly more aggressively than the growth exponent grows. There is no proportional-integral controller

and no conventional 0.9 safety-factor multiplier; acceptance is the plain threshold $\text{err} \leq \text{tol}$ `sim/RK45Solver.h:124`, and the protection against step-size oscillation comes from the 0.1/5.0 clamps and `hMax`. Because `h` is a member that persists across calls `sim/RK45Solver.h:159`, the adaptation carries from step to step across the whole flight. The accepted step returns the *fifth*-order estimate — `StepResult{y5State, y5Rate, usedStep}` `sim/RK45Solver.h:151` — the standard practice known as local extrapolation: the error was controlled on the fourth-order member, but the better number is the one worth keeping.

The vacuum step-runaway clamp

`setTimeStep` does more than seed the guess: it sets $\text{hMax} = \text{maxStepFactor} \times \text{inTs}$ with $\text{maxStepFactor} = 10$ `sim/RK45Solver.h:66`, and that clamp is mandatory. A constant-acceleration vacuum coast is a polynomial of degree two in time, which both embedded orders integrate *exactly*: `err` sits at rounding level every step, the grow branch quintuples `h` indefinitely, and “the integrator leaps past apogee and the ground in a few giant steps” `sim/RK45Solver.h:144`. The motivating regression is recorded in the integration tests — the flight “leapt megameters past the ground in only 6-8 steps” `tests/PhysicsIntegrationTests.cpp:147` — and the unit test pins the fix: under exactly-integrated dynamics every accepted step stays at or below `hMax` and the largest one *equals* it `sim/tests/RK45SolverTests.cpp:75`. The clamp keeps apogee and the ground crossing resolved on exactly the dynamics where the error estimator is blind.

The solver’s remaining knobs are constructor-enforced but unplumbed. The constructor throws `std::invalid_argument` for a non-positive tolerance `sim/RK45Solver.h:54`; `setErrorTolerance` and `setMaxStepSize` ignore non-positive values `sim/RK45Solver.h:73`, `sim/RK45Solver.h:70`. But no path from `QtRocket`, the GUI, or the CLI reaches either setter — the only caller of `setMaxStepSize` anywhere is a unit test `sim/tests/RK45SolverTests.cpp:112` — so every production RKF45 flight runs at $\text{tol} = 10^{-6}$ with the default clamp (Section 13 lists the missing plumbing). End to end, the payoff is pinned by the comparison flight: on the same motor, RKF45 finishes in far fewer steps than RK4 (the test comment records roughly 530 against 4050) with apogee agreement within 2% asserted and about 0.05% observed `tests/PhysicsIntegrationTests.cpp:163`. The same family documents why node-time sampling matters to the pair: with thrust frozen at the step start, RKF45’s apogee ran roughly 8% high; evaluating each stage at $t + C_i h$ lets the two estimates diverge across a sharp burn, raising `err` and shrinking the step to resolve it `tests/PhysicsIntegrationTests.cpp:149`, `sim/RK45Solver.h:33`.

7.6 Recording subtleties

Two quiet details of the loop’s bookkeeping deserve their own statement, because every consumer of the recorded history inherits the first and every direct user of `Integrator` can be bitten by the second.

The one-step timestamp offset

`appendState(currentTime, nextState)` and the statistics fold both run *before* the clock advance `sim/Propagator.cpp:133`, `sim/Propagator.cpp:166`, so a sample stamped `t` holds the state the rocket reaches at $t + \text{stepSize}$ — every recorded time is one step early, and the initial state itself is never appended (`launchRocket()` clears the history first `QtRocket.cpp:57`). Sample *spacing* is unaffected under RK4 — consecutive stamps are still exactly `dt` apart, and the integration suite pins it `tests/PhysicsIntegrationTests.cpp:131` — but stamped quantities such as `timeOfMaxAltitude` inherit the shift, and the test suite acknowledges it in print: “Time-to-apogee is quantized to `dt` and carries the recorder’s one-step time-stamp offset,” with the tolerance widened to 3 `dt` for exactly

this reason `tests/PropagatorTests.cpp:181`.

Function before model, and the silent dt reset

`Integrator::setIntegratorFunction` only stores the ODE callback `sim/Integrator.h:72`; it never updates an already-built solver. The stored function reaches a solver only when `setIntegratorModel` constructs a fresh one from it `sim/Integrator.h:57`, so the function must be set before the model — exactly the sequence the `Propagator` constructor uses `sim/Propagator.cpp:47`. A caller that inverts the order steps a solver holding a null `std::function`, which surfaces as `std::bad_function_call` — inside the loop, an `IntegratorError`. The second edge is quieter: switching models constructs a fresh solver whose step is unset (RK4's NaN sentinel `sim/RK4Solver.h:89`) or default-valued (RK45's $h = 0.01$, $h_{\text{Max}} = 0.1$ `sim/RK45Solver.h:159`), because `Integrator::setTimeStep` reaches only the currently active instance `sim/Integrator.h:77`. Production is rescued by the first line of `runUntilTerminate()`, which re-asserts `timeStep` at the top of every run `sim/Propagator.cpp:61`; the hazard is real only for direct `Integrator` users, and the selection test documents it — “the freshly (re)created solver needs its own step size” `sim/tests/IntegratorTests.cpp:69`.

7.7 Flight statistics and their consumers

`TrajectoryStatistics::update` folds one (time, state) sample into the running summary `sim/TrajectoryStatistics.h:26`: `maxAltitude` tracks the greatest z and `timeOfMaxAltitude` its stamp — time to apogee — `maxSpeed` the greatest $|velocity|$ with its stamp, and `totalFlightTime` is simply the stamp of the last folded sample, matching `getStates().back().first` `sim/TrajectoryStatistics.h:22`. Both comparisons use strict $>$, which records the *first* time each maximum is attained and is NaN-safe — `NaN > x` is false, so a bad sample is ignored rather than propagated `sim/TrajectoryStatistics.h:24`. `reset()` reassigns a default-constructed struct `sim/TrajectoryStatistics.h:44`, and the `Propagator` calls it at the top of every run `sim/Propagator.cpp:65`. Because the fold runs unconditionally `sim/Propagator.cpp:125`, the summary exists even when the history is not retained, and `maxAltitude` does double duty as the input to the no-liftoff guard `sim/Propagator.cpp:147`. The statistics are not a convenience layer over the history — they are the loop's own memory, and the integration suite asserts they agree with a post-hoc scan of the recorded series to 10^{-9} `tests/PhysicsIntegrationTests.cpp:292`.

QtRocket surfaces the whole record as three read paths: the state history via `getStates()` `QtRocket.h:48`, the summary via `getTrajectoryStatistics()` `QtRocket.h:55`, and the verdict via `getTerminationReason()` `QtRocket.h:62`. The CLI's `launch` command reads the live statistics rather than re-deriving them from the series `cli/Repl.cpp:703` and renders the verdict through its exhaustive reason-text switch `cli/Repl.cpp:39`; how the front ends present all of this — and which of these read paths the GUI actually consumes — is the subject of [Section 10](#).

The simulation core, in sum, is a small machine with sharp contracts: one lambda of physics, two interchangeable solvers behind one interface, a loop that cannot exit without a typed verdict, and a per-step record that already carries the mass properties a 6-DOF integrator will need. The models feeding its force evaluations — gravity, atmosphere, and the geoid — are the subject of [Section 8](#); the aerodynamic machinery waiting to enrich `getForces`, and to give `getTorques` something to say, is examined in [Section 9](#); the dormant orientation seams catalogued here reappear, consolidated, in [Section 13](#).

8 Environment models

Of the three forces the propagator integrates — thrust, gravity, and drag (Section 7) — the last two are the environment's to answer. `sim::Environment` is the container through which the flight dynamics see the outside world: it holds one live instance of each of three pluggable physics models — a `GravityModel`, an `AtmosphericModel`, and a `GeoidModel` `sim/Environment.h:27` — selected by display-name string and defaulting to "Constant Gravity" plus "Constant Atmosphere" `sim/Environment.h:31`. The container itself is deliberately thin: a 123-line header-only class `sim/Environment.h` whose whole job is to own the selected models and hand them out. The chapter walks the container first, then each model family. The centerpiece is the US Standard Atmosphere 1976 — a closed-form, seven-layer implementation validated to 0.1% (0.5% at a handful of density points) against the published report tables — and the chapter closes with the two deliberately simple atmospheres and the one model that exists but is consumed by nothing: wind.

One asymmetry frames everything that follows. The environment can produce six physical quantities — a gravity vector and five atmospheric properties — but the flight dynamics consume exactly two of them: gravitational acceleration `model/RocketModel.cpp:77` and air density `model/RocketModel.cpp:86`. Pressure, temperature, speed of sound, and dynamic viscosity are computed, oracle-validated, and unconsumed — they are the natural inputs for a future Mach-dependent drag coefficient and Reynolds-dependent skin friction (Section 9), and until that machinery lands they are dormant outputs behind a live interface. Figure 10 shows the arrangement.

8.1 The container: registries, selection, and injection

`Environment` is singleton-by-construction per application: its copy and move constructors and assignment operators are all deleted `sim/Environment.h:37`, and the one instance is created by the QtRocket controller at startup `QtRocket.cpp:43`. Internally it keeps two `std::maps` from display-name string to `std::shared_ptr` — three atmosphere keys ("Constant Atmosphere", "US Standard 1976", "Vacuum") `sim/Environment.h:103` and two gravity keys ("Constant Gravity", "Spherical Gravity") `sim/Environment.h:108` — plus a selection string per registry `sim/Environment.h:112` and the single eagerly constructed geoid `sim/Environment.h:117`. The maps are declared with *null* pointers; the constructor immediately calls both setters so that the two default selections point at live instances from the first moment `sim/Environment.h:31`, and the defaults' observable behavior — sea-level constants at every altitude, constant (0, 0, $-g_0$) gravity — is pinned by `DefaultsToConstantAtmosphereAndConstantGravity` `sim/tests/EnvironmentTests.cpp:45`.

Selection is a string comparison, nothing more. `setAtmosphereModel` is an if/else-if chain over the three known names `sim/Environment.h:74`; each matching branch records the name and builds a fresh instance — `atmosphereModels[atmosphereModel].reset(new sim::USStandardAtmosphere);` `sim/Environment.h:84` — and no `else` branch exists, so an unknown name changes nothing. The gravity setter is symmetric `sim/Environment.h:60`, with the spherical branch injecting the geoid into the new model's constructor `sim/Environment.h:70`. Retrieval is a map lookup on the current selection string `sim/Environment.h:93`, and enumeration copies the map keys out with `std::transform` `sim/Environment.h:42` — `std::map` iteration order makes both lists alphabetical, and a test pins the exact names and their order `sim/tests/EnvironmentTests.cpp:58`.

Unknown names are no-ops; known names are fresh instances

Two selection semantics are easy to miss and both are load-bearing. First, the setters' if/else-if chains have no fallback branch `sim/Environment.h:60` `sim/Environment.h:74`: an unrecognized

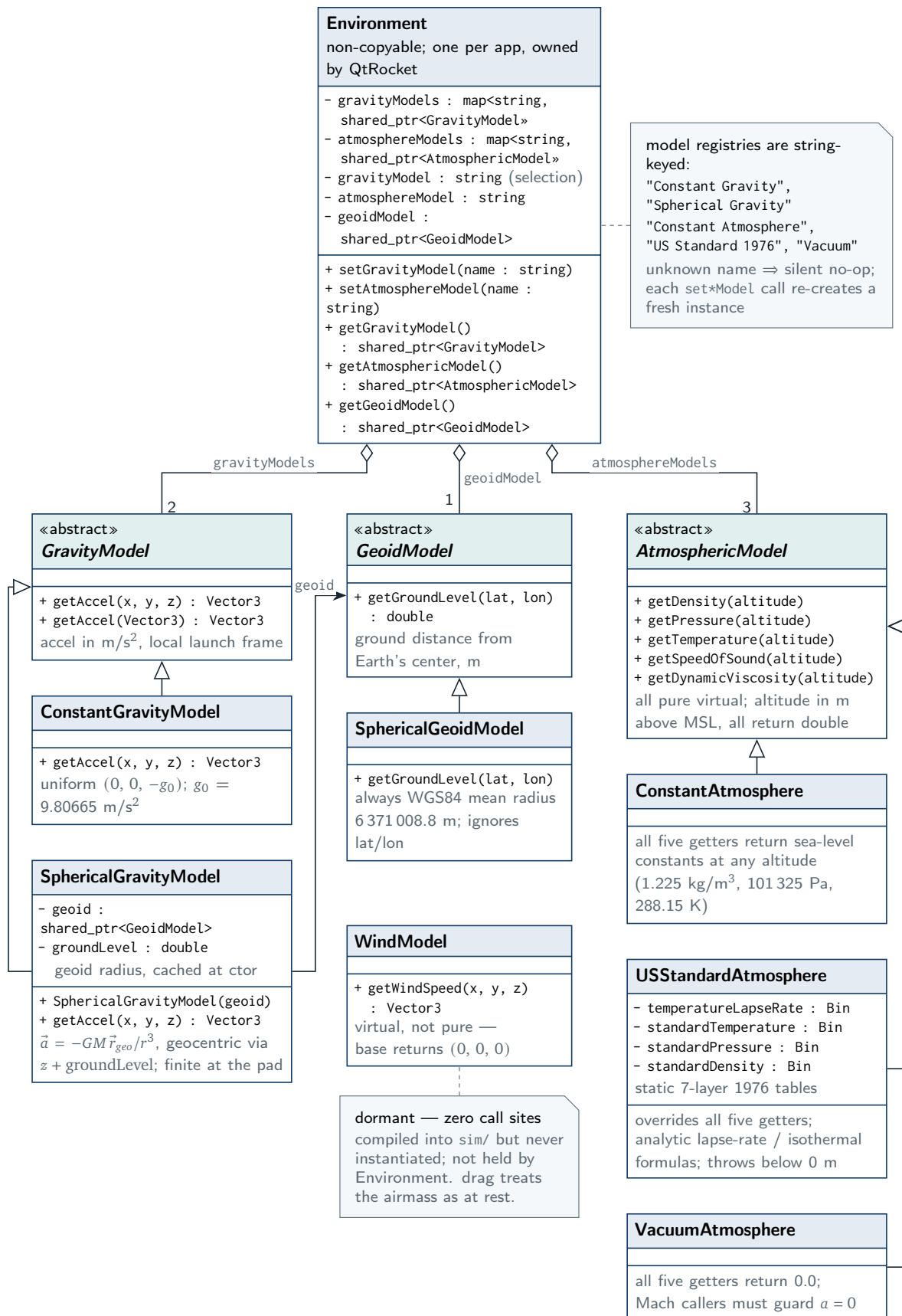


Figure 10. The environment subsystem. Environment owns one selected instance behind each of its two string-keyed registries plus the single geoid `sim/Environment.h:28`; the abstract interfaces fan out to the concrete models named in the registries. The consumption chain runs from the QtRocket controller, which owns the one shared instance, through the Propagator into `RocketModel::getForces`, which consumes exactly two quantities: gravitational acceleration and air density. The atmosphere's other four properties are computed and unconsumed, and `WindModel` stands entirely outside the container.

name silently leaves the current selection untouched, down to pointer identity — the tests assert the *same* `shared_ptr` remains selected after a bogus request `sim/tests/EnvironmentTests.cpp:102`. Second, every successful `set` call constructs a new model object, even when re-selecting the current name. All models are stateless (the spherical gravity model’s cached ground radius reconstructs identically), so this is harmless today — but a client that caches a fetched `shared_ptr` keeps the *old* instance alive and stops observing selection changes. Correct usage is to re-fetch per use, which is exactly what `RocketModel::getForces` does on every call `model/RocketModel.cpp:75` `model/RocketModel.cpp:83`. One related hazard: `getAtmosphericModel()` looks up the selection with `std::map::operator[]` `sim/Environment.h:95`, which would silently insert a null entry if the selection string were ever not a key — unreachable through today’s public API, but armed the day a setter branch is added without a matching registry entry.

Silent rejection at this layer is safe because the layer above never guesses at names. The registries are, in the GUI’s own words, “the source of truth for valid names” `gui/SimOptionsTab.cpp:31`: the GUI populates its combo boxes from the `getAvailable*Models` lists `gui/SimOptionsTab.cpp:33`, the CLI validates a requested name against them before calling the setter and reports `ERR` on a miss `cli/Repl.cpp:585` `cli/Repl.cpp:612` (Section 10), and the CLI’s prompt-state defaults carry a “matches Environment default” comment so the two layers cannot drift silently `cli/Repl.h:45`.

Environment by injection, not by lookup

The environment reaches the physics by parameter, never by global lookup. QtRocket owns the single shared instance (`QtRocket.h:82`, exposed at `QtRocket.h:34`); each `Propagator` receives it at construction `sim/Propagator.h:44` and dereferences it into the force callback on every integrator stage — `object->getForces(t, state, rate, *environment)` `sim/Propagator.cpp:41`; and `RocketModel::getForces` takes it as `sim::Environment&` `model/RocketModel.cpp:68`. Front ends mutate the same shared instance in place — the GUI’s change slots write through it precisely so that changing the atmosphere cannot clobber the gravity selection `gui/SimOptionsTab.cpp:91` — and every holder observes the change on its next fetch.

8.2 Gravity

The constant model is one line of physics: at any position, `return Vector3(0.0, 0.0, -utils::math::Constants::g0)`; `sim/ConstantGravityModel.h:20`, a uniform field of 9.80665 m/s^2 pointing down the world z axis. The value is not arbitrary: $g_0 = 9.80665 \text{ m/s}^2$ `utils/math/Constants.h:10` is *standard gravity*, the conventional sea-level weight acceleration, and it quietly bundles more than gravitation — see the box below.

Inverse-square gravity, and why GM is one constant

Newton’s law of gravitation for a spherically symmetric Earth gives

$$\vec{a} = -\frac{GM}{r^3} \vec{r},$$

where $\vec{r}$ is the geocentric position and $r = |\vec{r}|$. The product GM — the *gravitational parameter* — is kept as a single constant because it is known far more precisely than either factor alone: spacecraft tracking measures GM directly, while G is among the least precisely known fundamental constants. For Earth, $GM = 3.986004418 \times 10^{14} \text{ m}^3/\text{s}^2$. At the surface this gives $GM/R^2 \approx 9.820 \text{ m/s}^2$ — about 0.14% above standard gravity’s 9.80665, because the conven-

tional value folds in the centrifugal relief of Earth’s rotation, which a purely gravitational model does not.

`SphericalGravityModel` implements exactly that law, with one preparatory step. The simulation state lives in a local launch frame — origin at the pad, z up, ground at $z = 0$ `sim/SphericalGravityModel.h:19` — so before applying an inverse-square law the model must re-express the position geocentrically. Its constructor caches the launch site’s ground radius once, because the radius is constant for a site and `getAccel` runs on every integrator stage `sim/SphericalGravityModel.cpp:18`; the geoid is asked at placeholder coordinates $(0, 0)$ “until a real launch site is plumbed in” `sim/SphericalGravityModel.cpp:19`. The acceleration itself is five lines `sim/SphericalGravityModel.cpp:33`: in the code’s names, with $gz = z + \text{groundLevel}$ and $\vec{r}_{\text{geo}} = (x, y, gz)$,

$$\vec{a} = -\frac{\text{earthGM}}{r^3} \vec{r}_{\text{geo}}, \quad r = |\vec{r}_{\text{geo}}|, \quad \text{earthGM} = 3.986004418 \times 10^{14} \text{ m}^3/\text{s}^2 \quad (25)$$

`sim/SphericalGravityModel.cpp:36` `utils/math/Constants.h:26`, written as a single scalar `factor = -earthGM/(r2 · r)` so the vector multiply distributes over components. Tests pin the surface magnitude to GM/R^2 within 10^{-12} relative error `sim/tests/EnvironmentTests.cpp:123`, the $GM/(R+z)^2$ falloff at 10 km `sim/tests/SphericalGeoidAndGravityTests.cpp:67`, and the small inward horizontal component one kilometer downrange — the vector points at Earth’s center, not straight down `sim/tests/SphericalGeoidAndGravityTests.cpp:82`.

The ground-radius offset fixed a real hang

The line `const double gz = z + groundLevel;` is not a refinement; it is the fix for a located defect. An earlier `SphericalGravityModel` treated the pad-relative coordinates as geocentric and computed GM/r^3 at the pad’s $r = 0$ — producing NaN and hanging the propagator loop in an infinite iteration (the F1 defect recorded in the test file’s header `sim/tests/SphericalGeoidAndGravityTests.cpp:1`). Today the invariant is stated where it binds — “ $|rvec| \geq \text{groundLevel}$ so it’s never 0” `sim/SphericalGravityModel.cpp:32` — and defended twice: the `FiniteAtLaunchOrigin` regression test asserts finite acceleration at the exact origin `sim/tests/SphericalGeoidAndGravityTests.cpp:48`, and the propagator’s `NonFiniteState` guard aborts rather than loops if a NaN ever reaches the state again `sim/Propagator.cpp:94` (Section 7).

The ground radius comes from the geoid. A geoid, in theory, is the equipotential surface of Earth’s gravity field — “mean sea level” extended under the continents — and the interface asks it a single question: how far is the ground from Earth’s center at a given latitude and longitude `sim/GeoidModel.h:14`. The only implementation, `SphericalGeoidModel`, ignores both arguments and returns `meanEarthRadiusWGS84 = 6 371 008.8 m` `sim/SphericalGeoidModel.cpp:18` `utils/math/Constants.h:24` — the WGS84 (World Geodetic System 1984) mean of Earth’s polar and equatorial radii `sim/SphericalGeoidModel.h:10`. There is no `setGeoidModel`; `Environment` constructs the one instance in-class and notes “Only one geoid today; a registry can follow if needed” `sim/Environment.h:116`. Constancy across latitudes is itself pinned by test `sim/tests/SphericalGeoidAndGravityTests.cpp:38`.

End to end, the two gravity models agree the way the physics box predicts: an integration test flies the same rocket under both and requires the apogees to land within 5% — spherical $g \approx 9.82 \text{ m/s}^2$ near the surface versus the constant model’s 9.80665 `tests/PhysicsIntegrationTests.cpp:331`.

8.3 The US Standard Atmosphere 1976

`USStandardAtmosphere` — registry name "US Standard 1976" — is the subsystem’s centerpiece: a closed-form implementation of the 1976 US Standard Atmosphere’s lower layers, from sea level through the mesosphere `sim/USStandardAtmosphere.cpp`. The reference document whose tables anchor both the implementation and its test oracles is bundled in the repository `docs/us-standard-atmosphere_st76-1562_noaa.pdf`. The physics behind the formulas fits in one box.

Two relations govern a standard atmosphere

A static column of air must support its own weight: the pressure drop across a horizontal slab of thickness dh equals the slab’s weight per unit area,

$$\frac{dP}{dh} = -\rho g \quad (\text{hydrostatic equilibrium}).$$

Density, pressure, and temperature are tied by the ideal gas law, $P = \rho \frac{R^*}{M} T$, with R^* the universal gas constant and M the mean molar mass of dry air. Substituting one into the other eliminates ρ :

$$\frac{dP}{P} = -\frac{g_0 M}{R^* T(h)} dh.$$

The 1976 standard makes this integrable in closed form by *prescribing* $T(h)$ as piecewise linear: seven layers, each with a constant lapse rate $L_b = dT/dh$. Two layer types result. In a *gradient* layer ($L_b \neq 0$) the integral yields a power law, $P = P_b [T(h)/T_b]^{-g_0 M / (R^* L_b)}$; in an *isothermal* layer ($L_b = 0$) it yields an exponential, $P = P_b e^{-g_0 M (h-h_b) / (R^* T_b)}$. Density then follows from the gas law — dividing the pressure law by one more factor of $T(h)/T_b$ — which is why the density exponent is the pressure exponent minus one, and why the isothermal branches of P and ρ share the same exponential. Physically: pressure falls with height because less air sits above you, roughly exponentially with an 8 km scale height (ρ drops ~70% by 11 km and 99.99% by 71 km); temperature zig-zags — the troposphere cools with altitude, the stratosphere warms because ozone absorbs ultraviolet, the mesosphere cools again.

The constants. The formulas run on the 1976 standard’s own constants, not modern CODATA values: `Rstar = 8.31432 J mol-1 K-1`, `airMolarMass = 0.0289644 kg/mol`, and `g0 = 9.80665 m/s2` `utils/math/Constants.h:9`, plus `gamma = 1.4` for the speed of sound `utils/math/Constants.h:15` and the Sutherland pair `beta = 1.458 × 10-6 kg s-1 m-1 K-1/2` — “empirically measured,” the header notes, per the 1976 report — and `S = 110.4 K` `utils/math/Constants.h:19`. Three neighbors in the same header are dead: `standardTemperature`, `standardDensity`, and `earthGM_km` have no references outside their definitions `utils/math/Constants.h:22` — the first two share names with `USStandardAtmosphere`’s static tables, which are distinct entities and are what the formulas actually read.

The lookup primitive. Each per-layer quantity lives in a `utils::Bin`, a small piecewise-bin table: “each inserted key is a bin’s lower bound” and `operator[]` returns the value of the bin containing the query key `utils/Bin.h:15`. Insertion appends and re-sorts `utils/Bin.cpp:41`; lookup walks forward until a bin base exceeds the key, and “falling off the end means key is in the last bin” `utils/Bin.cpp:57` — so the topmost layer extends upward without bound. Keys below the lowest base throw `std::out_of_range` rather than silently clamping `utils/Bin.cpp:52`. A companion `getBinBase` returns the containing bin’s lower-bound key instead `utils/Bin.cpp:72`; the formulas below need both the layer’s anchored value and its base altitude.

The tables. Four static `utils::Bin` members — `temperatureLapseRate`, `standardTemperature`, `standardDensity`, `standardPressure` `sim/USStandardAtmosphere.h:28` — are populated once at static-initialization time `sim/USStandardAtmosphere.cpp:78` from the seven layer anchors in [Table 9](#) `sim/USStandardAtmosphere.cpp:21`. The layer-base pressures and densities are hard-coded from the report rather than derived recursively from the sea-level values, so each layer’s formula is anchored directly to published numbers rather than to accumulated floating-point error.

h_b (m)	T_b (K)	coded rate (K/m)	standard L_b (K/km)	P_b (Pa)	ρ_b (kg/m ³)
0	288.15	+0.0065	−6.5	101325	1.225
11000	216.65	0.0	0	22632.1	0.36391
20000	216.65	−0.001	+1.0	5474.89	0.08803
32000	228.65	−0.0028	+2.8	868.02	0.01322
47000	270.65	0.0	0	110.91	0.00143
51000	270.65	+0.0028	−2.8	66.94	0.00086
71000	214.65	+0.002	−2.0	3.96	0.000064

Table 9. The seven layers as coded `sim/USStandardAtmosphere.cpp:21`: base altitude, base temperature, the stored `temperatureLapseRate`, the standard’s lapse rate $L_b = dT/dh$ it corresponds to, and the hard-coded pressure and density anchors. The coded rate is the *negative* of L_b — positive where temperature falls with altitude — and every formula in the implementation uses that flipped convention consistently.

Sign convention. The stored `temperatureLapseRate` is the negative of the standard’s L_b : the troposphere is coded +0.0065 K/m — temperature *falls* 6.5 K per kilometer — while the warming layers based at 20 and 32 km are coded negative `sim/USStandardAtmosphere.cpp:39`. The convention is harmless because all three formulas use it consistently; with $L = -L_b$ the coded power law below is algebraically identical to the report’s equation (33a).

Temperature. With $h_b = \text{standardTemperature.getBinBase}(h)$, temperature is linear within the layer `sim/USStandardAtmosphere.cpp:114`:

$$T(h) = \text{standardTemperature}[h] - (h - h_b) \cdot \text{temperatureLapseRate}[h]. \quad (26)$$

Pressure. `getPressure` branches on whether the layer is isothermal, testing `floatingPointEqual` (`temperatureLapseRate[altitude]`, `0.0`) `sim/USStandardAtmosphere.cpp:123` — a ULP-scaled floating-point comparison `utils/math/UtilityMathFunctions.h:21` that is exact here only because the two zero rates are stored as literal `0.0` `sim/USStandardAtmosphere.cpp:40`. The isothermal branch (layers based at 11 and 47 km) is the exponential `sim/USStandardAtmosphere.cpp:125`:

$$P(h) = \text{standardPressure}[h] \exp\left(\frac{-g_0 \cdot \text{airMolarMass} \cdot (h - h_b)}{R_{\text{star}} \cdot \text{standardTemperature}[h]}\right), \quad (27)$$

and the gradient branch is the power law of [Listing 17](#): with $T_b = \text{standardTemperature}[h]$, $L = \text{temperatureLapseRate}[h]$, and $P_b = \text{standardPressure}[h]$,

$$P(h) = P_b \cdot \text{base}^{\text{exponent}}, \quad \text{base} = \frac{T_b - L(h - h_b)}{T_b}, \quad \text{exponent} = \frac{g_0 \cdot \text{airMolarMass}}{R_{\text{star}} \cdot L}. \quad (28)$$

Note that `base` is just $T(h)/T_b$ — the ratio of local to layer-base temperature — so [Eq. \(28\)](#) is the physics box’s power law in the code’s flipped-sign convention.

```

double base = (standardTemperature[altitude] - temperatureLapseRate[altitude] *
               (altitude - standardPressure.getBinBase(altitude))) / standardTemperature[altitude];

double exponent = (Constants::g0 * Constants::airMolarMass) /
                  (Constants::Rstar * temperatureLapseRate[altitude]);

return standardPressure[altitude] * std::pow(base, exponent);

```

Listing 17. The gradient-layer pressure power law as coded `sim/USStandardAtmosphere.cpp:132`: base is the temperature ratio $T(h)/T_b$, and four Bin lookups anchor the layer.

Density. `getDensity` is structurally identical — the same isothermal test, the same two branches — reading the `standardDensity` anchors instead of `standardPressure` `sim/USStandardAtmosphere.cpp:93`, with exactly one difference: the gradient-layer exponent subtracts one `sim/USStandardAtmosphere.cpp:107`,

$$\rho(h) = \rho_b \cdot \text{base}^{\frac{g_0 \cdot \text{airMolarMass}}{R_{\text{star}} \cdot L} - 1}. \quad (29)$$

The -1 is the ideal gas law’s doing: $\rho = PM/(R \cdot T)$, and dividing the pressure power law by one extra factor of $T(h)/T_b$ lowers the exponent by exactly one. In the isothermal branch T is constant, so density and pressure share the same exponential form. Density is the one atmospheric number flight actually consumes, entering the drag force $\vec{F}_d = -\frac{1}{2}\rho |\vec{v}| C_d A \vec{v}$ at `model/RocketModel.cpp:86` (Sections 2 and 9).

Auxiliary properties. Speed of sound is the perfect-gas formula `sim/USStandardAtmosphere.cpp:143` and dynamic viscosity is Sutherland’s law `sim/USStandardAtmosphere.cpp:150`, both driven by Eq. (26):

$$a(h) = \sqrt{\frac{\text{gamma} \cdot R_{\text{star}} \cdot T(h)}{\text{airMolarMass}}}, \quad \mu(h) = \frac{\text{beta} \cdot T(h)^{3/2}}{T(h) + S}. \quad (30)$$

Sutherland’s law is kinetic theory with one correction: rigid-sphere molecules alone would give a viscosity growing as $\sqrt{T}$, and the empirical pair $\text{beta} = 1.458 \times 10^{-6} \text{ kg}/(\text{m s K}^{1/2})$ and $S = 110.4 \text{ K}$ `utils/math/Constants.h:19` models the weak intermolecular attraction that enlarges the effective collision cross-section when the gas is cold — below temperatures of order S the molecules are measurably “stickier,” and the $T^{3/2}/(T + S)$ form damps the hard-sphere growth accordingly. At 288.15 K these evaluate to 340.29 m/s and $1.7894 \times 10^{-5} \text{ Pa s}$ — matching the constant atmosphere’s hard-coded 340.294 and 1.78938×10^{-5} `sim/ConstantAtmosphere.h:19`. Both are the 1976 report’s own auxiliary formulas, and both are unconsumed: a whole-tree search finds no caller of `getSpeedOfSound` or `getDynamicViscosity` outside the models and their tests — no Mach number, no Reynolds number, nothing (Sections 9 and 13).

Validated against the report, not against itself

The oracle suite compares the implementation to the published NOAA tables, never to its own output, and every assertion is a *relative* bound: `EXPECT_NEAR(computed / published, 1.0, _tol)`. Pressure and temperature must match to 0.1% at twenty altitudes from 0 to 80 000 m `sim/tests/USStandardAtmosphereTests.cpp:36` `sim/tests/USStandardAtmosphereTests.cpp:64`; density must match to 0.1% at thirteen altitudes `sim/tests/USStandardAtmosphereTests.cpp:10` and to 0.5% at seven more — 8–10 km and 50–80 km, where the file comments record a “Slight deviation of calculated density from the given density in the report table” `sim/tests/`

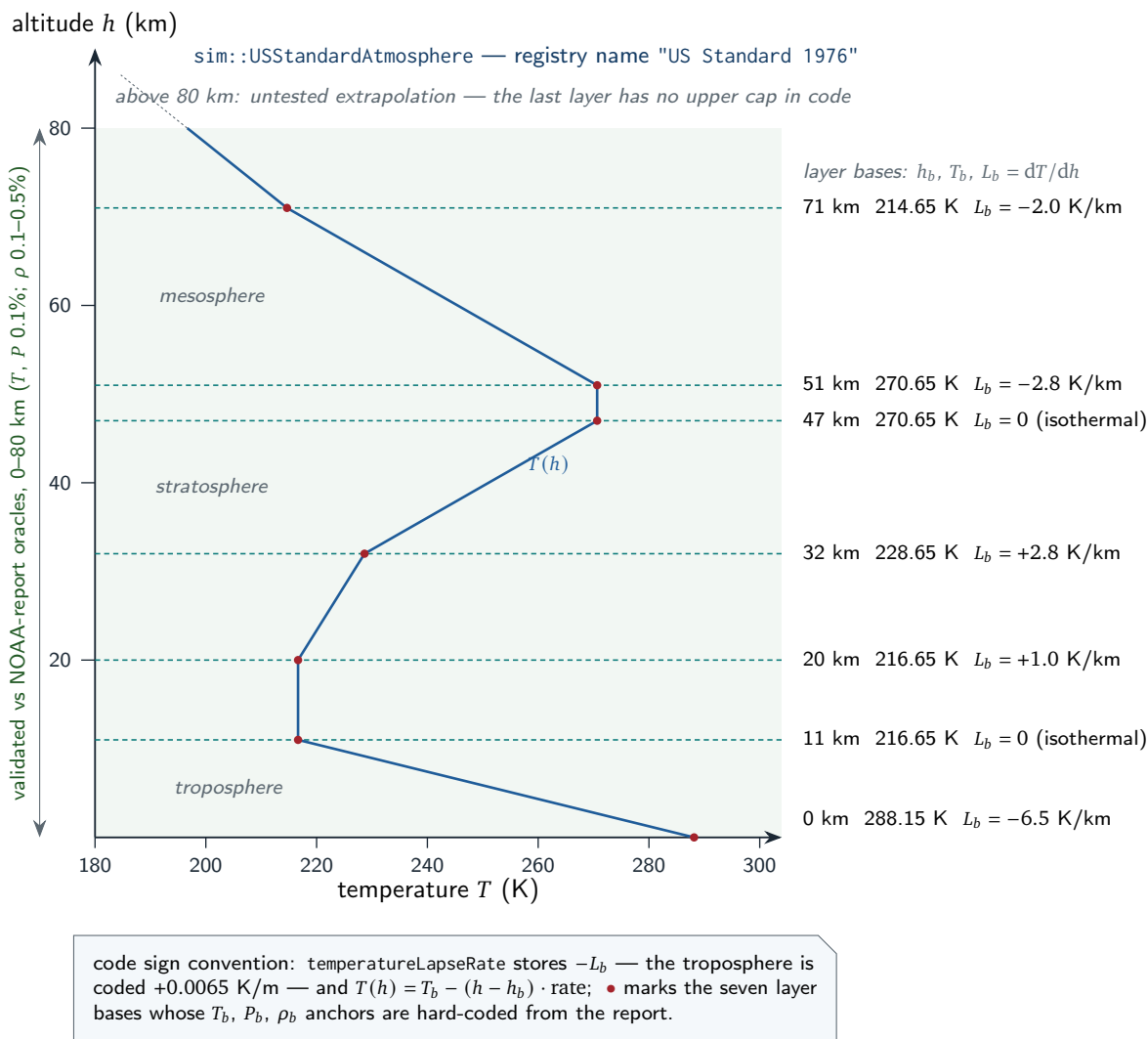


Figure 11. The temperature profile $T(h)$ of the US Standard Atmosphere 1976 as coded: piecewise linear between the seven layer bases whose temperature, pressure, and density anchors are hard-coded from the NOAA report `sim/USStandardAtmosphere.cpp:21`. The shaded band marks the oracle-validated range, 0–80 km; above it the last layer’s formulas extrapolate without bound. The bottom note records the sign convention: the stored `temperatureLapseRate` is the negative of the standard’s L_b .

USStandardAtmosphereTests.cpp:24. The 80 km points sit in the *extrapolated* last bin (base 71 000 m) and still meet the 0.1% pressure oracle. One loose end: `getDensity` still carries an open “@todo Verify against the 1976 US Standard Atmosphere / NOAA paper” `sim/USStandardAtmosphere.h:18` — verification that now exists.

The negative-altitude trap, and who guards it

The model’s domain is altitude ≥ 0 : any query below the lowest bin base propagates `std::out_of_range` from `utils::Bin::operator[]` `utils/Bin.cpp:52`. This was a real crash, not a hypothetical — integrator trial states dip below $z = 0$ near landing, and before the fix a full flight under “US Standard 1976” died there. The guard lives on the *consumer* side: `RocketModel::getForces` clamps — “Clamp altitude to ≥ 0 : trial states crossing $z=0$ fall outside the atmosphere models’ domain” `model/RocketModel.cpp:84` — and the regression is pinned end-to-end by `USStandardAtmosphereCompletesWithoutCrash` `tests/PhysicsIntegrationTests.cpp:280`. The trap hides behind a shared interface — the constant and vacuum models are total functions over the same inputs (Section 8.4), so a caller tested only against those two would pass every test and still crash here. Any new consumer of `AtmosphericModel` owes the same clamp.

Two further edges are honest limitations rather than defects. Above the last layer base there is no cap: the standard’s tables end at 84 852 m and switch to a kinetic model at 86 km, but the code’s 71 km layer extrapolates its closed forms to arbitrary altitude because the last `Bin` is unbounded `utils/Bin.cpp:57` — accurate at 80 km per the oracles, untested and increasingly wrong beyond. And the implementation feeds geometric altitude straight into tables the standard defines in *geopotential* height — no conversion $H = zR/(R + z)$ exists anywhere in the tree. Below 11 km the difference is under 0.2%, which is negligible at model-rocket altitudes; at 50–80 km it is part of what the 0.5% density tolerances absorb. Both belong to the consolidated inventory in Section 13.

8.4 The constant and vacuum atmospheres

The other two atmospheres are deliberately trivial, and each earns its keep in testing. `ConstantAtmosphere` — the default selection — returns the five sea-level standard values at every altitude: density 1.225 kg/m^3 , pressure 101 325 Pa, temperature 288.15 K, speed of sound 340.294 m/s, and dynamic viscosity $1.78938 \times 10^{-5} \text{ Pa s}$ `sim/ConstantAtmosphere.h:15`. For flights that top out a few hundred meters up, a constant-density column is a fine first approximation, and it makes hand-checkable physics: the `TerminalVelocityForceBalance` integration test pulls ρ and g from the live default models and verifies the net force vanishes at $v_t = \sqrt{2mg/(\rho C_d A)}$ `tests/PhysicsIntegrationTests.cpp:246`.

`VacuumAtmosphere` returns zero for all five properties `sim/VacuumAtmosphere.h:17`: “drag vanishes and the flight reduces to thrust + gravity,” making it the baseline that isolates integrator behavior from the aero model `sim/VacuumAtmosphere.h:9`. That role is exercised constantly. `DragReducesApogeeVersusVacuum` flies the same rocket under vacuum and constant atmosphere and requires the drag apogee to fall below 75% of the vacuum apogee `tests/PhysicsIntegrationTests.cpp:227`; the design-matrix sweep flies each reference airframe’s motor ladder twice — once under “Vacuum” with $C_d = 0$, once under “US Standard 1976” with $C_d = 0.75$ — and asserts drag always costs altitude, under both integrators `tests/DesignMatrixTests.cpp:542` `tests/DesignMatrixTests.cpp:559` (one motor per impulse class in the fast suite, the complete ladder under the heavy label; Section 12). One forward-looking wrinkle is documented in place: a vacuum has no sound, so `getSpeedOfSound` returns 0 and “Callers computing a Mach number must guard against 0” `sim/VacuumAtmosphere.h:21`.

— a division hazard that becomes live the day Mach-dependent drag arrives. Both simple models are total functions over pathological inputs; the test grid includes -100 m and NaN `sim/tests/SimpleAtmosphereTests.cpp:13`.

8.5 Wind: a dormant seam

`WindModel` completes the environment’s inventory, and it is the subsystem’s one fully dormant class: built, compiled, and consumed by nothing — with no unit test naming it, it even falls short of the series’ strict sense of *dormant* (built *and* tested). It is a concrete base class — virtual but not pure, because the base is the “no wind” model — with a single method `virtual Vector3 getWindSpeed(double x, double y, double z); sim/WindModel.h:16` whose implementation returns `Vector3(0.0, 0.0, 0.0) sim/WindModel.cpp:18`. It is compiled into the `sim` library `sim/CMakeLists.txt:24`, and that CMake listing plus its own two source files are the only places its name appears in the tree: no subclasses, no `Environment` member, no registry entry, and zero call sites for `getWindSpeed`. Consequently the drag law computes drag from the vehicle’s inertial velocity alone `model/RocketModel.cpp:88` — the air mass is implicitly at rest.

Plumbing the seam is a bounded job, and the container shows the pattern. Aerodynamically, drag opposes the *air-relative* velocity $\vec{v} - \vec{w}$, so consuming wind means: a wind registry and setter in `Environment` mirroring the other two registries `sim/Environment.h:103`; one subtraction in `RocketModel::getForces`, replacing `velocity` with the relative velocity before the drag product `model/RocketModel.cpp:88`; and a selection surface in the front ends. Under today’s 3-DOF point mass a steady wind would add only a lateral drag drift; the payoff that motivates wind in a rocket simulator — weathercocking into the breeze off the launch rod — additionally requires the angle-of-attack and torque machinery of [Section 9](#). The consolidated inventory in [Section 13](#) carries both entries.

The pattern across this chapter is the document’s in miniature. What flight consumes from the environment is two numbers per integrator stage — one gravity vector, one density — and everything behind those numbers is small, closed-form, and validated well past the consumed surface: five-decade pressure oracles for a drag law that only reads ρ , a speed of sound nothing divides by yet, a wind vector nothing subtracts. When Mach-dependent drag or wind arrives, the environment will not need to grow; its consumers will.

9 Aerodynamics

QtRocket’s aerodynamics is two layers that share a codebase but not a call graph. The *live* layer is small: every flight, from every front end, is flown by exactly three force terms — thrust along the world vertical, gravity, and a single quadratic drag term with a user-set constant drag coefficient and a constant scalar reference area `model/RocketModel.cpp:68`. The *dormant* layer is large: a genuine per-part Barrowman implementation — normal-force slopes, a composite center of pressure folded onto the same datum as the composite center of gravity, per-part drag fields — built, unit-tested against 10^{-12} and 10^{-9} oracles, and consumed by nothing in production; the aero test suite says so verbatim `model/tests/AeroTests.cpp:18`. [Figure 12](#) shows both layers at once. This section walks the live path end to end ([Section 9.1](#)), documents the wiring defect that keeps the drag law blind to the rocket’s geometry ([Section 9.2](#)), gives the Barrowman method the full treatment that [Section 2](#) only sketched ([Section 9.3](#)), reads the dormant implementation type by type and formula by formula ([Section 9.4](#)), verifies its dormancy ([Section 9.5](#)), and closes with the one verified defect inside it — the fin set’s mirrored chordwise frame ([Section 9.6](#)) — and with what consuming the seam would take ([Section 9.7](#)).

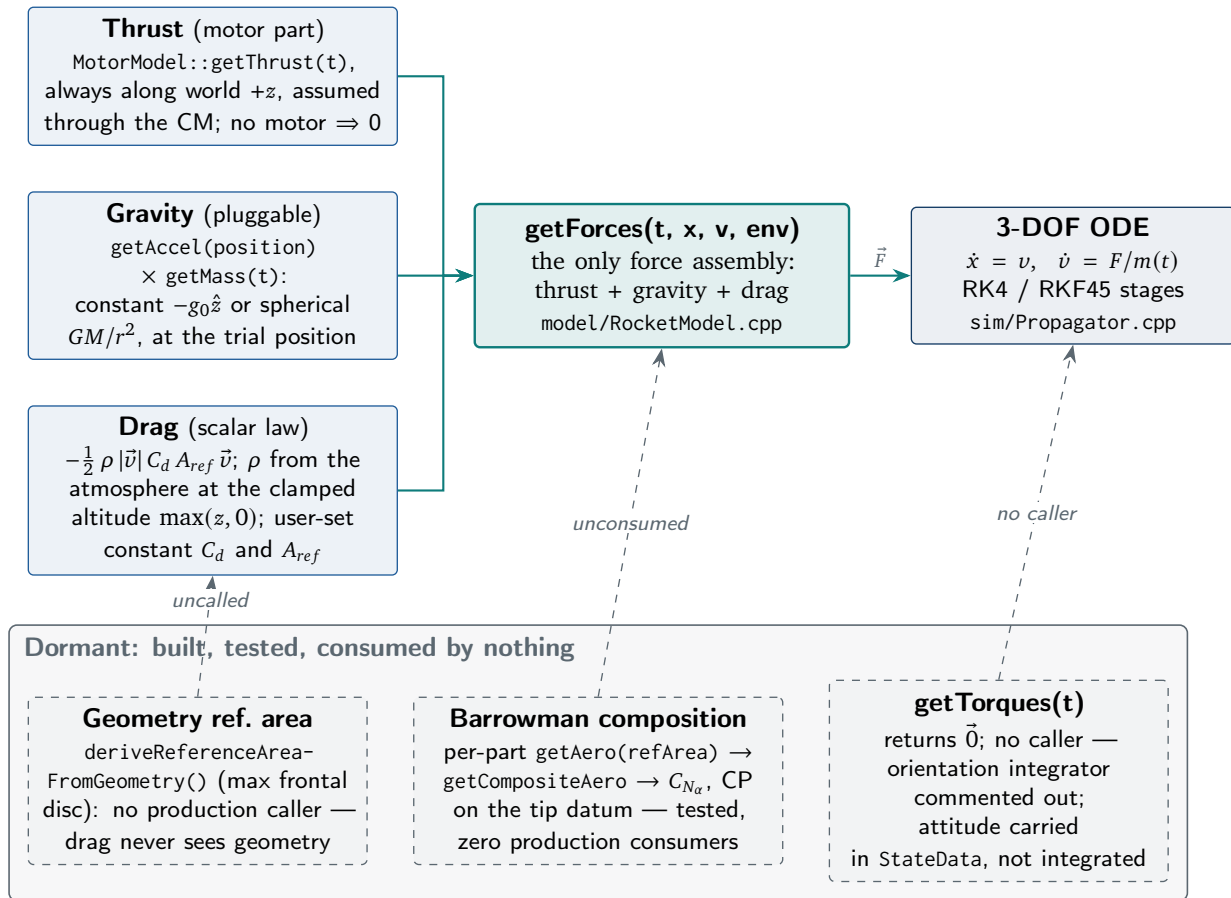


Figure 12. The force-assembly path. The three consumed terms — motor thrust always along world +z, pluggable gravity evaluated at the integrator’s trial position, and the scalar quadratic drag law with density sampled at the clamped altitude — are assembled by `RocketModel::getForces()` `model/RocketModel.cpp:68` and divided by $m(t)$ inside the Propagator’s ODE lambda `sim/Propagator.cpp:36`. The gray dashed row is the dormant machinery: the geometry-derived reference area has no production caller, the Barrowman composition feeds no force term, and `getTorques()` returns zero to a simulation loop that never calls it.

9.1 The live force path

The physics that actually flies enters the simulation as a closure built once in the `Propagator` constructor: $\dot{x} = v$, $\dot{v} = \text{getForces}(t, x, v, env) / \text{getMass}(t)$ `sim/Propagator.cpp:36`, with the division at `sim/Propagator.cpp:41`. The `position` and `velocity` arguments are the integrator’s *trial* stage values, not the committed state — each RK stage sees a self-consistent snapshot. [Listing 18](#) is the entire production force model, quoted whole because there is nothing else: no other function contributes a Newton to any QtRocket flight.

```
Vector3 RocketModel::getForces(double t, const Vector3& position, const Vector3& velocity,
    ↪ sim::Environment& environment)
{
    // Thrust along the rocket's z-axis, assumed through the CM.
    Vector3 forces{0.0, 0.0, motorPart ? motorPart->getMotorModel().getThrust(t) : 0.0};

    // Evaluate gravity at the integrator's trial position (not currentState) so each RK4 stage sees
    // a consistent state.
    auto gravityModel = environment.getGravityModel();

    Vector3 gravity = gravityModel->getAccel(position)*getMass(t);

    forces += gravity;

    // Drag: F = -1/2 * rho(altitude) * |v| * Cd * A * v, opposing velocity. Written with |v|*v (not
    // v^2 * vhat) so v = 0 gives zero drag with no division. With the Vacuum model rho = 0.
    auto atmosphere = environment.getAtmosphericModel();
    // Clamp altitude to >= 0: trial states crossing z=0 fall outside the atmosphere models' domain.
    const double altitude = position[2] > 0.0 ? position[2] : 0.0;
    const double rho = atmosphere->getDensity(altitude);
    const double speed = velocity.norm();
    const Vector3 drag = -0.5 * rho * speed * dragCoefficient * referenceArea * velocity;
    forces += drag;

    return forces;
}
```

Listing 18. The entire production force model `model/RocketModel.cpp:68`: thrust along world +z, gravity at the trial position, and one quadratic drag term. Everything this section calls dormant is dormant because it does not appear here.

Thrust. The first line constructs the force vector as $(0, 0, \text{getThrust}(t))$ — thrust is a world-frame +z component for the entire burn, “assumed through the CM” `model/RocketModel.cpp:70`, and zero when no motor is installed `model/RocketModel.cpp:71`. The thrust magnitude comes from the motor subsystem’s interpolated curve ([Section 6](#)).

Thrust is a world vector, not a body vector

The comment says “along the rocket’s z-axis,” but no attitude state is integrated anywhere in the dynamics ([Section 7](#)), so the rocket’s body z-axis is the world z-axis, permanently. This is the 3-DOF simplification stated plainly: launch angle is a velocity, not an attitude. Both front ends decompose the initial speed into v_x/v_z components `gui/CannonballTab.cpp:105`, `cli/Repl.cpp:680`, and nothing downstream knows the angle existed — for an angled launch, thrust

stays world-vertical while the rocket translates sideways, so there is no gravity turn and no thrust-along-velocity. The model is physically consistent only for vertical flights; for everything else the misalignment is a known consequence of integrating three linear degrees of freedom, not a bug in the force assembly.

Gravity. The second term is `getAccel(position) × getMass(t)` `model/RocketModel.cpp:77`, evaluated at the trial position so each RK stage sees a consistent state `model/RocketModel.cpp:73`. The acceleration comes from whichever pluggable gravity model the environment holds — constant $-g_0\hat{z}$ or Newtonian inverse-square ([Section 8](#)).

Drag. The remainder of the function is the entire production drag model. Air density is sampled from the pluggable atmosphere at the trial altitude, clamped to $z \geq 0$ because trial states crossing the ground fall outside the atmosphere models' domain `model/RocketModel.cpp:84` — the table-backed US Standard Atmosphere throws below its lowest bin ([Section 8](#)), and the clamp is regression-pinned by a flight that descends through $z = 0$ under that model `tests/PhysicsIntegrationTests.cpp:277`. The drag term itself is one line, quoted verbatim from `model/RocketModel.cpp:88`:

```
const Vector3 drag = -0.5 * rho * speed * dragCoefficient * referenceArea * velocity;
```

In the code's own variable names,

$$\vec{F}_{\text{drag}} = -\frac{1}{2} \rho \cdot \text{speed} \cdot \text{dragCoefficient} \cdot \text{referenceArea} \cdot \vec{v}, \quad (31)$$

where every factor is worth naming:

- $\rho = \rho(\max(z, 0))$, the air density in kg/m^3 returned by the selected `AtmosphericModel`: sea-level constant 1.225, US Standard 1976, or vacuum (in which case $\rho = 0$ and drag vanishes identically) `model/RocketModel.cpp:86`, `sim/ConstantAtmosphere.h:15`, `sim/VacuumAtmosphere.h:17`;
- $\text{speed} = \|\vec{v}\|$, the norm of the full trial velocity `model/RocketModel.cpp:87` — there is no wind, so airspeed equals ground velocity by construction ([Section 8](#));
- $\text{dragCoefficient} = C_d$, a dimensionless stored scalar, default `{1.0}` `model/RocketModel.h:138`;
- $\text{referenceArea} = A_{\text{ref}}$ in m^2 , a stored scalar, default `{1.134e-3}` — $\pi (0.019 \text{ m})^2$, the frontal disc of a 38 mm body tube, and the header says so `model/RocketModel.h:141`;
- $\vec{v}$, the full velocity vector, so the magnitude is the textbook $\frac{1}{2}\rho |\vec{v}|^2 C_d A_{\text{ref}}$ opposing all three velocity components — no angle-of-attack decomposition, no axial/normal split.

The $|\vec{v}| \cdot \vec{v}$ form (rather than $v^2\hat{v}$) gives exactly zero drag at $v = 0$ with no division `model/RocketModel.cpp:81`. C_d is a single constant with no Mach or Reynolds dependence — the atmosphere interface exposes speed of sound and viscosity `sim/AtmosphericModel.h:17`, but no production code computes a Mach or Reynolds number from them, so an upper-class flight passes Mach 1 under subsonic-constant drag without comment. And the force path is where the rotational story ends before it begins: `getTorques()` returns the zero vector unconditionally `model/RocketModel.cpp:94`, so no aerodynamic moment of any kind reaches the simulation.

Where the two scalars come from. Both drag inputs are plain stored state on `RocketModel`. The CLI stages the same default (`double referenceArea{1.134e-3};`, commented “matches Rocket-Model default (38 mm tube)” `cli/Repl.h:44`) and exposes `setdrag` and `setarea` `cli/Repl.cpp:497`, `cli/Repl.cpp:510`; the GUI's cannonball tab reads both from text fields `gui/CannonballTab.cpp:100`. Design

files persist all three drag inputs — `dragCoefficient`, `referenceArea`, and the override flag discussed below — `model/DesignSerializer.cpp:233` (Section 11). One validation asymmetry is worth recording: `setReferenceArea` rejects negatives at both the model and the REPL `model/RocketModel.h:74`, `cli/Repl.cpp:510`, but `setDragCoefficient` accepts any double at both layers `model/RocketModel.h:69`, `cli/Repl.cpp:497` — a negative C_d silently turns drag into thrust along $+\vec{v}$.

For all its simplicity, this is the best-tested physics in the tree. The analytic terminal-velocity balance $v_t = \sqrt{2mg/(\rho C_d A_{\text{ref}})}$ is derived in the test and checked for sign and magnitude of the net force below, at, and above v_t with no hard-coded constants `tests/PhysicsIntegrationTests.cpp:241`; and the heavy design sweeps fly every reference airframe across the full 1/4A→M motor ladder twice — once in vacuum with $C_d = 0$, once through the US Standard 1976 atmosphere with $C_d = 0.75$ — asserting that drag always lowers the apogee `tests/DesignMatrixTests.cpp:536`, `tests/DesignMatrixTests.cpp:552`.

9.2 The reference area: derived correctly, delivered nowhere

The constant A_{ref} was never meant to stay constant. `deriveReferenceAreaFromGeometry()` on `RocketModel` delegates to `Part::maxFrontalReferenceArea()` `model/RocketModel.cpp:48`, which takes the *maximum* — not the sum — of `getReferenceArea()` over the part tree `model/parts/Part.cpp:251`. Each part reports its own frontal disc: a tube πr_o^2 `model/parts/BodyTube.h:52`, a cone πR^2 `model/parts/ConicalNoseCone.h:56`, and a fin set deliberately the *body* disc πr_b^2 , not its tip extent — “fins don’t inflate the reference area” `model/parts/FinSet.h:71` — even though `getMaxRadius()` separately reports $r_b + s$ `model/parts/FinSet.h:72`. The base default is zero `model/parts/Part.h:164`, so the boot placeholder `HollowSphere`, which overrides neither `getAero` nor `getReferenceArea`, derives a 0 m² disc — pinned by the one test that calls the function `model/tests/AeroTests.cpp:200`. This is the shared single-body-disc convention of Barrowman’s method and of OpenRocket — whose own reference disc is the nose-cone base (Section C) — never a sum and never inflated by fins, and the headers state it `model/RocketModel.h:79`, `model/parts/Part.h:166`. The derivation is correct. It is also unreachable.

The geometry-derived reference area never reaches the drag law

`deriveReferenceAreaFromGeometry()` has no production caller: its only call sites are its own definition `model/RocketModel.cpp:48` and one unit test `model/tests/AeroTests.cpp:201`, verified by repository-wide search over `gui/`, `cli/`, `sim/`, `visualizer/`, and the controller. The header’s promise that “the geometry default applies only when it hasn’t” been manually overridden `model/RocketModel.h:76` is implemented by no code path. The consequence chain is short. Installing a new airframe root clears the override flag `model/RocketModel.cpp:155`; on design load, the saved area is applied *only when* the file carries `referenceAreaOverridden="true"` `model/DesignSerializer.cpp:263`; and all 25 design fixtures under `tests/data/designs/` carry `referenceAreaOverridden="false"` (verified by search across the 25 files). So every design load leaves whatever area was previously staged — by default the 38 mm disc — regardless of the airframe’s real diameter, and the CLI then syncs its staged copy *from* the rocket `cli/Repl.cpp:964`, cementing the stale value. A 75 mm airframe flies on a 38 mm disc: drag does not scale with airframe class at this commit. The sweep suite even believes otherwise — its header comment claims “The design’s own geometry-derived reference area (per-class frontal disc) is used, so drag scales with the airframe” `tests/DesignMatrixTests.cpp:518`, but the sweep body issues `loaddesign`, `setmotor`, `setatmosphere`, and `setdrag` and never any area command `tests/DesignMatrixTests.cpp:536`. The physics assertion the sweep makes (drag lowers apogee versus vacuum) still holds; the stated mechanism does not. One correct function, one persisted flag,

and one missing call between them. **FAIL**

9.3 The Barrowman method

The dormant layer implements the calculation [Section 2](#) introduced through the arrow intuition: where the center of pressure sits, computed from geometry alone. Because that layer is this section's subject, the method deserves a fuller account than the primer gave it.

Barrowman's method, from assumptions to weighted average

The problem is to predict the aerodynamic *normal* force — the sideways force that appears when the airflow meets the rocket at a small angle of attack α — and the point where it effectively acts, the center of pressure (CP). The solution used by essentially every hobby-rocketry tool was presented by James and Judith Barrowman in 1966 and formalized in James Barrowman's 1967 master's thesis. It is a linearized potential-flow calculation built on three assumptions.

Small angle of attack. For small α the normal-force coefficient is linear, $C_N \approx C_{N\alpha} \alpha$, so the only aerodynamic quantity a component needs to report is its slope $C_{N\alpha}$ (per radian). Coefficients are made dimensionless the standard way, $C_N = N/(q A_{\text{ref}})$, with dynamic pressure $q = \frac{1}{2} \rho v^2$ and a single agreed reference area A_{ref} .

Slender axisymmetric body. For a body that is long compared to its diameter, slender-body potential flow gives a remarkably simple result: the normal force per unit length is proportional to the rate of change of cross-sectional area, $dN/dx \propto dS/dx$. Three consequences follow directly. A cylinder ($dS/dx = 0$) contributes *zero* normal-force slope. Any nose shape rising from a point to base area $S_b = \pi R^2$ contributes exactly $C_{N\alpha} = 2$ per radian referenced to its base area — independent of profile, because the integral of dS/dx depends only on the endpoints. And the nose's CP sits at $\bar{x} = L - V/S_b$ from the tip, where V is the nose volume; for a cone, $V = \frac{1}{3} \pi R^2 L$ gives $\bar{x} = \frac{2}{3} L$. The CP is a property of the *external shape* only — a solid cone and a thin conical shell with identical silhouettes have different centers of mass but the same CP.

Thin flat fins. Fins are treated as thin flat plates. Their lift slope comes from a closed-form lifting-surface approximation in which the length of the mid-chord line captures the effect of sweep, and a multiplicative fin-body interference factor accounts for the extra lift the body carries when fins are mounted on it.

Composition is then linear by construction: every component shares the same α , the same q , and the same A_{ref} , so the slopes simply add, $C_{N\alpha} = \sum_i C_{N\alpha,i}$, and taking moments about any datum gives the composite CP as the slope-weighted average of the component CPs:

$$x_{cp} = \frac{\sum_i C_{N\alpha,i} x_{cp,i}}{\sum_i C_{N\alpha,i}}. \quad (32)$$

The stability verdict is the distance between this point and the center of gravity, conventionally quoted in *calibers* (body diameters): $SM = (x_{CP} - x_{CG})/d$ with stations measured tip-aft-positive, one to two calibers of positive margin being the hobby rule of thumb ([Section 2](#)). The method is exact in no regime and useful in exactly one: slender, axisymmetric, small- α subsonic flight — the regime sport rockets occupy, which is why a 1960s closed-form hand calculation remains the foundation of OpenRocket and its peers today ([Section C](#)).

9.4 The dormant implementation

QtRocket’s implementation of the method lives in two places: value types in `sim/Aero.h` (a deliberately header-only seam; the accompanying translation unit is an empty placeholder `sim/Aero.cpp:3`), and per-part `getAero()` overrides folded by one composition walk in the part tree.

The value types. Table 10 lists both structs. The design decision that makes everything downstream simple sits in the second field: `AeroComponent` stores the slope-weighted CP *moment* `cnAlphaXcp = cnAlpha ·  $x_{cp}$` rather than a raw station, so composing the weighted average of Eq. (32) is pure field addition, and a zero-lift body ($C_{N\alpha} = 0$) drops out of the average with no special case — the stated rationale in the seam’s own documentation `sim/Aero.h:17`. `AeroProfile::operator+=` accumulates the three fields and maintains `cpValid` `sim/Aero.h:53`; `cp()` is the guarded quotient `cnAlphaXcp/cnAlpha`, returning 0.0 when no part generates lift — a body-tube-only rocket has no defined CP `sim/Aero.h:51`.

Field	Type	Meaning
<code>sim::AeroComponent</code> — one part’s contribution <code>sim/Aero.h:32</code>		
<code>cnAlpha</code>	double	slope $C_{N\alpha}$ (per rad), referenced to the shared A_{ref}
<code>cnAlphaXcp</code>	double	<code>cnAlpha · x_{cp}</code> (m); x_{cp} from the part’s own CM, +z = forward
<code>cd</code>	double	drag-coefficient contribution, normalized to A_{ref}
<code>sim::AeroProfile</code> — the assembled whole-rocket profile <code>sim/Aero.h:43</code>		
<code>cnAlpha, cnAlphaXcp, cd</code>	double	the same three fields, summed over parts
<code>refArea</code>	double	the single shared reference area (m^2)
<code>cpValid</code>	bool	false when <code>cnAlpha = 0</code> (CP undefined) <code>sim/Aero.h:49</code>
<code>cp()</code>	method	<code>cnAlphaXcp/cnAlpha</code> behind the <code>cpValid</code> guard <code>sim/Aero.h:51</code>

Table 10. The Barrowman value types. Storing the moment instead of the station makes composition a field addition and lets zero-lift parts drop out of the CP average with no special case `sim/Aero.h:17`.

Per-part contributions. Every override is a pure function of geometry — computed on demand, never stored `model/parts/Part.h:114` — normalized to the single caller-supplied `refArea`, with x_{cp} reported from the part’s own CM in the +z = forward local frame. The base default is aerodynamically inert, `return {}`; `model/parts/Part.h:117`, inherited by `HollowSphere` and `Motor` — a burning motor changes the mass properties every step but neither the reference disc nor the aero profile, and a test pins exactly that `model/tests/AeroTests.cpp:210`. Table 11 summarizes; the two nonzero contributors follow.

Part	$C_{N\alpha}$ (shared A_{ref})	x_{cp} from own CM	c_d	Source
base Part default	0	—	0	<code>model/parts/Part.h:117</code>
BodyTube	0	— (drops out)	0	<code>model/parts/BodyTube.cpp:38</code>
ConicalNoseCone	$2\pi R^2/A_{ref}$	$L/3 - \bar{h}$	0	<code>model/parts/ConicalNoseCone.cpp:57</code>
FinSet	Eq. (35)	$x_{cp,LE} - x_c$ (mirrored)	0	<code>model/parts/FinSet.cpp:60</code>

Table 11. Per-part Barrowman contributions. `HollowSphere` and `Motor` inherit the inert default. Every `cd` is a literal zero — the per-part drag build-up is designed but unpopulated. The `FinSet` CP conversion carries the mirrored sign analyzed in Section 9.6.

The **cone** `model/parts/ConicalNoseCone.cpp:57` codes the classic nose result:

$$\text{cnAlpha} = \frac{2\pi \text{baseRadius}^2}{\text{refArea}}, \quad \text{xcpFromCm} = \frac{L}{3} - \bar{h}, \quad \bar{h} = \begin{cases} L/4 & \text{solid} \\ L/3 & \text{shell} \end{cases} \quad (33)$$

The first factor is the Barrowman $C_{N\alpha} = 2$ referenced to the base area πR^2 `model/parts/ConicalNoseCone.cpp:64` — the body-caliber normalization of the theory box — rescaled to the shared area. The second converts the textbook CP, $\frac{2}{3}L$ aft of the tip, into the part’s own-CM datum: in the middle-relative $+z$ =forward frame the tip is at $+L/2$, so the CP sits at $L/2 - \frac{2}{3}L = -L/6$; subtracting the cone’s own CM station $\bar{h} - L/2$ `model/parts/ConicalNoseCone.cpp:51` gives $L/3 - \bar{h}$ `model/parts/ConicalNoseCone.cpp:66`: $+L/12$ for a solid cone, exactly 0 for a shell, whose CM and CP coincide at $\frac{2}{3}L$ from the tip. The derivation is worked in the comment above the code `model/parts/ConicalNoseCone.cpp:59`, and the unit tests pin $C_{N\alpha} = 2$ at base area, the $+L/12$ and 0 CM-relative CP stations (through their moments), and the frame-independent tip station $-\frac{2}{3}L$ for both variants `model/tests/NoseConeTests.cpp:123`.

The **body tube** returns `{}` `model/parts/BodyTube.cpp:38`: a constant-diameter body has $dS/dx = 0$, so in the linear theory it produces no normal force at small α , contributes $C_{N\alpha} = 0$, and drops out of the composite CP weighted average — the moment-storage design makes that automatic. The comment adds that `cd` “stays 0 – skin friction over `getWettedArea()` is its eventual contribution” `model/parts/BodyTube.cpp:40`: the wetted-area getters exist for a drag build-up that has not been written `model/parts/BodyTube.h:51`, `model/parts/FinSet.h:70`, and every concrete part returns a literal zero `cd` `model/parts/ConicalNoseCone.cpp:67`, `model/parts/FinSet.cpp:89`. All three zero fields are pinned by test `model/tests/BodyTubeTests.cpp:108`.

The **fin set** `model/parts/FinSet.cpp:60` codes the standard N -fin Barrowman term. With root chord c_r , tip chord c_t , span s , sweep length X_t , body radius r_b , and $d = 2r_b$ `model/parts/FinSet.cpp:62`:

$$l_m = \sqrt{s^2 + \left(X_t + \frac{1}{2}(c_t - c_r)\right)^2}, \quad K_{fb} = 1 + \frac{r_b}{s + r_b}, \quad (34)$$

$$(C_{N\alpha})_{\text{body}} = K_{fb} \frac{4N (s/d)^2}{1 + \sqrt{1 + \left(\frac{2l_m}{c_r + c_t}\right)^2}}, \quad \text{cnAlpha} = (C_{N\alpha})_{\text{body}} \cdot \frac{\pi r_b^2}{\text{refArea}}, \quad (35)$$

where l_m is the mid-chord line length `model/parts/FinSet.cpp:74`, K_{fb} the fin-body interference factor `model/parts/FinSet.cpp:76`, and the slope is first referenced to the body disc πr_b^2 and then rescaled to the shared area `model/parts/FinSet.cpp:77`. The algebraic shape is Barrowman’s published result for an N -finned set: the numerator $4N (s/d)^2$ is the linearized slender-configuration normal-force slope for N thin flat-plate fins referenced to the body diameter d , and the square-root denominator is the mid-chord-sweep correction that reduces the effective slope of a swept planform — the derivation, and the trapezoid center-of-pressure station of Eq. (36) below, are developed in the same 1967 thesis as the nose-cone results, under the same small- α , thin-fin assumptions. A degenerate geometry is guarded, not propagated: fins on the axis ($r_b = 0$) would make the $(s/d)^2$ term singular, so $\text{refArea} \leq 0$ or $d \leq 0$ returns the inert component rather than a NaN `model/parts/FinSet.cpp:63`. The fin CP is the standard trapezoid station aft of the root leading edge, converted to the set’s CM datum through the axial mass centroid x_c :

$$x_{cp,LE} = \frac{X_t}{3} \frac{c_r + 2c_t}{c_r + c_t} + \frac{1}{6} \left((c_r + c_t) - \frac{c_r c_t}{c_r + c_t} \right), \quad x_c = \frac{c_r^2 + c_r c_t + c_t^2 + X_t (c_r + 2c_t)}{3 (c_r + c_t)}, \quad (36)$$

both coded at `model/parts/FinSet.cpp:84` — and both *aft-positive distances from the root leading edge*, which is where the sign defect of Section 9.6 enters. One fidelity caveat that is not a bug: N enters

only through the $4N$ factor `model/parts/FinSet.cpp:78`, so $C_{N\alpha}$ is exactly proportional to fin count (six fins report twice three, pinned to 10^{-12} `model/tests/FinSetTests.cpp:211`). Classic Barrowman derives the three- and four-fin cases; for more fins, real interference reduces per-fin effectiveness, so a comparison against OpenRocket for the `mid29_eightfin` fixture should be expected to diverge by construction.

The composition walk. `Part::getCompositeAero(refArea)` folds the components over the resolved placement of [Section 5 \(Listing 19\)](#). For each placed part, the CM station in the root frame is `cmStationZ = pose.origin.z + (-L/2 + cmOffsetz)` `model/parts/Part.cpp:244` — the identical expression the mass walk uses `model/parts/Part.cpp:203` — and the component is shifted onto the shared datum by adding `cnAlpha · cmStationZ` to its moment `model/parts/Part.cpp:246`. Adding the CM station back *exactly cancels* the part’s own CM that `cnAlphaXcp` carried, so the composite CP depends on external shape only, never on mass distribution `model/parts/Part.cpp:233` — pinned by the test that swaps a solid cone for a shell (different CMs, bit-identical composite CP) `model/tests/NoseConeTests.cpp:163`. The walk is a pure-geometry pass, gated only by structural dirtiness and deliberately separate from the time-aware mass walk: “it reads no time-varying state” `model/parts/Part.h:119`, so a burning motor cannot move the CP `model/tests/AeroTests.cpp:210`.

```
sim::AeroProfile Part::getCompositeAero(double refArea) const
{
    // Re-express every part's x_cp (reported from its own CM) onto the shared sub-tree-root (tip) datum:
    // the part's CM station = pose.origin.z + (-L/2 + getCenterMassOffset().z()). Adding cmStation back
    // cancels the part's own CM that cnAlphaXcp carried, so the composite cp depends on external shape
    // only, not mass distribution. cp() and cg() then share the tip datum, so cp() - cg() (the static
    // margin) is datum-independent.
    ensurePlacementCache();
    sim::AeroProfile profile;
    profile.refArea = refArea;
    for(const Placed& pl : resolvedCache)
    {
        const sim::AeroComponent c = pl.part->getAero(refArea);
        const double cmStationZ =
            pl.pose.origin.z() + (-pl.part->getLength() / 2.0 + pl.part->getCenterMassOffset().z());
        profile += sim::AeroComponent{c.cnAlpha, c.cnAlphaXcp + c.cnAlpha * cmStationZ, c.cd};
    }
    return profile;
}
```

Listing 19. The composition walk `model/parts/Part.cpp:231`: one shared reference area, one shared datum, one field addition per part. The station expression is identical to the mass walk’s `model/parts/Part.cpp:203`, which is what puts CP and CG on the same datum by construction.

The static margin is a subtraction by construction

Because the aero walk and the mass walk thread the identical station expression, the composite CP and the composite CG are both stations on the root part’s fore-plane (tip) datum with $+z$ = forward — for a nose-rooted rocket, distances from the nose tip, aft stations negative. The seam’s own documentation states the payoff: “static margin = `cp() - cg()`” `sim/Aero.h:25`, `model/parts/Part.h:119`. The datum is pinned three ways: a bare cone’s composite `cp()` is $-\frac{2}{3}L$ `model/tests/NoseConeTests.cpp:175`; the assembled-rocket oracle applies the root’s own CM station as the tip-datum shift `model/tests/AeroTests.cpp:92`; and the placement-invariance suite names

“the tip datum” outright `tests/PlacementInvarianceTests.cpp:110`. Two cautions for the eventual consumer. First, `sim/Aero.h:24` still says the walk re-expresses onto “the root part’s CM” — the same stale wording [Section 4](#) flagged for the composite CM; the code and `model/parts/Part.h:119` use the tip datum, and the load-bearing invariant (one shared datum) holds either way. Second, the sign: with $+z$ forward, aft stations are negative, so a *stable* design yields $cp() < cg()$ and the documented $cp() - cg()$ comes out negative — a calibers readout must fix the sign convention explicitly. Nothing yet does: no production code computes the subtraction, and the word “caliber” appears nowhere in the C++ sources (failed case-insensitive search).

Beyond the per-part formulas, the composition’s invariants are individually pinned: the CP is a pure ratio, invariant under any choice of shared reference area, while $C_{N\alpha}$ scales as $1/A_{\text{ref}}$ `model/tests/AeroTests.cpp:129`; composite `cd` is a plain sum, exercised through a fixed-value stub part because every real part’s is zero `model/tests/AeroTests.cpp:106`; a lift-less rocket reports `cpValid == false` with the guarded quotient returning zero `model/tests/AeroTests.cpp:120`; and nested sub-trees thread cumulative stations, with the test constructed so that a parent-only-offset bug would be measurably wrong `model/tests/AeroTests.cpp:242`.

9.5 Dormancy, verified

None of this machinery is reachable from any production code path. A repository-wide search for `AeroProfile`, `AeroComponent`, `getCompositeAero`, `aeroData`, and `deriveReferenceAreaFromGeometry` across `gui/`, `cli/`, `sim/`, `visualizer/`, and the controller matches only the type definitions in `sim/Aero.h` and its empty translation unit; every other reference lives in the model layer that *produces* the values — the part implementations, `RocketModel`’s uncalled derivation, `Propagatable`’s unread member — and in five test files that pin them. The test suite states its own status at file scope: “nothing in production consumes this in P2 – these tests pin the seam for P5” `model/tests/AeroTests.cpp:18` (the phrase spans lines 18–19; P2 and P5 are the project’s internal phase tags). The one production-adjacent landing pad confirms the finding: `Propagatable`, the model–sim bridge, carries a protected `sim::Aero` `aeroData`; member `model/Propagatable.h:61` that is default-constructed and never read or written anywhere — it exists today only as the reason the `using Aero = AeroProfile`; back-compatibility alias is kept `sim/Aero.h:63`. In the vocabulary of this document the layer is *dormant*, not partial: the math is present, tested, and — one frame defect aside ([Section 9.6](#)) — correct at the component level, and the flight it was built to inform is untouched by it. Even the placement-invariance baseline treats it that way, deliberately excluding the static margin from its legacy comparison because the legacy CP was CM-contaminated and “cp is unused until 6-DOF” `tests/PlacementInvarianceTests.cpp:318`.

9.6 The fin set’s mirrored chordwise frame

One verified defect sits inside the dormant layer, and it is the reason consumption cannot begin with a wiring change alone.

The FinSet chordwise frame is mirrored — mass live today, CP latent

The frame contract. A part occupies $z \in [-L, 0]$ with $+z = \text{forward}$ `model/parts/Part.h:130`; for a `FinSet`, L is the root chord `model/parts/FinSet.h:61`, and `sweep` is documented as the axial tip-LE offset *aft* of the root LE `model/parts/FinSet.h:45`. The fin root leading edge therefore belongs at the fore plane with the sweep running aft — exactly what the visualizer draws: `zRootLE = cr * 0.5`; and `zTipLE = cr * 0.5 - sweep`; in the middle-centered frame `visualizer/`

RocketMesh.cpp:350. Both x_c and $x_{cp,LE}$ of Eq. (36) are aft-positive distances from the root LE, so their middle-relative stations in the $+z$ =forward frame must be $+c_r/2 - x$.

The defect, at two sites. The mass-centroid offset returns `Vector3{0.0, 0.0, xcMass - cr / 2.0}` model/parts/FinSet.cpp:57 — that is $x_c - c_r/2$, the mirror image of the correct $c_r/2 - x_c$. The aero CP conversion returns `xcpFromCm = xcpFromRootLE - xcMass` model/parts/FinSet.cpp:88 — in the $+z$ =forward frame the correct CM-relative CP is $(c_r/2 - x_{cp,LE}) - (c_r/2 - x_c) = x_c - x_{cp,LE}$, the negative of what is returned. The cone performs the same conversion correctly ($\bar{h} - L/2$, model/parts/ConicalNoseCone.cpp:51); the fin set is the only part inconsistent with the frame documentation.

The errors do not cancel. The composite walk places the part at fore-plane origin z_0 and computes the station $z_0 - c_r/2 + \text{cmOffset}_z$ model/parts/Part.cpp:244. Threading the mirrored offsets through: the fin-set mass lands at $z_0 + x_c - c_r$ where truth is $z_0 - x_c$ — an error of $2x_c - c_r$ forward of truth; the fin CP lands at $z_0 + x_{cp,LE} - c_r$ where truth is $z_0 - x_{cp,LE}$ — an error of $c_r - 2x_{cp,LE}$ aft of truth. For the reference test fin ($c_r = 0.10$, $c_t = 0.05$, $s = 0.05$, $X_t = 0.04$ m model/tests/FinSetTests.cpp:21), $x_c = 0.0566667$ m and $x_{cp,LE} = 0.0372222$ m, so the fin mass sits $2x_c - c_r = 13.3$ mm forward of truth in the composite CM and inertia, and the fin CP sits $c_r - 2x_{cp,LE} = 25.6$ mm aft of truth. For an unswept rectangular fin ($c_t = c_r$, $X_t = 0$), $x_{cp,LE} = c_r/4$ and the CP error is half a chord.

The tests enforce the defect. The regression suite pins the mirrored values as anchors: `off.z() == xc - EX.cr/2` model/tests/FinSetTests.cpp:107; the aero oracle is built as `xcpRootLE - axialMassCentroid(EX)` model/tests/FinSetTests.cpp:199; and the independent literal anchors `cnAlpha = 11.944064334257781` and `cnAlphaXcp = -0.23224569538834566` model/tests/FinSetTests.cpp:206 bake the mirrored sign into an offline-computed constant ($-0.23224 \dots = 11.944 \dots \times (0.0372222 - 0.0566667)$). The suite enforces the defect rather than detecting it; a fix must update all three oracles together. The independent volume-mesh cross-check confirms only the from-LE centroid magnitude, not the sign convention model/tests/FinSetTests.cpp:112. *Blast radius.* The mass half is live today: `finSetCmOffset` feeds the composite CM and inertia that the mass walk computes model/parts/Part.cpp:203 and the simulation loop records into every saved state `sim/Propagator.cpp:132`, so every swept-fin design carries its fins $2x_c - c_r$ forward of truth — about 13 mm for the reference fin, small in absolute terms, because fins are light. The CP half is latent only because nothing consumes the aero profile; once consumed, an aft-displaced fin CP would inflate the apparent static margin — non-conservative for exactly the stability readout the seam exists to provide. **FAIL**

9.7 From seam to consumer

What stands between this layer and a flight it influences is well-defined. Consuming the seam requires an angle of attack — the angle between the body axis and the relative airflow, which needs an attitude state that `StateData` carries but nothing integrates (Section 7); a normal-force term $N = \frac{1}{2}\rho v^2 A_{\text{ref}} C_{N\alpha} \alpha$ applied at `cp()` alongside the axial drag of Listing 18; and the resulting restoring moment about the CG delivered through `getTorques()`, which today returns zero to a loop that never calls it model/RocketModel.cpp:94. The fin-frame fix of Section 9.6 and the reference-area wiring of Section 9.2 precede all of it. Until then, the honest summary stands: QtRocket’s aerodynamic fidelity is a constant- C_d point mass flying straight up, with the machinery for much more sitting tested beside it. Section 13 places both gaps in the consolidated inventory.

10 User interfaces

QtRocket presents three user-facing surfaces, all built over the same Qt-free core (Section 3) but at very different levels of maturity. The Qt6 Widgets application `qtrocket` is a *motor-and-launch* front end: it imports motors, stages a point-mass launch, flies it, and plots the result — but it cannot author, open, or save a rocket design. The headless command-line interface (CLI) `qtrocket-cli` is the most complete surface and the only one that can author designs: a line-oriented read-eval-print loop (REPL) whose stdout doubles as a machine-parseable protocol, making it scriptable from a pipe or a test harness. The standalone `qtrocket-visualizer` is a self-contained OpenGL viewer for saved `.qrd` design files that positions every rendered part with the *same* placement resolver the simulator’s mass pass consumes (Section 5). This section walks each surface in turn and closes with the asymmetry between them; Fig. 13 maps the classes involved and the core services each surface actually drives.

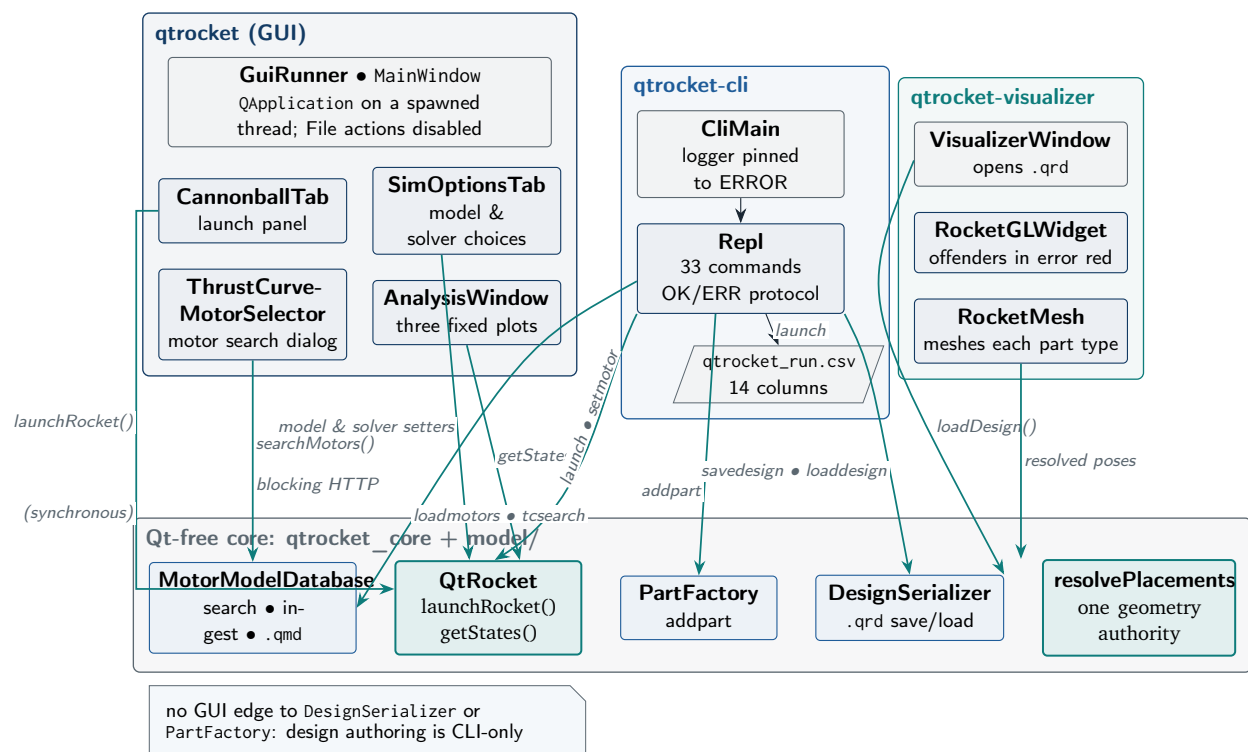


Figure 13. The three front ends and the core services each one drives. Solid edges are the load-bearing call paths at the pinned commit; the GUI’s missing edge to the design serializer — its File actions ship disabled — is the maturity asymmetry this section documents.

10.1 The Qt6 Widgets GUI

The `gui/` directory is the only Qt-dependent code in the main tree — roughly a thousand lines of hand-written widget code compiled directly into the `qtrocket` executable (`CMakeLists.txt:159`), plus the vendored 43,000-line QCustomPlot 2.1.1 plotting library (`gui/qcustomplot.h:23`), which is compiled with `-w` so the rest of the project can keep `-Werror` (`CMakeLists.txt:152`) and is excluded from coverage metrics (`CMakeLists.txt:262`). No GUI source is ever built into a library or a test binary; no test links `Qt6::Widgets` (Section 12).

10.1.1 The entry point: QApplication on a spawned thread

`main()` creates the `Logger` and the `QtRocket` controller singleton on the process main thread — no Qt yet (`main.cpp:20`) — and then blocks in `gui::run(qtrocket, argc, argv)` (`main.cpp:25`). The entire GUI entry point is seven lines ([Listing 20](#)).

```
int run(QtRocket* qtRocket, int argc, char* argv[])
{
    int ret = 0;
    std::thread guiThread(guiWorker, qtRocket, argc, argv, std::ref(ret));
    guiThread.join();
    return ret;
}
```

Listing 20. The whole GUI entry point: spawn a worker thread, join it immediately, and return the exit code through an out-parameter. `QApplication` is constructed inside `guiWorker`, on the spawned thread. `gui/GuiRunner.cpp:59`

`guiWorker` constructs `QApplication` on that spawned thread (`gui/GuiRunner.cpp:30`), shows the `MainWindow`, and runs the entire Qt event loop there (`gui/GuiRunner.cpp:51`); the exit code travels back through an `int&` passed via `std::ref` (`gui/GuiRunner.cpp:62`). The reason is layering, not concurrency, and the header says so: the function “lives in the gui layer so the `QtRocket` controller stays Qt-free and links into the headless CLI” (`gui/GuiRunner.h:11`). Keeping every Qt construction out of `main()`’s call chain is what lets the same controller object serve `qtrocket-cli` without a display.

Qt’s GUI thread is not the process main thread

Qt treats whichever thread constructs `QCoreApplication` as its main/GUI thread; the documented convention is to construct it in `main()`’s thread. `QtRocket` inverts this: the spawned thread owns the event loop (`gui/GuiRunner.cpp:30`) while the true main thread does nothing but sleep in `join()` (`gui/GuiRunner.cpp:63`). Because the join is immediate, there is no concurrency benefit — the process is behaviorally single-threaded-at-a-time. The pattern buys layering purity and imports a portability risk: it works on Linux/xcb, but platforms whose native event loop must own the first thread (macOS/Cocoa) do not support a GUI on a secondary thread.

Two smaller facts about `guiWorker` are worth recording. The application-level icon is set from `:/qtrocket.png` (`gui/GuiRunner.cpp:31`), a path absent from the resource file — the only embedded image is `:/images/resources/rocket64.png` (`qtrocket.qrc:4`), which the window-level icon in the `.ui` correctly uses (`gui/MainWindow.ui:16`) — so the application icon is silently null. And the translation loop tries to load `:/i18n/` resources the `qrc` does not contain (`gui/GuiRunner.cpp:40`, `qtrocket.qrc:1`): internationalization is dead plumbing at this commit ([Section 13](#)).

10.1.2 MainWindow: the shell, the disabled File menu, and the stub panes

`MainWindow`’s constructor builds the `.ui` skeleton and adds its two working tabs programmatically: `CannonballTab` and `SimOptionsTab` (`gui/MainWindow.cpp:30`). The skeleton around them is a vertical splitter holding a horizontal splitter — a `RocketTreeView` beside the tab widget (`gui/MainWindow.ui:49`) — above a `RocketModelerView` (`gui/MainWindow.ui:66`). Both custom widgets are Qt Designer promoted classes (`gui/MainWindow.ui:187`), and both are verified constructor stubs: each `.cpp` is six lines with an empty constructor body (`gui/RocketTreeView.cpp:3`, `gui/RocketModelerView.cpp:3`). No `setModel` call exists anywhere in `gui/`, so the tree view renders an empty pane and the modeler view a blank

widget. More broadly, nothing in the GUI observes part-tree edits at all: a repository-wide search for any structure-changed callback — and for callback members in `QtRocket.h` or `model/RocketModel.h` — comes back empty at this commit. The rocket-structure UI is a frame with nothing wired behind it (Section 13).

The File menu makes the same point from the other direction. It lists New, Open, Save, Save As, Close, and Quit (`gui/MainWindow.ui:82`), but five of the six ship permanently disabled in the .ui — `actionNew` (`gui/MainWindow.ui:117`), `actionOpen` (`gui/MainWindow.ui:129`), `actionSave` (`gui/MainWindow.ui:141`), `actionClose` (`gui/MainWindow.ui:153`), and `actionSave_As` (`gui/MainWindow.ui:170`) — and none has a slot. Only Quit is live, wired to `close()` (`gui/MainWindow.cpp:102`). The consequence is stated plainly: *the GUI cannot open a .qrd design*. The shared `model::DesignSerializer` (Section 11) is consumed by the CLI (`cli/Repl.cpp:957`) and the visualizer (`visualizer/VisualizerWindow.cpp:184`), and a search of `gui/` for any design-persistence reference returns nothing.

The two menus that do work are small: Tools → Save Motor Database opens a *.qmd save dialog (start directory hard-coded to the literal `/home`, `gui/MainWindow.cpp:76`), appends the .qmd suffix when missing (`gui/MainWindow.cpp:84`), and calls `saveMotorDatabase` inside a try/catch that surfaces failures in a message box (`gui/MainWindow.cpp:91`); Help → About shows a modal copyright dialog (`gui/MainWindow.cpp:66`).

10.1.3 CannonballTab and SimOptionsTab

CannonballTab is the launch panel for the point-mass (“cannonball”) flight model: the rocket is a mass with one drag coefficient and one reference area, so the tab exposes exactly five numbers and a motor. The line edits and their shipped defaults are: initial velocity 5.0 m/s (`gui/CannonballTab.ui:29`), launch angle 0.0° (`gui/CannonballTab.ui:46`), mass 1.0 kg (`gui/CannonballTab.ui:60`), drag coefficient 0.5 (`gui/CannonballTab.ui:74`), and reference area 0.001134 m² — the cross-section of a 38 mm-diameter airframe (`gui/CannonballTab.ui:91`). Only two fields carry validators: the angle is clamped to [0, 90] degrees measured *from vertical*, so the default 0° means straight up (`gui/CannonballTab.cpp:34`), and the area’s validator bottom is zero (`gui/CannonballTab.cpp:39`). The other three parse through `QString::toDouble()`, which silently yields 0.0 on junk; the model-side mass setter then silently ignores non-positive values (`model/RocketModel.h:87`) and the area setter negative ones (`model/RocketModel.h:74`), so the display and the model can disagree with no feedback.

Calculate Trajectory reads the five fields, decomposes the initial speed by the angle-from-vertical convention,

$$v_x = v_0 \sin \theta, \quad v_y = 0, \quad v_z = v_0 \cos \theta, \quad (37)$$

with degrees converted through the rounded constant $57.2958 \approx 180/\pi$ (`gui/CannonballTab.cpp:105`), builds the initial `StateData` at the origin (`gui/CannonballTab.cpp:107`), pushes mass, drag coefficient, and reference area into the `RocketModel` (`gui/CannonballTab.cpp:111`), and then calls `setInitialState` followed by `launchRocket` (`gui/CannonballTab.cpp:115`). The launch is fully synchronous: the entire ODE integration (Section 7) runs inside the button slot (`QtRocket.cpp:54`), so the window is frozen for the duration of the flight. On return the tab opens the `AnalysisWindow`; the `setModal(false)` preceding `exec()` is ineffective, since `QDialog::exec()` always runs application-modal (`gui/CannonballTab.cpp:118`).

The tab’s enable logic is deliberately centralized: the launch button is enabled iff the rocket has a motor, `isMotorSet()` (`gui/CannonballTab.cpp:85`, contract at `gui/CannonballTab.h:47`), and every motor-selection path re-invokes the same helper rather than toggling the button ad hoc (`gui/CannonballTab.cpp:77`, `gui/CannonballTab.cpp:170`, `gui/CannonballTab.cpp:241`). The motor paths all route through the `MotorModelDatabase` (Section 6): a file loader that dispatches on extension (.eng to the RASP importer, everything else to the RockSim .rse importer, `gui/CannonballTab.cpp:136`),

.qmd database load/save dialogs (`gui/CannonballTab.cpp:173`), and a Set Motor button that resolves the combo's common name against the database (`gui/CannonballTab.cpp:231`). The engine combo is always rebuilt from `listMotors()` and cleared first — the database is the single source of truth for its contents (`gui/CannonballTab.cpp:153`, contract at `gui/CannonballTab.h:44`).

SimOptionsTab applies every field live — there is no Ok/Apply button (`gui/SimOptionsTab.h:19`). The timestep validator is strictly positive because a non-positive *dt* would hang the run loop (`gui/SimOptionsTab.cpp:25`). The three combos are populated from the model registries — the source of truth for valid names: atmosphere {Constant Atmosphere, US Standard 1976, Vacuum} and gravity {Constant Gravity, Spherical Gravity} from `sim::Environment` (`sim/Environment.h:103`; [Section 8](#)), integrator {Runge-Kutta 4th Order, Runge-Kutta-Fehlberg} from `sim::Integrator` (`sim/Integrator.h:32`; [Section 7](#)). Signals are connected only *after* population so the `addItem` calls do not fire spurious applies, and one apply of all four displayed values is forced at construction so the engine always matches the display (`gui/SimOptionsTab.cpp:48`, `gui/SimOptionsTab.cpp:69`). The atmosphere and gravity slots mutate the shared *Environment in place* — changing the atmosphere model must not clobber gravity, and the rest of the application holds the same `shared_ptr` (`gui/SimOptionsTab.cpp:89`); timestep and integrator forward through the controller to the propagator (`gui/SimOptionsTab.cpp:101`, `QtRocket.h:36`).

10.1.4 Motor search over the network

`ThrustCurveMotorSelector` is the dialog behind the Get Thrustcurve Motor Data button: three facet combos (manufacturer, diameter, impulse class), a search button, a motor list, and an embedded QCustomPlot thrust-curve preview (`gui/ThrustCurveMotorSelector.ui:71`). Get Metadata fetches the search facets from `thrustcurve.org` and appends them to the combos (`gui/ThrustCurveMotorSelector.cpp:48`); the combos are never cleared first, so repeated clicks duplicate every entry, and the manufacturer facet displays the API's short *code* rather than the display name because the search API keys on the code (`model/MotorModelDatabase.cpp:151`). Search builds a `MotorQuery` from the non-empty combos only, leaving unset facets unconstrained (`gui/ThrustCurveMotorSelector.cpp:63`); every result is merged into the local database (`model/MotorModelDatabase.cpp:171`), which is why Set Motor can later resolve it by common name and plot its curve (`gui/ThrustCurveMotorSelector.cpp:83`).

The network path beneath those slots is fully synchronous, end to end, on the GUI thread. The dialog calls `MotorModelDatabase`, which lazily constructs the real `ThrustCurveClient` behind the `ThrustCurveAPI` interface (`model/MotorModelDatabase.cpp:137`; [Section 6](#)); the client issues HTTP GETs through `utils::CurlConnection::get` (`model/ThrustCurveClient.cpp:270`, `model/ThrustCurveClient.cpp:301`), which is a blocking `curl_easy_perform` with a 10 s connect timeout and a 30 s total timeout (`utils/CurlConnection.cpp:54`, `utils/CurlConnection.cpp:58`).

One click, up to 101 blocking round trips

`searchMotors` fans out: after the search response it loops the results and fetches each motor's thrust-curve samples with a separate `download.json` request (`model/ThrustCurveClient.cpp:320`), and the default result cap is `maxResults=100` (`model/ThrustCurveClient.cpp:298`). One Search click is therefore $1 + N$ blocking HTTP round trips with $N \leq 100$, each bounded only by the 30 s libcurl timeout, all executed in the slot body on the Qt event-loop thread. The code states this plainly on both sides of the seam: “each result costs one `download.json` round trip below, performed synchronously on the caller's (often GUI) thread” (`model/ThrustCurveClient.cpp:293`), and “these calls run synchronously on the caller's thread (including the GUI thread), so a hung connection must time out rather than wedge the app” (`utils/CurlConnection.cpp:52`). There is

no `QThread`, `QtConcurrent`, or `QNetworkAccessManager` anywhere in `gui/`: the timeouts are the only thing standing between a slow server and a multi-minute frozen window.

10.1.5 AnalysisWindow: exactly three fixed plots

`AnalysisWindow` is the post-flight view: three buttons drawing into a single promoted `QCustomPlot` widget (`gui/AnalysisWindow.ui:16`), each clearing the previous graph and enabling drag/zoom. Plot Altitude reads the recorded $(t, \text{StateData})$ history from `QtRocket::getStates()` (`gui/AnalysisWindow.cpp:39`) and plots `position[2]` — altitude z in meters — against time (`gui/AnalysisWindow.cpp:49`). Plot Velocity plots `velocity[2]`, the vertical velocity component, not speed (`gui/AnalysisWindow.cpp:74`). Motor Curve plots the motor's thrust samples in Newtons against time (`gui/AnalysisWindow.cpp:86`). These three are the entire analysis surface: the plotted variables are hard-coded, and there is no downrange, speed, acceleration, or mass plot. Axis ranges are spanned from the data (`gui/AnalysisWindow.cpp:55`); the labels are minimal (the altitude axis is labeled literally “Z”, `gui/AnalysisWindow.cpp:54`). Since only the three linear degrees of freedom are integrated at this commit (Section 7), the plot set matches what the state actually contains — but the CSV the CLI exports carries strictly more (Section 10.2.3).

10.2 The command-line REPL

`qtrocket-cli` is the headless front end: a 26-line `main` (`cli/CliMain.cpp`) wrapping `cli::Repl` (`cli/Repl.cpp`, 985 lines), a line-oriented REPL over the same `QtRocket` API the GUI drives. It reads line by line, so the identical binary works interactively, from a pipe, or from a redirected script (`cli/Repl.h:22`). The `Repl` is compiled into its own `cli` static library (`CMakeLists.txt:235`) so that the test suites drive the exact object code a user types into (Section 12); the executable links that library and nothing else (`CMakeLists.txt:297`).

10.2.1 Process model and the stdout protocol

```
int main(int /*argc*/, char* /*argv*/[])
{
    // Logger shares stdout with the CLI's machine-readable output, so run at ERROR. The "[ERROR]"
    // prefix keeps those lines distinct from the CLI's own "OK"/"ERR"/"CSV".
    utils::Logger::getInstance()->setLogLevel(utils::Logger::ERROR_);
```

Listing 21. The CLI's first act is pinning the `Logger` to `ERROR`: `stdout` is a protocol surface, and log chatter would corrupt it. The rest of the 26-line `main` prints the `#`-prefixed banner and hands `std::cin/std::cout` to the `Repl` (`cli/CliMain.cpp:13`).

The `stdout` contract is simple and consistently enforced. Every command's first response line is machine-parseable: `OK <cmd>[: detail]` on success or `ERR <cmd>: reason` (or `ERR usage: ...`) on failure. Payload rows — motor listings, model-name lists, CSV rows, the part tree, the `status` block — follow the `OK` line unprefixed. Human commentary is `#`-prefixed: the startup banner (`cli/CliMain.cpp:21`) and the entire `help` text, which itself ends with `OK help` (`cli/Repl.cpp:259`). Input lines beginning with `#` are treated as script comments (`cli/Repl.cpp:256`). Two extra line types exist only for `launch`: a `WARN:` line on non-nominal termination (`cli/Repl.cpp:737`) and a `CSV <absolute-path>` line naming the flight artifact (`cli/Repl.cpp:740`). This is why the `Logger` is pinned to `ERROR` before

anything else runs (`cli/CliMain.cpp:15`): the protocol and the log share one stream, so the log must be nearly silent and visually distinct when it does speak.

Parsing is equally deliberate. Only the command word is lowercased — arguments keep their case so motor codes like `G80T` and file paths survive (`cli/Repl.cpp:252`). Numeric arguments parse strictly: trailing garbage (e.g. `0.5abc`) rejects the token rather than truncating it (`cli/Repl.cpp:119`). An unknown *command* is an `ERR` (`cli/Repl.cpp:981`), but an unknown *design key* in `newdesign/addpart` warns and continues, so a newer file or CLI’s extra keys do not break an older one (`cli/Repl.cpp:205`).

Automation must parse `ERR`, not `$?`

The Repl’s run loop reads until EOF or `quit/exit` and then unconditionally returns 0 (`cli/Repl.cpp:232`), which `main` returns as the process exit code (`cli/CliMain.cpp:25`). A script consisting entirely of failing commands still exits 0. Pipelines therefore cannot use the shell exit status to detect failure; they must scan stdout for `ERR` lines. One edge case complicates even that: `launch` is the one command that can emit both — if the hardcoded CSV artifact cannot be opened, an `ERR launch: cannot open ...` line prints and the `OK launch: summary` still follows (`cli/Repl.cpp:717`), so a strict “first line decides” parser must treat `launch` specially.

10.2.2 The command inventory

The dispatch chain in `Repl::execute` (`cli/Repl.cpp:245`) contains 32 handler branches accepting 33 command words — `quit` and `exit` are aliases sharing one branch (`cli/Repl.cpp:975`). Table 12 is the complete inventory, grouped by function. Two session contracts pinned in the `help` text apply across the groups: part ids reset on reload, and the `launch/atmosphere` settings are session configuration, never saved in a design file (`cli/Repl.cpp:294`); and `drag` uses the active atmosphere’s density, so selecting the Vacuum atmosphere reduces the force model to thrust plus gravity (`cli/Repl.cpp:298`).

Table 12. The complete `qtrocket-cli` command inventory at commit `254cd41`: 33 command words across 32 handler branches. Angle brackets mark required arguments, square brackets optional ones. The right column gives the opening line of each command’s handler branch in `cli/Repl.cpp`.

Command and arguments	Semantics	Line
Session and output		
<code>help</code>	Prints the <code>#</code> -prefixed command list, then <code>OK help</code> .	259
<code>status</code>	Dumps the staged configuration: motor, dry mass (kg), drag coefficient, reference area (m ²), velocity (m/s), angle (deg from vertical), atmosphere, gravity, integrator, database size.	652
<code>states [stride]</code>	Prints the last run’s state series as CSV to stdout, keeping every stride-th sample (clamped ≥ 1); <code>ERR</code> when no run exists.	744
<code>save <path.csv></code>	Writes the full series (stride 1) to a caller-chosen file; echoes the absolute path.	759
<code>quit exit</code>	Prints <code>OK bye</code> and ends the session.	975
Motor database		
<code>loadmotors <file.rse file.eng></code>	Imports a motor file; <code>.eng</code> dispatches to the RASP importer, anything else to the RockSim <code>.rse</code> importer; reports the count added.	303
<code>savedb <file.qmd></code>	Saves the motor database as XML.	327

continued on next page

Table 12, continued

Command and arguments	Semantics	Line
loaddb <file.qmd>	Additive database load; reports new and total motor counts.	348
tcfacets	Fetches the thrustcurve.org search facets (manufacturers, diameters, impulse classes).	371
tcsearch <k=v>...	Online search; keys <code>manufacturer</code> , <code>diameter</code> , <code>impulseClass</code> . Results merge into the local database.	392
listmotors [substr]	Lists common names with average thrust (N) and total impulse (Ns); optional substring filter.	429
setmotor <code>	Exact common-name lookup; installs the motor on the rocket and stages the session mirror.	453
Staged flight configuration		
setmass <kg>	Must be > 0; applied immediately as a direct mass override on the root part.	479
setdrag <cd>	Drag coefficient; no range check; applied immediately.	497
setarea <m ² >	Must be ≥ 0; applied immediately and sets the manual-override flag.	510
setvelocity <m/s>	Staged in the Repl; consumed at <code>launch</code> .	528
setangle <deg>	Degrees from vertical (0 = straight up); staged.	540
setimestep <s>	Must be > 0; forwarded to the propagator.	552
listatmospheres	Lists the atmosphere-model registry names.	569
setatmosphere <name>	Validates the (possibly multi-word) name against the registry, then applies it to the shared environment.	577
listgravity	Lists the gravity-model registry names.	596
setgravity <name>	Validated against the registry; applied in place.	604
listintegrators	Lists the integrator registry names.	623
setintegrator <name>	Validated; forwarded to the propagator.	632
Simulation control		
launch	Flies the staged configuration; prints a summary block, an optional <code>WARN:</code> line, and a CSV <code><path></code> artifact line (Section 10.2.3).	671
Design authoring		
newdesign <type> [k=v...]	Builds a new root part via the factory and installs it with <code>setRoot</code> ; drops any staged motor.	784
addpart <parentId root> <type> [k=v...] [z=<m>]	Builds a child via the factory and attaches it to the parent with <code>abut(z)</code> .	825
listparts	Prints the indented part tree (one <code>[id]</code> type "name" m=<kg> line per part) plus a composite mass/CG footer at $t = 0$.	880
removepart <id>	Removes a subtree by id; refuses the root; re-checks the motor, which may have been in the removed subtree.	894
cleardesign	Resets the rocket to the boot placeholder part and clears the staged motor.	817
savedesign <file.qrd>	Writes the design via <code>DesignSerializer</code> (Section 11).	926
loaddesign <file.qrd>	Atomic load via <code>DesignSerializer</code> ; re-syncs the staged mirrors (drag, area, motor) from the loaded rocket.	946

10.2.3 Launching and the CSV artifact

`launch` guards on a staged motor (`cli/Repl.cpp:673`), decomposes the staged speed and angle exactly as the GUI does — Eq. (37), with the same rounded 57.2958 constant (`cli/Repl.cpp:35`) — and then calls the same two controller entry points, `setInitialState` and `launchRocket` (`cli/Repl.cpp:687`).

The summary statistics are read back from the propagator’s live-tracked trajectory statistics rather than re-derived (`cli/Repl.cpp:701`), and the summary block prints steps, final time, apogee, maximum speed, downrange distance $\sqrt{x^2 + y^2}$, and landing position at fixed three-decimal precision (`cli/Repl.cpp:728`).

Every successful launch also writes a hardcoded side effect: the full state series goes to `qtrocket_run.csv` in the current working directory (`cli/Repl.cpp:712`), and the CSV protocol line reports its absolute path. One writer serves the `launch` artifact, the `states` stdout dump, and the `save export` (`cli/Repl.cpp:102`), so all three emit the identical schema at nine significant digits: `t,x,y,z,vx,vy,vz,mass,cg_x,cg_y,cg_z,Ixx,Iyy,Izz` — time in seconds, position in meters, velocity in m/s, mass in kg, center of gravity in meters, and the inertia-tensor diagonal in kg m², all SI (`cli/Repl.cpp:104`). Note that the CSV already carries the time-varying mass, CG, and inertia diagonal that the GUI’s three plots never show.

Non-nominal terminations map to text through an exhaustive switch with no default case, so a new `TerminationReason` enumerator fails to compile in the CLI until it is handled (`cli/Repl.cpp:40`); the `NoLiftoff` message even carries its diagnostic criterion in prose — “never cleared 1 m after 3 s (check thrust vs weight)” (`cli/Repl.cpp:46`).

10.2.4 Design authoring from the REPL

The design commands make the CLI the only surface that can author a rocket. Geometry arrives as whitespace-delimited `key=value` tokens parsed into the shared `PartParams` struct — the same vocabulary the factory, the serializer, and the disk format use, so field names cannot drift between front ends (Section 4); `newdesign` and `addpart` then drive `model::part::makePart`, whose exceptions (missing required field, unknown type) surface directly as ERR lines (`cli/Repl.cpp:803`, `cli/Repl.cpp:863`).

Attachment, however, has exactly one spelling. `addpart` always seats the child with `model::part::abut(offset.z())` — the child abuts its parent’s aft plane with an optional forward standoff `z` (`cli/Repl.cpp:872`); the `x=/y=` tokens are parsed into a full vector and then dropped, since off-axis placement is meaningless in 3-DOF (`cli/Repl.cpp:189`). The other two placement verbs, `nestInBore` and `seatOnWall`, exist in the model layer (`model/parts/Placement.h:134`) and round-trip through the file format, but have no REPL spelling: the `.qrd` format is ahead of the CLI’s authoring vocabulary (Section 11).

`savedesign` and `loaddesign` are thin wrappers over the shared `DesignSerializer` (`cli/Repl.cpp:936`, `cli/Repl.cpp:957`); Section 11 covers the format and its fail-safe atomic load. After a successful load the Repl re-syncs its staged mirrors *from* the rocket — drag coefficient, reference area, and the motor name (`cli/Repl.cpp:964`); the displayed dry mass is not re-synced, one of several small session/model seams inventoried in Section 13. Structural edits keep the motor sound throughout: `newdesign` clears the staged motor because `setRoot` drops it (`cli/Repl.cpp:812`), and `removepart` re-checks `isMotorSet()` because the motor may have been in the removed subtree (`cli/Repl.cpp:921`).

10.3 The standalone visualizer

`qtrocket-visualizer` is a separate executable under `visualizer/` (namespace `viz`): a Qt6 OpenGL viewer for `.qrd` design files that links `Qt6::Widgets`, `Qt6::OpenGLWidgets`, `Qt6::OpenGL`, and `qtrocket_core` — and, by stated design, never touches `gui/` or `cli/` code (`visualizer/CMakeLists.txt:27`, `visualizer/CMakeLists.txt:5`). It shares exactly two things with the rest of the system, and they are the two that matter: the design loader and the placement resolver.

10.3.1 Startup and the GL-context workaround

The 41-line `main` does three deliberate things. It disables Qt’s rendering-hardware-interface (RHI) widget backing store, which on EGL/NVIDIA otherwise tries and fails to create a GLES2 context before the viewport exists (`visualizer/main.cpp:22`). It installs a minimal default surface format — a 24-bit depth buffer and nothing else: no OpenGL version, no core profile, no multisampling (`visualizer/main.cpp:24`). And it opens an optional `.qrd` path from `argv[1]` (`visualizer/main.cpp:35`). The empty format request is the point, not an omission: forcing a 3.3 core profile as the application-wide default poisons every context Qt creates, and on EGL (Wayland plus the NVIDIA driver) the driver then has no matching window `EGLConfig` — context creation fails with `EGL_BAD_MATCH` and the 3D view goes black, while GLX honors the same request, so the app would run on one machine and not another (`visualizer/main.cpp:8`).

Accepting whatever context the platform grants has two consequences the renderer carries throughout. Every shader ships as a twin pair — desktop GLSL 1.20 and OpenGL ES 1.00 (GLSL is the OpenGL Shading Language), written in the common `attribute/varying` subset — selected at initialization by `context()->isOpenGLES()` (`visualizer/RocketGLWidget.cpp:23`, `visualizer/RocketGLWidget.cpp:314`), with attribute locations bound by name before linking since neither dialect has layout qualifiers (`visualizer/RocketGLWidget.cpp:201`). And wireframe mode is implemented by expanding every triangle into its three edges as explicit `GL_LINES` vertices, because `glPolygonMode` does not exist on GLES2 (`visualizer/RocketGLWidget.cpp:518`).

10.3.2 From `.qrd` to pixels: the shared loader and the shared resolver

`VisualizerWindow` owns a headless `model::RocketModel` and a deliberately *empty* `MotorModelDatabase`: a design whose motor cannot be re-resolved loads anyway, with the miss reduced to a harmless warning (`visualizer/VisualizerWindow.h:29`). `openFile` delegates to the same static `model::DesignSerializer::load` the CLI uses (`visualizer/VisualizerWindow.cpp:184`; [Section 11](#)) inside a try/catch — a failed load pops a warning box and leaves any currently displayed model in place (`visualizer/VisualizerWindow.h:41`). On success the window rebuilds the render list via `viz::buildRocketMeshes` and updates the status bar with the file, the rocket name, and a recursive part count (`visualizer/VisualizerWindow.cpp:284`). The side panel offers four preset color schemes, a per-type color picker, and grid/axes/wireframe toggles wired directly to the viewport’s slots (`visualizer/VisualizerWindow.cpp:105`).

`RocketMesh` turns the part tree into triangles. Each part type maps to a primitive builder — cone, tube (solid or annular), UV sphere, or fin prism set — via `dynamic_cast` dispatch; a `Motor` or unknown type produces no geometry (`visualizer/RocketMesh.cpp:450`). All primitives are built in local coordinates centered on the origin with `+z` forward (`visualizer/RocketMesh.h:70`), so positioning is a pure translation. The positions themselves come from the placement resolver ([Listing 22](#)).

```
// Consume the same absolute placement the simulator does: resolve every part's pose once, then
// translate each origin-centered primitive so its fore plane lands at the resolved fore-plane
// origin (the build* primitives are centered about mid-length, half an axialLength forward of
// the aft plane).
const std::vector<model::part::Placed> placed =
    model::part::resolvePlacements(*root, model::part::Pose{});
```

Listing 22. The visualizer asks the model layer’s resolver — not its own arithmetic — where every part is. The comment names the contract. `visualizer/RocketMesh.cpp:429`

Rendered geometry cannot diverge from simulated geometry

`resolvePlacements` is documented as “the single authority for absolute placement: intent (StationLink) in, geometry (Pose) out. Both the simulator and the visualizer consume its output, so the two cannot disagree” (`model/parts/Placement.h:148`). The simulator side consumes it inside `Part::ensurePlacementCache`, which feeds the composite mass/CM/inertia pass (`model/parts/Part.cpp:160`; [Section 5](#)); the visualizer side is [Listing 22](#). What the user sees on screen and what the propagator integrates are, by construction, the same poses — there is no second placement computation that could go stale.

Since a resolved `Pose::origin` is the part’s *fore-plane* origin and a part occupies $z \in [-\text{axialLength}, 0]$ in its own frame (`model/parts/Part.h:144`), while the primitives are centered about mid-length, each mesh is translated by `p.pose.origin + Vector3(0.0, 0.0, -part.axialLength() / 2.0)` (`visualizer/RocketMesh.cpp:482`) — the mesh center lands half an axial length aft of the fore plane.

The overlap verdict is shared the same way. The mesh walk reads the cached `SolveResult` from `placementDiagnostics()` (`visualizer/RocketMesh.cpp:440`); where the simulator refuses a failed solve by throwing from the composite pass (`model/parts/Part.cpp:189`), the visualizer instead *renders* the failure: every offending part is painted a fixed error red (0.90, 0.10, 0.10) regardless of the active color scheme — the viewer ships four, `Classic` (contractually first, as the default), `Hi-Vis`, `Carbon`, and `Pastel` (`visualizer/ColorScheme.cpp:28`) — “so a self-intersecting design is unmistakable under any scheme” (`visualizer/RocketGLWidget.cpp:617`), and each located diagnostic is also logged (`visualizer/RocketMesh.cpp:446`). Same verdict object, two consumer policies — exactly as the `SolveResult` documentation prescribes (`model/parts/Placement.h:112`). The per-offender message text is carried on the render item but not yet surfaced in the UI — only the boolean reaches the GPU mesh (`visualizer/RocketGLWidget.cpp:515`; [Section 13](#)).

The rest of the pipeline is conventional and compact: two-sided directional lighting (culling is off because thin fins and tube bores expose back faces, `visualizer/RocketGLWidget.cpp:305`), a stand-up transform rotating body +z onto world +Y so the rocket points up on screen (`visualizer/RocketGLWidget.cpp:171`), and an orbit/pan/zoom camera that frames the model’s bounding sphere on load (`visualizer/RocketGLWidget.cpp:261`). One absence completes the picture: the visualizer has no automated test — its CMake declares only the executable (`visualizer/CMakeLists.txt`), no test binary references any `viz` symbol, and the coverage harness aggregates neither `visualizer/` nor `gui/` (`CMakeLists.txt:248`). The poses and overlap verdicts it renders are pinned upstream by the resolver and sweep tests ([Section 12](#)); the meshing, GL, and widget code paths are eyeball-verified against the committed `.qrd` fixtures only.

10.4 The maturity asymmetry

Read together, the three surfaces invert the usual expectation. The scriptable, headless CLI is the flagship: complete motor management, full design authoring, a disciplined stdout protocol, a documented flight artifact, and two test suites driving the identical object code ([Section 12](#)). The desktop GUI — the surface a newcomer would reach for first — is a capable motor-and-launch panel wrapped in the frame of a design tool that is not yet there: five permanently disabled File actions, two empty panes, and no path to a `.qrd`. The visualizer is complete for its narrow job and exemplary in how it stays consistent with the simulator; but ships without a single test. The gaps named in place here — the disabled menu, the stub views, the missing structure-change notification, the synchronous network and launch calls on the GUI thread — are collected in [Section 13](#).

11 Design persistence

QtRocket persists exactly two kinds of artifact, each with its own versioned XML format: rocket designs, written as `.qrd` files by `model::DesignSerializer` `model/DesignSerializer.h:31`, and the motor database, written as `.qmd` files by `MotorModelDatabase` `model/MotorModelDatabase.cpp:184`. Both ride Boost.PropertyTree XML; the design serializer’s class documentation declares the kinship — versioned save/load “mirroring MotorModelDatabase’s idiom” `model/DesignSerializer.h:17`. This section reads the `.qrd` format element by element against a committed fixture, walks the save and load paths, and states what round-trips, what is rejected, and what is silently lost. The motor format gets a one-paragraph recap (field semantics in [Section 6](#)); trajectory CSV files are run exports, not persisted state, and belong to [Section 10](#).

11.1 The `.qrd` design file

All design persistence goes through one stateless class of two static methods: `DesignSerializer::save(rocket, filename)` `model/DesignSerializer.h:31` and `DesignSerializer::load(rocket, motors, filename)`, which rebuilds a design in place `model/DesignSerializer.h:33`. The header states the format contract: a polymorphic part tree of `<part type=".." name=".."><params .../><link .../><children>...</children></part>` elements, the motor stored by common name, and the aero options (on disk, the `<sim>` block) `model/DesignSerializer.h:19`. [Listing 23](#) shows a complete committed fixture — 27 lines for a four-part design with a motor — and the rest of this subsection walks it top to bottom.

Version attribute and gate. The root carries `version="0.2"`, a `major.minor` pair with the policy stated at the stamp: “load accepts any minor within major 0, rejects other majors. 0.2 uses the `<link>` placement element (0.1 wrote a CM-to-CM `<offset>`, no longer supported)” `model/DesignSerializer.cpp:218`. The reader enforces exactly that: it splits the attribute at the first dot and throws on any major other than “0” before touching the rocket `model/DesignSerializer.cpp:250`; a missing attribute reads as the empty string and is rejected by the same branch. `<design name="...">` round-trips the display name `model/DesignSerializer.cpp:221`, `model/DesignSerializer.cpp:259` — though no CLI command calls `RocketModel::setName` `model/RocketModel.h:65`, so every committed fixture carries `name=""`.

Part elements via the factory reflector. Each `<part>` stores the factory type tag (`typeName()`) and human name as attributes, and its geometry as a `<params>` element produced by the factory reflector `part::params(node)` `model/DesignSerializer.cpp:101` — the inverse of `makePart`, reflecting only the node’s geometric scalars `model/parts/PartFactory.h:63`. `PartParams` is the same all-optional struct the CLI’s `key=value` parser fills and the factory consumes, “so the field vocabulary can’t drift between the front-ends and the on-disk format” `model/parts/PartFactory.h:23`; lengths are SI meters and densities kg/m^3 , never millimeters `model/parts/PartFactory.h:25`. The writer emits one attribute per *present* optional field and omits absent ones `model/DesignSerializer.cpp:34`; `finCount` writes as an integer, `solid` as `"true"/"false"`, “per the DB idiom” `model/DesignSerializer.cpp:49`. Geometry per part type therefore needs no schema of its own.

The `<link>` element: placement intent. Each non-root part may carry a `<link>` recording its `StationLink`: the seat kind (`Abut`, `NestInBore`, or `OnSurface` `model/parts/Placement.h:32`), fractional stations on parent and child (dimensionless, in $[0, 1]$ of live part length), and the gap in meters `model/parts/Placement.h:49`. Seat strings round-trip through one shared table “so reader and

```

<?xml version="1.0" encoding="utf-8"?>
<QtRocketDesign version="0.2">
  <design name=""/>
  <part type="NoseCone" name="Nose">
    <params baseRadius="0.0155" length="0.14999999999999999" wallThickness="0" density="2500"
      ↪ solid="true"/>
    <children>
      <part type="BodyTube" name="Airframe">
        <params innerRadius="0.0146" outerRadius="0.0155" length="0.40000000000000002" density="2500"/>
        <link seat="Abut" parentStation="0" childStation="1" gap="-0.03750000000000006"/>
        <children>
          <part type="FinSet" name="Fins">
            <params density="2500" rootChord="0.059999999999999998" tipChord="0.029999999999999999"
              ↪ span="0.040000000000000001" sweep="0.02" thickness="0.002500000000000001"
              ↪ bodyRadius="0.0155" finCount="6"/>
            <link seat="OnSurface" parentStation="0" childStation="0" gap="0.0077777777777778279"/>
            <children/>
          </part>
          <part type="BodyTube" name="Coupler">
            <params innerRadius="0.012" outerRadius="0.0146" length="0.050000000000000003" density="2500"/>
            <link seat="NestInBore" parentStation="1" childStation="0" gap="0.025000000000000008"/>
            <children/>
          </part>
        </children>
      </part>
    </children>
  </part>
  <motor commonName="G80T"/>
  <sim dragCoefficient="1" referenceArea="0.001134" referenceAreaOverridden="false"/>
</QtRocketDesign>

```

Listing 23. A complete .qrd 0.2 file, verbatim (tests/data/designs/mid29_multi.qrd): version attribute, design name, recursive part tree with present-only <params>, <link> elements exercising all three seat kinds, motor by common name, and the three-scalar <sim> block. The 17-digit doubles are Boost.PropertyTree writing max_digits10 decimals, so doubles round-trip bit-exactly through the text.

writer agree by construction” `model/DesignSerializer.cpp:53`. A link equal to the zero-config default (child fore plane abuts parent aft plane, no gap) is elided on write and recovered on read `model/DesignSerializer.cpp:104`, `model/DesignSerializer.cpp:202`, keeping the file proportional to authored intent rather than part count. The reserved 6-DOF (six-degree-of-freedom) quaternion `childRot` is deliberately not persisted while it is identity by construction in 3-DOF `model/DesignSerializer.cpp:85`.

Placement intent, never resolved coordinates

The file stores what the author meant — “this coupler nests 25 mm into that tube’s bore” — never where anything ended up: “Absolute pose is never serialized — the resolver re-derives it on load” `model/DesignSerializer.cpp:94`. Stations are fractions of *live* length `model/parts/Placement.h:42`, and every load re-derives every seam through the single shared resolver (Section 5). The divergence class that killed the legacy 0.1 format — a stored center-of-mass-to-center-of-mass (CM-to-CM) offset going stale against edited geometry — is unrepresentable in 0.2.

The motor reference. The motor is stored by common name only — `<motor commonName="G80T"/>`, written only when a motor is set `model/DesignSerializer.cpp:227`. It is never a tree part: the writer skips `Motor` nodes (“it cannot be rebuilt by the geometry factory,” `model/DesignSerializer.cpp:112`) and the reader rejects `type="Motor"` as a hand-edited or foreign file `model/DesignSerializer.cpp:157`.

Motor faithfulness is database-relative

A `.qrd` carries no thrust samples, no masses, no curve — one string. On load, a database hit installs the motor; a miss logs “motor ‘<name>’ not found in the database; loaded without a motor” and continues `model/DesignSerializer.cpp:279` — a warning, never an error. Both halves are pinned: against the same database, the reloaded motor’s burning-mass curve is exact at ignition, mid-burn, and post-burnout `tests/DesignPersistenceTests.cpp:161`; against an *empty* database, the geometry loads intact with `isMotorSet() == false` `tests/DesignPersistenceTests.cpp:180`. The file identifies a motor; it does not contain one.

The `<sim>` block. The final element carries exactly the three aerodynamic scalars that are `RocketModel` state: `dragCoefficient`, `referenceArea` (m^2), and the `referenceAreaOverridden` flag as a string boolean `model/DesignSerializer.cpp:233`. The comment above it fixes the scope: “Launch/environment options (velocity, angle, atmosphere, gravity, integrator) are not Rocket-Model state and are not serialized here — the CLI manages them as session config” `model/DesignSerializer.cpp:230`.

11.2 What is deliberately — and not so deliberately — absent

Three omissions are policy. Session and simulation configuration — initial velocity, launch angle, atmosphere, gravity, integrator, time step — is never written to a `.qrd`; the CLI’s help text repeats the contract: launch and atmosphere settings “are session config and are NOT saved in the design file” `cli/Repl.cpp:294`. A `.qrd` describes a rocket, not an experiment. Part identifiers reset on reload (the help note warns of it), and the design-matrix suite fingerprints reloaded trees with ids stripped `tests/DesignMatrixTests.cpp:308`. And `childRot` stays out of the schema until 6-DOF gives it a non-identity value to store. One omission is not policy but a silent loss:

setmass survives the session, not the file

The CLI's `setmass` applies a direct mass override to the root part — `rocket->setMass(m)` `cli/Repl.cpp:492` forwards to `topPart->setMass(m)` `model/RocketModel.h:87`, overwriting the part's geometry-derived mass `model/parts/Part.h:74`. But the writer reflects only the factory's geometry vocabulary: `PartParams` has no mass field, so `writeParams` `model/DesignSerializer.cpp:31` never sees the override, and on reload every part's mass is recomputed from geometry and density by its constructor. A user who trims mass with `setmass` and then `savedesign` gets a file that silently reloads at the geometry-derived mass — no warning at save time, no trace in the file, and no test pinning the behavior either way. **FAIL**

11.3 The save path

`DesignSerializer::save` is four steps `model/DesignSerializer.cpp:215`: stamp the version `model/DesignSerializer.cpp:220`, write the design name, recurse `writePart` from the top part (the root gets a placeholder link the loader ignores `model/DesignSerializer.cpp:223`, `model/DesignSerializer.cpp:94`), then emit the motor reference and `<sim>` block as two-space-indented XML `model/DesignSerializer.cpp:238`. One property-tree idiom in the children loop is load-bearing: siblings are appended with `add_child`, “NOT put” `model/DesignSerializer.cpp:115` — put would silently overwrite the previous same-keyed sibling, a trap the round-trip suite guards with a body tube carrying two children `tests/DesignPersistenceTests.cpp:59`. Nothing in the path resolves a pose or computes a mass: saving is pure reflection of authored intent.

11.4 The load path: build detached, install once

Loading is where the format's discipline pays off. The sequence in `DesignSerializer::load` `model/DesignSerializer.cpp:242`: parse (`read_xml` throws on an unreadable or malformed file `model/DesignSerializer.cpp:245`); gate on the major version before anything mutates `model/DesignSerializer.cpp:250`; recursively build the tree with `buildPart` `model/DesignSerializer.cpp:154`, which re-parses each `<params>` key-for-key against the writer `model/DesignSerializer.cpp:130` and hands them to `makePart` (throws on an unknown type or missing required field `model/parts/PartFactory.cpp:21`, `model/parts/PartFactory.cpp:73`); install with one `setRoot` `model/DesignSerializer.cpp:258`, which re-resolves the model's borrowed motor pointer and clears the reference-area override — a freshly installed airframe must not inherit a manual area `model/RocketModel.cpp:154`, `model/RocketModel.cpp:155`. The `<sim>` block is applied *after* `setRoot` precisely so a restored manual override wins over that reset `model/DesignSerializer.cpp:261`: the drag coefficient always applies, but the saved area applies only when the file's flag reads “true” `model/DesignSerializer.cpp:265`. A non-overridden file keeps the model's default area (the 38 mm disc, $1.134 \times 10^{-3} \text{ m}^2$ `model/RocketModel.h:141`), because the geometry-derived fallback the flag reserves semantics for has no production caller at this commit (Section 9). Last, the motor is re-resolved by name `model/DesignSerializer.cpp:271`.

Fail-safe in structure, fail-closed in vocabulary

The loader's own comment states the structural rule: “Build the whole geometry tree (detached) before touching the rocket, then install in one shot so a malformed file leaves the existing rocket untouched” `model/DesignSerializer.cpp:255`. Every throw in `buildPart` — unknown part type, missing geometry field, a `Motor` posing as a tree part, an unknown seat kind, a legacy `<offset>`, a rejected attach — fires while the new tree is still detached, and nothing after

setRoot can throw (only defaulted and optional reads remain), so loading is all-or-nothing; the version-rejection test proves it by checking that the boot placeholder body survives a failed load `tests/DesignPersistenceTests.cpp:204`. Within that structure the vocabulary is fail-closed: an unknown seat string throws “on the same path as makePart’s unknown-type rejection” `model/DesignSerializer.cpp:182`, pinned by the `seat="Bogus"` test `tests/DesignPersistenceTests.cpp:371` — the same posture as the placement diagnostics (Section 5): refuse to build a model from vocabulary the code does not understand, rather than simulate something silently wrong.

Two rejection semantics deserve explicit statement. First, *legacy 0.1 files have no migration path*: a 0.1 file stored per-child CM-to-CM `<offset>` vectors, and the reader throws “legacy 0.1 `<offset>` placement is no longer supported; this file predates the 0.2 `<link>` format and must be re-created” `model/DesignSerializer.cpp:196`, pinned by a hand-written 0.1 file in the tests `tests/DesignPersistenceTests.cpp:350`. Rejection stands in for migration — defensible for a format this young, but a decision to revisit once files accumulate outside the repository. (One nuance: the check runs per *child*, so strictly it is 0.1 *placements* that are rejected; a childless 0.1 root’s `<offset>` is never read, harmlessly.) Second, *attach failures convert silence into throws*: `Part::addChildPart` is a logged no-op on rejection, so the loader verifies every attach by child-count delta and throws if the count did not grow `model/DesignSerializer.cpp:204` — again before anything has mutated.

11.5 Round-trip guarantees

Fourteen serializer-level tests in `tests/DesignPersistenceTests.cpp` pin what the format preserves, with exact oracles wherever the mechanism permits; Table 13 summarizes them.

Two broader sweeps extend the guarantee. The CLI end-to-end test builds a rocket through real REPL (read-eval-print loop) commands, flies it, saves, clears, reloads, and re-flies: the `listparts` composite footer (mass and CG at $t = 0$) and the `apogee` line must be *byte-identical* across the round trip `tests/CliDesignCommandsTests.cpp:103`, `tests/CliDesignCommandsTests.cpp:106`, motor re-resolved by name `tests/CliDesignCommandsTests.cpp:100`. And the design-matrix suite runs save-clear-reload over every committed fixture its discovery admits (all but the poke-through offender, Section 11.6), requiring the exact composite footer and the id-stripped structure fingerprint `tests/DesignMatrixTests.cpp:417`, `tests/DesignMatrixTests.cpp:439`, `tests/DesignMatrixTests.cpp:440`; the seven `_multi` fixtures prove their baked-in motors re-resolve and fly to a string-equal apogee line `tests/DesignMatrixTests.cpp:482`, `tests/DesignMatrixTests.cpp:509`.

The format is ahead of its authoring tool

The serializer round-trips all three seat kinds through the shared string table `model/DesignSerializer.cpp:53`, and committed fixtures — including Listing 23 — carry `NestInBore` and `OnSurface` links. Yet the only placement verb the CLI can author is `abut(offset.z())` `cli/Repl.cpp:872`; `nestInBore` and `seatOnWall` exist in the placement vocabulary `model/parts/Placement.h:134` but have no REPL spelling (Sections 10 and 13). A design that nests a coupler can be loaded, simulated, re-saved, and visualized — it just cannot yet be *written* by any interactive front end. The on-disk format was finished ahead of its consumer.

11.6 The fixture corpus

Twenty-five `.qrd` files are committed under `tests/data/designs/` — and it is worth stating plainly that this is the *only* place designs ship: `data/` contains exactly one file, the `Aerotech.rse` motor data, and

Property pinned	Oracle
Geometry, tree structure, and multi-child siblings (the put-vs-add_child trap) tests/ DesignPersistenceTests.cpp:57	composite mass EXPECT_DOUBLE_EQ tests/ DesignPersistenceTests.cpp:94; CG per component to 10^{-9} tests/DesignPersistenceTests.cpp:97
Non-default links, all three seat kinds (Abut gap 0.012 m; NestInBore stations 1.0/0.0, gap 0.04 m; OnSurface parent station 0.30) tests/DesignPersistenceTests.cpp:380	seat EXPECT_EQ; stations and gap EXPECT_DOUBLE_EQ tests/ DesignPersistenceTests.cpp:420
Default-link elision tests/ DesignPersistenceTests.cpp:304	saved XML contains no <link substring tests/ DesignPersistenceTests.cpp:316; reloaded CG to 10^{-9} tests/DesignPersistenceTests.cpp:328
Motor identity, burning-mass model, and database miss tests/DesignPersistenceTests.cpp:138, tests/ DesignPersistenceTests.cpp:167	composite mass EXPECT_DOUBLE_EQ at $t \in \{0, 0.5, 5.0\}$ s (ignition, mid-burn, post-burnout) tests/ DesignPersistenceTests.cpp:161; thrust at 0.5 s EXPECT_DOUBLE_EQ after reload tests/ DesignPersistenceTests.cpp:282; a miss warns and loads the geometry motorless tests/ DesignPersistenceTests.cpp:180
Drag coefficient and reference-area override tests/ DesignPersistenceTests.cpp:100	$C_d = 0.55$ and 0.0421 m^2 exact tests/ DesignPersistenceTests.cpp:117; a non-overridden flag stays false tests/ DesignPersistenceTests.cpp:135
Version stamp; depth and degenerate shapes tests/ DesignPersistenceTests.cpp:288	raw file contains version="0.2" tests/ DesignPersistenceTests.cpp:298; childless root tests/DesignPersistenceTests.cpp:208 and four-deep chain (mass exact) tests/ DesignPersistenceTests.cpp:250 round-trip
Rejections are non-mutating: version "9.0", legacy <offset>, seat "Bogus" tests/ DesignPersistenceTests.cpp:189	each throws — asserted as <code>std::exception</code> for the version case tests/DesignPersistenceTests.cpp:201, <code>std::runtime_error</code> for the other two tests/ DesignPersistenceTests.cpp:350, tests/ DesignPersistenceTests.cpp:371; the boot placeholder body survives tests/DesignPersistenceTests.cpp:205

Table 13. Round-trip and rejection guarantees pinned by the fourteen `DesignPersistenceTests` cases. The CG (center-of-gravity) oracle is the load-bearing one for placement: “The CG match is what proves the placement (the body’s `StationLink`) round-tripped faithfully” tests/DesignPersistenceTests.cpp:91.

zero `.qrd` files. Any recollection of “bundled designs” in the data directory is wrong. The corpus is named `<class><diameter-mm><role>.qrd` across seven diameter classes (`micro13` through `xl75`) with three roles per class — `_basic` (nose, tube, three fins, no motor), `_shell` (thin-shell nose, four fins), and `_multi` (multi-child tree with a baked-in motor) — plus four one-offs: the three the README documents `tests/data/README.md:46`, and the deliberately self-intersecting `xl75_multi_pokethrough` `tests/DesignMatrixTests.cpp:329`. The README counts “24 rocket designs across 7 diameter classes” `tests/data/README.md:21`; the twenty-fifth, the poke-through offender, is excluded from fixture discovery because it must fail the overlap gate `tests/DesignMatrixTests.cpp:330` — it exists to prove the diagnostics fire, not to fly. These fixtures are working data: `design_matrix_tests` asserts the 24-file inventory is present and loads `tests/DesignMatrixTests.cpp:387`, then drives the flight matrix from it — each `_basic` airframe flies an ascending motor ladder spanning impulse classes 1/4A through M across 13–75 mm diameters, asserting nominal flights with apogee strictly monotonic in total impulse `tests/DesignMatrixTests.cpp:449`. The fast ctest entry flies one motor per class; the heavy-labeled entry re-runs the complete ladder under `QTROCKET_FULL_LADDER=1` (Section 12).

11.7 The motor database on disk: `.qmd`

The second persisted artifact needs only a paragraph here, since Section 6 owns the field semantics. `MotorModelDatabase` writes a `QtRocketMotorDatabase` root stamped `version="0.1"` `model/MotorModelDatabase.cpp:184` with one `<motor>` element per stored motor: every `MetaData` field as child elements (enums as their `str()` strings, with lossless `toEnum` inverses `model/MotorModelDatabase.cpp:190`), ejection delays as a CSV string, and the full thrust curve as `<thrustCurve><thrust time=".." force=".." />...</thrustCurve>` `model/MotorModelDatabase.cpp:222`. Unlike a `.qrd`, a `.qmd` carries the curve data itself, which lets the committed `tests/data/motors/estes_small.qmd` fixture pin online-sourced motors offline. The loader supplies per-field defaults, skips foreign nodes, and rebuilds each motor curve-first — “Order matters: `setMetaData()` triggers `computeMassCurve()`, which integrates the thrust curve, so the curve must be in place first” `model/MotorModelDatabase.cpp:303` — before merging into the name-keyed map `model/MotorModelDatabase.cpp:307`. Two contrasts with `.qrd` are instructive: the `.qmd` version attribute is written but never examined on load `model/MotorModelDatabase.cpp:249` (no gate, no migration hook), and loading is additive rather than replacing — the CLI’s `loaddb` reports new motors against the running total `cli/Repl.cpp:348`. The round trip reproduces the full 249-motor bundled import by count, with one motor pinned field-by-field exact — thrust samples and the derived mass curve included `tests/MotorDatabasePersistenceTests.cpp:297`.

11.8 Consumers and interoperability

At this commit `DesignSerializer` has exactly two production consumers. The CLI’s `savedesign` and `loaddesign` commands are thin wrappers over the two statics `cli/Repl.cpp:936`, `cli/Repl.cpp:957`, translating exceptions into the `ERR` protocol; after a successful load the REPL re-synchronizes its staged mirrors (drag coefficient, reference area, motor name) *from* the rocket `cli/Repl.cpp:964`. The standalone visualizer loads a `.qrd` passed on its command line through the same class `visualizer/VisualizerWindow.cpp:184`, so the viewer and the simulator cannot disagree about what a file means. The GUI — the front end a desktop user would expect to open and save files from — has no design persistence at all: a repository-wide search for `DesignSerializer` (and for any `.qrd` handling under `gui/`) finds call sites only in the CLI, the visualizer, and the tests (Section 10).

Interoperability with other tools’ design formats does not exist in any direction. QtRocket imports foreign *motor* data through three paths — RockSim `.rse`, RASP `.eng`, and the thrustcurve.org REST

API (Section 6) — but reads no foreign *design* format (no OpenRocket `.ork`, no RockSim `.rkt`) and exports its designs to none; even its own 0.1 files are locked out rather than migrated. Combined with the five-type part catalog (Section 4), this makes `.qrd` a closed world at this commit: complete and exactly tested within its boundary, and connected to nothing outside it. Section 13 carries that item in the gap inventory, and Section B sets it against the format surface a mature simulator ships.

12 Testing & infrastructure

This section describes the infrastructure a new engineer inherits at the pinned commit `254cd41`: six GoogleTest binaries registered with `ctest` as 246 entries, a small family of oracle patterns that the suites use instead of golden values, a committed fixture corpus the tests locate through compile-time absolute paths, a Clang-only `llvm-cov` coverage pipeline living in its own build tree, and one continuous integration (CI) workflow that drives five platforms through the same CMake presets developers use locally (Section 3 introduced the presets and the warning policy; this section covers the rest). The infrastructure is disciplined, but it has sharp edges, and each is marked where it lives: a coverage target that predates the newest test suite, one stale comment claiming tooling that does not exist, and an install rule that would ship the vendored dependencies alongside the application.

12.1 Six binaries, two registrations

Test sources live next to what they test: `model/tests/` and `sim/tests/` hold the per-layer unit suites, and the top-level `tests/` directory holds everything that crosses layers — integration, propagator behavior, and the two suites that drive the command-line interface (CLI). Table 14 lists the six binaries with their live per-case counts.

The link lines mirror the layering of Section 3: each binary links exactly the topmost layer it exercises. `model_tests` links `model` and `utils` `model/tests/CMakeLists.txt:22`; `sim_tests` links `sim` `sim/tests/CMakeLists.txt:12`; `propagator_tests` links `model` — which transitively pulls in `sim` and `utils` and supplies the out-of-line `Propagatable` constructor symbol `tests/CMakeLists.txt:37`; `integration_tests` links `qtrocket_core`, the same controller archive the executables link `tests/CMakeLists.txt:19`; and the two CLI-driving suites link the `cli` library, so they exercise the identical `Repl` object code users type into `tests/CMakeLists.txt:56`, `tests/CMakeLists.txt:78`.

Every binary is registered with `ctest` twice. First, per case: `gtest_discover_tests` runs each binary at build time and registers every `Suite.Case` as its own `ctest` entry `model/tests/CMakeLists.txt:29`, `sim/tests/CMakeLists.txt:18`, `tests/CMakeLists.txt:25` — 239 entries on this tree, which is what makes per-case filtering like `ctest -R PartTests` work. Second, in aggregate: one `add_test(NAME qtrocket_<suite>_tests ...)` per binary runs the whole executable as a single `ctest` entry `model/tests/CMakeLists.txt:31`, `sim/tests/CMakeLists.txt:20`, `tests/CMakeLists.txt:27`. Seven aggregates exist for six binaries — the design-matrix binary registers a second aggregate, as Section 12.3 explains — for a total of $239 + 7 = 246$ registered tests, counted live with `ctest -N` at the pinned commit.

Aggregate names are the selection contract

Both CI and the coverage pipeline run *only* the aggregates: CI filters with `ctest -R 'qtrocket_*` `.github/workflows/cmake-multi-platform.yml:74` and the coverage script defaults to `^qtrocket_.*tests$` `cmake/RunLlvmCoverage.cmake:14`. Note that `qtrocket_*` is a regular expression, not a glob — it means “qtrocket followed by zero or more underscores, anywhere in the name” — and it selects exactly the seven aggregates only because no discovered `Suite.Case` name

Binary	Cases	What it exercises	Defined at
<code>model_tests</code>	144	part composition and inertia, motors and thrust curves (with the RASP parser), the concrete part types, Barrowman aero, the part factory, placement types, the resolver sweep, the diagnostics gate	<code>model/tests/CMakeLists.txt:2</code>
<code>sim_tests</code>	27	the atmosphere models, <code>Environment</code> , the RK45 solver, spherical geoid and gravity, integrator selection	<code>sim/tests/CMakeLists.txt:2</code>
<code>integration_tests</code>	43	end-to-end physics through the controller, motor-database and design persistence, the placement-invariance gate, the poke-through regression	<code>tests/CMakeLists.txt:4</code>
<code>propagator_tests</code>	6	the propagator’s termination guards and trajectory statistics, driven by a mock <code>Propagatable</code>	<code>tests/CMakeLists.txt:32</code>
<code>cli_tests</code>	3	a scripted REPL build → save → reload → fly session	<code>tests/CMakeLists.txt:48</code>
<code>design_matrix_tests</code>	16	the committed fixture matrix, round-trip fidelity, and the 1/4A → M flight ladder	<code>tests/CMakeLists.txt:69</code>

Table 14. The six test binaries at the pinned commit, with per-case counts taken live from each binary’s `--gtest_list_tests` output (144 + 27 + 43 + 6 + 3 + 16 = 239 cases across 49 distinct gtest suites).

contains the substring `qtrocket`. The corollary binds new work: a future suite that forgets to register a `qtrocket_`-prefixed aggregate silently drops out of both CI and coverage, with no error anywhere.

The net has known holes, stated here and inventoried in [Section 13](#): no test binary covers `gui/` or `visualizer/` (the visualizer is also the only project target without a `qtrocket_enable_coverage` call — `visualizer/CMakeLists.txt` contains none, where every other target has one); and `utils/` has no test directory of its own `utils/CMakeLists.txt`, so `CurlConnection` is never executed by any suite. Each of these is an absence verified by searching the tree, not an impression.

12.2 Oracle patterns

The suites share a house style: assertions state *properties that must hold* — closed-form values derived independently, invariants, inequalities — rather than golden numbers snapshotted from the code under test. Four patterns recur, and each is worth learning by example because new tests are expected to follow them.

Independent numeric oracles. The inertia-tensor suite states the rule in its header: the tests pin the per-unit-mass helpers “against INDEPENDENT numeric oracles (disk / surface / volume-mesh integration) — never against the closed form they validate” `model/tests/InertiaTensorsTests.cpp:8`. The solid cone is checked against a Riemann stack of 400,000 thin disks `model/tests/InertiaTensorsTests.cpp:28`, the conical shell against a stack of hoops `model/tests/InertiaTensorsTests.cpp:55`, and the trapezoidal fin set against a brute-force volume mesh over N prisms `model/tests/InertiaTensorsTests.cpp:92` — the stated acceptance gate for the fin

set’s closed-form algebra. Agreement is demanded at 10^{-5} relative for the cone `model/tests/InertiaTensorsTests.cpp:157` and 0.5% for the mesh `model/tests/InertiaTensorsTests.cpp:209`; the textbook closed forms are then additionally pinned exactly, e.g. $I_{zz} = \frac{3}{10}R^2$ per unit mass at 10^{-15} `model/tests/InertiaTensorsTests.cpp:161`. The logic is the point: a closed form checked against its own algebra proves nothing, but an independent discretization of the defining integral converges to the same number only if both are right. (The tensors themselves are the subject of [Section 4](#).)

Literal-anchor regression pins. Where a bug was found and fixed, its exact numbers are frozen into an assertion with the derivation written beside it. The poke-through regression loads the one deliberately self-intersecting fixture through the real file reader and pins the diagnostic’s location and depth: `EXPECT_NEAR(d.zWorld, -0.26, 1e-9)` and penetration $0.0376 - 0.0395$ ($0.26/0.30$) ≈ 0.003367 m `tests/PokeThroughRegressionTests.cpp:63`, then asserts that the composite gate refuses the failed solve — `EXPECT_THROW` on `getCompositeI` `tests/PokeThroughRegressionTests.cpp:67` — the fail-closed posture of [Section 5](#), exercised end to end. The same file pins every resolved station of the whitepaper’s worked example to hand-computed literals `tests/PokeThroughRegressionTests.cpp:70`. The invariance suite carries two more: the solid nose cone’s local center of mass (CM) hand-pinned to -0.225 m at 10^{-12} , with the $\bar{h} - L = -3L/4$ derivation in the comment `tests/PlacementInvarianceTests.cpp:445`, and the fin-set CM recomputed from the fixture’s raw trapezoid geometry rather than read back through the part `tests/PlacementInvarianceTests.cpp:458`. Each pin defends itself against fixture drift — a guard assertion such as “fixture cone length changed — re-derive the pin” fails first if someone edits the geometry `tests/PlacementInvarianceTests.cpp:450`.

Invariance gates. The placement migration ([Section 5](#)) rests on the claim that it changed no physics, and the gate makes that claim falsifiable. A frozen baseline snapshots composite mass, CM, inertia, and every part’s resolved station for the 24-fixture corpus `tests/PlacementInvarianceTests.cpp:66`, formatted at `%.17g` so every double round-trips to the identical IEEE-754 value `tests/PlacementInvarianceTests.cpp:88`. The live tree must reproduce it: mass and inertia directly `tests/PlacementInvarianceTests.cpp:266`, CM and stations after the one known tip-datum shift `tests/PlacementInvarianceTests.cpp:286`, `tests/PlacementInvarianceTests.cpp:299`, all within 10^{-9} — a tolerance sized to admit the ULP-level drift of re-associated floating-point arithmetic and nothing larger `tests/PlacementInvarianceTests.cpp:151`, with a diagnostic test reporting the worst observed drift so the tolerance can be judged against reality `tests/PlacementInvarianceTests.cpp:328`. The baseline file declares its own epistemics: never regenerate it to make a failing migration pass, because “a diff here means the refactor changed the physics” `tests/data/placement-invariance-baseline.txt:4`. One quantity is deliberately excluded: the center of pressure, because the migration *corrected* a CM-contaminated legacy value, so baseline conformance there would pin the bug `tests/PlacementInvarianceTests.cpp:318`.

End-to-end flight sweeps. The design-matrix suite flies the whole engine — reader, resolver, motor database, propagator, both integrators — across seven reference airframes and asserts physical inequalities rather than trajectories. Per motor flown: apogee finite, above 1 m (“near-zero apogee implies a $t=0$ abort, not real flight” `tests/DesignMatrixTests.cpp:468`), below 10^6 m `tests/DesignMatrixTests.cpp:469`, and strictly increasing along the motor ladder — `EXPECT_GT(ap, prev)` `tests/DesignMatrixTests.cpp:470`. The atmosphere sweep flies each motor twice from identical state — vacuum with `setdrag 0`, then “US Standard 1976” with `setdrag 0.75` `tests/DesignMatrixTests.cpp:542` — and requires `EXPECT_LT(drag, vac)` `tests/DesignMatrixTests.cpp:552`: drag dissipates energy, so apogee under drag must be strictly lower.

The two integrators of [Section 7](#) must agree in vacuum to 2%: `EXPECT_NEAR(rk45, rk4, 0.02 * rk4)` [tests/DesignMatrixTests.cpp:599](#). Launch-pad behavior is pinned from both sides: the smallest motor in the corpus must lift the lightest airframe nominally [tests/DesignMatrixTests.cpp:610](#), while a 0.59 Ns motor under a 2000 kg/m³ tube must produce a “no liftoff” warning and an apogee below 1 m [tests/DesignMatrixTests.cpp:630](#). A units regression rides along: online-sourced motor mass must arrive in kilograms, not grams [tests/DesignMatrixTests.cpp:269](#) — the fix for a real bug in which a 6.1-gram motor loaded as 6.1 kilograms ([Section 6](#)). These oracles survive legitimate physics improvements — a better drag model still satisfies every inequality — but a sign error, a unit slip, or a broken ignition path fails them immediately.

Impulse classes and the motor ladder

Model-rocket motors are graded by total impulse $I_{\text{tot}} = \int F(t) dt$ (newton-seconds) into letter classes, each class doubling the ceiling of the one before: A up to 2.5 Ns, B up to 5, C up to 10, and so on up to M at 10240 Ns; fractional classes 1/4A and 1/2A sit below A. This is the standard NAR/TRA convention — domain background, not a table encoded in the repository. A “ladder” is a set of motors of one diameter arranged in ascending total impulse. For a fixed airframe in vacuum under fixed gravity, more delivered impulse means more kinetic energy at burnout and therefore a higher apogee, which is why apogee monotonicity along the ladder is a coarse but robust integral check on the entire motor $\rightarrow$ force $\rightarrow$ integrator chain.

12.3 The light/heavy registration split

The ladder sweeps are expensive by nature: they “fly every motor of each class’s full 1/4A $\rightarrow$ M ladder, in vacuum and through the atmosphere — with high-impulse motors coasting to tens of km this dominates the runtime” [tests/CMakeLists.txt:85](#). The build resolves the tension between coverage and iteration speed by registering the same binary a third time ([Listing 24](#)): a default (“light”) aggregate that runs all 16 cases, and a “heavy” aggregate that re-runs only the sweep cases — the `--gtest_filter=FlightMatrix.*:Atmosphere.*` filter selects 3 of the 16 — tagged with the ctest label `heavy` and the environment variable `QTROCKET_FULL_LADDER=1` [tests/CMakeLists.txt:96](#).

```
add_test(NAME qtrocket_design_matrix_tests COMMAND design_matrix_tests)

add_test(NAME qtrocket_design_matrix_heavy_tests
  COMMAND design_matrix_tests "--gtest_filter=FlightMatrix.*:Atmosphere.*")
set_tests_properties(qtrocket_design_matrix_heavy_tests PROPERTIES
  LABELS heavy
  ENVIRONMENT "QTROCKET_FULL_LADDER=1")
```

Listing 24. The double registration of the design-matrix binary [tests/CMakeLists.txt:94](#). The light entry runs the whole suite with one motor per class; the heavy entry re-runs just the two sweep suites under the full-ladder environment variable, labeled so ctest can include or exclude it by cost.

The switch lives in the test code, not the build ([Listing 25](#)): each of the seven `Ladder` entries pairs a reference airframe with its motors in ascending total impulse [tests/DesignMatrixTests.cpp:349](#), and `motorsToFly` returns either the full ladder or just its smallest rung:

$$\text{motorsToFly}(\text{lad}) = \begin{cases} \text{lad.motors} & \text{if QTROCKET_FULL_LADDER is set,} \\ \{ \text{lad.motors.front()} \} & \text{otherwise.} \end{cases}$$

The check is `std::getenv(...) != nullptr` [tests/DesignMatrixTests.cpp:377](#) — presence-only, so even `QTROCKET_FULL_LADDER=0` enables the full ladder. The light run keeps the coverage *topology* (every diameter class, every airframe, both integrators) while flying only one motor per class; under it the monotonicity loop degenerates to a single motor, so only the heavy entry actually exercises the apogee-ordering oracle [tests/DesignMatrixTests.cpp:461](#).

```
std::vector<std::string> motorsToFly(const Ladder& lad)
{
    if(std::getenv("QTROCKET_FULL_LADDER") != nullptr)
        return lad.motors;
    return { lad.motors.front() };
}
```

Listing 25. The entire light/heavy switch [tests/DesignMatrixTests.cpp:375](#). The build’s heavy ctest entry sets the environment variable; the code reads only its presence.

The selection algebra falls out of the label, and the build file spells it out [tests/CMakeLists.txt:92](#): `ctest -R "qtrocket_*` runs both entries (full coverage — 7 tests); `ctest -LE heavy` is the fast development loop (everything except the full-ladder sweeps — 245 of the 246 entries); `ctest -L heavy` runs only the sweeps (exactly 1 entry). Because the heavy entry is a strict re-run subset — three cases under a stronger environment — the plain aggregate filter gets full coverage without paying full-ladder cost on the thirteen cheap cases twice.

12.4 Test data and fixtures

Tests locate their committed data through two compile-time definitions: `QTROCKET_DATA_DIR` points at the bundled motor data in `data/` and `QTROCKET_TEST_DATA_DIR` at the fixture corpus in `tests/data/`, both expanded to absolute paths from `CMAKE_SOURCE_DIR` [tests/CMakeLists.txt:15](#), [tests/CMakeLists.txt:53](#), [tests/CMakeLists.txt:74](#).

Tests are cwd-independent; shipped binaries carry no source paths

The rationale is stated at the definition site: “Absolute paths so the tests run from any cwd” [tests/CMakeLists.txt:13](#) — a test binary invoked by ctest, an IDE, or by hand from any directory finds the same fixtures. The definitions are `PRIVATE` to the test targets, and a search across `model/`, `sim/`, `cli/`, and `gui/` finds no other consumer of either macro — so the executables users run embed no build-machine paths, and the application starts with an empty motor database (the bundled `data/Aerotech.rse` is test input, not runtime data; [Section 6](#)).

The corpus itself is small and self-describing [tests/data/README.md: 25](#) .qrd designs under `tests/data/designs/` — the 24-fixture invariance corpus spanning seven motor-diameter classes (13, 18, 24, 29, 38, 54, 75 mm, each airframe sized so its motor fits the body bore), plus `xl75_multi_pokethrough.qrd`, the deliberately self-intersecting offender — a motor-database fixture `motors/estes_small.qmd`, and the frozen invariance baseline. The corpus checks itself: `designFiles()` enumerates the directory at runtime, excluding the poke-through offender [tests/DesignMatrixTests.cpp:329](#), and a dedicated test pins the result against the 24-name expected list — failing with “fixture count drifted from the documented matrix” if anyone adds or removes a design without updating the manifest [tests/DesignMatrixTests.cpp:394](#).

12.5 Coverage

Coverage is Clang/LLVM source-based coverage, deliberately isolated from normal development. The `coverage-clang` preset inherits `debug-clang` but redirects to a separate `build-coverage/` tree and sets `QTROCKET_ENABLE_COVERAGE=ON` `CMakePresets.json:32`, so instrumented objects never pollute `build/`. The option is hard-gated to Clang: any other compiler hits a message(`FATAL_ERROR . . .`) naming the reason — the pipeline uses `llvm-cov`, whose instrumentation format GCC does not emit `CMakeLists.txt:101`. GCC remains fully supported for building and testing through the `debug-gcc/release-gcc` presets; it is only coverage that requires Clang. Tool discovery tolerates distribution-versioned binary names (`llvm-cov-21` down to `-17`) `CMakeLists.txt:105`.

Instrumentation is applied per target by one helper, `qtrocket_enable_coverage` `CMakeLists.txt:118`: a no-op unless the option is on, it adds the compile flags (`-O0 -g -fprofile-instr-generate -fcoverage-mapping` `CMakeLists.txt:108`) and — only when the target is not a static library — the matching link flags `CMakeLists.txt:121`. The distinction is load-bearing: static archives never see a link line, so the profiling runtime is linked once into each final test executable rather than uselessly requested on every archive.

The `coverage` target itself `CMakeLists.txt:256` runs the whole pipeline through a script-mode CMake driver, `cmake/RunLlvmCoverage.cmake`. In order: `ctest` runs the aggregate suites under `L` `LVM_PROFILE_FILE=<raw>/%p-%m.pro` `cmake/RunLlvmCoverage.cmake:33` — `%p` (process id) and `%m` (binary signature) give each test process its own profile file instead of clobbering one — and a nonzero `ctest` result is fatal `cmake/RunLlvmCoverage.cmake:39`: the pipeline is fail-closed, refusing to report coverage of a failing tree. The raw profiles are merged with `llvm-profdata` `merge -sparse` `cmake/RunLlvmCoverage.cmake:50`; `llvm-cov report` writes the text summary to `build-coverage/coverage/summary.txt` `cmake/RunLlvmCoverage.cmake:80`; and `llvm-cov show` emits the HTML report with per-region line counts and branch counts to `build-coverage/coverage/html/` `cmake/RunLlvmCoverage.cmake:98`. The ignore regex excludes `_deps`, the build trees, `tests/` itself, and the vendored `gui/qcustomplot.cpp` `CMakeLists.txt:262`, so the percentages measure project code exercised by the suites — not dependency code and not the tests' own bodies.

```
set(QTROCKET_COVERAGE_BINARIES
  "$<TARGET_FILE:sim_tests>"
  "$<TARGET_FILE:model_tests>"
  "$<TARGET_FILE:integration_tests>"
  "$<TARGET_FILE:propagator_tests>"
  "$<TARGET_FILE:cli_tests>")
```

Listing 26. The coverage target's binary list `CMakeLists.txt:248` — five binaries, not six. The same five names appear in the target's `DEPENDS` `CMakeLists.txt:267`, while the test-selection regex matches all seven aggregates `CMakeLists.txt:261`.

The coverage target predates the design-matrix suite

A verified latent defect: `design_matrix_tests` is missing from both the `-object` list and the `DEPENDS` of the `coverage` target (Listing 26), yet the selection regex `^qtrocket_.*tests$` `CMakeLists.txt:261` matches both of that binary's aggregate entries — the coverage stack landed in one commit (`2c9404b`) before the design-matrix suite existed, and the lists were never updated. Three consequences follow. In a fresh coverage tree, the target does not build the binary, the inner `ctest` run hits an unbuilt executable, and the script aborts at its fail-closed gate `cmake/RunLlvmCoverage.cmake:39` — unless the user has already built the whole tree. When the binary is

built, every coverage run pays for the heavy full-ladder sweep, which is slow by design `tests/CMakeLists.txt:85`. And although the binary is instrumented `tests/CMakeLists.txt:72` and its profiles are merged, it is never passed to `llvm-cov` as an `-object`, so code exercised *only* by it would be invisible in the report — benign today because it links the same `cli` and `qtrocket_core` archives whose mappings arrive through the other binaries, and its own translation unit sits under the ignored `tests/` path.

12.6 Continuous integration

CI is one GitHub Actions workflow, triggered on pushes and pull requests against `development` `.github/workflows/cmake-multi-platform.yml:12`. Its design statement leads the file: CI drives the build through the project’s presets, “so it exercises the same Ninja-based configure/build/test path that developers use locally instead of each platform’s default generator” `.github/workflows/cmake-multi-platform.yml:3`. There is no separate CI build system to drift out of sync — [Table 15](#) is simply the preset table of [Section 3](#) mapped onto runners.

Runner	Preset	Toolchain
ubuntu-latest	release-gcc	GCC
ubuntu-latest	release-clang	Clang
macos-latest	release-clang	AppleClang
windows-latest	release-msvc	MSVC (Ninja + <code>cl</code>)
ubuntu-latest (QEMU VM)	release-clang	FreeBSD 15.0, Clang/libc++

Table 15. The CI matrix `.github/workflows/cmake-multi-platform.yml:28`: four hosted-runner entries with `fail-fast` disabled — so one platform’s breakage does not mask the others — plus the separate FreeBSD virtual-machine job (lines 26 and 94 of the same workflow).

The matrix steps are few: install CMake and Ninja (Ninja is not preinstalled on the Linux/macOS images, and every preset requires it) `.github/workflows/cmake-multi-platform.yml:37`; install Qt 6.10.2 `.github/workflows/cmake-multi-platform.yml:42`; on Windows, set up the MSVC toolchain environment explicitly, because the Ninja generator does not run `vcvars` the way the Visual Studio generator implicitly did `.github/workflows/cmake-multi-platform.yml:48`; warm a ccache keyed on OS × preset, explicitly to cache the compiled FetchContent dependencies across runs `.github/workflows/cmake-multi-platform.yml:54`; then configure, build, and test through the presets `.github/workflows/cmake-multi-platform.yml:61`. The test step’s filter is the aggregate contract of [Section 12.1](#) — and because *both* design-matrix entries match it, every push and pull request runs the heavy full-ladder sweep in CI `.github/workflows/cmake-multi-platform.yml:74`. (The comment above that step still names only four suites — “(model, sim, integration, propagator)” `.github/workflows/cmake-multi-platform.yml:73`; it predates the CLI and design-matrix suites. Read the filter, not the comment.)

The fifth platform has no hosted runner, so the workflow boots a FreeBSD 15.0 virtual machine (QEMU, via `vmactions/freebsd-vm`) inside an `ubuntu-latest` runner `.github/workflows/cmake-multi-platform.yml:76`. The job is separate from the matrix because the toolchain setup differs: Qt comes from `pkg` and the base compiler is Clang with `libc++`, but the build itself reuses the `release-clang` preset unchanged — `pkg install -y cmake ninja git pkgconf qt6-base qt6-tools`, then the same three preset commands under `set -e` `.github/workflows/cmake-multi-platform.yml:101`. The hermetic curl configuration of [Section 3](#) pays off exactly here, and the workflow says so: “The hermetic curl config means OpenSSL + zlib (both in the FreeBSD base system) are the only external

libs curl needs” `.github/workflows/cmake-multi-platform.yml:99`. A 90-minute timeout bounds the run, since the VM has no ccache and compiles every `FetchContent` dependency from source `.github/workflows/cmake-multi-platform.yml:87`.

What CI does not run, verified by the single workflow file being the only one under `.github/workflows/`: no coverage job, no sanitizer job, no debug-preset job, no artifact upload, and no release or packaging step. [Section 13](#) carries the inventory.

12.7 Warning hygiene

[Section 3](#) stated the policy — `-Wall -Wextra -Wpedantic -Werror` on GCC and Clang `CMakeLists.txt:97`, `/W4 /WX` on MSVC `CMakeLists.txt:88` — and the detail that makes it livable is placement. The `add_compile_options` calls are directory-scoped and sit *after* all five `FetchContent_MakeAvailable` blocks but *before* the project’s own subdirectories and targets, so vendored dependencies compile with their own flags while every project target must be warning-clean; the history note records that the exemption was verified in `compile_commands.json` when the flags landed `TODO.md:23`. One vendored file lives inside a project target and is exempted per source: the third-party plotting widget `gui/qcustomplot.cpp` is compiled with `-w` — “qcustomplot throws some warnings, so silence them so we can turn on `-Werror` for our code” `CMakeLists.txt:152`.

On MSVC, `/WX` turns the compiler’s C4996 “use the `_s` variant” deprecation nags into hard errors, so the build defines `_CRT_SECURE_NO_WARNINGS` tree-wide, with the rationale recorded in place: those deprecations are non-standard editorializing against standard C++ functions like `std::getenv`, genuine `[[deprecated]]` uses still warn, and GCC/Clang remain the real safety net `CMakeLists.txt:89`. One line of that comment block has gone stale and should not be trusted: “We also run ASAN and clang-tidy for additional security” `CMakeLists.txt:93`. No `-fsanitize` flag, no `CLANG_TIDY` property, and no `.clang-tidy` configuration exists anywhere in the project’s CMake, presets, or CI — each a failed search of the tree. The warning flags are real and enforced; the sanitizer claim is a comment about infrastructure that does not exist.

12.8 What the infrastructure does not do

Distribution is where the infrastructure stops. The project contains exactly one `install()` rule, for the GUI executable alone — `install(TARGETS qtrocket BUNDLE DESTINATION . LIBRARY DESTINATION ...)` `CMakeLists.txt:308`. Nothing installs `qtrocket-cli`, `qtrocket-visualizer`, the motor data, or any header, and the one rule works on Linux only through side effects: it names no `RUNTIME DESTINATION`, so the executable lands in `bin/` by CMake’s implicit default, and `CMAKE_INSTALL_LIBDIR` is defined only because some dependency’s build includes `GNUInstallDirs` — no project CMake file includes it (a failed search). There is no CPack configuration anywhere in the project’s CMake files, hence no installers or packages of any kind.

cmake --install would ship the vendored dependencies

None of the five `FetchContent_Declare` blocks passes `EXCLUDE_FROM_ALL` `CMakeLists.txt:15`, so the generated root install script includes the `_deps` sub-installs for GoogleTest, jsoncpp, curl, Eigen, and Boost alongside the project’s own rule — observable in the generated `build/cmake_install.cmake`. Running `cmake --install build` today would install GoogleTest, curl, and Boost headers and archives into the prefix next to `qtrocket`. The install rule is a template stub, not a distribution story; treat it accordingly.

Internationalization is dead scaffolding from the same template: the build requires `LinguistTools`

`CMakeLists.txt:139`, lists the single translation source `qtrocket_en_US.ts` `CMakeLists.txt:145` among the GUI sources `CMakeLists.txt:187`, and runs `qt_create_translation(QM_FILES ...)` `CMakeLists.txt:201` — but `QM_FILES` appears nowhere else in any CMake file, so the compiled catalogs are never attached to a target or installed; the roadmap parks translations explicitly `TOD0.md:79`. The macOS bundle identifier is likewise still the template placeholder `my.example.com` `CMakeLists.txt:301`.

The shape of the whole is easy to state. Everything between a developer’s editor and a green five-platform CI run is built, pinned, and self-checking: six suites whose assertions are oracles, a fixture corpus that audits its own manifest, a coverage pipeline that refuses to report on a failing tree, and a warning wall that vendored code cannot breach. Everything between that green run and a user’s machine — installers, packages, translations, a runtime motor database — is not built yet, and [Section 13](#) inventories it with the rest of the unfinished work.

13 Yet to be implemented

Every expository chapter of this document has marked its unbuilt edges in place — a clause here, a callout there — and deferred the consolidated view to this chapter. This is that view: the complete inventory, at pinned commit `254cd41`, of what QtRocket has built but not wired in, what it has wired in with named gaps, and what it has not started. The companion Architecture Review ([docs/ArchitectureReviewLatex/](#)) calls the first category “a ring of machinery that is built, unit-tested, and consumed by nothing,” and that ring is the defining feature of the project’s frontier: QtRocket builds and tests its seams before their consumers exist, so most of what is missing is not missing code — it is a missing caller. The tree says so itself, in the aerodynamics suite’s own words: “nothing in production consumes this in P2 – these tests pin the seam for P5” `model/tests/AeroTests.cpp:18`.

This chapter is written as a map for the next engineer, not as a critique. Each entry records three things: what exists, where it lives, and what consuming it would require. The ordering within each category is roughly by size of the waiting machinery; the *ranking* by user impact belongs to the Architecture Review and is reproduced, with attribution, in [Section 13.5](#).

The status vocabulary, and how absence is grounded

Three status words partition this inventory, with the companion review’s exact senses. *Dormant*: built and tested, with zero production consumers — unreachable from any shipping code path. *Partial*: reachable from a production path, but with named gaps. *Absent*: no trace in the tree — and every absence claim below is grounded in a failed search of the source, not an impression. The distinction is a planning tool, not taxonomy for its own sake: consuming a dormant seam is weeks of work where building an absent subsystem is months.

13.1 Dormant seams

Seven seams sit finished, most of them unit-tested, with no production caller. Each names, implicitly or in an in-source comment, the work it is waiting for.

The Barrowman layer and `aeroData`. The largest dormant seam is the entire center-of-pressure (CP) pipeline. Per-part Barrowman contributions exist for the cone and the fin set, expressed as a normal-force slope `cnAlpha` and a slope-weighted CP moment `cnAlphaXcp` `sim/Aero.h:32`; `Part::getCompositeAero()` folds them additively over resolved placement onto the nose-tip datum `model/parts/Part.cpp:231`, the same datum as the composite center of gravity (CG), so the static margin is literally `cp() - cg()` — a subtraction by construction, and the seam’s own documentation states that intent `sim/Aero.h:25`. All of it is unit-tested (`model/tests/AeroTests.cpp`,

`model/tests/NoseConeTests.cpp`) and none of it is called from production: the only callers of `getCompositeAero` are three test files `tests/PlacementInvarianceTests.cpp:106`, and the `aeroData` member staged for the result on the model-sim bridge is default-constructed and never read `model/Propagatable.h:61`, kept compiling only by a back-compatibility alias `sim/Aero.h:66`. Consuming the seam requires an angle-of-attack computation (none exists anywhere in the tree), a normal-force or restoring-moment term in the force path, and a display surface; the cheapest first consumer is a live CP/CG/static-margin readout, which needs no new physics at all. Two latent defects gate that consumption and are covered with the 6-DOF cluster below ([Section 13.2](#)); the full analysis of the layer is [Section 9](#)'s.

The wind model. `WindModel` is the purest specimen of the pattern: compiled into the `sim` library `sim/CMakeLists.txt:24`, its entire behavior is `return Vector3(0.0, 0.0, 0.0);` `sim/WindModel.cpp:18`, and a repo-wide search finds no instantiation, no call site, and no slot for it in `Environment` (`sim/Environment.h` holds gravity, atmosphere, and geoid members only). The drag law consumes ground-relative velocity directly `model/RocketModel.cpp:88`, so airspeed equals ground speed by construction. Consuming it requires an `Environment` slot and a relative-velocity substitution in `getForces()` — but, as the review observes, wind is only worth plumbing after attitude exists: a point mass cannot weathercock, so wind would shift drag without producing any behavior wind exists to model.

Ignition offsets and ejection delays. The motor subsystem carries two time-related seams. First, arbitrary ignition times: `MotorModel::startMotor(double startTime)` accepts any start time `model/MotorModel.h:328`, and the mass model honors the offset by evaluating at `simTime - ignitionTime` `model/MotorModel.cpp:36` — but `getThrust` passes absolute simulation time through to the curve `model/MotorModel.cpp:82`, and `ThrustCurve::setIgnitionTime`, the piece that would shift the thrust samples, has zero call sites, with its `maxTime` adjustment commented out inside the definition `model/ThrustCurve.cpp:44`. The asymmetry is masked because the only production ignition is `startMotor(0.0)` at launch `model/RocketModel.cpp:108`; any future air-start would burn propellant on one clock and produce thrust on another until the two paths are unified ([Section 6](#)). Second, ejection delays: both file loaders parse them — the `RockSim` loader splits the CSV list `model/RSEDatabaseLoader.cpp:59`, the `RASP` loader splits on - with 1000 as the plugged sentinel `model/RASPLoader.cpp:119`, and the `thrustcurve.org` client leaves them an explicit TODO `model/ThrustCurveClient.cpp:198` — they are stored `model/MotorModel.h:303` and round-tripped through the motor database XML, and no simulation, GUI, or CLI code reads them. Their consumer is the absent recovery-event system ([Section 13.4](#)).

deriveReferenceAreaFromGeometry(). The geometry-derived reference area is built end to end — `RocketModel` exposes it `model/RocketModel.cpp:48` over `Part::maxFrontalReferenceArea()`'s widest-frontal-disc convention `model/parts/Part.cpp:251` — and has no production caller. The header's promise that the geometry default applies whenever the user has not overridden the area `model/RocketModel.h:76` is implemented by no code path: design load applies the saved area only when `referenceAreaOverridden="true"` `model/DesignSerializer.cpp:265`, and all 25 bundled fixtures say "false", so every loaded design flies on whatever scalar was last staged — by default the 38 mm disc. Drag therefore does not scale with airframe class; [Section 9](#) carries that verified defect in full. Consumption here is one call: wire the derivation into the non-overridden branch, and the seam's convention (fins deliberately report the body disc, not tip extent, `model/parts/FinSet.h:71`) already matches standard practice.

radialSeamCheck (the Layer-1 diagnostic). The placement design specifies three diagnostic layers; the shipped gate wires in only the second, the envelope sweep ([Section 5](#)). The first — a per-seam, $O(1)$ radius-compatibility check dispatched on seat kind — is implemented `model/parts/`

`Placement.cpp:56` and unit-tested `model/tests/ResolverSweepTests.cpp:194`, and nothing in production calls it: the placement cache’s re-resolve runs the sweep alone `model/parts/Part.cpp:170`. The function even documents its own missing caller — its `zWorld` field is left parent-local behind the comment “the root-frame z is added once a pose is known” `model/parts/Placement.cpp:87`, a promise no caller exists to fulfill. Consuming it means invoking it per attachment during resolve, converting a wrong-radius mate from a sweep-time overlap (or a silent pass) into an immediate, located seam diagnostic.

Per-sample RockSim mass and CG histories. The bundled motor file ships manufacturer-measured mass and CG values on every thrust sample `data/Aerotech.rse:19`, and the loader reads only `t` and `f` `model/RSEDatabaseLoader.cpp:91`. QtRocket instead always reconstructs burn mass with the 128-point constant- I_{sp} depletion curve `model/MotorModel.cpp:132` and holds the motor’s own center of mass fixed `model/parts/Motor.cpp:12` — the fallback a mature tool uses when a file carries no measured data, applied unconditionally. Consuming the discarded columns requires a per-motor measured mass curve in place of the reconstruction and intra-motor CG travel during the burn; the latter also depends on motor placement becoming real (Section 13.3), since a CG history acting at the airframe CM would move the right mass at the wrong station.

The `isp` member. The smallest seam, one line: specific impulse is computed whenever motor metadata is installed, $I_{sp} = I_{tot} / (g_0 \cdot m_{prop})$ `model/MotorModel.cpp:130`, stored `model/MotorModel.h:340`, and read by nothing. It is the natural input for a future propellant-performance display or a mass-flow-consistency check on ingested files; today it is dead weight computed on every metadata write.

13.2 The 6-DOF cluster

The largest coordinated group of dormant machinery is the scaffolding for six degrees of freedom (6-DOF) — rotational state, torques, and the inertia pipeline that would drive them. Five pieces exist, typed and in place, and none is consumed:

- **The torque interface.** `getTorques(t)` is a pure virtual on the model-sim bridge `model/Propagatable.h:30`; `RocketModel`’s override returns the zero vector unconditionally `model/RocketModel.cpp:94`. A repo-wide search finds no call site in `sim/`, `gui/`, `cli/`, or the controller — only the declaration, the zero override, and a test mock `tests/PropagatorTests.cpp:43`.
- **The inertia accessor.** `getCompositeInertiaTensor(t)` is declared `model/Propagatable.h:33` and implemented as a pass-through to the part tree `model/RocketModel.cpp:43`, and no production code calls the virtual; the per-step recording reaches the tree directly through `writeMassProperties` `model/RocketModel.cpp:59`. The composite $I(t)$ and $CG(t)$ that Section 4 shows the tree computing so carefully have exactly one time-varying consumer today: the per-step state snapshot (the CLI’s `listparts` also prints a static $t = 0$ composite).
- **The quaternion state slots.** `StateData` carries `orientation` and `orientationRate` quaternions, a direction-cosine matrix, and 3-2-1 Euler angles `sim/StateData.h:46`; no code anywhere writes or reads them. Both quaternions are initialized to the all-zero quaternion — not the identity rotation, not any rotation — and the members’ “(vector, scalar)” annotation sits one constructor-order hazard away from Eigen’s (w, x, y, z) value constructor (Section 7). The first consumer inherits an invalid orientation plus a documented convention trap.
- **The orientation integrator.** The rotational half of the propagator exists as a commented-out member whose interface is already worked out (Listing 27): the same `DESolver` machinery, instantiated for quaternions, with a time-keyed callback so torques are sampled at each stage’s

node time exactly as forces are. The solver templates admit the quaternion instantiation by `static_assert sim/RK45Solver.h:51`, though only `Vector3` instantiations exist `sim/Integrator.h:57`, and the stage arithmetic would not compile for `Eigen::Quaterniond` without adaptation — the template is a staged intention, not working rotational machinery.

- **The recorded seam.** Every accepted step snapshots mass, CG, and the full inertia tensor into the state history `sim/Propagator.cpp:132`, and the header names the purpose: these are “the seam the 6-DOF rotational ODE will read” `sim/StateData.h:59`. On the placement side, every `StationLink` carries a `childRot` quaternion reserved for 6-DOF, identity in 3-DOF `model/parts/Placement.h:55`, deliberately left out of the design file format while it is identity by construction `model/DesignSerializer.cpp:85`.

```
std::unique_ptr<sim::Integrator> linearIntegrator;
// The 6-DOF orientation integrator will use the same DESolver<Quaternion> interface, with a
// time-keyed ODE callback so getTorques() can be sampled at each stage's node time like getForces().
// std::unique_ptr<sim::RK4Solver<Quaternion>> orientationIntegrator;
```

Listing 27. The rotational half of the propagator, present as a commented-out member with its design already stated in place `sim/Propagator.h:100`. The dormant seam names its own consumer: a quaternion ODE fed by `getTorques()` at stage node times.

Two defects gate 6-DOF consumption

The Architecture Review names two latent defects that must be fixed *before* this cluster gains a consumer, because both leave the 3-DOF trajectory untouched and become flight errors the moment rotation is integrated. First, the fin set’s chordwise frame is mirrored: its mass and CP math use aft-positive from-root-leading-edge distances directly as $+z$ offsets `model/parts/FinSet.cpp:57`, placing the composite fin CP aft of truth and the fin-set mass forward of truth, and the unit tests pin the mirrored values as regression anchors `model/tests/FinSetTests.cpp:207` — the suite enforces the defect rather than detecting it (Section 9). Second, the composite-inertia walk deliberately omits the RIR^T child-tensor rotation — identity in 3-DOF, skipped “to stay bit-stable” — and the in-source TODO already prescribes the regression test that must accompany its addition `model/parts/Part.cpp:217`. Fixed in that order, the fin-set mirror is flushed out while it is still a display bug rather than a flight bug.

13.3 Partial areas

Six areas are reachable from production paths today, each with a named gap.

Placement authoring. The model and the serializer support the full placement vocabulary — three seat kinds `model/parts/Placement.h:32`, arbitrary fractional stations, and gaps, all round-tripped through the design file `model/DesignSerializer.cpp:54` — but the only authoring surface, the REPL’s `addpart`, attaches everything with `abut(offset.z())` `cli/Repl.cpp:872`, and parses `x=/y=` offsets only to discard them `cli/Repl.cpp:189`. `Abut` is the only seat kind reachable from any front end; `NestInBore` and `OnSurface` placements exist today only in hand-edited `.qrd` files and C++ test fixtures. The file format is ahead of the authoring vocabulary (Section 5).

The GUI design surface. At the pinned commit the main window’s design widgets are constructor-only stubs — `RocketTreeView` is an empty constructor `gui/RocketTreeView.cpp:3`, `RocketModelerView` the same `gui/RocketModelerView.cpp:3` — and the File menu’s

New/Open/Save/Save As/Close actions ship `enabled="false"` `gui/MainWindow.ui:117`; exactly three actions are wired: Quit, Save Motor Database, and About `gui/MainWindow.cpp:42`. Every capability the widgets promise exists one layer down in the CLI (Section 10). Two commits on the unmerged `RocketTreeViewImplementation` branch add a read-only part tree and `.qrd` open/save wiring; that work is in progress and is not reviewed here.

GUI responsiveness and change notification. Everything the GUI does, it does on the Qt event-loop thread: the launch slot flies the whole simulation synchronously inside the button handler `gui/CannonballTab.cpp:116`, and one motor-search click can issue up to 1 + 100 blocking HTTP round trips (Section 10) — there is no `QThread`, `QtConcurrent`, or `QNetworkAccessManager` anywhere under `gui/`. In the other direction the model cannot tell the GUI anything: no structure-change notification out of `RocketModel` exists at the pinned commit (a failed repo-wide search for any callback or signal), so a future part-tree widget has nothing to subscribe to; the unmerged `RocketTreeViewImplementation` branch prototypes such a callback.

Analysis and plotting. The GUI's analysis window draws exactly three fixed graphs — altitude `gui/AnalysisWindow.cpp:49`, vertical velocity `gui/AnalysisWindow.cpp:74`, and the motor thrust curve `gui/AnalysisWindow.cpp:86` — and never surfaces the flight statistics or the typed termination reason the controller exposes `QtRocket.h:55`: a user whose run aborted for no-liftoff gets an altitude plot and no explanation. The CLI prints the statistics and writes a fixed fourteen-column CSV `cli/Repl.cpp:104`; there is no configurable plotting, no event annotation, and no stability output on any surface.

The motor database ships empty. Exactly one motor file is bundled, `data/Aerotech.rse`, and only the test binaries can find it, through the `QTROCKET_DATA_DIR` compile definition `tests/CMakeLists.txt:16`; neither front end loads any motor data at startup (a failed search — no production reference to the data directory exists). Every GUI or CLI session begins with an empty database and a manual import (Section 6).

Single motor, single stage. One motor is a structural assumption: the model borrows a handle to the *first* `Motor` node found by depth-first search `model/RocketModel.cpp:13`, a second motor is silently ignored for thrust while still contributing mass, and `Stage.h` is a commented-out include under the comment “Not yet” `model/RocketModel.h:22`. The one motor is also pinned to the wrong place: it is attached CM-to-CM with the airframe under an offset documented “Zero today” with no setter `model/RocketModel.h:131`, so the CG-travel-during-burn machinery of Section 4 acts at the airframe CM rather than in an aft mount, and the motor's placement link is never persisted `model/DesignSerializer.cpp:114` — lossless today only because the offset cannot move.

13.4 Absent subsystems

The following have no trace in the tree. Each claim rests on a failed search of the source at the pinned commit, not on an impression.

Recovery and flight events. Nothing changes at apogee. Once off the pad, the only phase logic in the simulator is the terminate predicate — descending below the launch site `model/RocketModel.cpp:62` — so every flight ends ballistically, descending under the ascent's drag law; apogee exists only as a running maximum in the statistics `sim/TrajectoryStatistics.h:29`. There is no deployment event, no parachute or streamer model, no drag switch at any altitude. The ejection delays every loader ingests (Section 13.1) are this subsystem's staged input, and the review points at the typed termination taxonomy `sim/Propagator.h:35` as the natural seed of an event vocabulary — liftoff, burnout, apogee, ejection, touchdown.

Drag build-up, Mach, and Reynolds. The per-part drag field exists and every producer returns zero: `cd` stays 0 on the body tube — “skin friction over `getWettedArea()` is its eventual contribution” `model/parts/BodyTube.cpp:41` — on the nose cone `model/parts/ConicalNoseCone.cpp:67`, and on the fin set `model/parts/FinSet.cpp:89`; the wetted-area getters staged for the purpose have no production consumer `model/parts/BodyTube.h:51`. Above them, the atmosphere interface exposes speed of sound and dynamic viscosity `sim/AtmosphericModel.h:17`, and the US Standard Atmosphere implements both correctly `sim/USStandardAtmosphere.cpp:143` — those two overrides have no caller of any kind, production or test. No Mach number or Reynolds number is computed anywhere (failed search); the single user-set C_d is speed-independent, so an upper-class flight passes Mach 1 under subsonic-constant drag without comment (Sections 8 and 9).

Staging, clusters, and flight configurations. No `Stage` type exists anywhere under `model/` (the commented include above is the only mention); there is no motor cluster support — the single-motor DFS makes a second motor physically inconsistent rather than illegal — and no concept of named flight configurations pairing a design with alternative motors.

Launch rod, launch site, and Coriolis. The only pad modeling is a hold clamp — the rocket is pinned at rest at $z = 0$ until the first powered step lifts it `sim/Propagator.cpp:111` — and a no-liftoff abort `sim/Propagator.cpp:146`. There is no guided rod phase, rod length, or departure-velocity constraint; since attitude does not exist, a rod could not constrain the thrust direction anyway. There is likewise no launch-site concept: the geoid ignores its latitude/longitude arguments `sim/SphericalGeoidModel.cpp:18`, the spherical gravity model queries it once with a literal (0, 0) placeholder `sim/SphericalGravityModel.cpp:20`, altitude above the pad is implicitly altitude above sea level in every atmosphere lookup, and no Coriolis or Earth-rotation term appears anywhere (failed search).

Foreign design formats. QtRocket imports foreign *motor* data through three paths (Section 6) and foreign *design* data through none: there is no OpenRocket `.ork` reader or writer, no RockSim `.rkt` design import, and no export beyond the trajectory CSV `cli/Repl.cpp:104`. Its own legacy 0.1 design files are rejected fail-closed with “must be re-created” rather than migrated `model/DesignSerializer.cpp:194`. Combined with the five-type part catalog (Section 4), the `.qrd` format is a closed world at this commit (Sections B and 11).

Optimization and dispersion. No optimizer, no Monte-Carlo or dispersion analysis; searches for either return nothing. The deterministic core and the scriptable REPL make external parameter sweeps feasible today — the design-matrix suite is exactly such a sweep run as a test (Section 12) — but nothing in the product performs one.

Packaging and internationalization. Distribution is a single install rule covering the GUI target only `CMakeLists.txt:308`; no CPack configuration exists (failed search), the macOS bundle identifier is the Qt template placeholder `CMakeLists.txt:301`, and the CLI, the visualizer, and the motor data are not installed at all. Internationalization is dead scaffolding: the build generates translation catalogs nothing references `CMakeLists.txt:201`, and the GUI loads a translator from a resource prefix that is never populated `gui/GuiRunner.cpp:40`, under a TODO admitting English-only support `gui/GuiRunner.cpp:34`.

13.5 A suggested order of work

Ranking this inventory is deliberately not this document’s job; the Architecture Review does it, and its recommendation is reproduced here because it follows directly from the dormant-versus-absent distinction above. The review’s sequencing (`docs/ArchitectureReviewLatex/sections/11-roadmap.tex`) wires seams to consumers in dependency order:

1. **Consume the Barrowman seam.** Fix the fin-set chordwise mirror, add the RIR^T rotation with the pinned-identity regression test the TODO already prescribes [model/parts/Part.cpp:220](#), and give `deriveReferenceAreaFromGeometry()` its caller. The payoff is a CP/CG/static-margin readout in the CLI and GUI — no new physics, the first production exercise of `getCompositeAero`, and the sign defect flushed while it is still a display bug rather than a flight bug.
2. **Recovery and flight events.** Independent of rotation: grow the termination taxonomy [sim/Propagator.h:35](#) into an event vocabulary, consume the ejection delays, and switch drag at deployment. This replaces the ballistic terminate with a full flight — and a minutes-long parachute descent forces the simulation off the GUI thread, which in turn makes the controller's initialization race ([Section 3](#)) worth fixing first.
3. **Six degrees of freedom, then wind.** Reinstate the orientation integrator ([Listing 27](#)), fixing the quaternion zero-initialization and the constructor-order hazard before any consumer reads them; align thrust with the body axis; derive restoring torques from step 1's aero profile. Only then plumb the wind model — a point mass cannot weathercock, so connecting it earlier would shift drag without producing any behavior wind exists to model.
4. **The GUI design editor.** Pure consumption of a proven core: the CLI already demonstrates the full author-persist-fly loop against the same API, the 25 committed `.qrd` fixtures are a ready test corpus, and the in-flight branch supplies the tree view. What remains is the property panel, part add/remove/edit, and closing the CLI's seat-and-station authoring gap.

Consumption before construction (the review's verdict)

“Almost nothing in Tiers 1–2 requires new architecture. The aero profile, the torque interface, the quaternion state, the ejection delays, the termination taxonomy, the authoring API, and the serializer all exist and are tested. The highest-leverage sequencing wires existing seams to consumers in dependency order, fixing the two known latent defects before their numbers become user-visible” ([docs/ArchitectureReviewLatex/sections/11-roadmap.tex](#)).

Staging, clusters, the multi-host joint graph, and packaging stay off this critical path by design: none blocks the four steps, and each gets cheaper after them — staging needs the event system, clusters need the joint graph.

13.6 The placement system's forward hook

The Part Placement whitepaper ([docs/PartPlacementDesignLatex/](#)) closes with the same forward posture this chapter documents, and one of its designed pieces is still unbuilt: Layer 3 of its diagnostics ladder, the degrees-of-freedom accounting. In 3-DOF a single axial `StationLink` fully pins a child; under 6-DOF one link leaves a rotational degree of freedom unpinned, and the design specifies a cheap detector that warns when a child has only one link under free orientation ([docs/PartPlacementDesignLatex/sections/06-diagnostics.tex](#)). No such code exists in `model/parts/` — verified by reading `model/parts/Placement.cpp` and `model/parts/Part.cpp` end to end — so the detector is designed, not shipped: absent, with its specification already written.

The rest of the placement system's 6-DOF path is the inventory's most finished forward hook. The schema is already shaped for rotation — `childRot` rides in every stored link as an identity quaternion [model/parts/Placement.h:55](#), `Pose::compose()` is the full rotate-then-translate rigid-transform law that degenerates to vector addition under identity [model/parts/Placement.h:81](#) — so the design document reduces 6-DOF placement to three localized edits: stop forcing `childRot` to identity at authoring and load time, add the one RIR^T line to the composite-inertia pass, and feed the integrated attitude

into the resolver’s root pose ([docs/PartPlacementDesignLatex/sections/09-sixdof.tex](#)). Zero schema change, zero signature change, no second resolver.

That is the shape of this whole chapter, and the review closes with the image that fits it: the hard structural machinery — composite inertia, placement resolution, Barrowman composition, adaptive integration, the termination taxonomy — is built, tested, and waiting behind seams that name their consumers. In the review’s words, “the rooms are framed and inspected; the remaining work is opening the doors between them” ([docs/ArchitectureReviewLatex/sections/11-roadmap.tex](#)).

A OpenRocket as comparison baseline

The four appendices that follow read QtRocket against OpenRocket: a sixteen-area feature matrix ([Section B](#)), a method-by-method comparison of the flight physics ([Section C](#)), and a side-by-side reading of the two programs as software ([Section D](#)). This appendix frames all three. It states why OpenRocket is the baseline and what that framing does and does not license, describes OpenRocket itself in one page, and names the four philosophical differences between the projects that, more than any difference in effort, explain the shape of what the comparisons find.

A.1 Why this baseline, and what it is not

OpenRocket is the baseline for one reason: it is the de facto standard for what a mature open-source model-rocket simulator offers. No other freely available tool combines a seventeen-year development history, a published methods document with a validation chapter, a six-degree-of-freedom flight model, and a component catalog broad enough to describe most real airframes. Measuring against anything less would flatter QtRocket; measuring against a commercial tool would put the methods out of reach behind closed source. The framing is inherited from the companion Architecture Review ([docs/ArchitectureReviewLatex/](#)), which pinned it precisely, and it bears restating in this document’s own terms: QtRocket is an independent project with similar goals. It is not a port of OpenRocket, not a reimplementation, and at no point in this document is OpenRocket treated as a specification to clone. Where the two projects agree (SI units throughout, a nose-tip datum, a headless core beneath the GUI), they arrived by parallel reasoning; where they disagree, the disagreement is usually deliberate and the owning body section says why.

Baseline, not specification

The comparison measures distance, not conformance. A row that reads **Absent** in the feature matrix is a statement of how far QtRocket currently is from a mature tool’s surface, never a defect report against requirements QtRocket promised to meet; equally, an area where QtRocket is ahead of the baseline is a local engineering fact, not a claim of superiority over a finished product. OpenRocket calibrates the map. It does not draw it.

The two sides of every comparison also carry different kinds of evidence, and the distinction holds throughout the appendix block. QtRocket claims cite file and line into the pinned commit [254cd41](#), the same evidentiary standard as the body of this document. OpenRocket claims are researched from primary sources — the [openrocket/openrocket](#) repository at tag [release-24.12](#) and branch [unstable](#) (examined 2026-07-07), the published technical documentation, and the project’s developer documentation — and carry their sources in footnotes. They are researched, not adversarially line-verified the way QtRocket claims are, and the appendices that lean on them say so.

A.2 OpenRocket in brief

OpenRocket is a GPLv3 Java application¹ that began as Sampo Niskanen’s 2009 master’s thesis at Helsinki University of Technology and has been in continuous development since. The thesis, updated and extended, remains published as the *OpenRocket Technical Documentation*, version 13.05 (2013)² — still the latest revision as of this writing, and therefore the authoritative statement of the methods while being a decade stale in places. The staleness is real and flagged where it matters: techdoc §4.2 still describes a flat-Earth model with no Coriolis effect, whereas the current source offers flat, spherical, and WGS84 geodesy, with Coriolis acceleration in every force evaluation under the latter two.³ Section C cites the techdoc for the methods and the source wherever the code has moved past it.

The current stable release is 24.12 under a year.month scheme; the final 24.12 GitHub release is dated July 2025, and development happens on the repository’s default `unstable` branch.⁴ The runtime requirement is Java 17, enforced in code rather than by documentation: `Application.java` hard-codes `SUPPORTED_JRE_VERSIONS = { 17 }`.⁵ Structurally it is two Gradle subprojects that are also two Java Platform Module System modules: `info.openrocket.core`, the headless simulation engine, file formats, and design classes, reusable by other applications; and `info.openrocket.swing`, the GUI, which depends on core. Services are wired by Google Guice dependency injection behind a static locator.⁶ The 24.12 surface — quaternion 6-DOF flight, extended Barrowman aerodynamics, event-driven recovery and staging, a roughly two-dozen-type component catalog — is one QtRocket does not attempt to match today, which is precisely what makes it a useful yardstick.

A.3 Four philosophical differences

The companion review’s central observation about this comparison is that the gap pattern is explained less by effort than by philosophy. Four differences in engineering posture recur in every appendix that follows; a reader who holds them in mind can predict most verdicts before reading them.

Developmental order. QtRocket builds and tests its seams before wiring in their consumers; OpenRocket built features first and validated them against flight afterward. QtRocket’s Barrowman composition is implemented, unit-tested, and consumed by no production code — the test suite says so itself: “nothing in production consumes this in P2” (`model/tests/AeroTests.cpp:18`). OpenRocket’s equivalent machinery shipped inside a flying product and was then measured against flight telemetry and wind-tunnel data in the techdoc’s validation chapter, whose headline claim is CP and drag-coefficient accuracy of roughly ten percent at subsonic speeds (techdoc ch. 6–7). The consequence, developed in Section 13, is that QtRocket’s distance to the baseline is overwhelmingly a consumption gap, not a construction gap.

Failure posture. QtRocket is fail-closed where OpenRocket is warn-and-continue. A self-intersecting QtRocket design is a hard, located failure: the mass computation refuses to produce a number and throws with the first overlap diagnostic (`model/parts/Part.cpp:189`). OpenRocket’s posture is the opposite and is built into its event vocabulary: `SIM_WARN` is a first-class flight event, recorded in the

¹License and module overview: https://openrocket.readthedocs.io/en/latest/dev_guide/architecture.html.

²Sampo Niskanen, *Development of an Open Source model rocket simulation software*, Master’s thesis, Helsinki University of Technology, 2009; and *OpenRocket technical documentation, for OpenRocket version 13.05*, 2013-05-10, <https://openrocket.sourceforge.net/techdoc.pdf>. Both are indexed at <https://openrocket.info/documentation.html>.

³`util/GeodeticComputationStrategy.java` on the `unstable` branch of <https://github.com/openrocket/openrocket>.

⁴<https://github.com/openrocket/openrocket/releases>; the final release-24.12 is dated 2025-07-27.

⁵<https://raw.githubusercontent.com/openrocket/openrocket/release-24.12/core/src/main/java/info/openrocket/core/startup/Application.java>.

⁶Module split and entry point from the architecture page cited above and the root `build.gradle`; `com.google.inject:guice:7.0.0` in `core/build.gradle` on `unstable`.

flight data alongside ignition and apogee while the simulation proceeds.⁷ The trade is stated plainly in the body: fail-closed costs user forgiveness but guarantees that every produced number came from a coherent model.

Placement intent versus geometric offsets. OpenRocket positions a component by a stored (`AxialMethod`, `offset`) pair — top, middle, bottom, absolute, or after — whose arithmetic is purely geometric, computed from component lengths and never from mass or CM.⁸ QtRocket stores placement as physical intent: a `StationLink` names fractional stations on both ends of a seam, an axial gap, and a seat kind selecting the radial compatibility rule (`model/parts/Placement.h:49`), and a single resolver turns intent into geometry for simulator and visualizer alike ([Section 5](#)). The intent model is arguably richer; it is also younger, and at the pinned commit only its simplest verb is reachable from a front end.

Infrastructure-first engineering. Where OpenRocket’s effort visibly went into product surface — top-level `install4j/` and `snap/` installer trees sit beside `core/` and `swing/` in the repository⁹ — QtRocket’s went into the engineering substrate: a five-platform CI workflow building with `-Werror` tree-wide (`/WX` on MSVC) and flying full motor-ladder sweeps on every pull request (`.github/workflows/cmake-multi-platform.yml:29`; [Section 12](#)). The same posture shows up inside the numerics, where the young project is occasionally ahead of the mature one — QtRocket’s embedded-error adaptive RKF45 against OpenRocket’s heuristically stepped RK4 ([Section C](#)) — and in what each ships without: one has no headless scripting story, the other no installer.

A.4 Reading guide

The three comparisons are ordered from broad to deep, and each stands alone. [Section B](#) is the sixteen-area feature matrix: for every area from the component catalog to distribution, what OpenRocket offers and what QtRocket has, with each QtRocket cell carrying a status verdict and citations. Start there for the shape of the gap. [Section C](#) compares the flight physics method by method — dynamics, aerodynamic coefficients, drag, atmosphere and wind, gravity, integration, motors, recovery, and validation posture — and is written for the reader who wants the actual equations. [Section D](#) compares the two programs as software: module structure, the design tree, positioning, mass properties, simulation control, extension surfaces, persistence, motor data, and GUI architecture. All three inherit this appendix’s two conventions without restating them: distance, not conformance; and footnoted primary sources on the OpenRocket side against pinned `file:line` citations on the QtRocket side.

B Feature comparison with OpenRocket

[Section A](#) sets the terms of this comparison: OpenRocket is the baseline because it is the de facto standard for what a mature open-source model-rocket simulator offers, and the comparison measures distance, not conformance. This appendix is the inventory behind that judgment — a matrix over sixteen feature areas: the components catalog; mass and inertia modeling; CP and stability aerodynamics; drag decomposition; degrees of freedom and flight dynamics; atmosphere, gravity, and wind; recovery; staging, clusters, and flight configurations; motor data and selection; the design UI; 3D visualization; plotting and analysis; file formats and interoperability; extensibility and scripting;

⁷`FlightEvent.Type` in `simulation/FlightEvent.java`, <https://github.com/openrocket/openrocket/blob/unstable/core/src/main/java/info/openrocket/core/simulation/FlightEvent.java>.

⁸`rocketcomponent/position/AxialMethod.java`, <https://github.com/openrocket/openrocket/blob/unstable/core/src/main/java/info/openrocket/core/rocketcomponent/position/AxialMethod.java>.

⁹<https://github.com/openrocket/openrocket>, top-level directory listing.

optimization and dispersion; and documentation, localization, platform, and distribution. The areas and the QtRocket verdicts follow the companion architecture review’s matrix; the OpenRocket column has been refreshed against the source research gathered for [Sections C and D](#), and where the older review and the newer research disagreed, the research prevails and the claim is stated conservatively.

Two evidentiary standards apply, one per column, and the asymmetry is deliberate. QtRocket claims carry `file:line` citations into the pinned commit [254cd41](#) and are grounded in the body sections ([Sections 4 and 6 to 12](#)). OpenRocket claims are condensed from the project’s documentation, release notes, and source tree at tag [release-24.12](#), cited by footnote URL; they were checked against the repository in July 2026 but are not held to the file-and-line standard of the QtRocket column.¹⁰ Version pinning matters on both sides: 24.12 is the final stable GitHub release as of this writing — the beta shipped December 2024 and the final release is dated July 27, 2025¹¹ — while development continues on the `unstable` branch. The matrix names `unstable` explicitly wherever a claim would otherwise mislead.

The status markers grade the QtRocket column only; the OpenRocket column describes. **Present** means implemented and consumed by production code. **Partial** means built but incomplete or unconsumed — and in this codebase that marker deserves a second look, because several **Partial** verdicts cover machinery that is finished and unit-tested but wired to nothing ([Section 13](#)); the distance those rows measure is a consumption gap, not a construction gap. **Absent** means no trace at the pinned commit, each such verdict grounded in a failed search of the tree rather than an impression. **WIP** means under development on an unmerged branch, reported as such and never silently folded into the pinned-commit picture.

B.1 The sixteen-area matrix

Table 16. Feature-area comparison: OpenRocket 24.12 versus QtRocket at commit [254cd41](#).

Area	OpenRocket	QtRocket
Rocket components (part catalog)	Roughly two dozen concrete types under one abstract <code>RocketComponent</code> tree: six nose-cone shape families, transitions, inner tubes, couplers, centering rings, bulkheads, lugs, rail buttons, four fin classes including free-form, recovery devices, mass objects, pods, boosters. Per-class <code>isCompatible</code> child rules; per-component mass/CG/ C_d overrides; a commercial preset database.	Partial Five types (<code>model/parts/PartFactory.cpp:32</code>); geometry immutable after construction, so live dimension editing has no mutation path. Distinctive: intent-based <code>StationLink</code> placement (<code>model/parts/Placement.h:49</code>) whose overlap sweep hard-fails self-intersecting designs (<code>model/parts/Part.cpp:184</code>) — stronger than warn-and-simulate (Section 5).

continued on next page

¹⁰Primary sources: <https://openrocket.info/>; <https://openrocket.readthedocs.io/en/latest/>; the `openrocket/openrocket` repository at tag `release-24.12` (<https://github.com/openrocket/openrocket>); and the technical documentation at <https://openrocket.sourceforge.net/techdoc.pdf>.

¹¹GitHub releases of `openrocket/openrocket`: `release-24.12` published 2025-07-27; <https://github.com/openrocket/openrocket/releases>.

Table 16 (continued)

Area	OpenRocket	QtRocket
Mass / inertia modeling	Mass from volume and density per component; an external <code>MassCalculator</code> walks the tree each force evaluation; two scalar moments (longitudinal, axial) — the tensor is treated as diagonal and axisymmetric. Overrides: overridden mass rescales inertia linearly, overridden CG shifts it by parallel axis. Motor mass and CG interpolated between curve samples during burn.	Present Lazy two-pass time-aware composition with parallel-axis shifts (<code>model/parts/Part.cpp:175</code>); mass-delta cache freezes bit-stably after burnout (<code>model/parts/Part.cpp:130</code>); closed-form full 3×3 tensors pinned by numeric oracles (<code>model/tests/InertiaTensorsTests.cpp:186</code>). The RIR^T child-tensor rotation is omitted pending 6-DOF (<code>model/parts/Part.cpp:217</code>).
Aerodynamics: CP / stability	Extended Barrowman recomputed every force evaluation at the instantaneous angle of attack and Mach number ($C_{N_\alpha} \equiv C_N/\alpha$): Galejs body lift, free-form fin MAC integrals, fin-body interference, pitch damping, roll forcing and damping; live CP/CG markers refreshed by a background simulation while editing.	Partial Per-part Barrowman math exists and is unit-tested, with composite CP on a shared tip datum (<code>sim/Aero.h:25</code> , <code>model/parts/Part.cpp:231</code>) — but zero production consumers, and no surface shows CP, CG, or margin. Latent defect: mirrored <code>FinSet</code> chordwise sign convention (Section 9).
Aerodynamics: drag decomposition	Component build-up: roughness-aware fully-turbulent skin friction with a roughness-limited floor; per-shape transonic/supersonic nose pressure drag from experimental curves; fin leading/trailing-edge terms; base drag with motor area subtracted under thrust; parasitic lug drag; axial drag scaled empirically with angle of attack.	Absent One user-set scalar C_d (default 1.0) times one reference area (<code>model/RocketModel.cpp:81</code>); every part's drag contribution returns zero (<code>model/parts/BodyTube.cpp:41</code>); no Mach or Reynolds number exists. <code>deriveReferenceAreaFromGeometry()</code> is correct but uncalled (<code>model/RocketModel.cpp:48</code>), so all 25 design fixtures under <code>tests/data/designs/</code> fly on a stale or default 38 mm disc.
Degrees of freedom / flight dynamics	Quaternion 6-DOF ascent over a two-scalar inertia model, degrading deliberately to 3-DOF for recovery descent and tumbling stages; RK4 with a-priori heuristic step limits, no embedded error pair; validated against flight and wind-tunnel data (headline accuracy roughly 10% at subsonic speeds).	Partial 3-DOF point mass, heavily tested end-to-end. Thrust hardwired along world $+z$ (<code>model/RocketModel.cpp:70</code>); <code>getTorques()</code> returns zero with no caller (<code>model/RocketModel.cpp:94</code>); orientation integrator commented out (<code>sim/Propagator.h:103</code>). Ahead: embedded-error adaptive RKF45 (<code>sim/RK45Solver.h:75</code>); typed termination taxonomy (<code>sim/Propagator.h:35</code>).
Atmosphere / gravity / wind	Eight-layer ISA with launch-site overrides; pink-noise turbulence (two-pole Kasdin filter, $\alpha = 5/3$) and, new in 24.12, multi-level winds with CSV import; flat, spherical, or WGS84–Vincenty geodesy with Coriolis; Somigliana latitude-dependent gravity; launch rod/rail.	Partial Three atmospheres (US Standard 1976 validated to 0.1–0.5% up to 80 km) and true inverse-square spherical gravity (<code>sim/SphericalGravityModel.cpp:28</code>). Wind is dead code: returns (0, 0, 0), zero call sites (<code>sim/WindModel.cpp:16</code>); no rod/rail (pad-hold clamp only, <code>sim/Propagator.cpp:105</code>), no launch site, no Coriolis.
Recovery	Event-driven deployment over sixteen <code>FlightEvent</code> types ¹² with per-trigger delays; dual deployment via altitude events; parachute C_d default 0.8 and an empirical streamer drag law; dedicated 3-DOF descent and tumble steppers with exact apogee/ground root solves.	Absent Nothing changes at apogee; flight ends ballistically at the $z < 0$ -descending terminate (<code>model/RocketModel.cpp:62</code>). Ejection delays are parsed and persisted but consumed by no flight logic; apogee exists only as a post-hoc statistic.

continued on next page

¹²`FlightEvent.java` under `core/src/main/java/info/openrocket/core/simulation/:` LAUNCH, IGNITION, LIFTOFF,

Table 16 (continued)

Area	OpenRocket	QtRocket
Staging / clusters / flight configurations	Unlimited serial stages plus <code>ParallelStage</code> boosters and pods; stage separation forks the simulation, one <code>FlightDataBranch</code> per branch; motor clusters via <code>Clusterable</code> inner tubes; named flight configurations keyed by UUID with per-stage active flags.	Absent Single motor, single stage: first-match DFS (<code>model/RocketModel.cpp:13</code>); <code>Stage.h</code> is a commented-out “Not yet” include (<code>model/RocketModel.h:22</code>). Ignition timing is half-plumbed (<code>ThrustCurve::setIgnitionTime</code> has zero call sites); the motor sits at the airframe CM with a hardcoded zero offset (<code>model/RocketModel.cpp:124</code>).
Motor data & selection	Complete thrustcurve.org corpus bundled offline: 24.12 ships a serialized snapshot; the <code>unstable</code> branch replaces it with an SQLite database published from a separate motor-database repository (1458 motors, 1591 curves at the checked snapshot). ¹³ RASP <code>.eng</code> and RockSim <code>.rse</code> user files; rich chooser with search, filters, curve plots, delay presets.	Partial Three ingest paths behind one facade: RSE (<code>model/RSEDatabaseLoader.cpp:20</code>), strict RASP (<code>model/RASPLoader.cpp:253</code>), live thrustcurve.org search behind a fake-injectable seam (<code>model/ThrustCurveClient.h:68</code>) — live in-app search the baseline lacks. Ships with an empty database (the one bundled <code>.rse</code> is loaded only by tests); per-sample mass/CG data discarded (<code>model/RSEDatabaseLoader.cpp:89</code>); common-name keying replaces on collision — last write wins, distinguished only in the logs.
Design UI	Component tree with per-component config dialogs, drag-and-drop, and undo; live 2D schematic with CG/CP markers; a background-simulation thread pool re-runs the flight on every edit to keep the altitude and stability readouts live. ¹⁴	WIP At the pinned commit, a motor-and-launch front end only: design widgets are constructor stubs (<code>gui/RocketTreeView.cpp:3</code>), File actions ship disabled (<code>gui/MainWindow.ui:117</code>), authoring is CLI-only (<code>cli/Repl.cpp:784</code>). The unmerged <code>RocketTreeViewImplementation</code> branch adds a read-only tree and <code>.qrd</code> open/save; no add/edit UI, no property panel, no schematic (Section 10).
3D visualization	Embedded JOGL 3D finished/unfinished views; per-component appearance, textures, and decals; PhotoStudio photorealistic rendering.	Present Standalone <code>qtrocket-visualizer</code> meshes the tree (<code>visualizer/RocketMesh.cpp:404</code>) and positions meshes with the simulator’s own placement resolver (<code>visualizer/RocketMesh.cpp:433</code>), so rendered and simulated geometry cannot diverge; overlap offenders render in error red (<code>visualizer/RocketGLWidget.cpp:617</code>). Not embedded in the GUI; motors render no geometry; no tests.
Plotting / analysis	Fifty-plus typed flight-data series on configurable axes with event overlays; user-defined custom expressions; component analysis; CSV export with selectable columns.	Partial Three fixed plots (<code>gui/AnalysisWindow.cpp:36</code>); CLI launch summary plus a fixed-schema CSV auto-written to a hardcoded <code>.qtrocket_run.csv</code> (<code>cli/Repl.cpp:104</code>). No configurable plots, no event annotations, no stability outputs.

continued on next page

LAUNCHROD, BURNOUT, EJECTION_CHARGE, STAGE_SEPARATION, APOGEE, RECOVERY_DEVICE_DEPLOYMENT, GROUND_HIT, SIMULATION_END, ALTITUDE, TUMBLE, SIM_WARN, SIM_ABORT, EXCEPTION.

¹³<https://github.com/openrocket/motor-database>; on `unstable` the bundled datafiles/thrustcurves/initial_motors.db carries metadata `motor_count: 1458, curve_count: 1591` and a published update URL.

¹⁴`RocketPanel.java` (`swing/src/main/java/info/openrocket/swing/gui/scalefigure/`) owns a daemon background-simulation executor; every design edit triggers `updateExtras()`, which re-runs a simulation to refresh the on-canvas readouts.

Table 16 (continued)

Area	OpenRocket	QtRocket
File formats & interop	<code>.ork</code> ZIP-plus-XML with a content-sniffing loader (ZIP, gzip, and bare XML all load) and format versions back to 1.0; RockSim <code>.rkt</code> import and export; RASAero <code>.CDX1</code> import; OBJ, SVG, and CSV export.	Partial Versioned <code>.qrd</code> XML round-trips the tree with placement intent, re-resolved on load (<code>model/DesignSerializer.cpp:93</code>); fail-safe atomic loading (<code>model/DesignSerializer.cpp:255</code>). Legacy 0.1 files rejected with no migration path (<code>model/DesignSerializer.cpp:194</code>); mass overrides lost on save; no foreign import or export of any kind.
Simulation extensibility / scripting	Three listener interfaces — lifecycle, event veto, and paired pre/post hooks around every physics sub-model (gravity, wind, thrust, aero, mass) that can override results per step; shipped simulation extensions; JavaScript scripting via GraalJS; annotation-scanned plugin JARs.	Partial No listener, plugin, or scripting mechanism. Partial substitute the baseline lacks: a 33-command scriptable CLI over a machine-parseable OK/ERR protocol (<code>cli/Repl.cpp:232</code>), weakened as automation by an always-zero exit code.
Optimization / dispersion	Rocket Optimizer: parameter modifiers, altitude/velocity/stability goals, and stability-domain constraints evaluated over parallel simulation runs. Monte Carlo dispersion absent, on the project's own roadmap.	Absent Neither optimizer nor dispersion exists (greps return zero hits); since the baseline also lacks Monte Carlo, the gap is the optimizer only. The deterministic core and scriptable CLI make external sweeps feasible today, but nothing in-product supports them.
Docs / i18n / platform / distribution	GPLv3 Java 17 split into JPMS core and Swing modules; installers with bundled JRE for Windows/macOS/Linux plus a Snap package; localization into roughly fifteen languages; published technical documentation with a validation chapter.	Partial Strong engineering infrastructure: five-platform CI with <code>-Werror</code> (<code>.github/workflows/cmake-multi-platform.yml:28</code>), <code>llvm-cov</code> coverage, 239 tests. Distribution essentially absent: one install rule (<code>CMakeLists.txt:308</code>), no CPack or installers; i18n is dead scaffolding (<code>CMakeLists.txt:201</code>), English only.

B.2 Where QtRocket is ahead

Fairness cuts both ways. Sixteen rows of mostly **Partial** and **Absent** would misrepresent the comparison if the appendix did not also record the places where QtRocket offers something the baseline does not. Seven are defensible at the pinned commit.

Live motor search. Both front ends can search `thrustcurve.org` interactively — facet metadata, filtered search, per-result curve download — through one client behind a fake-injectable seam (`model/ThrustCurveClient.h:68`). OpenRocket's chooser is offline-first, working against a corpus bundled at release time (the 24.12 serialized snapshot; on `unstable`, the SQLite database above). The trade cuts both ways — OpenRocket is complete out of the box where QtRocket starts empty ([Section 6](#)) — but interactive search against the canonical registry is a capability the baseline's user surface does not offer.

Embedded-error adaptive integration. QtRocket's RK45 is a true embedded pair: fourth- and fifth-order estimates from shared stages, a per-step error norm, step rejection and re-integration, and a growth clamp pinned by a regression test (`sim/RK45Solver.h:75`). OpenRocket integrates ascent with RK4 whose step is chosen a priori as the minimum of heuristic caps — pitch-angle and roll-rate limits, event distance, a $1.5\times$ growth limiter — computed after evaluating only the first stage; the methods research confirmed there is no error estimate and no step rejection

anywhere in the stepper¹⁵ (Section C). In fairness, those heuristics are attitude-aware and tuned to 6-DOF oscillation, a problem QtRocket does not yet face.

A hang-proof, typed termination taxonomy. Every exit from the run loop is one of five typed verdicts — nominal, no-liftoff, non-finite state, time/iteration cap, integrator error (`sim/Propagator.h:35`) — and the loop is bounded by both a simulated-time cap and an iteration cap, so it cannot hang. The CLI consumes the enum in an exhaustive switch with no default, so a new verdict fails to compile until every handler is written (`cli/Repl.cpp:41`). OpenRocket reports aborts and warnings as flight events (`SIM_ABORT`, `SIM_WARN`) in the flight data; QtRocket’s distinction is the exhaustive partition and the compile-time obligation it imposes on consumers.

A scriptable headless REPL. Thirty-three commands, exactly one machine-parseable OK/ERR line per command, and designs authored, persisted, and flown entirely from a pipe (`cli/Repl.cpp:232`). OpenRocket’s core module is deliberately headless and reusable, but the project ships no interactive console: scripted use means writing Java against the module or using the separately maintained Python bindings.¹⁶

One resolver for physics and rendering. The visualizer positions every mesh from the same `resolvePlacements` output the mass and inertia composition consumes (`visualizer/RocketMesh.cpp:433`, `model/parts/Placement.h:148`), so rendered and simulated geometry cannot diverge by construction. OpenRocket’s 2D figure and 3D views are mature, separate rendering paths over the component tree; nothing makes their geometry and the simulator’s identical by construction.

Fail-closed geometry diagnostics. A self-intersecting design is a hard, located failure before any number is produced: the composite mass computation refuses with the first overlap diagnostic (`model/parts/Part.cpp:184`), and the visualizer paints the offender in error red from the same cached verdict (`visualizer/RocketGLWidget.cpp:617`). OpenRocket warns and simulates anyway — more forgiving, but a produced number does not guarantee a coherent model.

True inverse-square gravity. The spherical gravity option evaluates the central field $-GM\mathbf{r}/|\mathbf{r}|^3$ at the integrator’s trial position (`sim/SphericalGravityModel.cpp:28`). OpenRocket’s `WGSGravityModel` applies Somigliana latitude-dependent surface gravity with a spherical-approximation $(R/(R+h))^2$ altitude correction.¹⁷ At model-rocket altitudes the numbers are close, and OpenRocket’s latitude dependence is the more useful refinement there; the QtRocket option’s distinction is being an exact central vector field rather than a corrected scalar magnitude — the right shape for the high-altitude regime its US Standard Atmosphere already covers.

None of this reverses the overall verdict. OpenRocket is a mature product with a broad surface and a published validation record; QtRocket is a much smaller simulator whose engineering discipline runs ahead of its feature count. The matrix should be read with Section 13 in hand: much of its **Partial** column is finished machinery awaiting a consumer, so the distance it measures is overwhelmingly a consumption gap, not a construction gap.

¹⁵`RK4SimulationStepper.java` under `core/src/main/java/info/openrocket/core/simulation/`; the stepper ships at tag `release-24.12`, and the eight step-size candidates — none involving an error estimate — were verified in the class’s code comments on the `unstable` development branch (fetched 2026-07). See Section C.

¹⁶<https://github.com/openrocket/orhelper>.

¹⁷`WGSGravityModel.java` under `core/src/main/java/info/openrocket/core/models/gravity/`.

C Simulation-methods comparison with OpenRocket

Where [Section B](#) compares the two simulators feature by feature, this appendix compares them *method by method*: for each area of flight physics, what OpenRocket computes, what QtRocket computes, and a short assessment of the distance between the two. It is written for the reader who wants the actual equations. OpenRocket’s methods are taken from its technical documentation¹⁸ and, where the 2013 techdoc has been overtaken by later development (geodesy, Coriolis, multi-level wind), from the current source.¹⁹ As [Section A](#) establishes, OpenRocket-side claims are condensed from those sources — researched, but not line-verified in the way QtRocket claims are. Every QtRocket claim cites the pinned commit 254cd41. [Table 17](#) condenses the whole appendix into one table.

C.1 Flight dynamics

OpenRocket. Ascent is a genuine six-degree-of-freedom (6-DOF) simulation: world-frame position and velocity plus a unit orientation quaternion and body angular velocity, written as a first-order system and stepped together (techdoc §4.2.1, §4.2.4, eqs. 4.17–4.19). Every force evaluation recomputes local wind, angle of attack (AoA), and Mach number, then the full aerodynamic coefficient set at those instantaneous conditions, then thrust, gravity, mass, and moments of inertia (techdoc §4.2.4, steps 0–6). In the code the linear equation is assembled in rocket coordinates as $(-f_N, -f_{side}, T - D_{axial})/m$, rotated to the world frame, with gravity and Coriolis acceleration added; the pitch, yaw, and roll moments are referred to the center of gravity (CG) and divided by exactly *two* scalar moments of inertia — longitudinal for pitch/yaw, axial for roll — so the inertia tensor is treated as diagonal and axisymmetric, never as a full 3×3 tensor.²⁰ A deliberate ± 0.0005 uniform random perturbation is added to C_m and C_{yaw} at every evaluation “to prevent over-perfect flight”.²¹ The 6-DOF model is also deliberately abandoned when it stops earning its keep: “When a parachute is deployed the rocket typically splits in half, and it is no longer practical to compute the orientation of the rocket” (techdoc §4.2.5), so recovery descent and the descent of unstable, tumbling stages (techdoc §3.5) run as 3-DOF point masses under a different stepper. Fidelity is matched to flight phase.

QtRocket. The integrated state is three linear degrees of freedom, full stop. The entire physics reaching the integrator is one lambda, `state = rate, rate = getForces(t)/getMass(t) sim/Propagator.cpp:36`, with thrust hardwired along world `+z model/RocketModel.cpp:71` — there is no attitude for it to follow. `StateData` reserves quaternion orientation and rate, a direction-cosine matrix, and 3-2-1 Euler-angle slots, and nothing anywhere writes them `sim/StateData.h:46`; the orientation integrator exists as a commented-out member with a design note `sim/Propagator.h:103; getTorques()` is declared on the bridge interface `model/Propagatable.h:30`, implemented as the zero vector `model/RocketModel.cpp:94`, and called by no production code. All of this is dormant seam, inventoried in [Section 13](#) and described in [Section 7](#).

The inertia inversion

The data models run in the opposite direction from the dynamics. OpenRocket integrates real attitude but represents inertia as two scalars; QtRocket integrates no attitude but records

¹⁸Sampo Niskanen, *OpenRocket technical documentation, for OpenRocket version 13.05*, 2013-05-10, <https://openrocket.sourceforge.net/techdoc.pdf> — the updated form of Niskanen’s 2009 Master’s thesis. Cited hereafter as “techdoc” with its own section and equation numbers.

¹⁹OpenRocket source, `unstable` branch of <https://github.com/openrocket/openrocket> (fetched 2026-07-07), files under `core/src/main/java/info/openrocket/core/`. Cited hereafter by class name.

²⁰`simulation/RK4SimulationStepper.java, computeAcceleration()`.

²¹`RK4SimulationStepper, constant PITCH_YAW_RANDOM`.

the *full* 3×3 composite inertia tensor about the instantaneous composite CM into every retained state sample `sim/StateData.h:62`, refreshed each step through `writeMassProperties` `sim/Propagator.cpp:132` and pinned by a test that watches I_{xx} shrink through the burn and freeze after burnout `tests/PhysicsIntegrationTests.cpp:356`. QtRocket thus carries a strictly richer rotational data model than the one OpenRocket actually flies with — and integrates none of it. The recorded tensor is the declared seam the future 6-DOF rotational ODE will read.

The distance here is one of kind, not tuning: for weathercocking, gravity turns, spin, or dispersion, OpenRocket computes and QtRocket cannot yet.

C.2 Aerodynamic coefficients

OpenRocket. The normal-force machinery is an *extended* Barrowman method, re-evaluated at the instantaneous flight condition on every force call. Instead of the classic $\alpha \rightarrow 0$ derivative, OpenRocket defines, for $\alpha > 0$,

$$C_{N_\alpha} \equiv \frac{C_N}{\alpha}, \quad C_{m_\alpha} \equiv \frac{C_m}{\alpha} \quad (38)$$

so the whole coefficient set depends on the actual AoA and Mach number (techdoc §3.1.1, eq. 3.8). The extensions beyond classic Barrowman, each with its techdoc equation: body lift per Galejs, $C_N^{\text{lift}} = K (A_{\text{plan}}/A_{\text{ref}}) \sin^2 \alpha$ with $K \approx 1.1$, acting at the planform centroid (eqs. 3.26–3.27); free-form fin planforms via numerically integrated mean-aerodynamic-chord (MAC) integrals (eqs. 3.30–3.32); arbitrary fin counts with empirical interference multipliers falling to 0.750 beyond eight fins (eq. 3.54); fin–body interference $K_{T(B)} = 1 + r_t/(s + r_t)$ (eqs. 3.55–3.56); pitch and yaw damping proportional to ω^2/ν_0^2 (eqs. 3.59–3.60); and full roll forcing/damping strip theory for canted fins (eqs. 3.66, 3.69–3.72). Compressibility runs through everything: the Prandtl factor $1/\beta$ with $\beta = \sqrt{|M^2 - 1|}$ (eqs. 3.13–3.14), Busemann third-order strips for supersonic fins (eqs. 3.41–3.49), and a moving fin center of pressure (CP): fixed at quarter-MAC below M 0.5, the empirical $X_f/\bar{c} = (\mathcal{AR}\beta - 0.67)/(2\mathcal{AR}\beta - 1)$ above M 2, and a fifth-order polynomial in M with matched values and derivatives bridging the transonic gap between them (eqs. 3.35–3.36).

QtRocket. The Barrowman layer is the classic zero-AoA linearization, computed per part and folded over the placement-resolved tree (Section 9). A nose cone contributes $\text{cnAlpha} = 2\pi \text{baseRadius}^2/\text{refArea}$ with its CP at $2L/3$ aft of the tip `model/parts/ConicalNoseCone.cpp:64`; a fin set contributes the standard trapezoidal-fin slope `model/parts/FinSet.cpp:77` multiplied by an interference factor *identical in form to OpenRocket’s* eq. 3.55, $K = 1 + \text{bodyRadius}/(\text{span} + \text{bodyRadius})$ `model/parts/FinSet.cpp:76`; body tubes contribute zero slope, exactly as slender-body theory demands. Contributions are summed as slope-weighted CP moments so the composite CP is a guarded ratio `model/parts/Part.cpp:231`, `sim/Aero.h:51`. What the layer does *not* have: any α dependence beyond the linear slope, any body-lift term, any damping or roll coefficient, and any Mach dependence at all — nothing in the tree computes a Mach number, and the atmosphere’s speed-of-sound accessor has no production caller `sim/USStandardAtmosphere.cpp:143`. And the layer is dormant: the test suite states “nothing in production consumes this” verbatim `model/tests/AeroTests.cpp:18`.

The assessment writes itself: formula by formula the two projects share an ancestor, and where they overlap (fin slope, interference factor, CP-moment composition) they agree. OpenRocket then evaluates its extended version at flight conditions every step and flies on the result; QtRocket’s classic version is evaluated at $\alpha \rightarrow 0$, with no Mach anywhere, by tests alone.

C.3 Drag

OpenRocket. Zero-AoA drag is a per-component, semi-empirical build-up (techdoc §3.4): skin friction, body and fin pressure drag, base drag, and parasitic (launch-guide) drag, with interference drag explicitly neglected. Skin friction assumes a fully turbulent boundary layer, using the smooth-wall correlation and a roughness-limited floor

$$C_f = \frac{1}{(1.50 \ln R - 5.6)^2} \quad \text{and} \quad C_f = 0.032 \left(\frac{R_s}{L} \right)^{0.2} \quad (39)$$

selected piecewise by a roughness-critical Reynolds number, with separate subsonic and supersonic compressibility corrections (eqs. 3.78–3.84) and body/fin geometry factors when scaled to the reference area (eq. 3.85). Nose pressure drag is per-shape: a stagnation-based blunt-cylinder limit, a closed form for cones, an ogive shape-factor scaling, and stored experimental transonic curves for ellipsoid/power/parabolic/Haack profiles, interpolated in Mach and fineness ratio (techdoc App. B); high-subsonic values are bridged by a constrained power law (eq. 3.87). Base drag is empirical,

$$(C_{D\bullet})_{\text{base}} = \begin{cases} 0.12 + 0.13 M^2 & M < 1 \\ 0.25/M & M > 1 \end{cases} \quad (40)$$

with a thrusting motor’s cross-section subtracted from the base area (eq. 3.94); launch lugs are blunt cylinders at stagnation pressure with a flow-through correction (eqs. 3.95–3.96). Finally the zero-AoA total (eq. 3.97) is stretched over AoA by an empirical two-part polynomial $C_A(\alpha) = C_{D_0} f(\alpha)$ with $f(0) = 1$, a hump of 1.3 at $\alpha = 17^\circ$, and $f(90^\circ) = 0$ (techdoc §3.4.7).

QtRocket. The production drag model is one line `model/RocketModel.cpp:88`:

$$\vec{F}_{\text{drag}} = -\frac{1}{2} \rho(\text{altitude}) \|\text{velocity}\| \text{dragCoefficient} \cdot \text{referenceArea} \cdot \text{velocity}, \quad (41)$$

a single constant C_d (default 1.0, `model/RocketModel.h:138`) times a single constant reference area (default $1.134 \times 10^{-3} \text{ m}^2$, a 38 mm disc, `model/RocketModel.h:141`), opposing the full velocity vector, with density sampled at the trial altitude clamped to $z \geq 0$ `model/RocketModel.cpp:85`. There is no Mach or Reynolds dependence, no AoA dependence, and no per-component build-up: the `AeroComponent` value type carries a per-part `cd` field for exactly that purpose, and every concrete part returns zero — the body tube’s comment defers skin friction over its (unconsumed) wetted area to future work `model/parts/BodyTube.cpp:41`, `model/parts/ConicalNoseCone.cpp:67`, `model/parts/FinSet.cpp:89`, `model/parts/BodyTube.h:51`.

The reference area never reaches the drag law

QtRocket implements the max-frontal-disc reference-area convention correctly — `deriveReferenceAreaFromGeometry()` `model/RocketModel.cpp:48` walks the part tree taking the maximum frontal disc `model/parts/Part.cpp:251` — but the function has no production caller; its only call site outside its own definition is a test `model/tests/AeroTests.cpp:201`. Design load applies a saved area only when the design was saved with a manual override `model/DesignSerializer.cpp:263`, and all 25 bundled design fixtures carry `referenceAreaOverridden="false"`, so every loaded design flies on whatever scalar was previously staged — by default the 38 mm disc — regardless of the airframe’s real diameter. Drag does not scale with airframe class at this commit. **FAIL**

Distance: OpenRocket’s drag is five semi-empirical families evaluated per component per step; QtRocket’s is one user-supplied number, currently wired to one defective constant. This is the widest methods gap in the whole comparison.

C.4 Atmosphere and wind

Both atmospheres are members of the same family — the International Standard Atmosphere (ISA) and the US Standard Atmosphere 1976 are identical below 32 km — and this is the one area where the two simulators are nearly peers. OpenRocket linearly interpolates temperature between eight ISA layer bases (0 to 84.852 km) and integrates the hydrostatic/ideal-gas pair within each layer (techdoc §4.1.1, Table 4.1); the current code adds a launch-site override that inserts an extra layer with a back-computed sea-level lapse rate, converts geometric to geopotential height with $R = 6\,356\,766$ m, and precomputes layer conditions for speed.²² QtRocket’s `USStandardAtmosphere` hard-codes the same first seven layer anchors (0 to 71 km — it has no 84.852 km top row) with published base pressures and densities `sim/USStandardAtmosphere.cpp:78`, and evaluates the closed-form gradient-layer power law and isothermal exponential exactly within each layer `sim/USStandardAtmosphere.cpp:121`, the density exponent carrying the ideal-gas -1 `sim/USStandardAtmosphere.cpp:107`. The differences are at the edges: QtRocket has no launch-site override (the registry offers exactly Constant, US Standard 1976, and Vacuum, `sim/Environment.h:74`), no geopotential conversion, throws below 0 m `utils/Bin.cpp:52` (its one consumer clamps, `model/RocketModel.cpp:85`), and extrapolates the 71 km layer upward without bound `utils/Bin.cpp:59`. Against that, QtRocket’s implementation is the more aggressively *verified* of the two: pressure and temperature match the bundled NOAA report tables to 0.1% relative at twenty altitudes from 0 to 80 km `sim/tests/USStandardAtmosphereTests.cpp:36`, `sim/tests/USStandardAtmosphereTests.cpp:64`, and density to 0.1%/0.5% `sim/tests/USStandardAtmosphereTests.cpp:10`, `sim/tests/USStandardAtmosphereTests.cpp:24`.

Wind is not close. OpenRocket models wind as a steady average plus pink-noise turbulence with spectrum $1/f^\alpha$, $\alpha = 5/3$, generated by a two-pole Kasdin filter over Gaussian white noise at a fixed 20 Hz and scaled by the filter’s precomputed standard deviation 2.252 (techdoc §4.1.2, eqs. 4.5–4.6);²³ since release 24.12, arbitrary altitude levels each carry their own pink-noise model, with the instantaneous wind vectors of the two adjacent levels linearly interpolated component-wise.²⁴ QtRocket’s `WindModel` is a concrete class returning the zero vector `sim/WindModel.cpp:16` with zero call sites and no `Environment` slot `sim/Environment.h:103`; drag is computed from inertial velocity, so the air mass is implicitly at rest `model/RocketModel.cpp:88`.

C.5 Gravity and geodesy

The 2013 techdoc states flat-Earth Cartesian coordinates and “the coriolis effect ... is not simulated” (techdoc §4.2) — and is obsolete on this point. Current OpenRocket offers three geodetic strategies — flat, spherical great-circle (the default), and a full Vincenty direct geodesic on the WGS84 (World Geodetic System 1984) ellipsoid — and adds Coriolis acceleration $-2\boldsymbol{\Omega} \times \vec{v}$ to every force evaluation under the latter two.²⁵ Gravity is either a user constant or Somigliana normal gravity on the ellipsoid,

$$g_0(\lambda) = 9.7803267714 \frac{1 + 0.00193185138639 \sin^2 \lambda}{\sqrt{1 - 0.00669437999013 \sin^2 \lambda}}, \quad (42)$$

with an inverse-square altitude correction $g = g_0 (R_E/(R_E + h))^2$.²⁶

²²`models/atmosphere/ExtendedISAModel.java`.

²³Constants verbatim in `models/wind/PinkNoiseWindModel.java`: `ALPHA = 5/3`, `POLES = 2`, `STDDEV = 2.252`, `DELTA_T = 0.05`.

²⁴`models/wind/MultiLevelPinkNoiseWindModel.java`; OpenRocket ReleaseNotes.md, 24.12.

²⁵`util/GeodeticComputationStrategy.java`; `simulation/RK4SimulationStepper.java` and `simulation/AbstractEulerStepper.java`.

²⁶`models/gravity/WGSGravityModel.java`.

QtRocket offers two gravity models (Section 8): a constant field $(0, 0, -g_0)$ with $g_0 = 9.80665 \text{ m/s}^2$ `sim/ConstantGravityModel.h:18`, `utils/math/Constants.h:10` (the default, `sim/Environment.h:33`), and a true Newtonian central force over a spherical Earth:

$$\vec{a} = -\frac{\text{earthGM}}{r^3} \vec{r}_{\text{geo}}, \quad \vec{r}_{\text{geo}} = (x, y, z + \text{groundLevel}), \quad (43)$$

where the pad’s geocentric radius comes from a geoid model `sim/SphericalGravityModel.cpp:33`, `sim/SphericalGravityModel.cpp:36`. The only geoid returns the WGS84 mean radius 6 371 008.8 m at every latitude and longitude `sim/SphericalGeoidModel.cpp:18`, and the gravity model caches it at the placeholder site $(0, 0)$ `sim/SphericalGravityModel.cpp:20`. The spherical model is in one sense the more literal of the two central-force treatments — a downrange displacement picks up a genuine inward horizontal component, pinned by test `sim/tests/SphericalGeoidAndGravityTests.cpp:82`, and the surface magnitude matches GM/R^2 to 10^{-12} relative `sim/tests/EnvironmentTests.cpp:123` — but QtRocket has no latitude dependence, no Coriolis term, and no launch-site concept at all. For low-altitude model rockets both simulators’ options agree to well under 1%; the OpenRocket extras matter for long boosted flights and dispersion analysis, which QtRocket does not yet attempt.

C.6 Numerical integration

What an embedded pair buys

A Runge–Kutta step of order p commits an unknown local error of order h^{p+1} . An *embedded pair* evaluates one shared set of stage derivatives and combines them twice, with weight sets of order p and $p + 1$; the difference of the two results is a nearly free estimate of the lower-order solution’s local error, which a controller can compare against a tolerance to *reject and retry* a too-large step or grow a wastefully small one. A fixed-step or heuristically stepped integrator, however well tuned, never measures its own error; an embedded-pair integrator measures it every step. The cost is extra stages (six for Fehlberg’s 4(5) pair versus four for classical RK4) and the machinery of acceptance and rejection.

OpenRocket. The ascent stepper is classical fixed-formula RK4 (techdoc §4.2.4, eqs. 4.20–4.21) with the quaternion updated multiplicatively and renormalized each step. Step-size control is heuristic and a priori: after evaluating only k_1 , the stepper takes the minimum of eight candidate steps — the user step (default 0.05 s, divided by 5 on the launch rod), the time to the next scheduled flight event, a 3° default cap on pitch angle per step, a roll-angle step of $2 \times 28.32^\circ$ (deliberately an uneven division of the circle, to sample the wind), a 2° roll-rate-change cap, a 4° pitch/yaw-acceleration cap, a launch-rod resolution term, and a $1.5\times$ growth limiter — floored at 0.001 s.²⁷ There is no error estimate and no step rejection: the k_1 rates decide the step before it is taken, and the step is never re-integrated. (An RK6 stepper with the same heuristic selection exists on the development branch; it is not in the 24.12 release notes and should be treated as unreleased.) Recovery and tumble descend under a separate 3-DOF second-order Euler stepper with a 0.5 s base step, exact root-solves to land precisely on apogee and ground crossings, and a jerk-based oscillation guard.²⁸

QtRocket. The `Integrator` facade offers two runtime-selectable backends `sim/Integrator.h:32`: a fixed-step classical RK4 `sim/RK4Solver.h:27` and a true Runge–Kutta–Fehlberg 4(5) embedded pair whose Butcher tableau matches the textbook Fehlberg coefficients digit for digit `sim/RK45Solver.h:172`.

²⁷`simulation/RK4SimulationStepper.java, step()`; the candidate list is verbatim from the code comment. Defaults: `RECOMMENDED_TIME_STEP = 0.05 s`, `RECOMMENDED_ANGLE_STEP = 3^\circ`, `MIN_TIME_STEP = 0.001 s`.

²⁸`simulation/AbstractEulerStepper.java`.

Each attempt evaluates six stages at their proper node times, forms both the fourth- and fifth-order estimates, and accepts or rejects on the estimated local error (Listing 28): rejection shrinks h by $(\text{tol}/\text{err})^{1/4}$ floored at 0.1 and *retries the step*; acceptance grows the next guess by $(\text{tol}/\text{err})^{1/5}$ capped at $5 \times \text{sim/RK45Solver.h:141}$, clamps it to $h_{\max} = 10 \times$ the seeded step $\text{sim/RK45Solver.h:66}$, $\text{sim/RK45Solver.h:147}$ — the clamp is a correctness requirement, since exactly-integrated coasting drives $\text{err} \rightarrow 0$ and unbounded growth — and advances with the fifth-order estimate. Below $h_{\min} = 10^{-15} \text{ s}$ the solver throws $\text{sim/RK45Solver.h:80}$, which the run loop converts to a typed `IntegratorError` verdict $\text{sim/Propagator.cpp:171}$ (Section 7).

```
// Local error estimate: magnitude of the 5th-vs-4th-order difference across state and rate.
const double err = std::sqrt((y5State - y4State).squaredNorm()
    + (y5Rate - y4Rate).squaredNorm());

if(err > tol)
{
    // Reject: shrink the step and retry, flooring the shrink factor at 0.1.
    const double scale = std::pow(tol / err, 0.25);
    h *= (scale < 0.1) ? 0.1 : scale;
    continue;
}
```

Listing 28. The step-rejection core OpenRocket’s ascent stepper has no analogue of: an embedded-pair error estimate compared against a tolerance, with the step retried when it fails. `sim/RK45Solver.h:120`

What each scheme does and does not control

OpenRocket’s heuristics bound the *geometry* of a step — degrees of pitch or roll per step — which is an excellent proxy for the stiff part of a 6-DOF problem, but they never measure integration error and cannot detect a step that was numerically too coarse; accuracy rests on the caps being conservative. QtRocket’s RKF45 measures the local truncation error of exactly what it integrates and rejects steps that exceed tolerance — a genuine methodological advantage, and the one area where QtRocket’s numerics are strictly ahead — but three qualifiers bound the claim. The error norm is a single absolute Euclidean norm mixing meters and meters/second `sim/RK45Solver.h:121`, flagged in-source as a possible refinement `sim/RK45Solver.h:36`; there is no safety factor or PI controller, only the $0.1/5 \times$ clamps; and the tolerance (default 10^{-6} , `sim/RK45Solver.h:46`) is reachable from no front end — `setErrorTolerance/setMaxStepSize` have no production callers `sim/tests/RK45SolverTests.cpp:112`. And with only three linear DOF integrated, the adaptive machinery currently controls a far easier problem than OpenRocket’s.

Head to head on the same 3-DOF flight, QtRocket’s RKF45 reproduces its RK4 apogee to about 0.05% while taking roughly 530 steps where RK4 at the same seed takes about 4050 `tests/PhysicsIntegrationTests.cpp:164`, `tests/PhysicsIntegrationTests.cpp:171`. QtRocket’s default fixed step is 0.01 s `sim/Propagator.h:110` against OpenRocket’s default 0.05 s — the finer default compensating for the absence of angle-based caps.

C.7 Motors

Both simulators represent thrust identically in kind: a measured thrust curve, linearly interpolated between samples, zero outside the burn. OpenRocket’s `ThrustCurveMotor` carries (time, thrust,

CG+mass) sample lists and linearly interpolates all three;²⁹ QtRocket’s `ThrustCurve` interpolates thrust the same way `model/ThrustCurve.cpp:83`, including a regression-pinned ramp from an implicit (0, 0) origin when a curve omits its first sample `model/ThrustCurve.cpp:76` (Section 6).

Mass depletion is where they part. When a RockSim `.rse` file supplies measured per-sample mass and CG, OpenRocket uses those samples directly; only for RASP `.eng` files (which carry just total and propellant mass) does it fall back to distributing mass loss proportional to burned impulse, $\Delta m_i \propto \frac{1}{2}(f_i + f_{i+1}) \Delta t_i$, scaled to consume exactly the propellant mass.³⁰ QtRocket *always* reconstructs mass under the constant-Isp assumption, even when measured data is present: the RSE loader reads only each sample’s `t` and `f` attributes and discards the file’s per-sample `m` and `cg` `model/RSEDatabaseLoader.cpp:88`, and `computeMassCurve` builds a 128-point table `model/MotorModel.cpp:134` by left-Riemann depletion proportional to delivered impulse,

$$\text{propMass}_{i+1} = \text{propMass}_i - F(t_i) \Delta t \frac{\text{propWeight}}{\text{totalImpulse}}, \quad \Delta t = \frac{\text{burnTime}}{127}, \quad (44)$$

`model/MotorModel.cpp:141`, forcing the final point to exact depletion so post-burn mass is exactly the casing mass `model/MotorModel.cpp:143`. The physical content is the same as OpenRocket’s RASP fallback — mass flow proportional to thrust — so the practical difference appears only for motors whose measured burn deviates from constant effective exhaust velocity. Two QtRocket-side quirks are worth restating from Section 6: mass and thrust gate burnout on different clocks (metadata `burnTime` versus the curve’s last sample time, `model/MotorModel.cpp:36`, `model/MotorModel.cpp:72`), and a derived `isp` member is computed and never read `model/MotorModel.cpp:130`.

Ignition and delays: OpenRocket treats motor ignition, burnout, and ejection charges as flight events, each trigger optionally carrying a delay time — this is how staging and dual deployment are expressed (techdoc §4.2.6, Table 4.2). QtRocket ignites its single motor exactly once, at $t = 0$, from `RocketModel::launch()` `model/RocketModel.cpp:105`; `ThrustCurve::setIgnitionTime` has zero call sites `model/ThrustCurve.h:31`, and ejection-delay lists are parsed by every loader and consumed by nothing `model/MotorModel.h:303`.

C.8 Recovery physics

OpenRocket has a complete recovery subsystem. A parachute is a drag device with default $C_D = 0.8$ on its canopy area;³¹ a streamer’s drag comes from Niskanen’s own wind-tunnel experiments,

$$C_D = 0.034 \cdot \frac{\rho_m + 25 \text{ g/m}^2}{105 \text{ g/m}^2} \cdot \frac{l + 1 \text{ m}}{l}, \quad (45)$$

good to roughly 15% in descent rate (techdoc App. C, eq. C.6); tumbling stages use a two-component drop-test model with $C_{D,f} = 1.42$ and $C_{D,bt} = 0.56$ (techdoc §3.5, eq. 3.99). Deployment is driven by the flight-event system — apogee detection, altitude-threshold main deployment, ejection-charge delays — and switches the active stepper from 6-DOF RK4 to the 3-DOF Euler landing stepper (techdoc §4.2.5–4.2.6).

QtRocket has none of this: no recovery device exists in the part vocabulary, no deployment event exists in the simulation loop, and descent is simply the same constant- C_d point mass falling until the nominal termination condition — descending *and* below the launch site, $z < 0 \wedge v_z < 0$ `model/RocketModel.cpp:65` — ends the flight. The only event-like machinery is the typed termination taxonomy itself `sim/Propagator.h:35`. Recovery is an absence, not a dormancy; Section 13 carries it in the consolidated inventory.

²⁹`motor/ThrustCurveMotor.java`.

³⁰`file/motor/AbstractMotorLoader.calculateMass()`.

³¹`rocketcomponent/Parachute.java, DEFAULT_CD`; techdoc §4.2.5.

C.9 Validation posture

OpenRocket published flight-data validation. Against altimeter flights of a small model rocket, apogee error was +16% and +7% (versus RockSim’s +24% and +19% on the same flights); a hybrid-powered flight came in at −16%; a magnetometer-instrumented rolling rocket spun at 16 rev/s where the simulation predicted about 8 — roll rate under-predicted by roughly 2×, cause unknown; wind-tunnel comparison put the simulated CP within a few percent of body length at subsonic speeds. The headline claim is CP and drag-coefficient accuracy of “approximately 10% at subsonic velocities” (techdoc ch. 6–7).

QtRocket, at this commit, has no whole-flight field validation: no test compares a simulated trajectory against a recorded flight, and every oracle in the six test binaries is analytic or table-derived. What it has instead is unusually dense component-level *verification*: atmosphere properties to 0.1–0.5% of the NOAA report tables `sim/tests/USStandardAtmosphereTests.cpp:36`; the RKF45 solver against exact solutions — constant acceleration to 10^{-9} `sim/tests/RK45SolverTests.cpp:43`, a harmonic oscillator to 10^{-4} `sim/tests/RK45SolverTests.cpp:126`, exponential decay to 10^{-5} `sim/tests/RK45SolverTests.cpp:150`, a thrust-ramp analogue to 10^{-6} `sim/tests/RK45SolverTests.cpp:175`; a ballistic arc’s apogee against $v_0^2/2g$ to 0.5% `tests/PropagatorTests.cpp:178`; the drag law against the analytic terminal-velocity balance to 10^{-6} N `tests/PhysicsIntegrationTests.cpp:262`; inverse-square surface gravity to 10^{-12} relative `sim/tests/EnvironmentTests.cpp:123`.

Verification is not validation

The two projects’ accuracy numbers answer different questions and should not be compared digit for digit. OpenRocket’s ~10% is *validation*: model versus nature, with all modeling error included — and its published misses (roll off 2×, supersonic drag over-predicted) are part of the claim’s honesty. QtRocket’s 10^{-4} -to- 10^{-12} oracles are *verification*: code versus its own stated mathematics, guaranteeing the implementation computes the declared model correctly while saying nothing about whether that model matches a real flight. Given the force model this appendix has documented — constant C_d on a defective reference area, no wind, no attitude — QtRocket could not currently support an OpenRocket-style 10% whole-flight claim, and it does not make one. The honest statement is: OpenRocket’s physics is validated to ~10%; QtRocket’s physics is verified to compute exactly what it says, and what it says is still a point mass.

C.10 Summary

Read column-wise, the table repeats the conclusion of [Section B](#) in numerical form: the gap is concentrated in what QtRocket *consumes*, not in what it has built. The one method where QtRocket is ahead — a genuine embedded-pair integrator with step rejection — is also the one whose advantage will matter most on the day the dormant machinery in the other rows is wired in.

D Architecture comparison with OpenRocket

The two preceding appendices compared what the tools offer ([Section B](#)) and how their physics is computed ([Section C](#)). This appendix compares them as software: how each is built and modularized, how each represents a rocket in memory, how each positions parts and computes mass properties, how each structures a simulation run, where each lets outside code in, and how each persists its work. The framing of [Section A](#) applies throughout — QtRocket is an independent project, OpenRocket is a calibration baseline, and the comparison measures distance, not conformance. It is also worth saying once what the comparison is *for*: the two architectures were shaped by different pressures (a

Table 17. Methods at a glance. OpenRocket per its techdoc and current source; QtRocket per the pinned commit 254cd41. “Dormant” marks machinery that is built and tested with zero production consumers.

Area	OpenRocket	QtRocket
Degrees of freedom	6-DOF quaternion ascent, two scalar inertias; deliberate 3-DOF degradation for recovery and tumble	3-DOF point mass; quaternion slots and full 3×3 recorded inertia dormant
Aero coefficients	Extended Barrowman at instantaneous AoA/Mach each step: body lift, interference, damping, roll, transonic fin CP	Classic Barrowman at $\alpha \rightarrow 0$, no Mach dependence; complete, tested, dormant
Drag	Per-component semi-empirical build-up: friction with roughness floor, per-shape wave drag, base $0.12 + 0.13M^2$, lug, $C_A(\alpha)$ hump	Single constant $C_d \times$ constant reference area; geometry-derived area implemented but never called
Atmosphere	ISA, 8 layers, launch-site override, geopotential conversion	US Standard 1976, 7 layers, closed-form; no overrides; verified 0.1–0.5% to 80 km
Wind	Average + 2-pole pink-noise turbulence, 20 Hz; multi-level profiles (24.12)	Zero-wind stub, zero call sites
Gravity/geodesy	Somigliana latitude gravity, flat/spherical/WGS84-Vincenty positions, Coriolis	Constant g_0 or true inverse-square over mean-radius geoid; no latitude, no Coriolis, no launch site
Integration (ascent)	RK4, heuristic a-priori step caps (8 candidates, 3° angle), no error estimate, no rejection	RK4 or embedded-pair RKF45 with acceptance/rejection, $h_{\max}$ clamp; tolerance not user-reachable
Integration (descent)	Adaptive 2nd-order Euler, exact apogee/ground root-solves	Same 3-DOF loop as ascent; termination at $z < 0 \wedge v_z < 0$
Motor mass	Manufacturer per-sample mass/CG when available; impulse-proportional fallback for RASP	Constant-Isp 128-point reconstruction always; measured RSE mass/CG discarded
Recovery	Parachute/streamer/tumble models, event-driven deployment with delays	Absent
Validation	Flight-data validated: $\sim 10\%$ apogee subsonic; roll off $\sim 2\times$	Component-level analytic/table oracles (10^{-4} – 10^{-12}); no whole-flight field data

seventeen-year-old product with a large user base; a young core built seams-first), and reading them side by side shows which of QtRocket’s choices are genuine alternatives and which are simply earlier points on the same road.

Provenance differs by side, as everywhere in these appendices. OpenRocket statements were researched from the project’s repository at tag [release-24.12](#) and branch [unstable](#) (examined 2026-07-07) and from its published developer documentation³²; they are not file-and-line verified the way QtRocket claims are. QtRocket statements carry [file:line](#) citations into the pinned commit [254cd41](#). [Table 18](#) is the map; the subsections walk it row by row.

Dimension	OpenRocket 24.12	QtRocket 254cd41
Language / build	Java 17, Gradle, two JPMS modules	C++23, CMake, layered static libraries
Wiring	Guice injector behind a static locator	Controller singleton; constructor injection at the composition root
Design tree	23 concrete RocketComponent classes, per-class child-compatibility rules	5 Part types in one composite, 4 factory-constructible
Mass properties	External MassCalculator , recomputed per step, hookable	In-tree composite cache behind dirty flags and a mass-delta gate
Positioning	AxialMethod + RadiusMethod offsets; nose-tip origin, +x aft	StationLink placement intent + one resolver; nose-tip datum, +z = forward
Simulation control	Event queue, 16 event types, four phase steppers, forking per stage	Single loop, typed termination, no events
Extensibility	Per-step listener hooks, GraalJS scripting, annotation-scanned plugins	No plugin surface; machine-scriptable REPL protocol
Persistence	Versioned .ork ZIP+XML; byte-signature multi-format import	Single-format .qrd XML; placement intent; fail-closed loads
Motor data	Bundled snapshot (SQLite pipeline on unstable)	Ships empty; three ingest paths including live REST
GUI	Mature Swing product with live background simulation	Thin Qt front end; authoring is CLI-first

Table 18. The architecture comparison at a glance. Each row is expanded, with sources, in the subsection that follows it.

D.1 Language, runtime, and build

OpenRocket is Java: [Application.java](#) hard-codes `SUPPORTED_JRE_VERSIONS = { 17 }`, and CI builds against Temurin 17. The build is Gradle, producing two subprojects that are also two Java Platform Module System (JPMS) modules, each with its own `module-info.java`: `info.openrocket.core` — the simulation engine, file formats, and design classes, deliberately headless and reusable — and `info.openrocket.swing`, the GUI, which depends on core and carries the application entry point. Object wiring uses Google Guice 7: a static service locator, `info.openrocket.core.startup.Application`, holds the injector, and code obtains services through it (`FlightConfiguration`, for example, calls `Application.getInjector().getInstance(RocketDescriptor.class)`); the older generation of static

³²<https://github.com/openrocket/openrocket> and https://openrocket.readthedocs.io/en/latest/dev_guide/architecture.html. Class, enum, and package names quoted below are exact; short code excerpts are verbatim from the tagged sources.

accessors is marked deprecated in favor of the injector (“@deprecated Fetch the db from Guice instead”).

QtRocket is C++ `CMakeLists.txt:5` under CMake, and its modularity is the link graph rather than a module system: `utils` links only third-party math and networking `utils/CMakeLists.txt:14`, `sim` links only `utils` `sim/CMakeLists.txt:28`, `model` links `utils` and `sim` `model/CMakeLists.txt:44`, and the controller compiles once into the `qtrocket_core` static library that every executable links `CMakeLists.txt:224`. Qt is confined to the GUI executable and the visualizer; the core, like OpenRocket’s, is headless by construction (Section 3). Wiring is manual: the controller is a process-wide singleton `QtRocket.cpp:19` acting as the composition root, and the one dependency that must vary per run — the `Environment` — is injected into the `Propagator`’s constructor there `QtRocket.cpp:48`, never reached through a global.

The honest version of the “DI container versus singletons” contrast is narrower than the labels suggest. OpenRocket’s injector is itself resolved through a static locator, so most call sites still reach a global — what the container buys is *rebindability*: tests and plugins can install different bindings without touching call sites, and the plugin framework (Section D.5) depends on exactly that. QtRocket’s object graph is small enough to wire by hand; it pays for its two singletons with a known blemish (the controller’s double-checked initialization reads its flag outside the mutex `QtRocket.cpp:21`, where the `Logger` one layer down already uses the safe Meyers form `utils/Logger.cpp:19`) and gets its rebinding from ordinary constructor seams instead — the injected `Environment`, and the fake-injectable `ThrustCurveAPI` interface behind the motor database `model/ThrustCurveClient.h:68`.

One instinct, two enforcement mechanisms

Both projects enforce the same first-order rule — a headless core that must not know about its GUI. OpenRocket enforces it with the module system: `swing` requires `core`, never the reverse, and the JVM checks the boundary. QtRocket enforces it with the link graph and one comment stated where it binds: “Bottom layer: utils must not depend on model/ or sim/” `utils/CMakeLists.txt:13`, with Qt appearing only in front-end link lists `CMakeLists.txt:286`. A module system checks the rule mechanically; a link graph makes violating it a build failure. Either way, the invariant is load-bearing, not documentation.

D.2 The design tree and mass properties

Both projects model a rocket as a tree of components; the trees differ in breadth and, more interestingly, in where the rules live. OpenRocket’s `info.openrocket.core.rocketcomponent` package defines roughly 23 concrete classes under three abstract families — `ComponentAssembly` (the `Rocket` root, stages, pods), `ExternalComponent` (body tubes, transitions, nose cones, four fin-set classes, lugs, rail buttons), and `InternalComponent` (rings, couplers, bulkheads, mass objects, recovery devices) — with cross-cutting capability interfaces such as `MotorMount` and `RingInstanceable`. Tree legality is enforced per class: every component implements `isCompatible(Class<? extends RocketComponent> type)`, and `addChild()` throws when the answer is no. `Rocket` accepts only `AxialStage`; `BodyTube` accepts pods, internals, and any external component that is not itself a body — fins attach to tubes, tubes do not nest in tubes. The type system carries the domain rules.

QtRocket’s composite has five concrete `Part` types registered in one umbrella header `model/parts/Parts.h:4`, four of them constructible through a factory that dispatches on the type name and validates required fields `model/parts/PartFactory.cpp:27` over a single type-agnostic parameter struct `model/parts/PartFactory.h:27` (the `Motor` is deliberately excluded `model/parts/PartFactory.cpp:73`). Attachment rules are structural rather than typed: `addChildPart` rejects null, already-parented, and

cycle-forming children `model/parts/Part.cpp:71`, but any part may parent any other — there is no `isCompatible` analogue. What QtRocket has instead is a *geometric* legality gate: the placement sweep of [Section 5](#) checks the resolved tree for self-intersection and fails the mass computation closed when it finds one `model/parts/Part.cpp:184`. OpenRocket asks “may this class contain that class?” at edit time; QtRocket asks “does this geometry physically fit?” at resolve time. The two checks are complementary, and neither project currently has both.

The mass-properties split is the genuinely different philosophy. OpenRocket keeps mass *out* of the components: the `masscalc` package is deliberately small — `MassCalculator`, `MassCalculation`, and `RigidBody`, the aggregate mass-and-inertia result — and the calculator walks the component tree afresh whenever the simulation needs mass properties; nothing is cached inside components. QtRocket computes composite mass, CM, and inertia *inside* the tree: a two-pass, time-aware walk `model/parts/Part.cpp:175` whose per-part tensors follow the per-unit-mass convention `model/parts/Part.h:30` and whose results are cached behind a structural dirty flag and the mass-delta gate `model/parts/Part.cpp:137` ([Section 4](#)).

OpenRocket: components describe, a calculator computes

Because mass computation is a free-standing service rather than a component method, it is naturally an interception point: the listener interface of [Section D.5](#) exposes `preMassCalculation/postMassCalculation` hooks that can replace or adjust the `RigidBody` a step sees. The components stay a pure description of the rocket; every consumer — simulator, analysis dialogs, plugins — goes through one calculator. The price is recomputation on every step and no natural home for a cache.

QtRocket: the cache that can never be wrong — under one assumption

Storing the composite in the tree buys QtRocket a property OpenRocket’s per-step calculator does not have: after burnout, when `getMass(t)` repeats bit-for-bit, the cache freezes and repeated or rejected integrator stage queries are served bit-stably `model/parts/Part.h:248`, while the ungated live mass sum stays cheap as the ODE divisor `model/parts/Part.cpp:118`. The price is that the gate keys on *total* mass alone, so its soundness is coupled to the single-motor assumption `model/RocketModel.cpp:13` — two simultaneous burners holding total mass constant while shifting CM would be served stale properties. Neither design dominates: OpenRocket pays per step for generality; QtRocket pays in coupled assumptions for bit-stability.

D.3 Positioning models

OpenRocket positions components with a pair of enums. Each component stores an (`AxialMethod`, `offset`) pair, where `AxialMethod` is one of `ABSOLUTE`, `AFTER`, `TOP`, `MIDDLE`, `BOTTOM`; each value implements `getAsPosition(offset, innerLength, outerLength)` returning the component’s front-face coordinate in the parent’s frame (`MIDDLE`, for instance, is `offset + (outerLength - innerLength)/2`). `AFTER` is the internal sibling-stacking mode for body components in sequence; the GUI exposes the other four. Radial placement is a second enum, `RadiusMethod` (`COAXIAL`, `FREE`, `RELATIVE`, `SURFACE` — the last a flush mount on the parent’s outer surface). The arithmetic is purely geometric: component lengths only, never mass or CM. The global frame puts the origin at the nose tip with `+x` pointing *aft*.

QtRocket stores a `StationLink` per attachment — fractional stations on both ends of the seam, a gap, and a seat kind `model/parts/Placement.h:49`, with the seat kind closed at three values because a rigid axisymmetric stack admits only three radial relationships `model/parts/Placement.h:32` — and derives every absolute pose through one resolver, “intent (`StationLink`) in, geometry (`Pose`) out”

`model/parts/Placement.h:148`, whose core is two lines of arithmetic `model/parts/Placement.cpp:20`. The frame puts the datum at the nose tip with $+z$ = forward: each part occupies $z \in [-L, 0]$ in its own frame `model/parts/Part.h:130`. The frame-convention difference is therefore concrete: both projects plant the origin at the nose tip, but their longitudinal axes run in *opposite directions* — a station aft of the tip has positive x in OpenRocket and negative z in QtRocket, and OpenRocket uses x where QtRocket uses z . Any future `.ork` import will live or die on getting that sign flip right once, in one place.

The deeper distinction is what is stored. OpenRocket stores geometry directly: the offset is a signed length between anchor planes, computed by the author, and the method names which planes. QtRocket stores intent and derives the geometry: the link names which feature of the child seats against which feature of the parent, the seat kind selects the gap’s sign and the radial compatibility rule, and the resolver — pure geometry, “No CM, time, or mass” `model/parts/Placement.h:158` — turns it into coordinates on every read (Section 5). The gap between the two is smaller than it first appears: OpenRocket’s TOP/MIDDLE/BOTTOM anchors are themselves re-evaluated against live lengths at query time, so both systems survive a parent-length edit. What OpenRocket’s scheme does not carry is radial semantics in the axial store (radius is a separate method with no compatibility meaning) or a validation gate coupled to placement; what QtRocket’s scheme does not carry is OpenRocket’s FREE/RELATIVE radial freedom — off-axis seating is deliberately deferred to 6-DOF (Section 13). Both projects converged independently on the same load-bearing rule: placement arithmetic must never involve mass. QtRocket arrived there the hard way, by replacing a CM-coupled legacy scheme (Section 5); OpenRocket appears to have started there.

D.4 The simulation engine

This is the largest structural difference between the two codebases. OpenRocket’s engine, `BasicEventSimulationEngine`, is event-driven: a time-ordered `EventQueue` rides on the mutable `SimulationStatus`, the run begins by pushing a LAUNCH event, and the loop alternates between handling due events and advancing the ODE, clamping each step so it never overshoots the next queued event. The `FlightEvent.Type` enum names sixteen event kinds — among them IGNITION, LIFTOFF, LAUNCHROD, BURNOUT, EJECTION_CHARGE, STAGE_SEPARATION, APOGEE, RECOVERY_DEVICE_DEPLOYMENT, GROUND_HIT, TUMBLE, and SIMULATION_END. Integration is delegated to a `SimulationStepper` chosen per flight phase (Listing 29): full 6-DOF RK4 under power and coast, a simplified 3-DOF stepper under a deployed recovery device, a tumble stepper, and a ground stepper after landing. Stage separation *forks the simulation*: the engine pushes a copy of `SimulationStatus` onto a `toSimulate` stack and later pops it to fly the separated booster, each branch producing its own `FlightDataBranch` in the result.

```
private final SimulationStepper flightStepper = new RK4SimulationStepper();
private final SimulationStepper landingStepper = new BasicLandingStepper();
private final SimulationStepper tumbleStepper = new BasicTumbleStepper();
private final SimulationStepper groundStepper = new GroundStepper();
```

Listing 29. The four phase steppers of OpenRocket’s event-driven engine, quoted from `BasicEventSimulationEngine.java` at tag `release-24.12`. The engine swaps steppers as events change the flight phase; the fidelity of the dynamics is a per-phase decision, not a global one.

QtRocket’s engine is a single loop: `Propagator::runUntilTerminate()` `sim/Propagator.cpp:58` advances one solver (RK4 or the embedded-error RKF45, selected at runtime through the integrator facade `sim/Integrator.h:52`) until the model’s own predicate says the flight is over — descending below

the launch site `model/RocketModel.cpp:62` — or one of the guard exits fires. There is no event queue, no phase steppers, and no forking; what QtRocket has instead is the typed five-value termination taxonomy `sim/Propagator.h:35` that partitions every possible exit into a diagnosis (Section 7). The two designs are honest reflections of their feature sets: OpenRocket needs an event architecture because recovery, staging, and tumbling *are* events; QtRocket’s every flight is single-phase and ballistic, so a loop with a predicate is the whole requirement. The event system is precisely the architectural piece QtRocket’s planned recovery and staging work will need, and the termination taxonomy is its natural seed (Section 13).

D.5 Extension surfaces

OpenRocket lets outside code into the running simulation. Three listener interfaces attach to a run: `SimulationListener` (start/end/pre-step/post-step, with a boolean pre-step veto), `SimulationEventListener` (observe or veto flight events), and — the key mechanism — `SimulationComputationListener`, which brackets *every physics sub-computation* with paired hooks: `pre/post` around acceleration, the atmospheric model, wind, gravity, flight conditions, aerodynamics, mass calculation, and thrust. A `pre*` hook returning non-null replaces the model’s output for that step; a `post*` hook may modify the result. On top of the listeners sit simulation extensions (the tree ships a `JavaCode` extension and script-backed ones), JavaScript scripting through GraalJS behind the standard JSR-223 API, and a plugin framework in which providers are discovered by classpath annotation scan (`@Plugin`, `ClassGraph`) and bound through Guice modules — runtime plugin JARs, enabled by the container architecture of Section D.1.

QtRocket has no listener, plugin, or scripting mechanism of any kind. Its pluggability is internal and compile-time-shaped: string-keyed registries select among the built-in gravity, atmosphere `sim/Environment.h:31`, and integrator implementations `sim/Integrator.h:52`, and the front ends enumerate those registries so their vocabulary cannot drift from the core’s `gui/SimOptionsTab.cpp:32`. What QtRocket offers outside code instead is the REPL: 33 commands over a machine-parseable stdout protocol — exactly one `OK/ERR` line per command, `CSV` paths, `#` comments `cli/Repl.cpp:245` — deliberately kept clean by silencing the logger to `ERROR` in the CLI binary `cli/CliMain.cpp:17` (Section 10). These are different answers to different questions. OpenRocket extends *inward*: injected code runs between physics sub-steps and can replace a model mid-flight, which is what active control experiments, custom wind fields, or air-starts need. QtRocket extends *outward*: an external program drives whole runs over a text protocol, which is what parameter sweeps, CI harnesses, and reproducible batch studies need — and which OpenRocket itself has no interactive equivalent for (its headless story is a separate Python helper module, `orhelper`³³). Neither surface subsumes the other: an outward protocol cannot touch a flight mid-step, and an inward hook is no substitute for a scriptable process boundary.

D.6 Persistence

OpenRocket’s design file, `.ork`, is a ZIP archive containing a versioned XML document (`rocket.ork`) plus attached resources, and its format history is explicit: version 1.8 landed with release 22.02, 1.10 with 24.12, and 1.11 (embedding thrust curves in the archive) is staged for the next release³⁴. The loader, `GeneralRocketLoader`, auto-detects what it is fed by byte signature — GZIP, ZIP, bare

³³<https://github.com/openrocket/orhelper> — an org-maintained Python module “to facilitate interacting and scripting with OpenRocket.”

³⁴Format-version table and archive layout: https://openrocket.readthedocs.io/en/latest/dev_guide/file_specification.html.

<openrocket XML, RockSim’s <RockSimDoc, RASAero’s <RASAeroDoc — so files from two foreign ecosystems and every historical OpenRocket version load through one entry point; RockSim is supported for both import and export. QtRocket persists designs in exactly one format: .qrd XML at version 0.2 `model/DesignSerializer.cpp:220`, which stores placement *intent* — the <link> elements — and re-resolves geometry on every load `model/DesignSerializer.cpp:93`. Its loader is fail-safe in structure and fail-closed in vocabulary: the tree is built detached and installed atomically so a bad file leaves the current design untouched `model/DesignSerializer.cpp:255`, an unknown seat kind is rejected outright `model/DesignSerializer.cpp:184`, and the retired 0.1 format is refused with an explicit “must be re-created” rather than migrated `model/DesignSerializer.cpp:194` (Section 11).

The postures could not be more different, and each is right for its context. OpenRocket’s loader is maximally forgiving because two decades of user files and three neighboring ecosystems exist; rejecting any of them would strand real work. QtRocket’s loader is maximally strict because almost no files exist yet, so rejection is cheaper than a migration framework and guarantees that anything loaded is coherent. The asymmetry has a shelf life: the .ork version table is a precedent for exactly the machinery .qrd will need once users accumulate files, and OpenRocket’s content-sniffing entry point is the natural shape for any future foreign-format import (Section 13).

D.7 Motor data

OpenRocket ships its motor corpus with the application: release 24.12 bundles a 3.3 MB serialized snapshot of thrustcurve.org data (`thrustcurves.ser`), and the `unstable` branch replaces it with an SQLite database (`initial_motors.db`, 1,458 motors and 1,591 curves per its metadata) published from a dedicated data-pipeline repository³⁵ — motor data curation has been moved out of the application into a data engineering pipeline. QtRocket inverts the default: the shipped database is empty (the bundled .rse file is wired into test binaries only, via a compile definition `tests/CMakeLists.txt:16`), and motors arrive at run time through three ingest paths behind one facade — the RockSim .rse loader `model/RSEDatabaseLoader.cpp:24`, the strict validating RASP .eng parser `model/RASPLoader.cpp:137`, and a live thrustcurve.org REST client hidden behind the fake-injectable `ThrustCurveAPI` seam `model/ThrustCurveClient.h:68`. The loaders are owned inside the database and never touched by front ends `model/MotorModelDatabase.cpp:63`; storage is a common-name-keyed map `model/MotorModelDatabase.h:129` (Section 6).

Architecturally this is offline-first versus online-first. OpenRocket’s user is never blocked by the network and never sees an empty chooser; the cost is a release-time curation pipeline and data that ages between releases. QtRocket’s user gets live search against the canonical source — a workflow the baseline’s in-app selection does not lead with (OpenRocket’s core carries thrustcurve.org client classes, but its chooser is fed by the bundled corpus) — at the cost of a first run that begins with nothing until a file is imported or the network answers. The designs are converging from opposite ends: OpenRocket’s new SQLite pipeline adds in-app updating to a bundled corpus, and QtRocket’s natural next step is bundling a snapshot in front of its live client.

D.8 GUI architecture

OpenRocket’s Swing GUI is the product: one document window hosting the 2-D rocket figure and JOGL-based 3-D view, per-component configuration dialogs, plotting, printing, and an optimizer. Its most architecturally interesting habit is *live background simulation*: `RocketPanel` owns a static `ExecutorService` — “a fixed-sized thread pool for all background simulations with all threads in

³⁵<https://github.com/openrocket/motor-database> — “a motor database for OpenRocket based on thrustcurve.org data and other sources,” republished as `motors.db.gz` for in-app updates.

daemon mode and with minimum priority” — and every design edit re-runs a simulation off the interaction thread to refresh the altitude, velocity, and stability readouts drawn on the design canvas, alongside continuously recomputed CP/CG markers. Simulation is treated as a rendering input, cheap enough to run speculatively on every edit.

QtRocket’s GUI is a thin front end over the same core its CLI drives, and at the pinned commit it is the narrowest of the three surfaces: the design widgets are constructor-only stubs `gui/RocketTreeView.cpp:3`, the File actions ship disabled `gui/MainWindow.ui:117`, and design authoring lives in the REPL (`newdesign cli/Repl.cpp:784`, `addpart cli/Repl.cpp:825`). Its threading posture is the inverse of OpenRocket’s twice over: the entire Qt event loop runs on a *spawned* `std::thread` that constructs `QApplication` `gui/GuiRunner.cpp:62` — an inversion of Qt’s main-thread convention whose consequences [Section 10](#) examines — while simulation runs *synchronously* on that GUI thread from the launch button’s slot `gui/CannonballTab.cpp:116`. OpenRocket pushes simulation off the interaction thread by default; QtRocket has not yet separated the two at all. The one place the QtRocket rendering story is architecturally ahead is outside the GUI proper: the standalone visualizer positions geometry with the exact resolver the simulator’s mass pass consumes ([Section 10](#)), a single-source-of-truth property OpenRocket’s separate figure-rendering path does not have.

D.9 What each architecture makes easy

Read as wholes, the two architectures optimize for different verbs. OpenRocket’s makes *extending the flight* easy: a new flight phenomenon is a new event type absorbed by an engine already built around a queue; a new physics variant is a listener hook away; a foreign file is one more byte signature; and the container wiring lets plugins rebind services without touching call sites. The costs are the ones mature products accept — behavior assembled at run time by injection and annotation scan is harder to trace statically, the loader’s forgiveness dilutes invariants, and the simulation’s warn-and-continue posture ([Section A](#)) means a produced number does not always certify a coherent model. QtRocket’s architecture makes *trusting the number* easy: the link graph is the layering, placement has one resolver shared by physics and rendering, incoherent designs fail closed with located diagnostics, and a deterministic core behind a text protocol makes every result reproducible from a script. Its costs are the mirror image: no way into a flight mid-step, a closed persistence world, an empty first run, and — the expensive one — no event architecture, so the recovery and staging features that define the baseline’s flight realism require growing a new control structure, not just new physics ([Section 13](#)). Neither project would be improved by adopting the other’s architecture wholesale; each is shaped correctly for its stage. The useful reading of OpenRocket for QtRocket’s purposes is a preview of which seams get consumed next — the event queue, the background-simulation executor, the format-version table — because the baseline demonstrates, in shipped code, what those seams are for.

E Glossary

The vocabulary this document leans on, in two parts: general rocketry, aerodynamics, and numerical-methods terms first, then QtRocket’s own terms of art — the vocabulary shared with the companion Architecture Review — quoting the project’s in-tree definitions verbatim where they exist. Entries are alphabetical within each part; each cross-references the chapter where the concept lives, and code citations anchor to the pinned commit [254cd41](#).

E.1 Rocketry and physics terms

Adaptive step control. Letting an ODE solver choose its step size from a local error estimate: shrink and retry above tolerance, regrow while the dynamics are smooth. QtRocket’s RKF45 shrinks with exponent 1/4 and grows with exponent 1/5, under fixed clamps `sim/RK45Solver.h:124` (Section 7).

Angle of attack (AoA). The angle between the body axis and the velocity through the air; at small angles each component’s aerodynamic normal force is proportional to it. Computed nowhere at the pinned commit (Section 9).

Apogee. Peak altitude, where vertical velocity crosses zero and real flights deploy recovery. Recorded as a running maximum every step `sim/TrajectoryStatistics.h:26` (Section 7).

Ballistic. Flight under gravity and drag alone. Every QtRocket flight ends ballistically: descent runs under the ascent drag law until the vehicle is below the launch site and descending `model/RocketModel.cpp:65` (Sections 7 and 13).

Barrowman method. The standard model-rocketry stability calculation (James and Judith Barrowman, 1966): each component contributes a normal-force slope C_{N_α} and a center of pressure at small angle of attack, composed by a C_{N_α} -weighted average. Implemented per part and per composite; dormant (Section 9).

Burnout. The end of the propellant burn: zero thrust afterward `model/MotorModel.cpp:72`, casing mass only. See *two burnout clocks* in Section E.2 (Section 6).

Caliber. One body diameter, the conventional unit for static margin; one to two calibers is the hobby rule of thumb. The word appears nowhere in the tree (Section 9).

Center of gravity / center of mass (CG, CM). The mass-weighted mean position of the vehicle’s material. Computed time-aware from the part tree, so the burning motor moves it every step `model/parts/Part.cpp:196` (Section 4).

Center of pressure (CP). The station where the aggregate aerodynamic normal force acts; a rocket is statically stable when it sits aft of the CG. Composed as the guarded moment ratio `cnAlphaXcp / cnAlpha` `sim/Aero.h:51`, unconsumed in flight (Section 9).

Degrees of freedom (3-DOF, 6-DOF). The coordinates a simulation integrates: three translational, or six with attitude. QtRocket integrates exactly the three linear DOF `sim/Propagator.cpp:36`; the rotational half is typed, never-integrated scaffolding (Section 7).

Dynamic pressure. $q = \frac{1}{2}\rho v^2$, the pressure scale of aerodynamic forces; present inside the drag law `model/RocketModel.cpp:88`, never a standalone variable (Section 9).

Ejection charge; ejection delay. The charge a motor fires a fixed delay after burnout to deploy recovery. The RSE and RASP loaders parse delay lists and the motor database persists them — 1000 is the sentinel for a plugged motor with no charge `model/MotorModel.h:303`; the thrustcurve.org client leaves them empty, a marked `TODO` — and nothing consumes them (Sections 6 and 13).

Geoid. The equipotential surface of Earth’s gravity field (“mean sea level”). QtRocket’s only geoid returns the WGS84 mean radius, 6 371 008.8 m, for any coordinates `sim/SphericalGeoidModel.cpp:18` (Section 8).

Impulse class. The letter designations (A, B, C, ...) each doubling the previous class’s total-impulse ceiling, with 1/4A and 1/2A below A; derived from the motor code `model/MotorModel.cpp:97` (Section 6).

Inertia tensor. The 3×3 symmetric matrix mapping angular velocity to angular momentum — how a body resists rotation about every axis. Carried in two conventions; see *per-unit-mass tensor* in Section E.2 (Section 4).

- Lapse rate.** The rate of temperature change with altitude. The 1976 US Standard Atmosphere prescribes one constant lapse rate per layer, which makes pressure and density integrable in closed form; the coded sign is the negative of the standard's `sim/USStandardAtmosphere.cpp:114` (Section 8).
- Mach number.** Speed over the local speed of sound. The atmosphere can supply the speed of sound `sim/USStandardAtmosphere.cpp:143`, but no Mach number is computed anywhere and C_d is speed-independent (Sections 8 and 9).
- Parallel-axis theorem.** (Huygens–Steiner.) The rule for shifting a centroidal inertia tensor to a point displaced by $\vec{d}$: $I = I_{\text{cm}} + m((\vec{d} \cdot \vec{d}) I_3 - \vec{d} \vec{d}^T)$. Coded as `parallelAxisTerm model/parts/Part.cpp:19` (Section 4).
- Quaternion.** A four-component, gimbal-lock-free representation of 3-D rotation, stored (x, y, z, w) project-wide. `StateData`'s orientation quaternions are zero-initialized — not a rotation at all — and never written `sim/StateData.h:46` (Section 7).
- Recovery; dual deployment.** Parachute or streamer descent; dual deployment fires a drogue at apogee and the main chute at low altitude. No deployment event of any kind exists (Section 13).
- Reynolds number.** The ratio of inertial to viscous forces in a flow, which governs skin friction. Sutherland viscosity is implemented `sim/USStandardAtmosphere.cpp:150`; no Reynolds number is computed (Section 9).
- Runge–Kutta 4 (RK4).** The classical fixed-step fourth-order integrator: four derivative evaluations per step, combined with weights $\frac{1}{6}(1, 2, 2, 1)$. QtRocket's default solver `sim/RK4Solver.h:45` (Section 7).
- Runge–Kutta–Fehlberg 4(5) (RKF45); embedded pair.** An adaptive integrator forming a fourth- and a fifth-order estimate from the *same* six stages; their difference is a free local-error estimate that drives the step controller `sim/RK45Solver.h:120` (Section 7).
- Specific impulse (I_{sp}).** Impulse per unit weight of propellant, $I_{tot}/(g_0 m_{prop})$, in seconds. Computed and never read `model/MotorModel.cpp:130`; the constant- I_{sp} assumption nonetheless underlies the derived motor mass curve (Section 6).
- Standard atmosphere.** A reference model of temperature, pressure, and density versus altitude. The US Standard Atmosphere 1976 uses seven piecewise-linear temperature layers; QtRocket implements the closed forms, validated to 80 km `sim/USStandardAtmosphere.cpp:93` (Section 8).
- Static margin.** The CP–CG separation in calibers; positive (CP aft) means a small angle of attack produces a restoring moment. In QtRocket a subtraction by construction — `cp() - cg()` on a shared datum `sim/Aero.h:25` — computed nowhere in production (Section 9).
- Thrust curve.** The measured (time, thrust) sample table from a motor's certification test. Interpolated linearly, zero outside the burn window `model/ThrustCurve.cpp:47` (Section 6).
- Total impulse.** $I_{tot} = \int F dt$ (N·s), which fixes the impulse class; the RASP loader derives it from the samples by trapezoidal integration `model/RASPLoader.cpp:189` (Section 6).
- Weathercocking.** A statically stable rocket's rotation into the wind during boost. It requires wind and attitude dynamics; QtRocket has neither — the wind model returns the zero vector and has no call sites `sim/WindModel.cpp:16` (Sections 9 and 13).

E.2 QtRocket terms of art

- Abut.** The seat kind meaning rim-to-rim at equal radii, with the gap a signed forward standoff `model/parts/Placement.h:34`. The default seat, and the only one reachable from the REPL (Section 5).
- Dirty flag.** Two staleness flags propagated up the parent chain: `needsRecomputing` gates the compos-

ite mass-properties cache `model/parts/Part.h:239`; `placementDirty` gates the placement cache `model/parts/Part.h:283` and is set only by structural edits — a burn moves no geometry ([Section 4](#)).

Dormant. Status vocabulary inherited from the Architecture Review: *built and tested but unreachable from any production code path*. Distinct from *partial* (reachable, with named gaps) and *absent* (no trace in the tree); [Section 13](#) carries the inventory.

Fail-closed. Refusing to produce a number from an incoherent model: a failed placement solve throws with the first located diagnostic rather than returning a silently wrong mass or inertia `model/parts/Part.cpp:182`; legacy design files are rejected, never migrated ([Sections 4](#) and [11](#)).

Hard, located failure. A refusal that carries *who, into what, where, and by how much* — the `OverlapDiagnostic` fields `model/parts/Placement.h:102`. The antonym throughout this document is silent miscomputation ([Section 5](#)).

Heavy/fast test split. The ctest label `heavy` plus `QTROCKET_FULL_LADDER=1` `tests/CMakeLists.txt:96`: the fast loop flies one motor per impulse class; the heavy registration re-runs the flight sweeps over the complete 1/4A→M ladder ([Section 12](#)).

Implicit-origin ramp. When a query precedes a thrust curve’s first sample, thrust ramps linearly from an implicit (0, 0) origin `model/ThrustCurve.cpp:76` — the guard that replaced an out-of-bounds read which made apogee vary run to run ([Section 6](#)).

Mass-delta cache. The composite mass/CM/inertia cache, rebuilt iff structurally dirty or the composite mass changed — an exact `!=` against a NaN-seeded sentinel `model/parts/Part.cpp:137`. It recomputes every step during a burn and freezes bit-stably after burnout ([Section 4](#)).

Motor DFS. `findMotorInTree`, the first-match depth-first search that re-borrows the single `Motor` handle after every tree edit `model/RocketModel.cpp:15`. It embodies the single-motor assumption: a second motor is silently ignored by thrust ([Section 6](#)).

One-step timestamp offset. The recorder appends the state reached at $t + h$ against timestamp t `sim/Propagator.cpp:133`; every consumer of the flight history inherits the offset ([Section 7](#)).

Pad hold. The kinematic clamp standing in for a pad’s normal force: until the first step ending above $z = 0$ under power, position and velocity are held at rest `sim/Propagator.cpp:105`, so the implicit-origin ramp’s near-zero first step cannot terminate the flight ([Section 7](#)).

Per-unit-mass tensor. A part’s own tensor is geometric — inertia divided by mass, m^2 — while composites are full mass-weighted $\text{kg}\cdot\text{m}^2$ about the composite CM `model/parts/Part.h:30`. Factoring density out lets a motor’s tensor stay constant while its mass burns down ([Section 4](#)).

Pinned commit. Commit `254cd41`, the revision every code citation anchors to ([Section 1](#)); reproduction recipe in [Section F](#). Line numbers are anchors into that revision, not absolutes.

Placement intent. What the tree stores about position: a feature-to-feature physical relationship (“this rim seats against that rim”), never an opaque offset or a resolved coordinate. Realized as the `StationLink` ([Section 5](#)).

Resolver. `resolvePlacements`, “The single authority for absolute placement: intent (`StationLink`) in, geometry (`Pose`) out” — both the simulator and the visualizer consume its output, “so the two cannot disagree” `model/parts/Placement.h:148` ([Section 5](#)).

Seam. An interface or capability built and tested ahead of its consumer — the load-bearing metaphor of this document and its companions. The canonical in-tree example: per-step mass/CG/inertia snapshots are “the seam the 6-DOF rotational ODE will read” `sim/StateData.h:59` ([Section 13](#)).

Seat kind. The closed three-value enum governing a link’s radial-compatibility check and the sign of its gap `model/parts/Placement.h:32`: `Abut` (rim to rim), `NestInBore` (child outer diameter inside

the parent bore; the gap an insertion depth moving the child aft — the one sign-flipping seat), and `OnSurface` (child on the parent’s outer wall; the gap an axial standoff). Closed because a rigid axisymmetric stack admits exactly three radial relationships (Section 5).

Solid-host rule. A host’s radius budget at a station is its bore when hollow and its outer skin when solid `model/parts/Part.cpp:314`; an offender fits iff its outer radius stays within that capacity — one predicate for both nesting and poke-through (Section 5).

Station. A position along a part’s axis as a fraction of *live* length in $[0, 1]$: 0 the aft plane, 1 the fore plane, mapped to $z = (s - 1)L$ on every read `model/parts/Part.cpp:305` — so editing a length moves the seam (Section 5).

StationLink. The only stored spatial relationship: a parent station, a child station, a seat-signed gap in meters, a seat kind, and a quaternion reserved for 6-DOF `model/parts/Placement.h:49`. A default-constructed link means “child fore plane abuts parent aft plane” `model/parts/Placement.h:46`, so the common stack costs zero authored numbers (Sections 5 and 11).

stdout protocol. The CLI’s machine-readable contract: one OK/ERR line per command, WARN: on non-nominal termination, CSV paths, with logging demoted to keep those lines distinct `cli/CliMain.cpp:16`. The termination-reason switch has no default, so a new reason fails to compile until handled `cli/Repl.cpp:39` (Section 10).

Termination taxonomy. The five-value `TerminationReason` enum — `Nominal`, `NoLiftoff`, `NonFiniteState`, `MaxSimTimeExceeded`, `IntegratorError` — partitioning every exit from the run loop `sim/Propagator.h:35` (Section 7).

Tip datum. The whole-assembly reference frame planted at the root part’s fore plane — the nose tip. Composite CM and CP are both reported on it, which is what makes the static margin a plain subtraction `model/parts/Part.cpp:231` (Sections 5 and 9).

Two burnout clocks. The verified asymmetry that `getMass` gates the burn on the metadata burn time `model/MotorModel.cpp:36` while `getThrust` gates on the curve’s last sample time `model/MotorModel.cpp:72`; RASP motors are immune by construction, RockSim motors are not (Section 6).

ULP tolerance. The tests’ floating-point idiom: error budgets measured in units in the last place, the spacing between adjacent representable doubles. `utils::floatingPointEqual` defaults to a 4-ULP band `utils/math/UtilityMathFunctions.h:21` — the same width GoogleTest’s `EXPECT_DOUBLE_EQ` enforces — and the idiom recurs from factory round-trips (Section 4) to burnout mass lookups (Section 6).

Vacuum step-runaway clamp. The RKF45 cap $h \leq h_{max}$, default ten times the seeded step `sim/RK45Solver.h:144`. Required, not cosmetic: vacuum coasting is integrated exactly by both embedded orders, so the error estimate vanishes and the step would otherwise grow five-fold indefinitely (Section 7).

F Reproducibility & colophon

This whitepaper is itself a small, self-contained build: a contributor who clones the repository can regenerate the PDF, with every diagram, from the sources checked into `docs/DesignWhitepaperLatex/`. The layout and style contract are inherited from the Architecture Review (`docs/ArchitectureReviewLatex/`), which in turn inherited them from the Part Placement design whitepaper (`docs/PartPlacementDesignLatex/`), so the three documents read as one series. This appendix records the build pipeline, the revision every claim is pinned to, and how the document was produced and verified.

F.1 Directory layout and pipeline

The master file `DesignWhitepaper.tex` carries only the document scaffolding — title block, front matter, and an ordered list of `\input` directives — while the prose lives in `sections/`, one file per section or appendix. The shared style contract is isolated in `preamble.tex`: the color palette, the fonts (XCharter text, `newtxmath`, Inconsolata mono), the callout environments (including the `physbox` added for this document, which keeps physics background visually separate from statements about the code), and the domain macros (`\src`, `\srcf`, `\tikzfig`, the feature-status markers) that the section files depend on. A section file is just a body, never a standalone $\text{\TeX}$ program.

Every figure is hand-built TikZ under `diagrams/tikz/`, included directly by `\tikzfig` and compiled in line by the main engine — there is no PlantUML stage and no external image asset. The UML class, sequence, and component figures use dedicated styles added to `diagrams/tikzstyles.tex` for this document (three-compartment class boxes, inheritance/realization/composition arrowheads, lifelines). The build is two commands:

```
make          # latexmk -pdf -shell-escape DesignWhitepaper.tex
make veryclean # remove aux files, the minted cache, and the PDF
```

The engine is `pdflatex`, driven by `latexmk` so the multi-pass dance (cross-references, the table of contents, `cleveref`, the bookmark tree) converges automatically. The one non-default requirement is `-shell-escape`: listings are typeset by `minted`, which shells out to `pygmentize`. Prerequisites are a $\text{\TeX}$ distribution providing `latexmk`, `pdflatex`, and `minted` with its `pygmentize` helper.

F.2 The revision under description

The document describes one tree state: commit `254cd41` (“Major comment cleanup”), the tip of the `development` line at the time of writing, described against a byte-identical working tree in July 2026. Every `\src` citation refers to that revision; line numbers will drift as the tree moves, so a reader on a later commit should treat them as anchors into `254cd41`, not absolutes.

The ground-truth policy is the one stated in [Section 1](#): the source at the pinned commit and the two companion documents — the Architecture Review and the Part Placement design whitepaper — are authoritative; every other artifact under `docs/` is historical and was deliberately not relied on. Where a companion document and the code disagree, this document reports the code and says so in place.

F.3 How this document was produced and verified

The document was produced in three staged passes, each a fan-out of independent workers.

Deep reads. Twelve parallel deep-reads produced working dossiers: nine full reads of the code subsystems (the part tree and placement machinery, the controller and `utils` foundation, motors, the simulation core, the environment models, aerodynamics, the GUI and visualizer, the CLI and persistence, and the build/test/CI stack), a digest of the two companion documents, and two research dossiers on OpenRocket — architecture and simulation methods — compiled from primary sources (`openrocket.info`, the OpenRocket documentation, Niskanen’s thesis and the technical documentation, and the `openrocket/openrocket` repository at `release-24.12` and `unstable`, fetched July 2026). Each dossier carried a citation ledger of exact `file:line` anchors re-checked against the source before use.

Writing. Each section was then written from its dossiers — writers were forbidden to assert anything about the code not present in a dossier or re-verified in the source directly, with general

physics theory admitted only as clearly-framed background. The diagrams were authored the same way and test-compiled standalone before inclusion.

Adversarial verification. Finally, one verifier per section re-checked the finished text against the source itself: every citation opened and its line number and substance confirmed, every listing diffed verbatim, every equation re-derived against the code, every count and constant re-taken (over two thousand checks; roughly eighty line-number corrections and seventy substantive claim corrections were applied). Three cross-cutting passes followed: a consistency sweep that adjudicated shared facts across sections against the source, a diagram pass that re-verified twelve of the thirteen figures element-by-element and recompiled them (the thirteenth, the front-end component map, was authored afterward from already-verified section text and test-compiled the same way), and a completeness critique whose findings were folded back into the text. OpenRocket-side claims were verified for faithful transmission from the research dossiers; the few that could not be pinned to a release tag are marked as branch-verified where they appear.

Two behaviors were confirmed by execution during the underlying review series rather than by reading: the CLI command inventory ([Section 10](#)) and end-to-end flight behavior through the built `qtrocket-cli` binary. The dossiers themselves are working notes, not part of the document, and live untracked under `notes/`.